CHAPTER 4

INTRODUCTION TO .NET FRAMEWORK

Learning Objectives

4.1 INTRODUCING ASP.NET

When Microsoft created .NET, it wasn't just dreaming about the future—it was also worrying about the headaches and limitations of the current generation of web development technologies. The .NET Framework allows programmers to develop Internet applications with an ease that was never provided before. To develop Internet applications, the .NET Framework is equipped with ASP.NET.

ASP.NET is a web application development framework. It is a **server side scripting technology** and ASP.NET scripting code is executed by **web server**. ASP.NET allows programmers to build dynamic web sites, web applications and web services using compiled

4.2

languages like VB.NET and C#. It was released in January 2002 with version 1.0. ASP.NET is descendant to Microsoft's Active Server Pages technology. Furthermore, it is built on the Common Language Runtime (CLR) giving freedom to programmers to write their code using any supported .NET language. CLR also allows objects written in different languages to interact with each other. The CLR makes development of web applications simple.

An ASP.NET application can includes:

Web pages (.aspx files): These are the main building blocks of ASP.NET applications. Web pages are also called as **Web Forms**. It is used to create User Interface between a Web application and its user.

Web services (.asmx files): A web service is a web application which is basically a class consisting of methods that could be shared and used by the applications on other computers and other platforms. A Web service does not have a user interface.

Code-behind files: These files provide the ability to separate the executable code from the presentation code. We can write the HTML code in a page file with a **.aspx** extension, and we can write the C# or VB.NET code in the code-behind file with a **.cs** or **.vb** extension, depending on its language.

Configuration file (web.config): This file is used to create database connections, to modify user authentication and authorization, to set image path for the uploading images and managing larger size files while uploading to the web server. It contains application-level settings that configure everything from security to debugging and state management.

Global.asax: The Global.asax file, sometimes called the ASP.NET application file, provides a way to respond to application or module level events in one central location. You can use this file to implement application security, as well as other tasks. The Global.asax file is configured so that any direct HTTP request, through URL, is rejected automatically, so users cannot download or view its contents.

Other components. Components are the compiled assemblies with useful functionality. Components allow us to separate business and data access logic and create custom controls.

> **Note:** A virtual directory can hold additional resources that are used by ASP.NET web applications, It includes style sheets, images, XML files, etc. In addition, you can extend the ASP.NET model by developing specialized components known as HTTP handlers and HTTP modules, which can plug into your application and take part in the processing of ASP.NET web requests.

4.2.1 Features of ASP.Net

There are following features of ASP.Net:

Object-Oriented: ASP.NET features a completely **object-oriented programming model.** It includes an **event-driven, control-based** architecture that encourages code encapsulation and code reuse. It reduces the amount of code required to build large applications and makes development simpler and easier to maintain.

Integrated Development Environment: ASP.NET is integrated with Visual Studio .NET, which provides a GUI designer, a rich toolbox, and a fully integrated debugger. This allows the development of applications in a **What You See is What You Get (WYSIWYG)** manner. Therefore, creating ASP.NET applications is much simpler.

Language Independent: ASP.NET is **Language Independent.** It allows programmers to code in any supported .NET language such as **C#, VB.NET, J#.NET,** and many other languages that have third-party compilers.

Validation: ASP.NET **validates** information entered by the user without writing a single line of code **using validation controls.**

High Performance: ASP.NET is dedicated to **high performance**. ASP.NET pages and components are compiled on demand instead of being interpreted every time they're used. It provides high performance by using early binding, just-in-time compilation, native optimization, and caching services.

Secure: ASP.NET applications are **secure** and use a set of default authorization and authentication schemes. However, we can modify these schemes according to the security needs of an application.

4.2 SETTING UP THE DEVELOPMENT ENVIRONMENT

ASP.NET is based on the **CLR**, class libraries, and other tools integrated with the Microsoft .NET Framework. Therefore, to develop and run the ASP.NET applications, we need to install the **.NET Framework.** The .NET Framework is available in two forms:

- .NET Framework SDK (Software Development Kit).
- Microsoft Visual Studio .NET Integrated Development Environment (IDE).

To develop any web application, we need Internet Information Server (IIS) configured on either the development machine or another machine on the network. The .NET Framework must be installed on the machine on which IIS is configured.

We can create ASP.NET applications by just installing the .NET Framework SDK and configuring an IIS server. In this case, we need to use a text editor, such as Notepad, to write the code. Therefore, we have to work without the IDE and other integrated tools that come with Microsoft Visual Studio .NET. In other words it means that text editors do not offers many features that would make ASP.NET development easier. So, text editors are not the ideal applications for creating ASP.NET pages.

Hence, installing Microsoft Visual Studio .NET is recommended, to get the full benefit of the .NET features. Microsoft Visual Studio .NET allows us to manage entire web sites, and it provides features such as IDE, creating and deleting virtual directories, working with the databases, and dragging and dropping HTML components. To make code easy to read, it colors the ASP.NET code automatically.

The development machine must have a **Web Browser** to run web applications. It must have a **Database Management System** to develop ASP.NET Database Applications.

4.3 ELEMENTS OF AN ASP.NET PAGE

Before we start creating a web page, we should be familiar with basic ASP.NET syntax.

4.3.1 Directives

ASP.Net directives are instructions to specify optional settings, such as registering a custom control and page language. These settings describe how the web forms (.aspx) or user controls (.ascx) pages are processed by the .Net framework. The syntax for declaring a directive is:

```
<%@ directive_name attribute=value [attribute=value] %>
```

For example, to specify the language as C# for any code output to be rendered on the page, use the following line of code:

```
<%@ Page Language = "C#" %>
```

This line indicates that any code in the block, <% %>, on the page is compiled by using C#.

4.3.2 Code Declaration Block

Code declaration block used to process ASP.NET pages. This is the block where we provide the whole functionality. This code is also compiled into MSIL. The Syntax is:

```
<script runat="server" [language=codelanguage]>      //CODE DECLARATION BLOCK
      // Code Here
</script>
```

In this Syntax:

- **runat="server"** indicates that the code is executed at the server side.
- **[language=codelanguage]** indicates the language that is used. We can use VB, C#, or JScript.
- The square brackets indicate that this attribute is optional.

For example:

```
<script runat="server">      //CODE DECLARATION BLOCK
        void SetMessage(Object Sender, EventArgs e)
        {
                lblMessage.Text = "Your Name is " + txtMessage.Text;
        }
</script>
```

4.3.3 Code Render Block

Code render block contains the instructions that ASP.NET uses to produce output. Code render block is denoted by <% %>. The syntax used in the block, <% %>, must correspond to the language specified in the @ Page directive. Otherwise, an error is generated when we display the page in a web browser.

To render the output on a page, we can use the **Response.Write()** method. For example, to display the text **"Working With ASP.NET"** on a page, use the following code for **<%@ Page Language="C#" %>**.

```
<% Response.Write("Working With ASP.NET"); %>
```

We can use HTML tags in the argument passed to the Response.Write() method. For example, to display the text in bold, we use the following code:

```
<% Response.Write("<B> Working With ASP.NET </B>"); %>
```

4.3.4 HTML Control Syntax

We can declare several standard HTML elements as HTML server controls. Use the HTML element and add the attribute **runat=server**. This causes the HTML element to be treated as a server control. It is now programmatically accessible by using a unique ID. HTML server controls must reside within a <form> section that also has the attribute **runat=server**.

Note: Before proceeding with the discussion on ASP.NET, Let's have a brief introduction about the **Visual Studio Integrated Development Environment**.

4.4 THE VISUAL STUDIO IDE

Microsoft Visual Studio is an Integrated Development Environment (IDE) for creating, documenting, running and debugging programs written in a variety of .NET programming languages. It provides a complete set of development tools for building web applications, web services, desktop applications and mobile applications. The components stay the same regardless of language, making it very easy to switch projects and languages.

The Visual Studio IDE consists of several sections, or tools, that the developer uses while programming. In the following section, we have discussed some of the important tools of **Visual Studio IDE** to be familiar with the Visual Studio IDE. As we view in figure **(Figure 4.1)**, we generally have three main sections in view:

- The Toolbox on the left
- The Solution Explorer on the right
- The Code/Design view in the middle

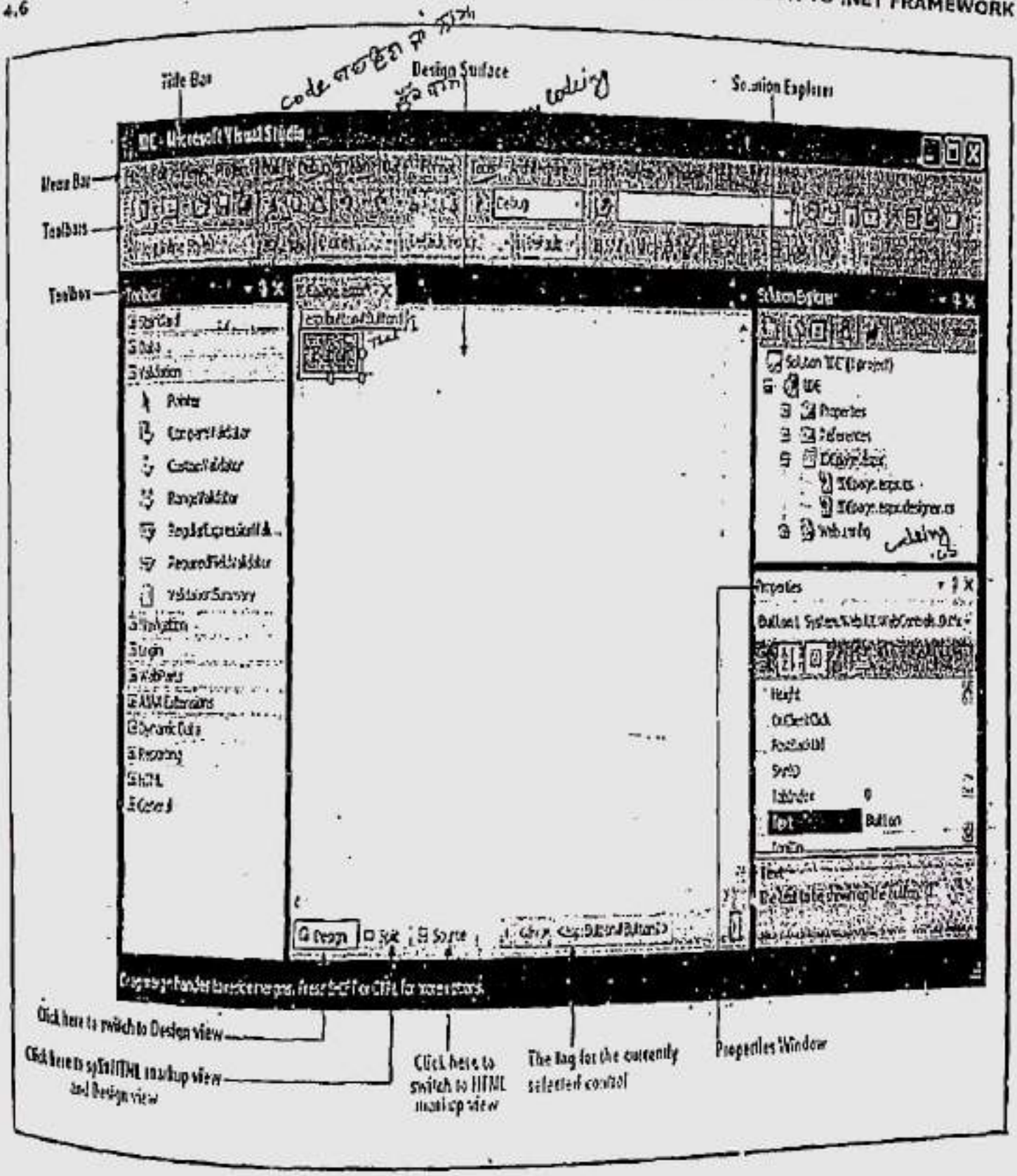


Figure 4.1: Visual Studio IDE

4.4.1 Toolbox

The Toolbox works in conjunction with the design view. Its primary use is providing the controls that we can drag onto the design surface of a form. However, it also allows us to add controls that we create ourselves (controls can be created and used in any language). The content of the Toolbox depends on the current designer we are using as well as the project type. For example, when designing a web page, we will see the set of tabs shown in figure (Figure 4.2).

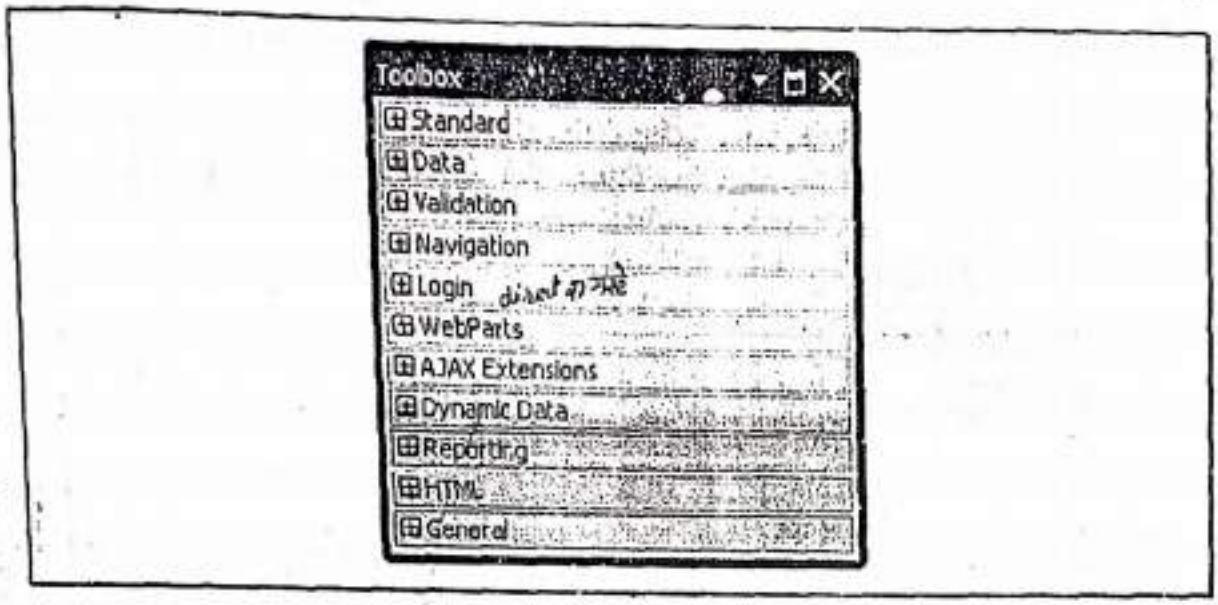


Figure 4.2: Toolbox Tabs

Each tab contains a group of controls To view a tab, click the heading, and the control will be shown as in following figure (**Figure 4.3**).

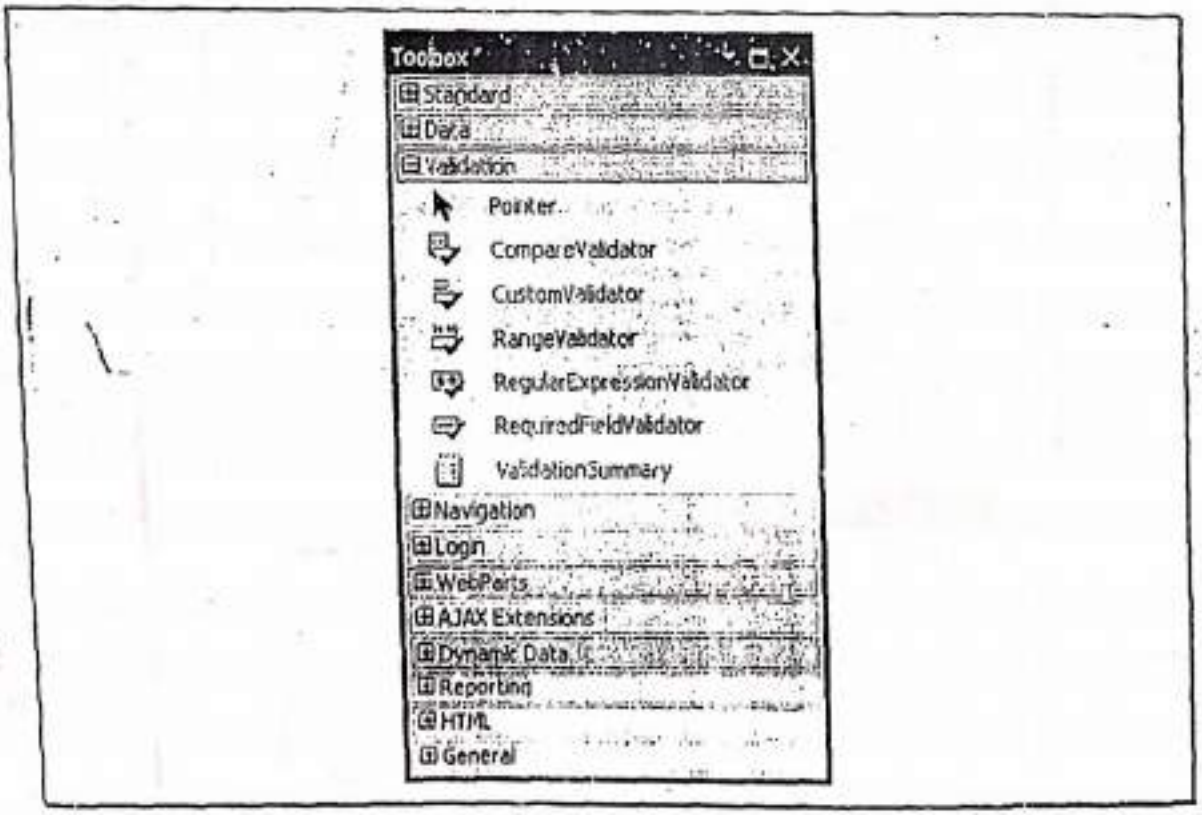


Figure 4.3: Tabs containing controls

4.4.2 Solution Explorer

This is a section that is used to view and modify the contents of the project. The Solution Explorer is the most basic visual filing system. It allows us to see the files that are in the application directory. For example, a Visual Studio Web Application Project, generally have a Web Form object with a code behind file, a Web.config file, references to System components and possibly other modules with special code that is used by the application, shown in figure (Figure 4.4).

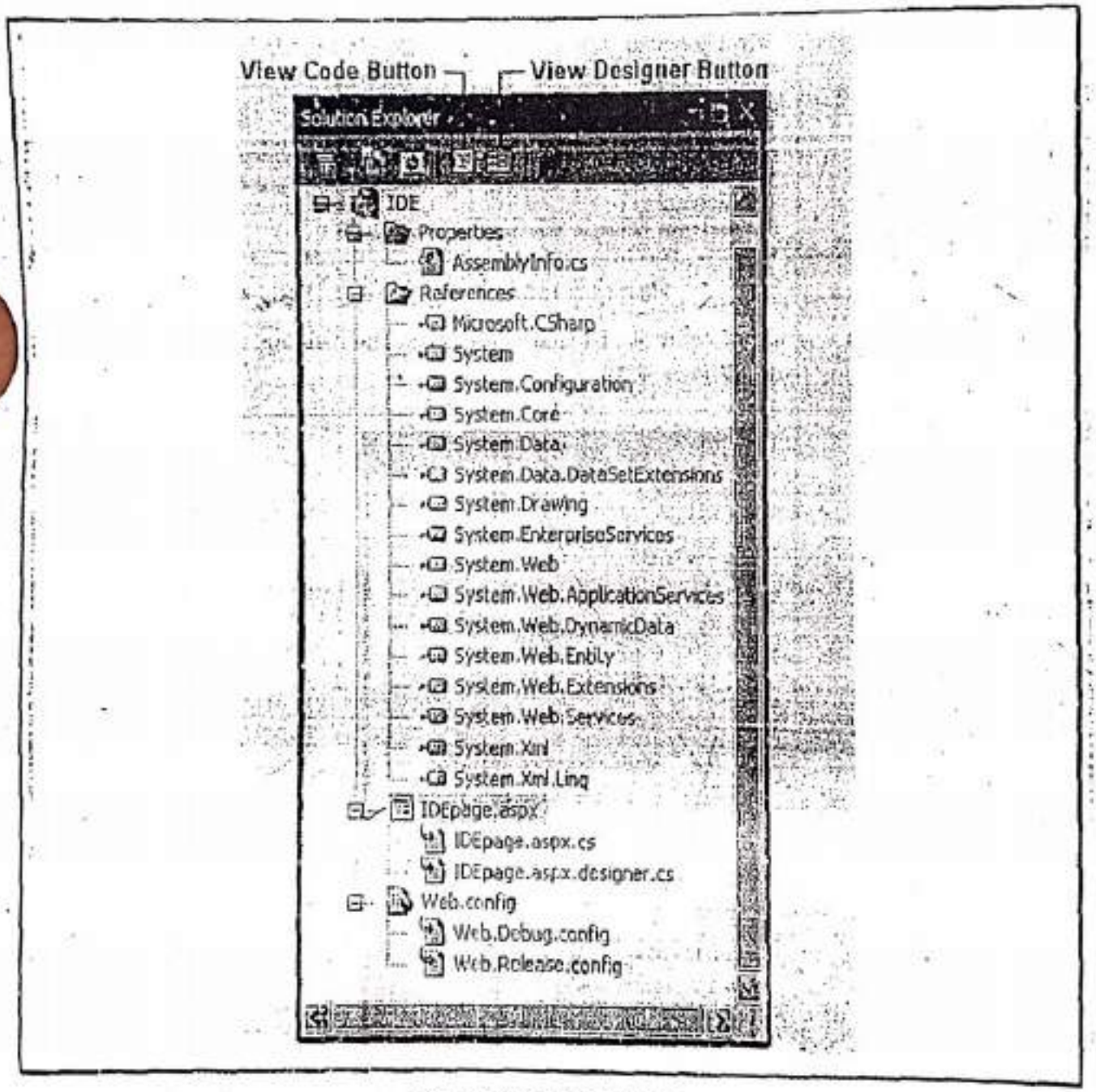


Figure 4.4: Solution Explorer

View Designer Button: Used to switch to the Design View.

View Code Button: Used to switch to the Code View or Code behind file.

4.4.3 Design view

This is where the magic takes place. Forms are designed graphically and this is the area or the surface where the forms can be designed. In other words, the developer has a form on the screen that can be sized and modified to look the way it will be displayed to the application users. Controls are added to the form from the Toolbox, the color and caption can be changed along with many other items.

4.4.4 Code view

Many of Visual Studio's most welcome enhancements appear when we start to write the code that supports our user interface. To start coding, we need to switch to the code-behind view. To switch back and forth, we can use two buttons that are placed just above the Solution Explorer window. The **tooltips** identify these buttons as **View Code** and **View Designer**. When we switch to code view, the code behind file will appear where we can write code for our application. It will be shown as in following figure **(Figure 4.5)**

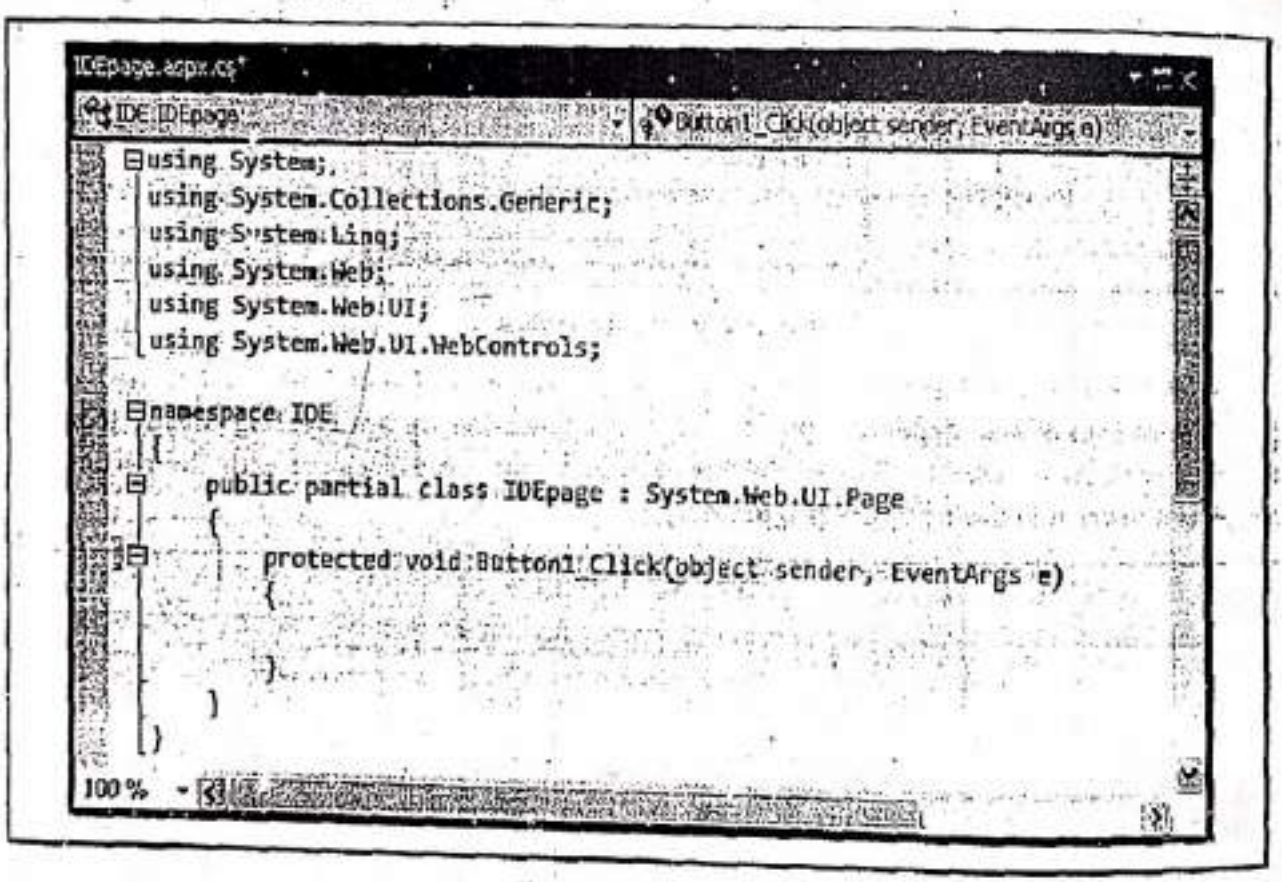


Figure 4.5: Code View

4.4.5 Properties Windows

Each control has its own properties and events. The properties window shows all the control's properties to be changed at design time. These properties can also be changed at run time but with some code. These properties change the way the control is displayed in our application. The properties window for a button control in a web application can be shown as in following figure (Figure 4.6).

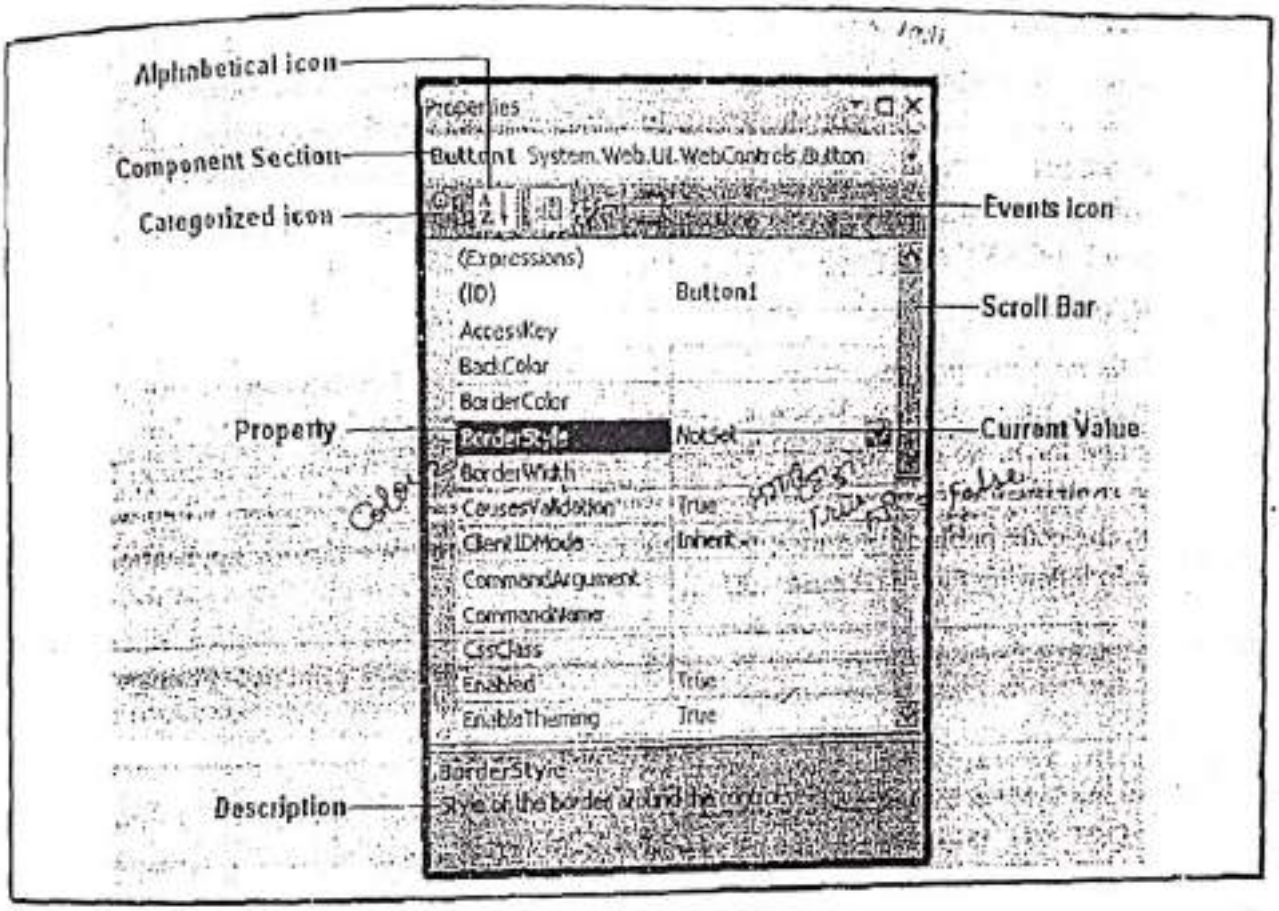


Figure 4.6: Property Window

Besides properties, different controls may have same or different events associated with them. List of events also depends upon the type of application we are creating. **Event Icon** of properties window is used to display the list of events that are associated with the particular control as shown in following figure **(Figure 4.7)**.

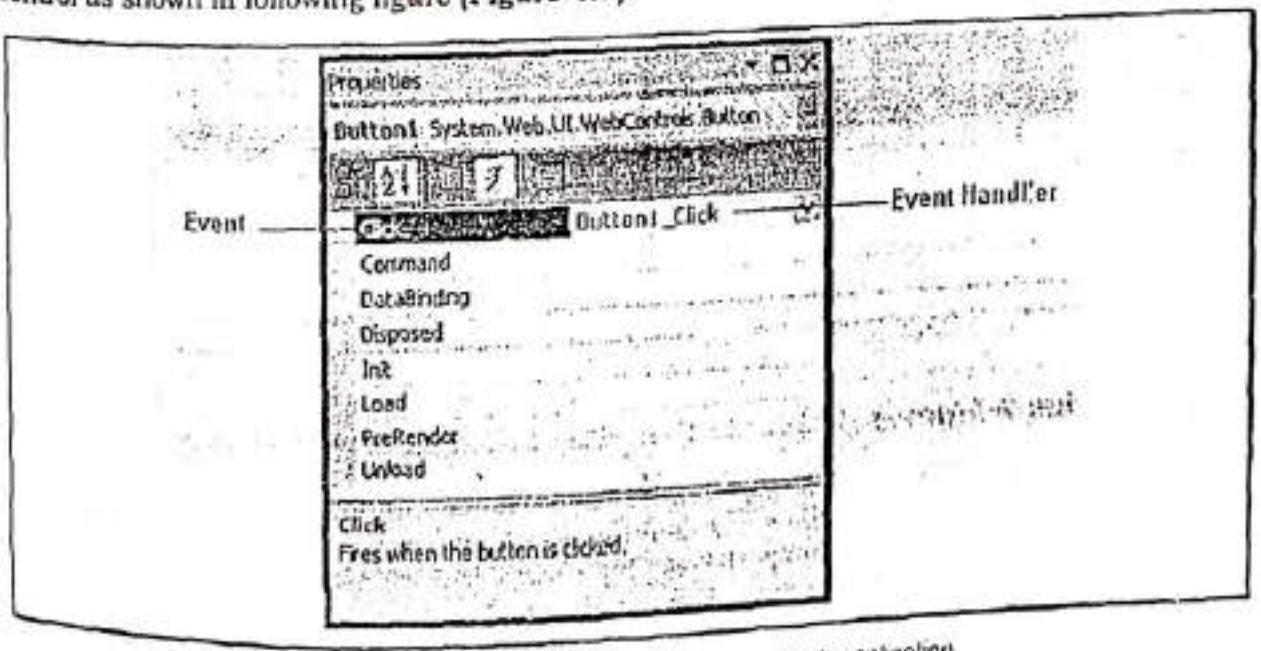


Figure 4.7: Events list of Button control for a Web application

4.4.6 Error List and Task List

The Error List and Task List are two versions of the same window. The Error List catalogs error information that's generated by Visual Studio when it detects problematic code. The Task List shows a similar view with to-do tasks and other code annotations you're tracking. Each entry in the Error List and Task List consists of a text description and, optionally, a link that leads you to a specific line of code somewhere in your project. With the default Visual Studio settings, the Error List appears automatically whenever you build a project that has errors **(see Figure 4.8)**.

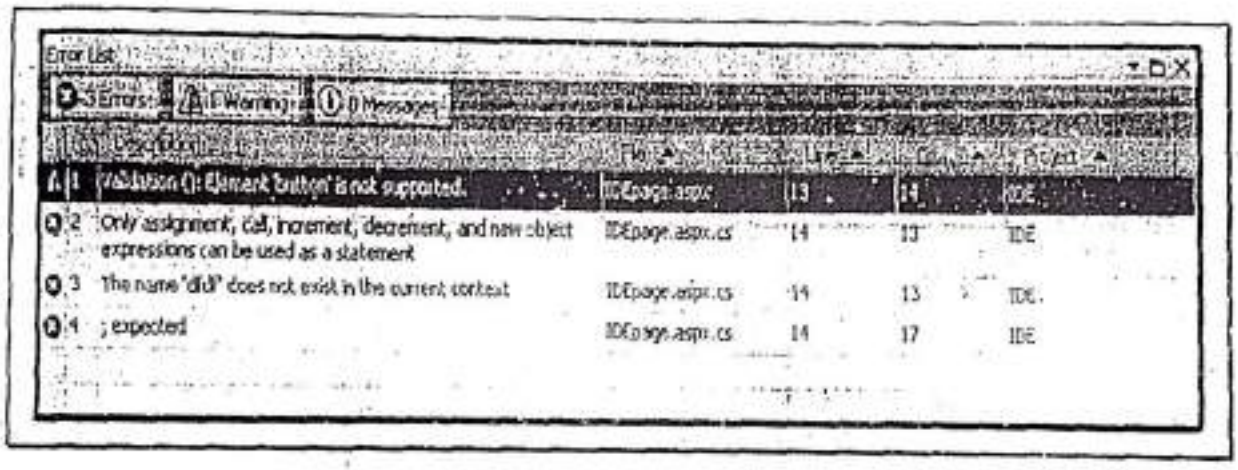


Figure 4.8: Viewing build errors in a project

To see the Task List, choose **View** menu and select option **Task List**. Two types of tasks exist i.e. **user tasks** and **comments**. We can choose which we want. We can see them from the drop-down list at the top of the Task List. User tasks are entries we have specifically added to the Task List. We create these by clicking the Create User Task icon (which looks like a clipboard with a check mark) in the Task List. We can give our task a basic description, a priority, and a check mark to indicate when it's complete **(see Figure 4.9)**.

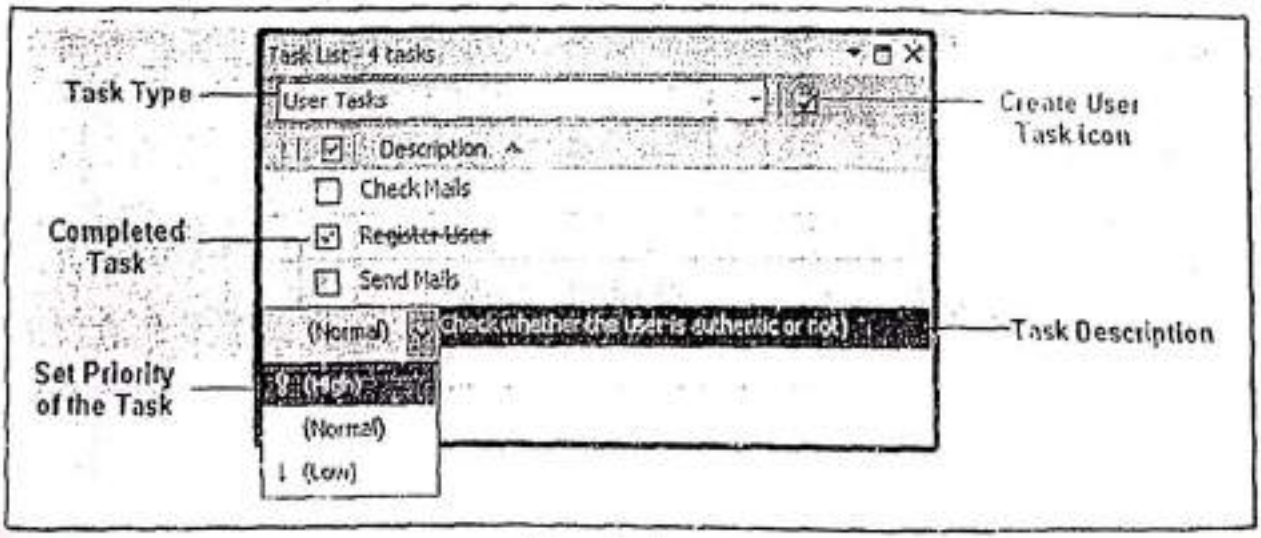


Figure 4.9: Task List

4.4.7 IntelliSense and Outlining

As we are going to program with Visual Studio, we have to be familiar with its many timesaving conveniences. IntelliSense and Outlining are the most important features of Visual Studio IDE. These can be described as follows:

4.4.7.1 Outlining

Outlining allows Visual Studio to "collapse" a subroutine, block structure, or region to a single line. It allows us to see the code that is of our interest, while hiding unimportant code. To collapse a portion of code, click the minus box as shown in figure (**Figure 4.10**). Click the box again (which will now have a plus symbol) to expand it.

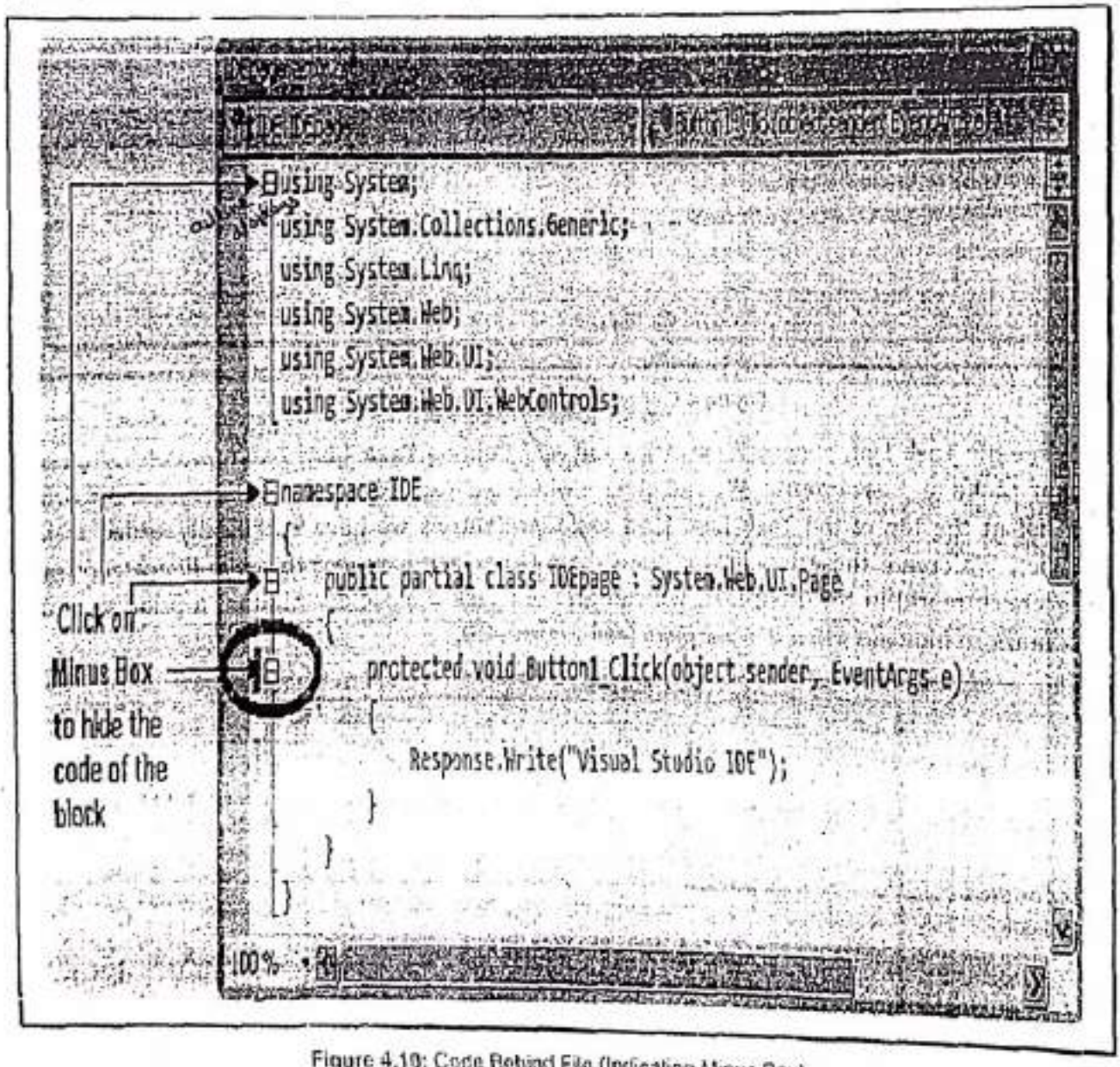


Figure 4.10: Code Behind File (Indicating Minus Box)

When we click on minus box the code written in the block will be collapsed as shown in following, figure (**Figure 4.11**).

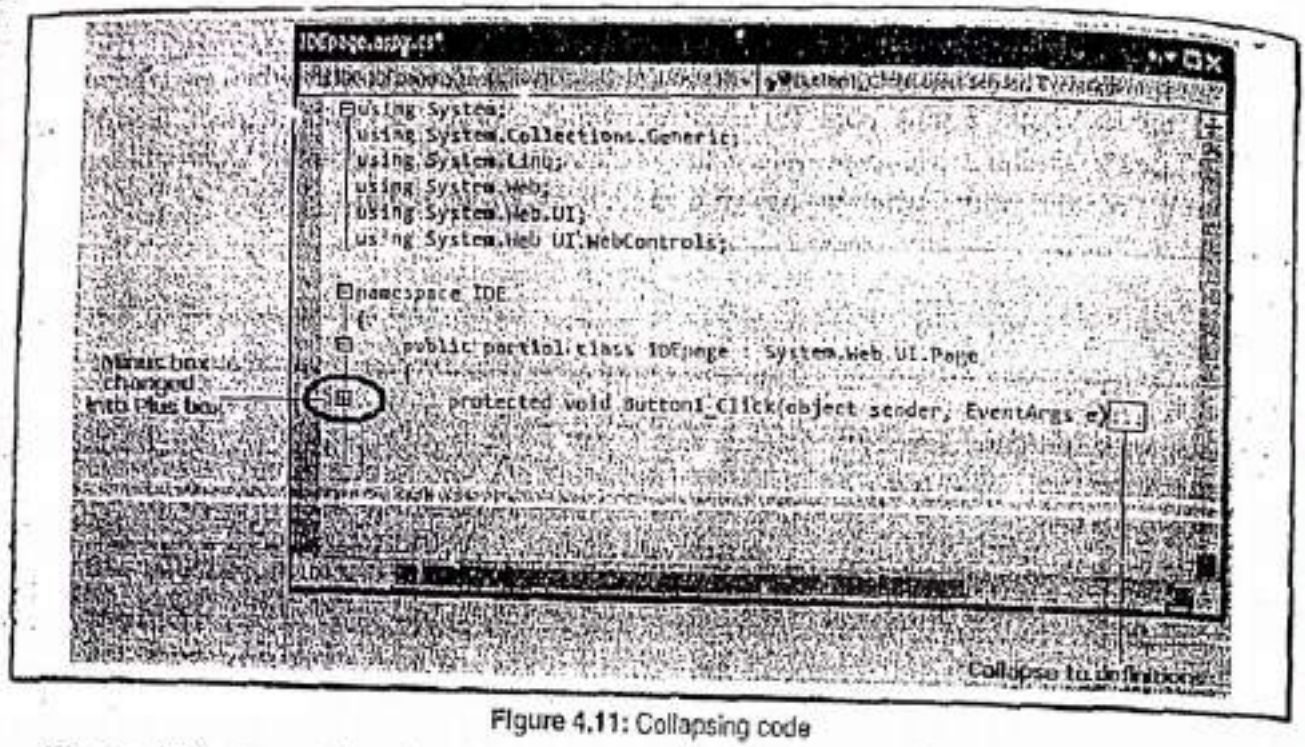


Figure 4.11: Collapsing code

We can hide any section of code. To do so, simply select the code, right-click the selection, and choose **Outlining >> Hide Selection** as shown in following figure **(Figure 4.12)**.

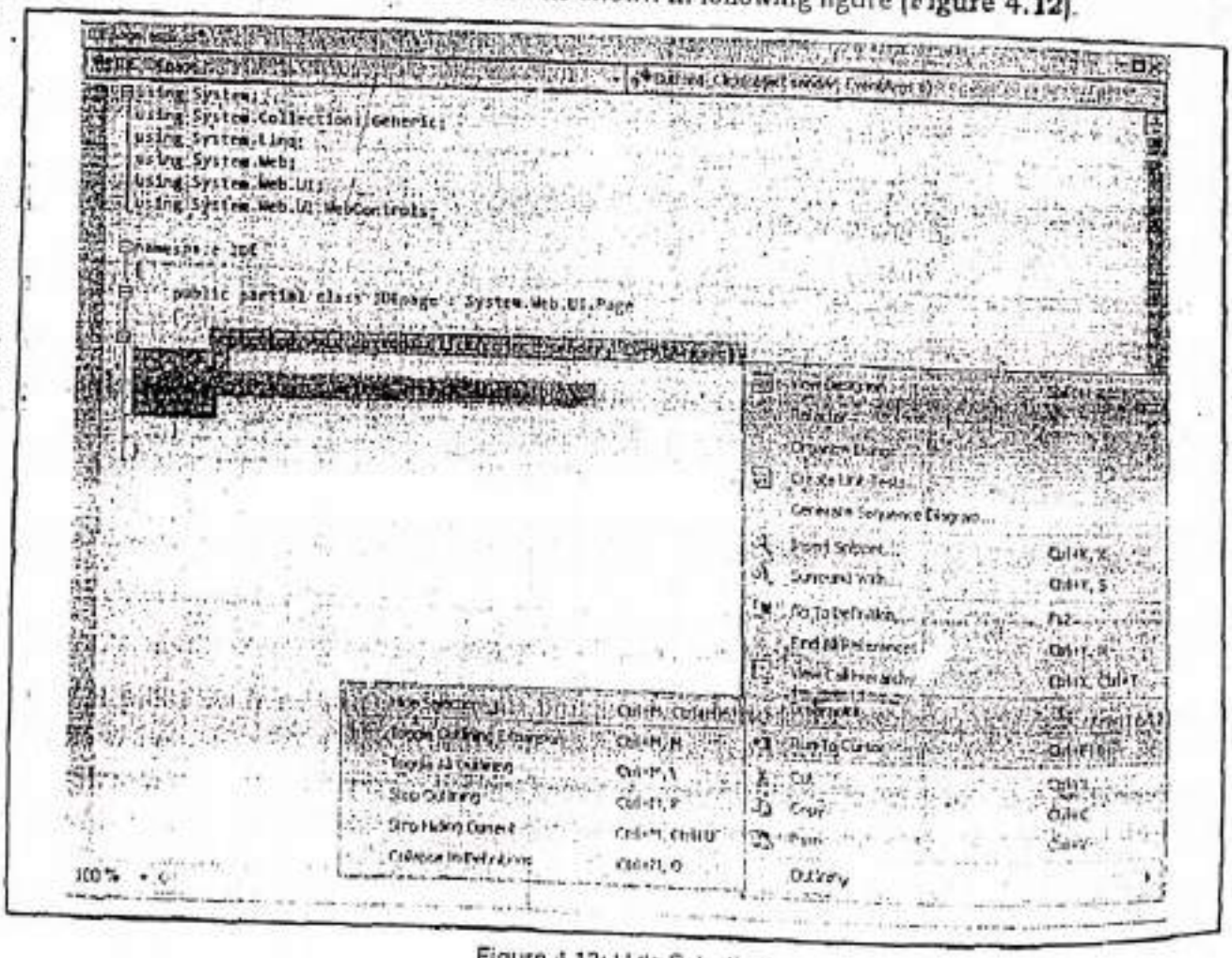


Figure 4.12: Hide Selection

4.14

4.4.7.2 Member List

Visual Studio makes it easy for us to interact with controls and classes. When we type a class or object name, Visual Studio pops up a list of available properties and methods (see Figure 4.13). It uses a similar trick to provide a list of data types when we define a variable and to provide a list of valid values when we assign a value to an enumeration.

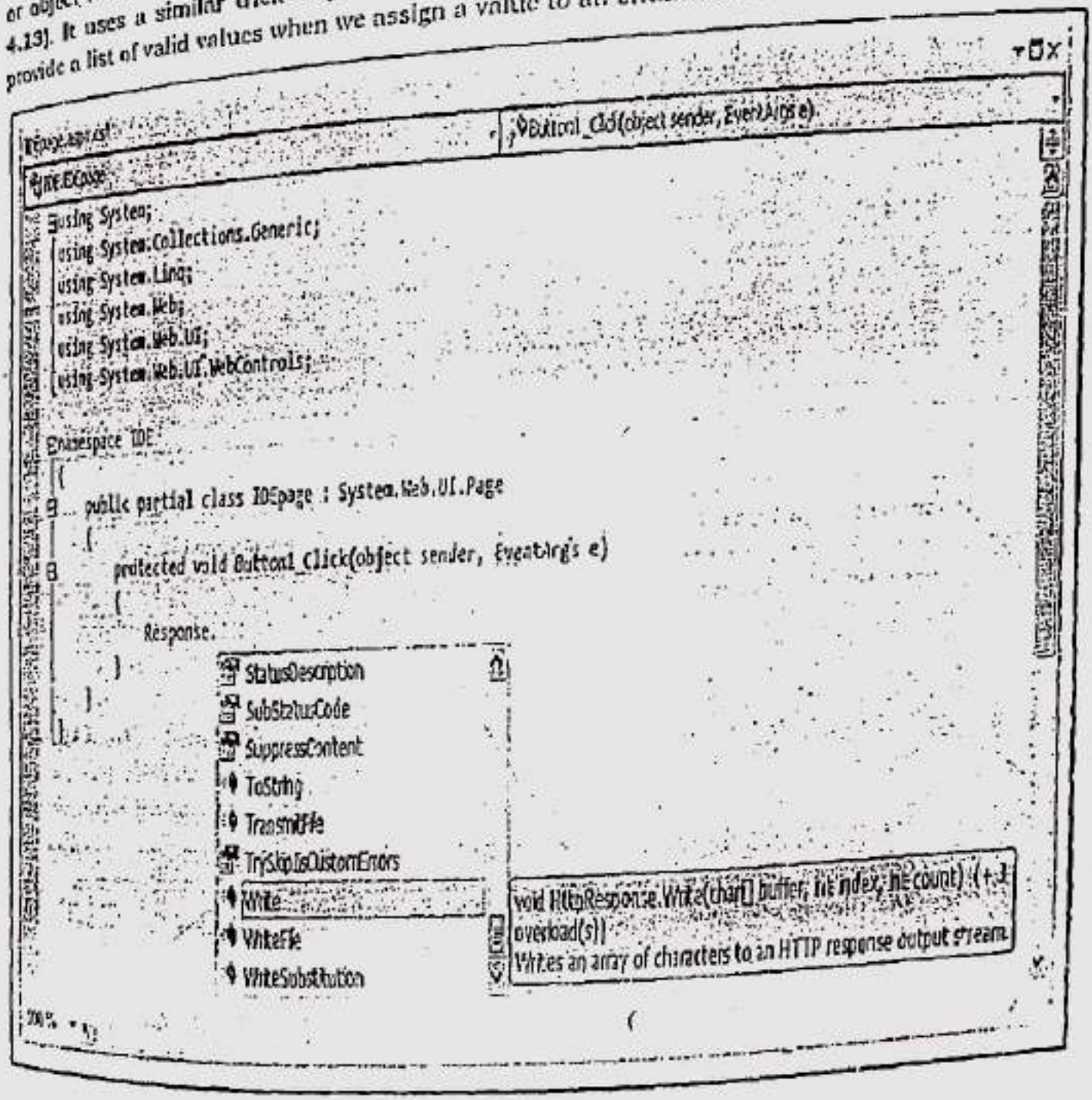


Figure 4.13: IntelliSense at work

Visual Studio also provides a list of parameters and their data types when we call a method or invoke a constructor. This information is presented in a tooltip above the code and is shown as we type. Because the .NET class library heavily uses function overloading, these methods may have multiple different versions. When they do, Visual Studio indicates the number of versions and allows us to see the method definitions for each one by clicking the small up (▲) and down (▼) arrows in the tooltip. Each time we click the arrow, the tooltip displays a different version of the overloaded method (see Figure 4.14).

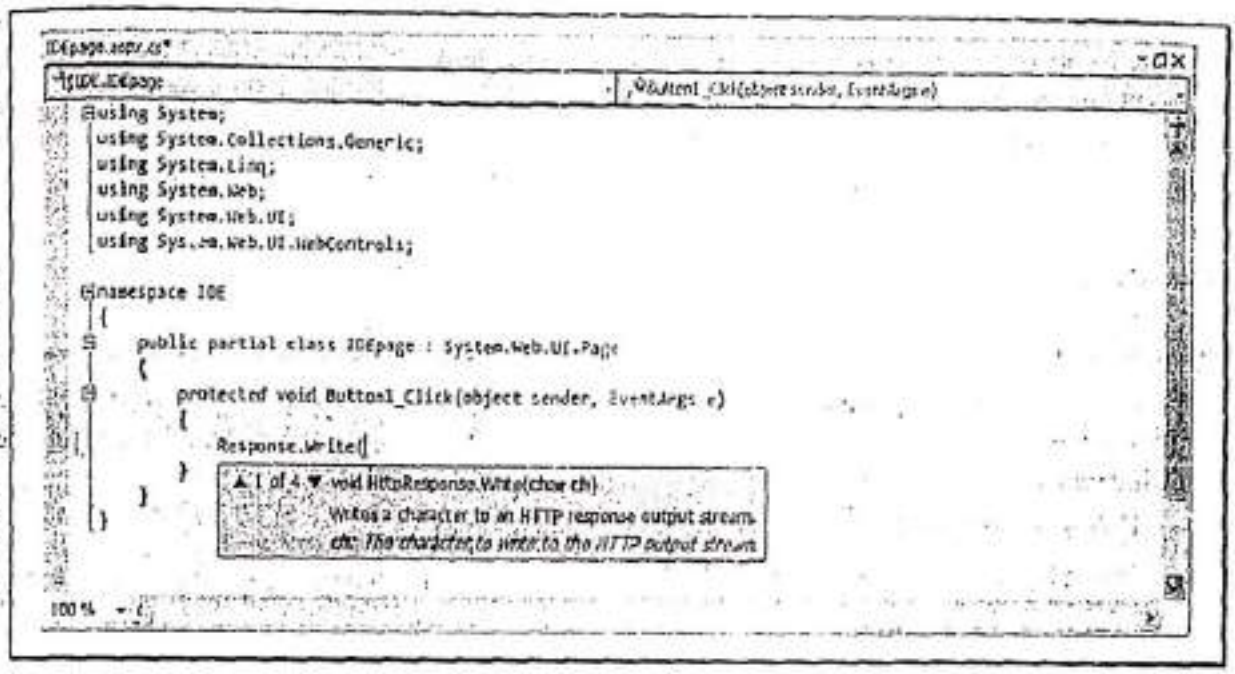


Figure 4.14: IntelliSense with overloaded methods

4.4.7 Error Underlining

One of the code editor's most useful features is error underlining. Visual Studio is able to detect a variety of error conditions, such as undefined variables, properties, methods, invalid data type conversions, and missing code elements. Rather than stopping us, to alert us that a problem exists, the Visual Studio editor quietly underlines the offending code. We can hover our mouse over an underlined error to see a brief tooltip description of the problem (**see Figure 4.15**).

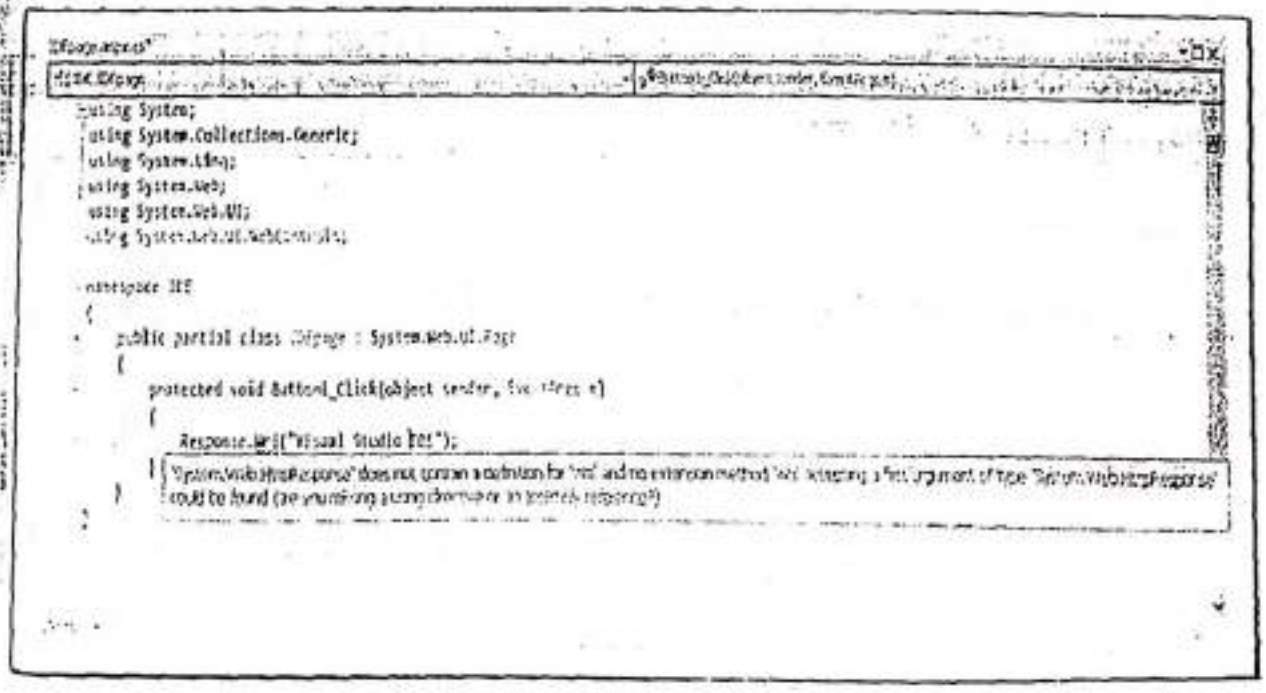


Figure 4.15: Highlighting errors at design time

Visual Studio won't flag our errors immediately. Instead, it will quickly scan through our code as soon as we try to compile it and mark all the errors it finds. If our code contains at least one error, Visual Studio will ask us whether it should continue. At this point, we will almost always decide to cancel the operation and fix the problems Visual Studio has reported. (If we choose to continue, we will actually wind up using the last compiled version of our application, because the .NET compilers can't build an application the has errors.)

4.5 CREATING AN ASP.NET APPLICATION

After gaining and understanding of the basic ASP.NET page syntax, we can now create a simple ASP.NET web application. We can create an ASP.NET web application in one of the following ways:

- Using a Text Editor.
- Using Microsoft Visual Studio .NET IDE.

4.5.1 Using a Text Editor

We can write the ASP.NET code in a text editor, such as **Notepad**, and save the code as an **.aspx** file. We can save the .aspx file in the directory **C:\inetpub\wwwroot**. Then, to display the output of the web page in a Web Browser (**IE, Firefox**), we simply need to type

http://localhost/<filename>.aspx

in the Address box. If the IIS server is installed on some other machine on the network, replace "**localhost**" with the name of the **server.**

Let's take a look on **Program 4-1** to demonstrate how we can create an ASP.NET application using a Text Editor (Notepad).

Program 4-1

Description of the Program 4-1: In this program, we are going to demonstrate the various elements of an ASP.NET page.

Step-1: Open a Text Editor such as Notepad or WordPad.

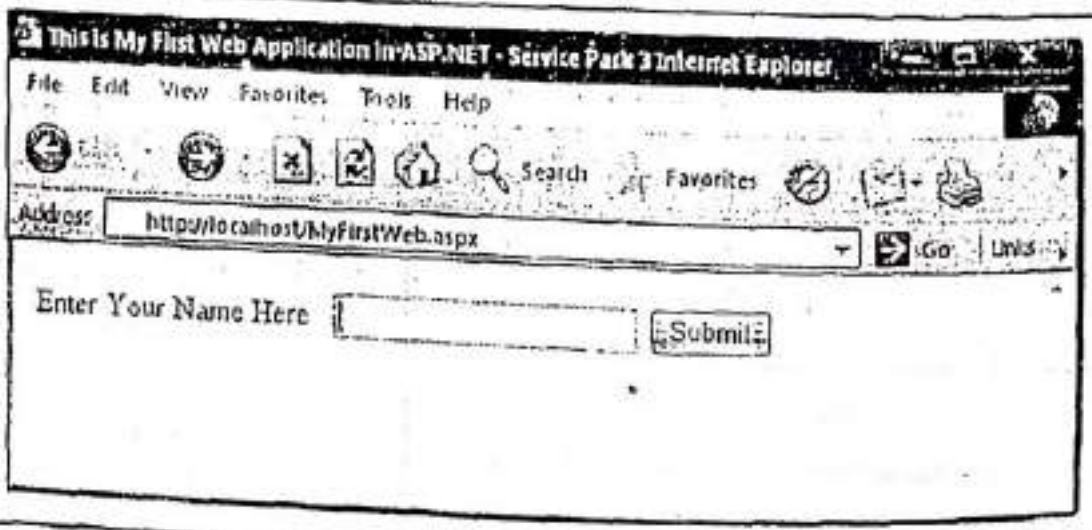
Step-2: Write the following code in the Text Editor.

```
<%@Page Language="C#"%>
    <script runat="server">
        void SetMessage(Object Sender, EventArgs e)
        {
            lblMessage.Text = "Your Name is " + txtMessage.Text;
        }
    </script>
    <html>
      <head id="Head1" runat="server">
        <title>This is My First Web Application in ASP.NET</title>
```

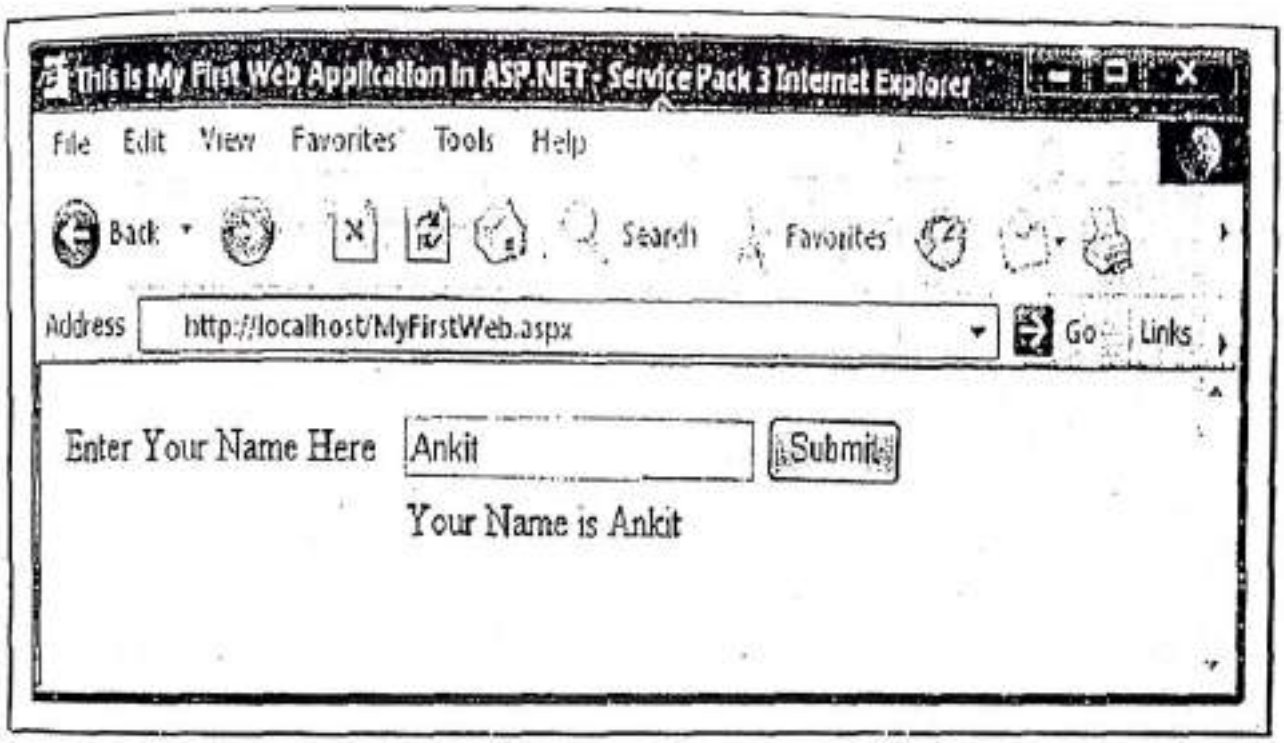
```
    </head>

    <body>
        <form id="form1" runat="server">
        <div>
        <table>
          <tr>
            <td style="width: 151px"> Enter Your Name Here</td>
              <td style="width: 157px">
               <asp:TextBox ID="txtMessage" OnTextChanged="SetMessage"
        runat="server">
                </asp:TextBox>
              </td>
               <td style="width: 185px">
                 <asp:Button ID="btnSetMessage" runat="server" Text="Submit" />
              </td>
           </tr>
                 <tr>
                <td style="width: 151px"> </td>
                 <td colspan="2"><asp:Label ID="lblMessage"
runat="server"></asp:Label></td>
            </tr>
                 </table>
          </div>
          </form>
    </body>
</html>
```

Step-3: Save the code as MyFirstWeb.aspx file in the directory C:\inetpub\wwwroot.

Step-4: To run this application in a web browser, write **http://localhost/MyFirstWeb.aspx** n address bar of the web browser and you can see the output as follows:



Step-5: Enter your name in the textbox and click on submit. You can see the output as follows:



Here's, the end of the **Program 4-1.**

4.5.2 Using Microsoft Visual Studio .NET IDE

We can use Microsoft Visual Studio .NET IDE to create web pages. Microsoft Visual Studio .NET IDE consists of various tools and windows, which provide the user friendly environment, and make development easy, fast and efficient. Whenever we create a web application in Microsoft Visual Studio .NET IDE, the application is automatically created on a web server i.e. IIS server. Now, in the following sections, we are going to demonstrate how to create an ASP.NET application using C#.NET and VB.NET.

4.5.2.1 Creating an ASP.NET application using C#.NET

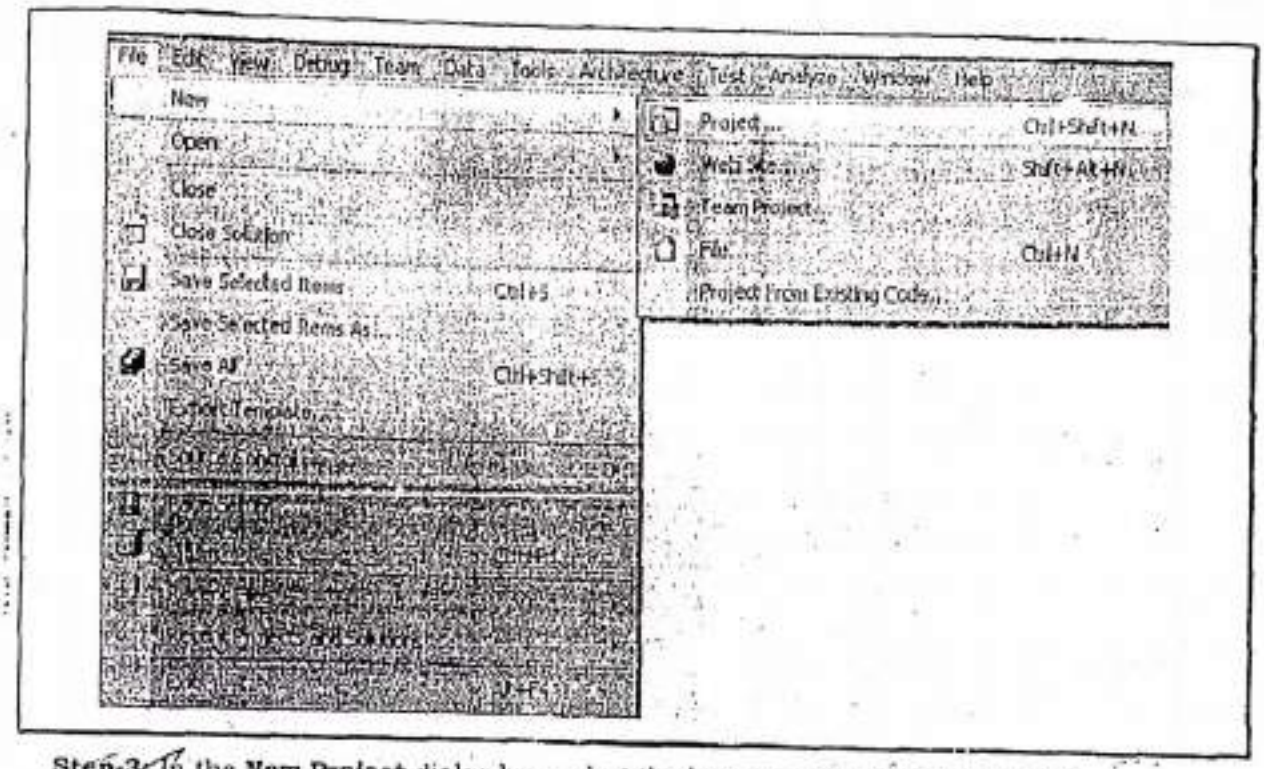
This can be demonstrated by a **Program 4-2.**

Program 4-2

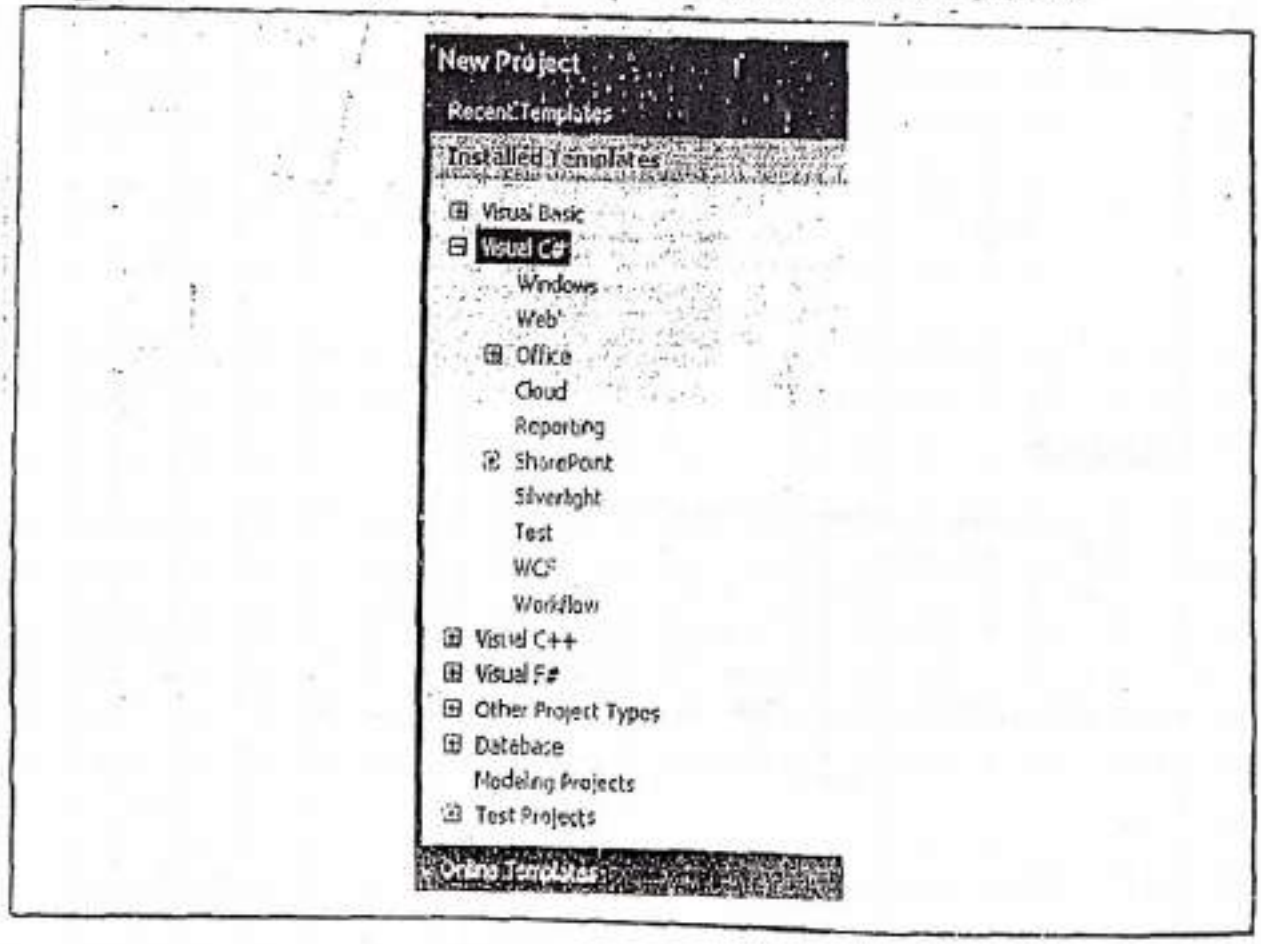
Description of the Program 4-2: In this program, we are going to create a Web Application, displays a message on button click. To do so, follow the steps given below.

Step-1: Start an instance of **Visual Studio.**

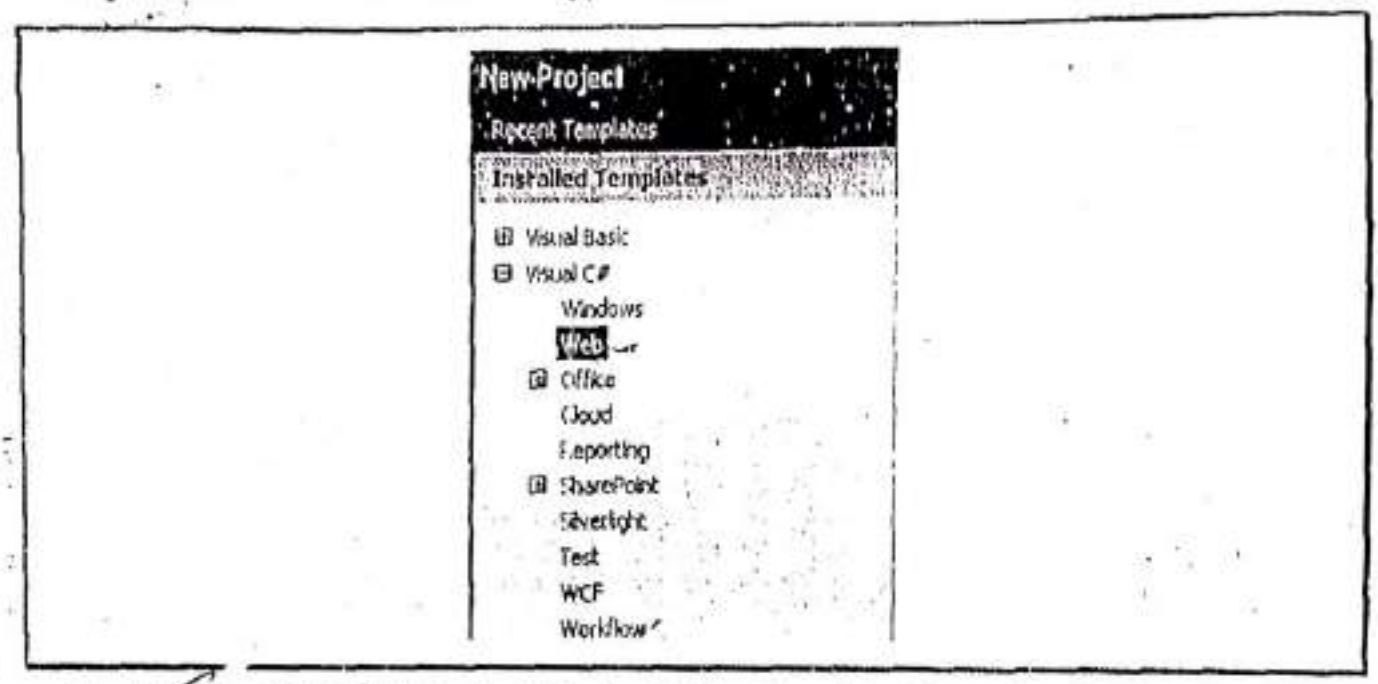
Step-2: Click on File menu and select New >> Project.



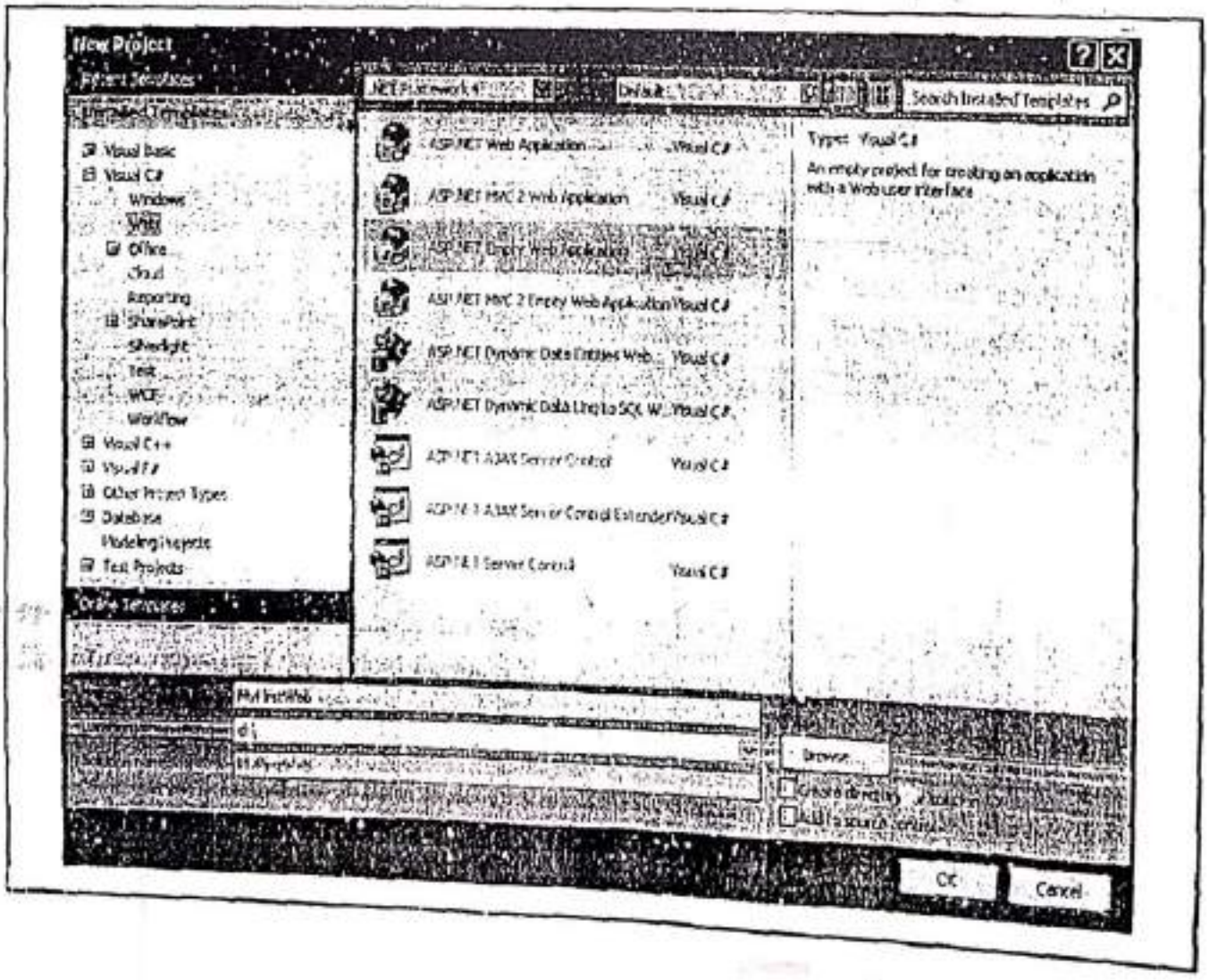
Step-3: In the **New Project** dialog box, select the language **Visual C#** to be used.



Step-4: Select **Web** from **Visual C#** applications.

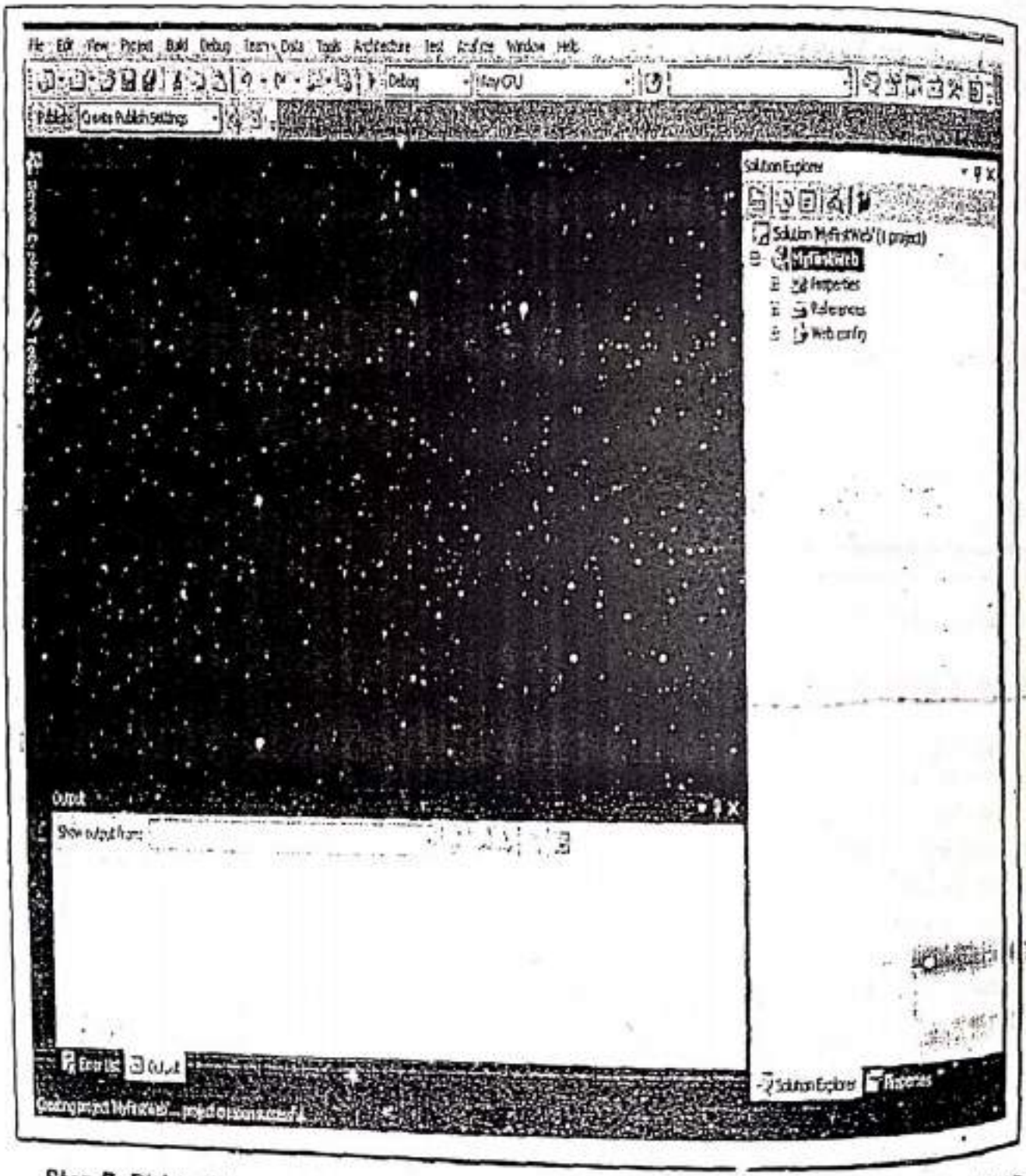


Step-5: Select **ASP.NET Empty Web Application** template. Name it as **"MyFirstWeb"** and set **Location** (here it is d :\).

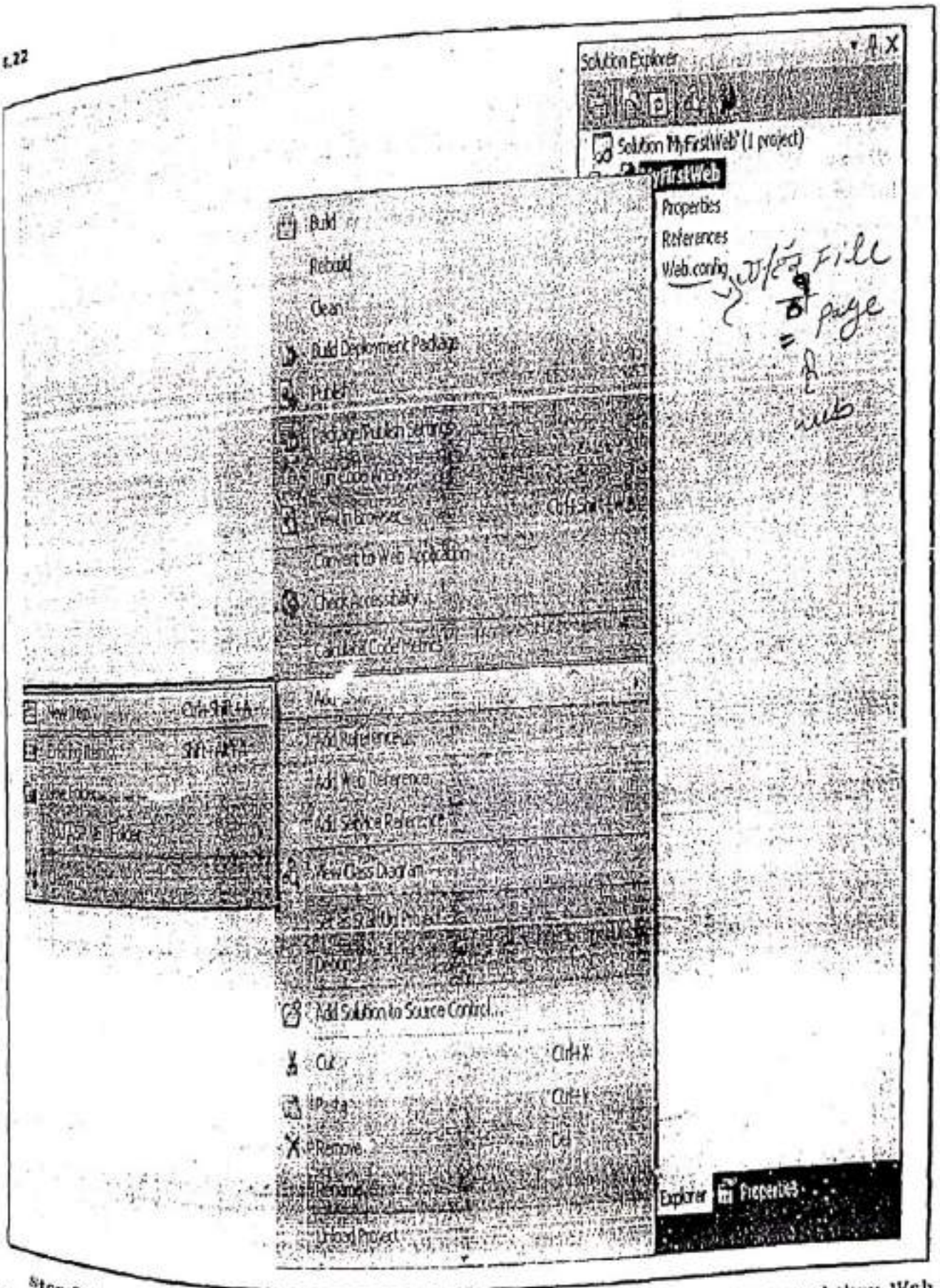


Note: In **New Project** dialog, we have deselected an option "**Create directory for solution**", which means a namespace (*a new directory*) will not be created. If you want to create a namespace then select this option and a namespace (*a new directory in given location*) will be created automatically whose name will be same as the name of the application (*i.e. MyFirstWeb*).

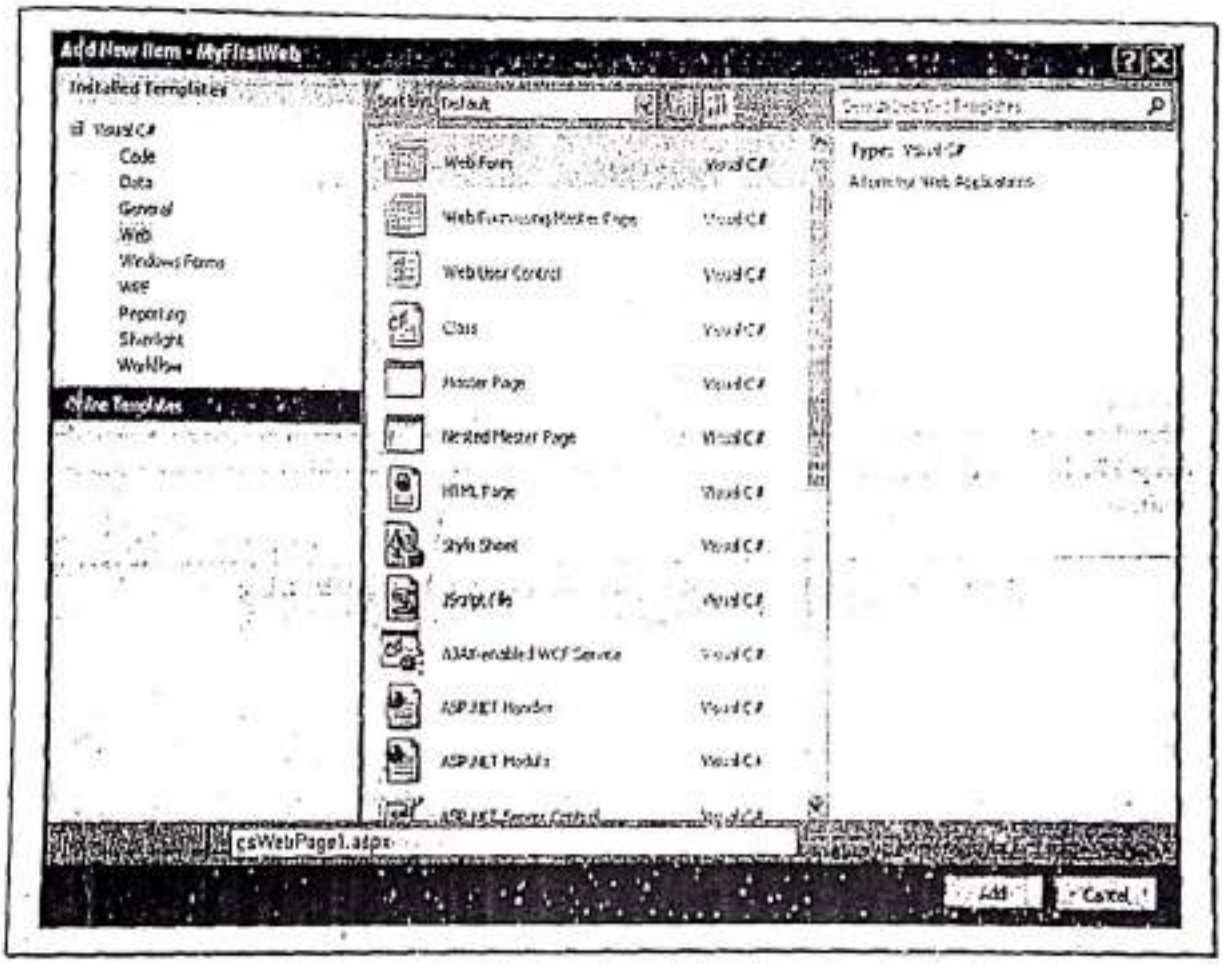
Step-6: Click on **Ok** button of **New Project** dialog. The following window will appear. In **Solution Explorer**, verify that the new **MyFirstWeb** application (folder) is created.



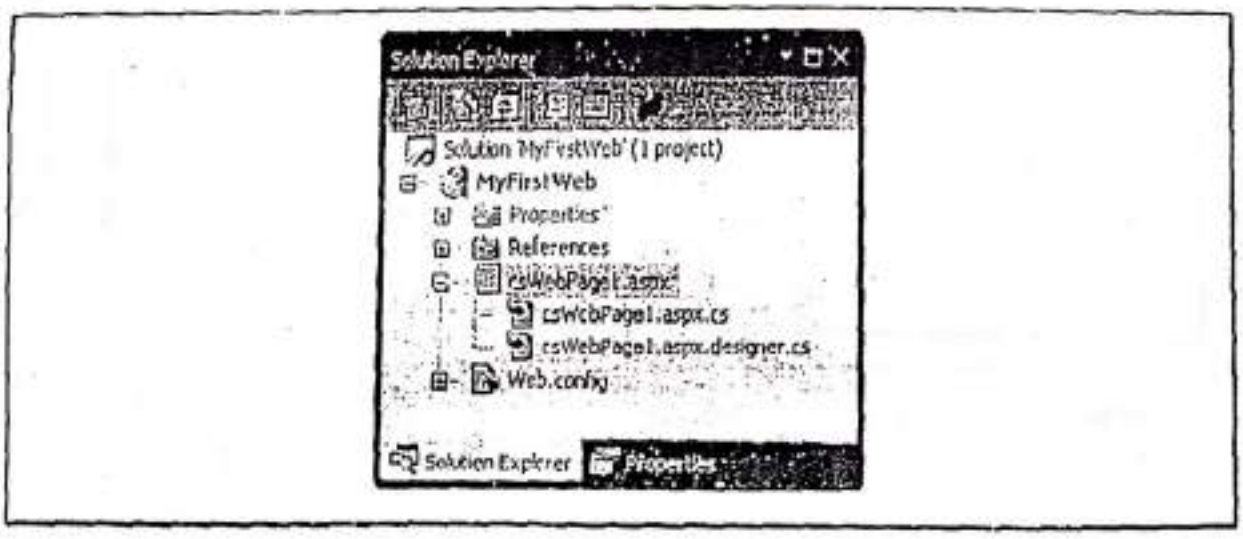
Step-7: Right click on project folder '**MyFirstWeb**' in **Solution Explorer**. Then select **Add >> New Item**



Step-8: In **Add New Item** dialog box, Select **Web** from **Visual C#** applications and then **Web Form** from the templates. Name it as **"csWebPage1.aspx"**.

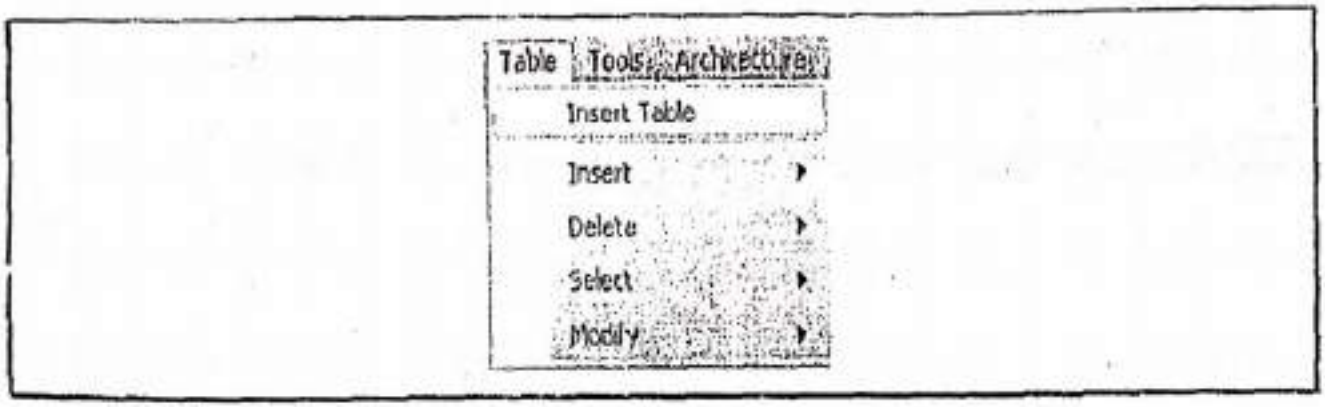


Step-9: Click on **Add** button in **Add New Item** dialog box. See the **Solution Explorer** will consist of a web page **csWebPage1.aspx**, **csWebPage1.aspx.cs** (code behind file), and a **Web.config** file.

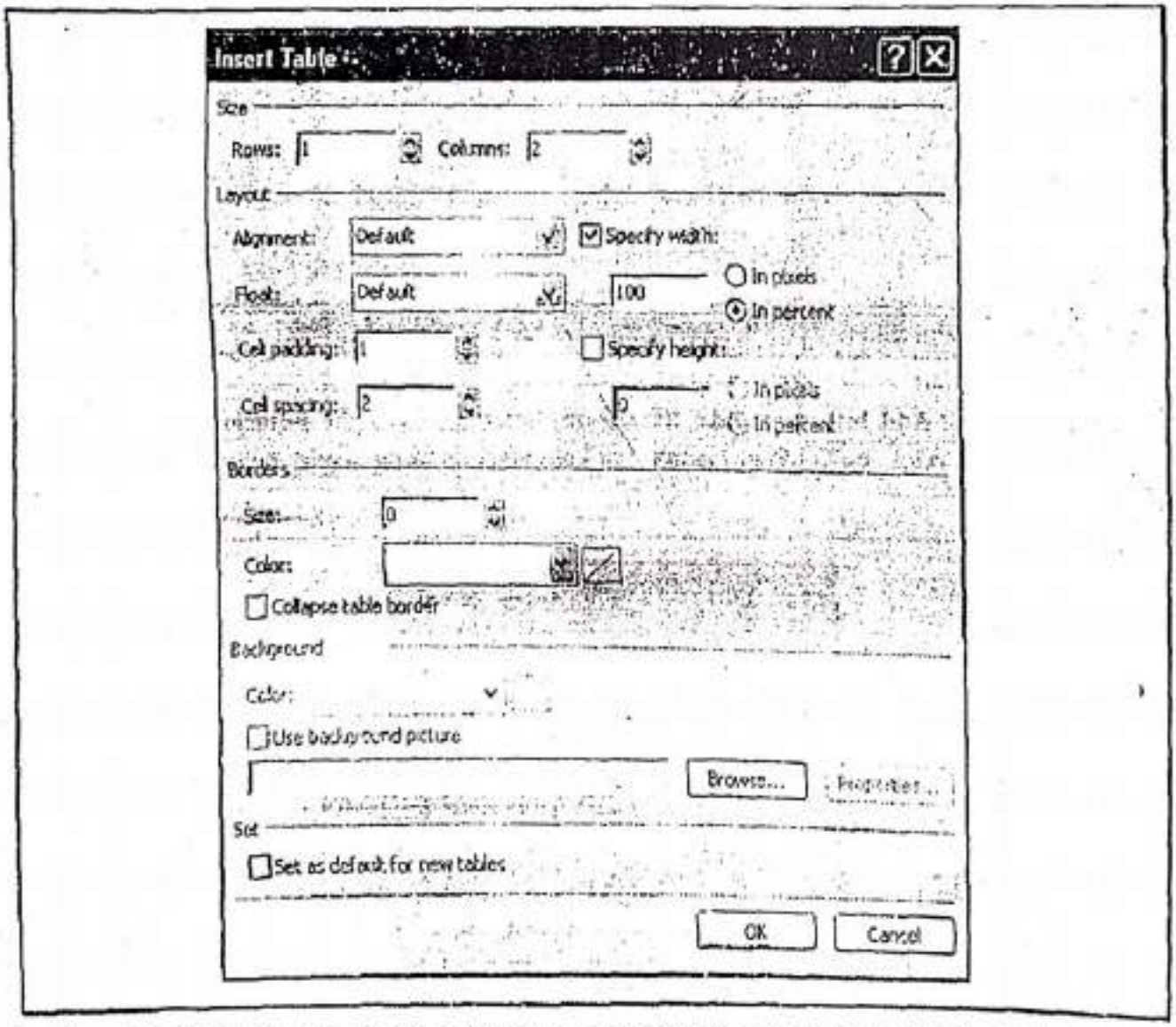


Step-10: In **Solution Explorer**, click on the **View Designer** button: . The **Design Window** for **csWebPage1.aspx** will appear.

Step-11: Click on **Table** menu and then click on its option **Insert Table.**

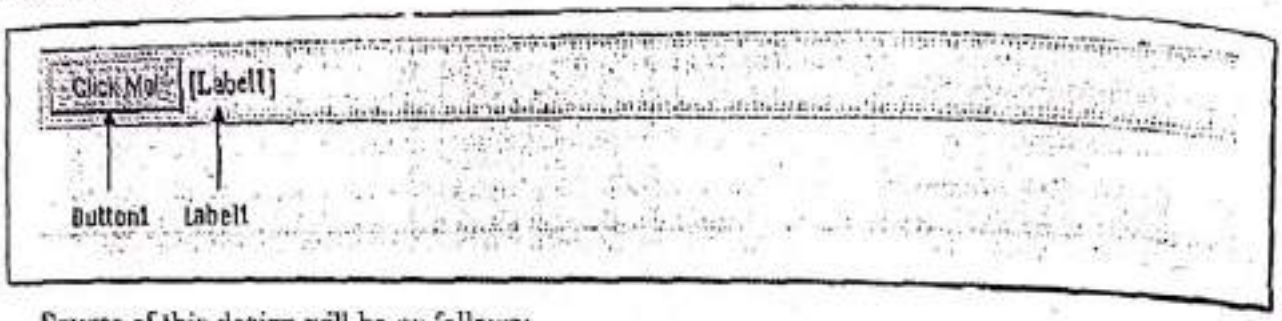


Step-12: In **Insert Table** dialog set value of **Rows** as **"1"** and **Columns** as **"2"** and click on **Ok** button.



Step-13: Place **Button** in first column and a **Label** in the second column of the inserted table. Right click on the **Button** and click on **Properties**. A **Property window** will appear. Set

the **Text** property of the **Button** as **"Click Me!"**. Similarly, set the **Text** property of the **Label** as blank. The design will be look like as follows:



Source of this design will be as follows:

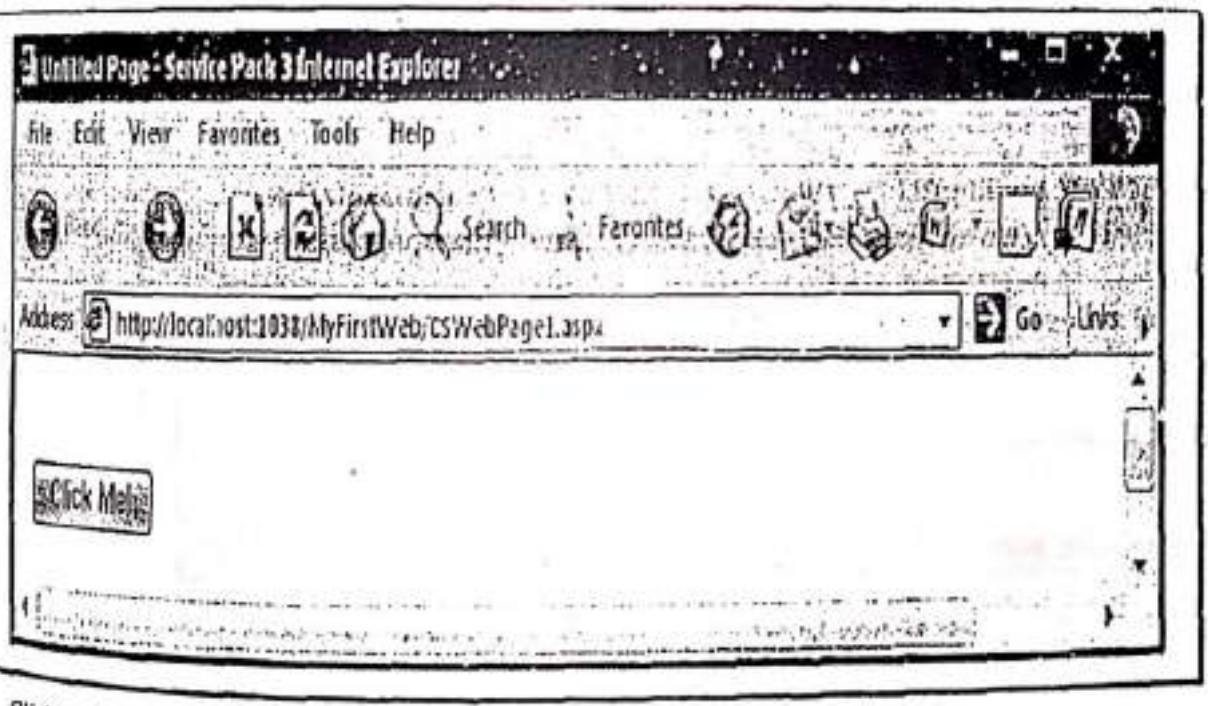
```
<%@Page Language="C#" AutoEventWireup="true"
    CodeBehind="csWebPage1.aspx.cs" Inherits="csWebPage1"%>

<!DOCTYPE html PUBLIC"-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title></title>
</head>
<body>

<form id="form1" runat="server">
<div>
    <table>
        <tr>
          <td>
            <asp:Button ID="Button1" runat="server" Text="Click Me!"/>
          </td>
          <td>
            <asp:Label ID="Label1" runat="server"></asp:Label>
          </td>
        </tr>
    </table>
</div>
</form>
</body>
</html>
```

Step-14: Double click on **Button**. The code behind file will appear. Add the following code in **C#.NET code behind file.**

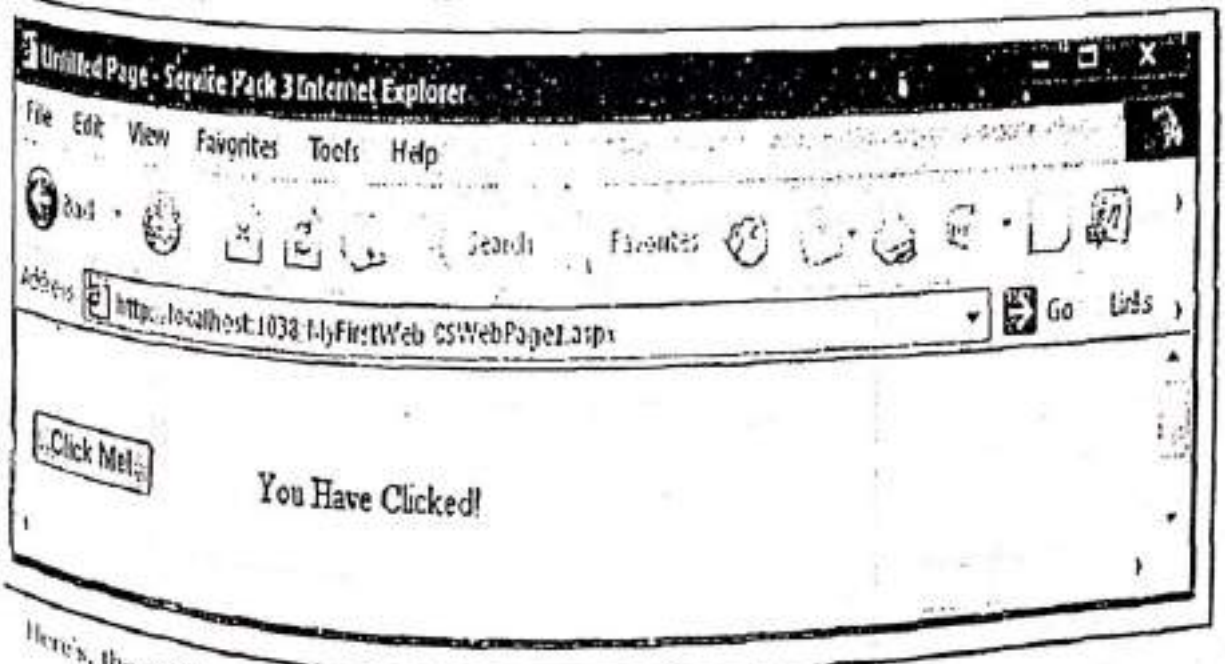
```
using System;
public partial class csWebPage1 : System.Web.UI.Page
```

```
protected void Button1_Click(object sender, EventArgs e)
{
    //Setting text of Label1, when we click on Button1
    Label1.Text = "You Have Clicked!";
}
```

Step-15: To run the application, press F5. The following window will appear.



Click on the button and the message will be shown as follows:



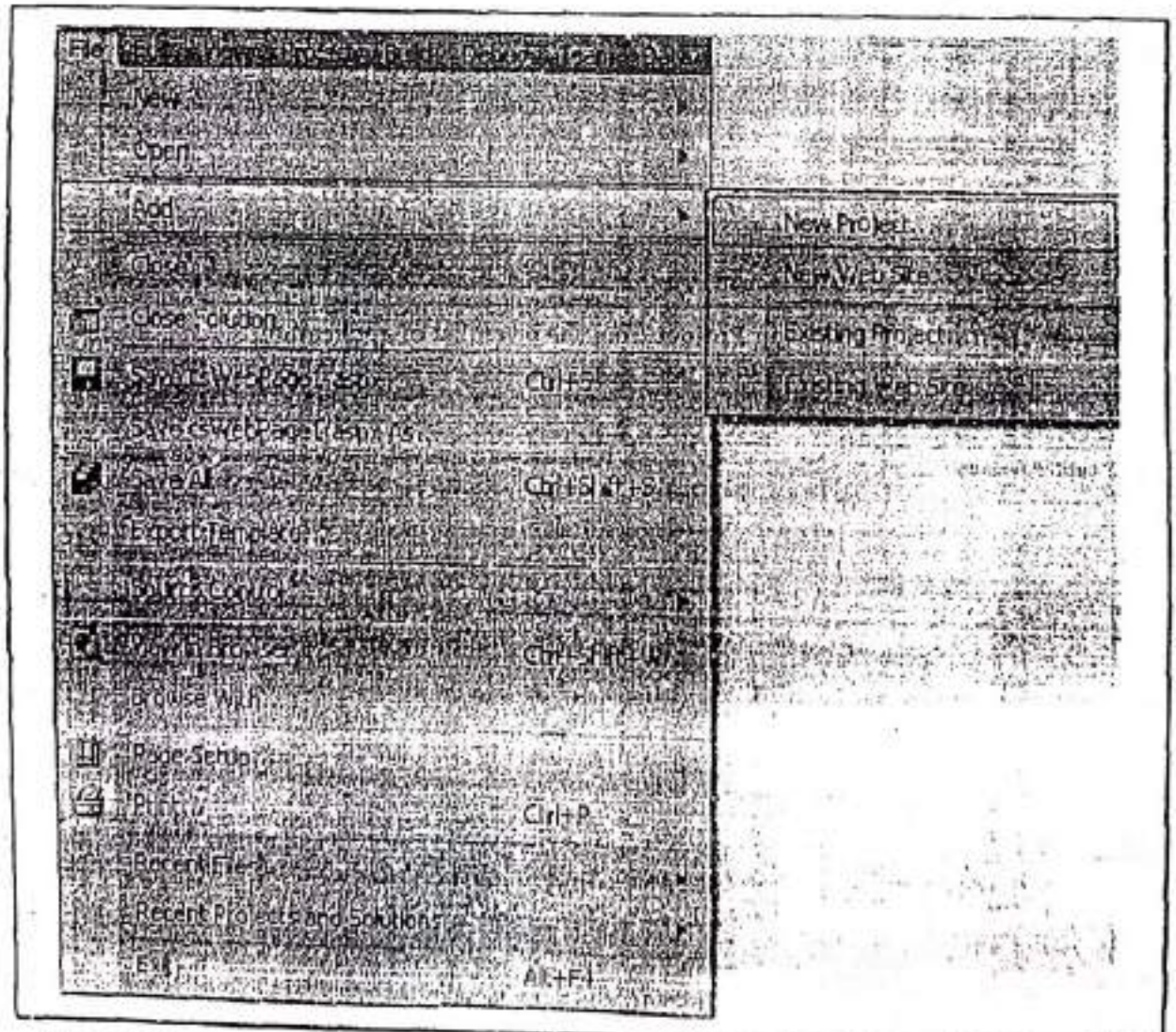
Here's the end of the Program 4-2.

4.6 DEPLOYING AN ASP.NET WEB APPLICATION

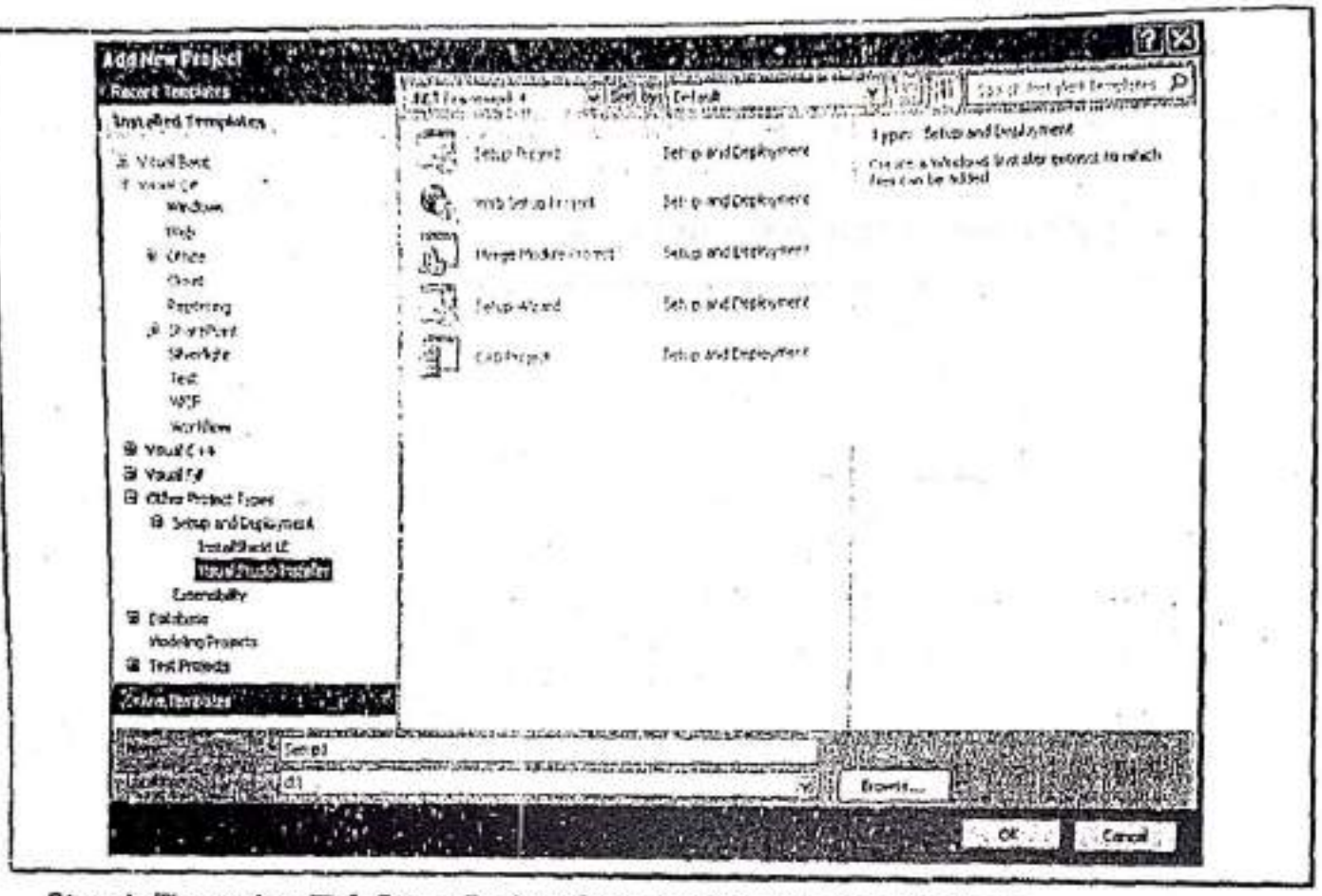
After creating and testing our ASP.NET web applications, the next step is **deployment**. Deployment is the process of distributing the finished applications (without the source code) to be installed on other computers. In Microsoft Visual Studio .NET, the deployment mechanism is the same irrespective of the programming language and tools used to create applications. In this section, we will deploy the web application demonstrated above in **Program 4-2**. We can deploy any of the application that was created by using VB.NET or C#.NET. To do so, follow the steps:

Step-1: Open the web application project that we want to deploy. Here, it is **MyFirstWeb.**

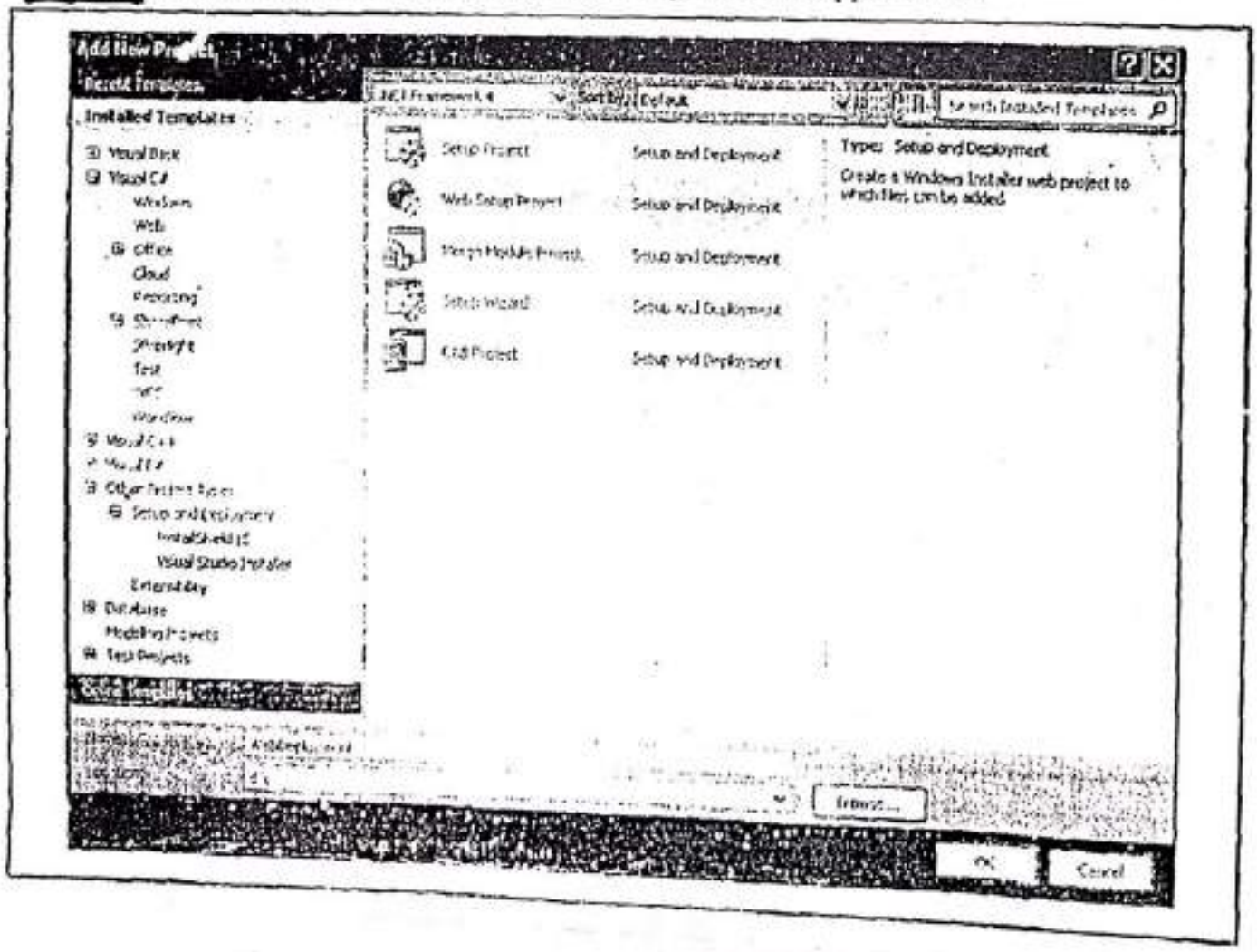
Step-2: Click on **File** menu and select **Add >>New Project**. The Add New Project dialog box will appear.



Step-3: From the **Installed Templates** pane, select **Other Project Types>>Setup and Deployment>>Visual Studio Installer.**

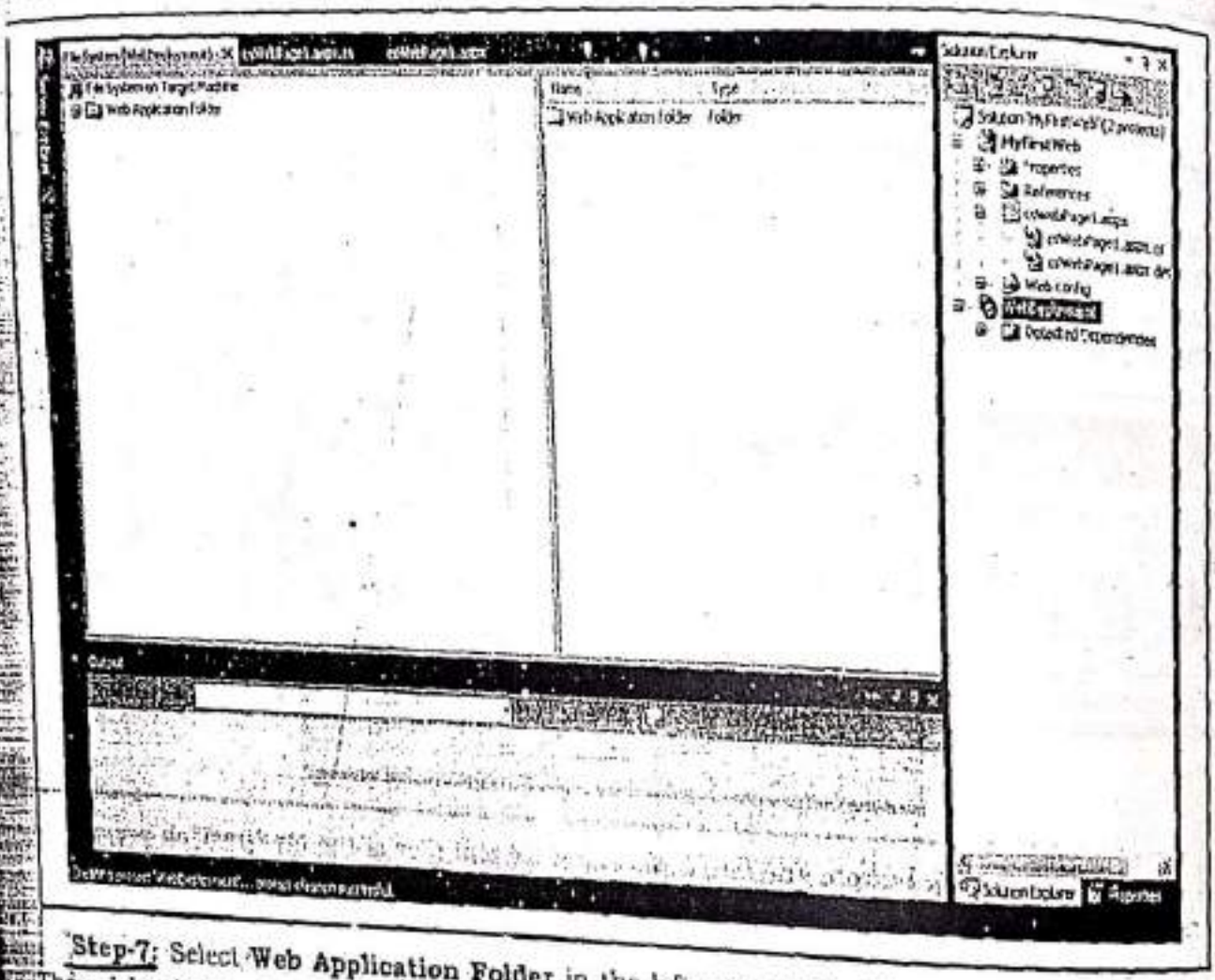


Step-4: Then select **Web Setup Project** from the given list of applications.

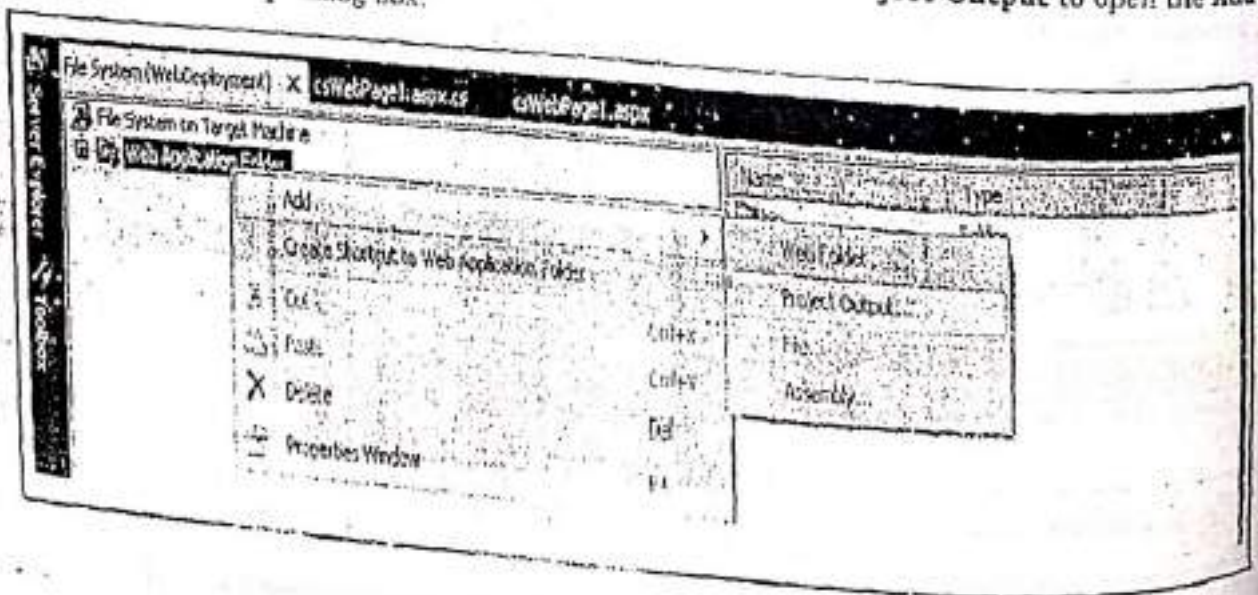


Step-5: Change the default name of the project. Here it is changed to **"WebDeployment"**.

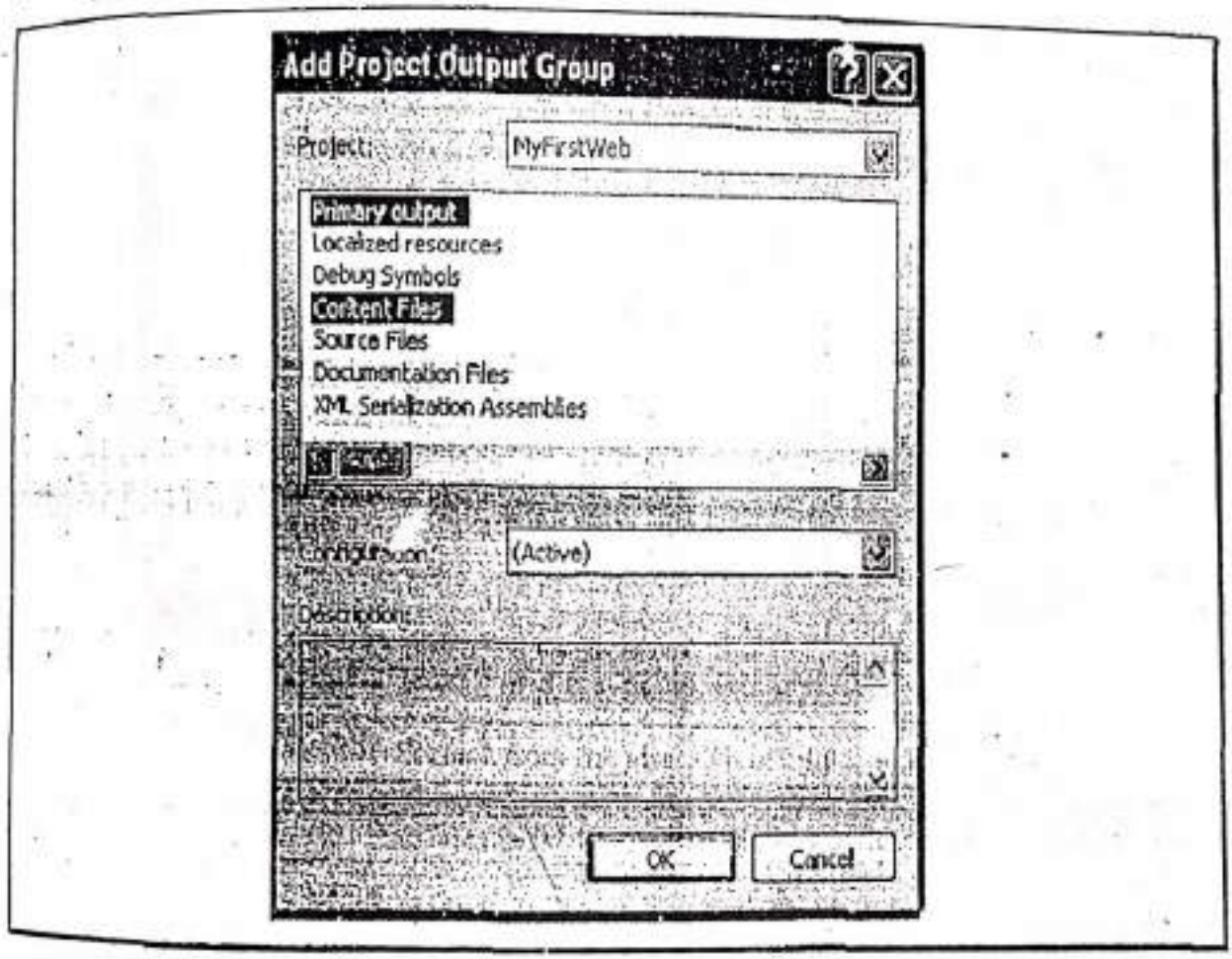
Step-6: Click on **Ok** button. The project is added in the **Solution Explorer** window. Also, a **File System** editor window appears to the left. The editor window has two panes. The left pane displays different items. The right pane displays the content of the item selected in the left pane.



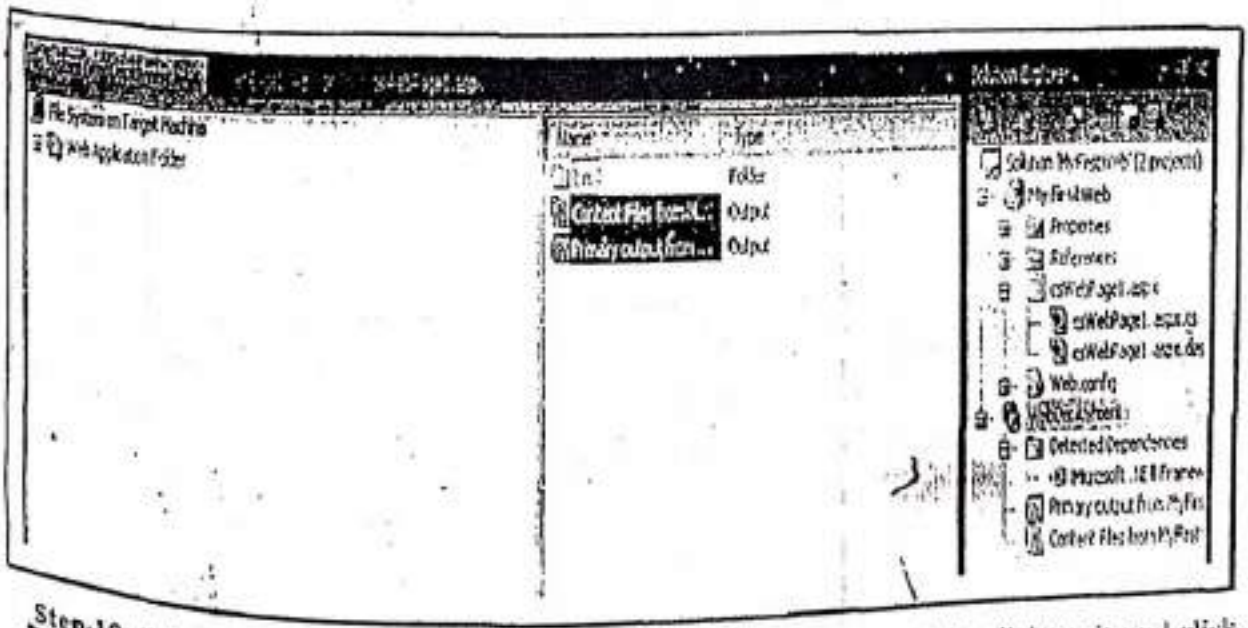
Step-7: Select **Web Application Folder** in the left pane of the **File System** editor window. Then right click on the **Web Application Folder**, select **Add >>** **Project Output** to open the **Add Project Output Group** dialog box.



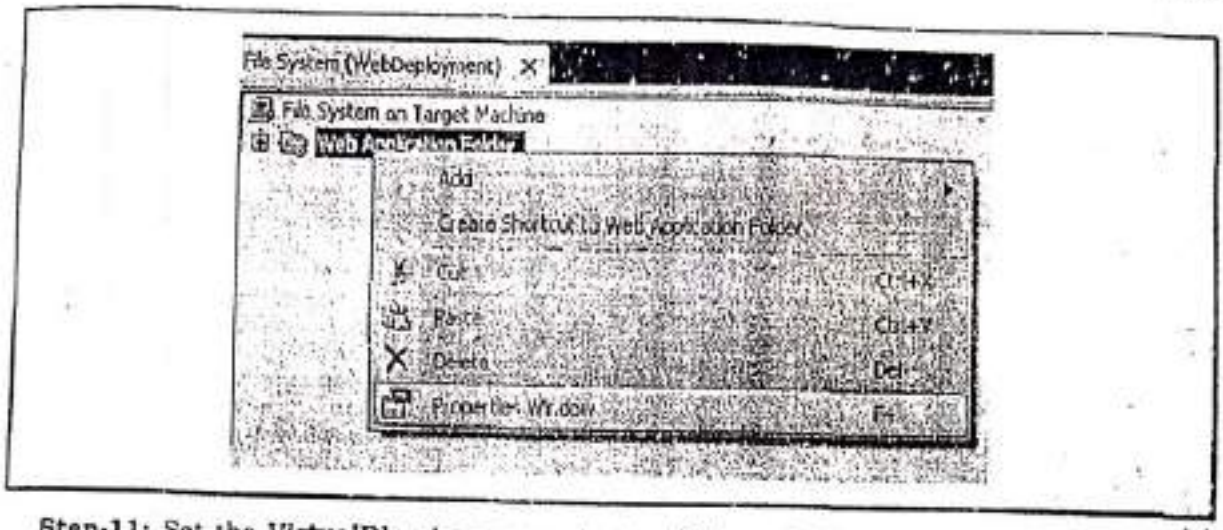
Step-8: Verify that **MyFirstWeb** is selected in the Project drop-down list. Then, select **Primary Output** and **Content Files** from the list.



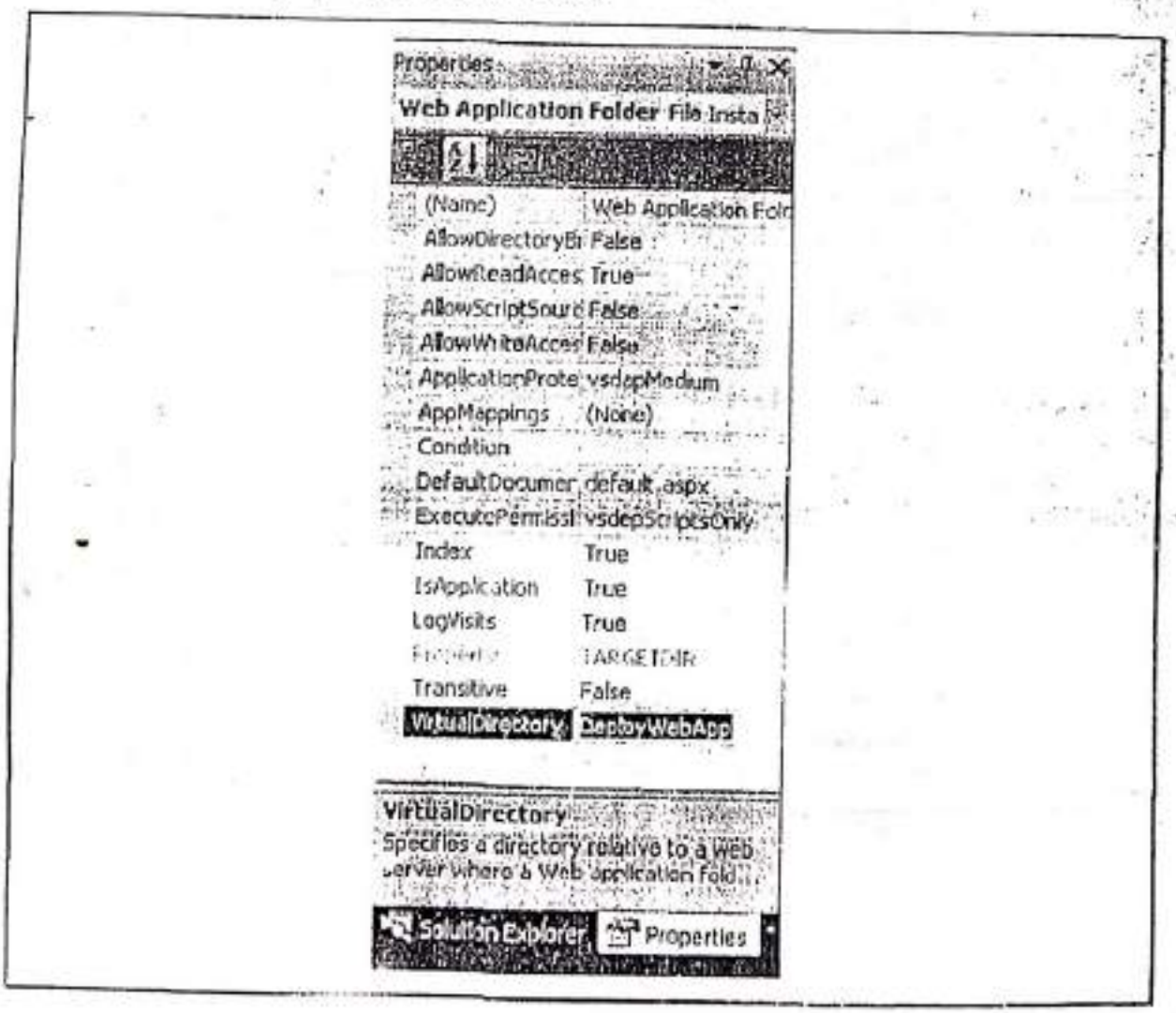
Step-9: Click on **Ok** button. The output files and content files of the **MyFirstWeb** project are added to the solution.



Step-10: Select **Web Application Folder** in the **File System** editor, right click on it and click on **Properties window** option to open the **Properties window**.



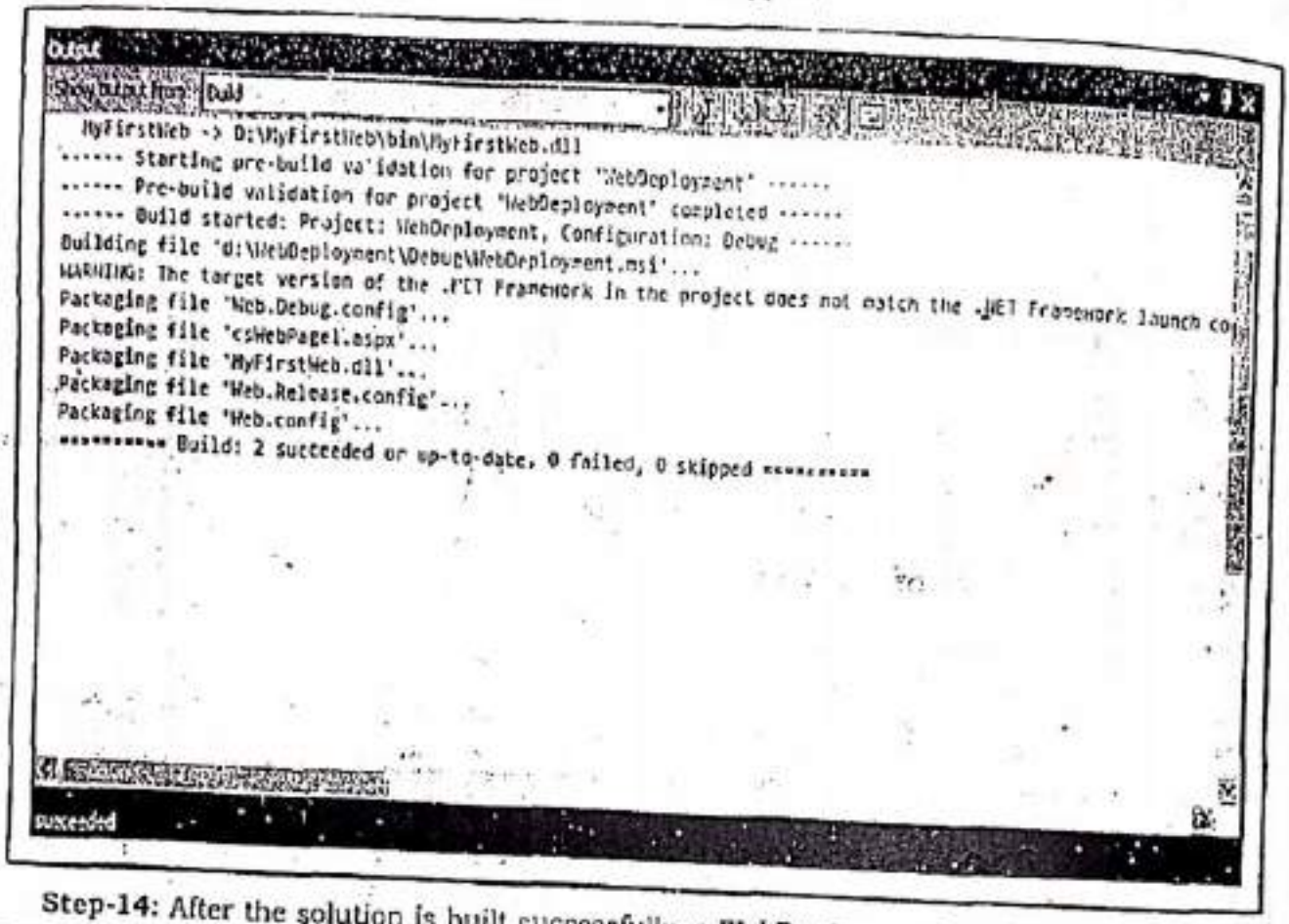
Step-11: Set the **VirtualDirectory** property to a folder, **<folder name>**, that would be the virtual directory on the target computer where we want to install the application. By default, this property is set to **WebDeployment**, which is the name of the Web Setup project that we added. Here, we have set the property to **DeployWebApp**.



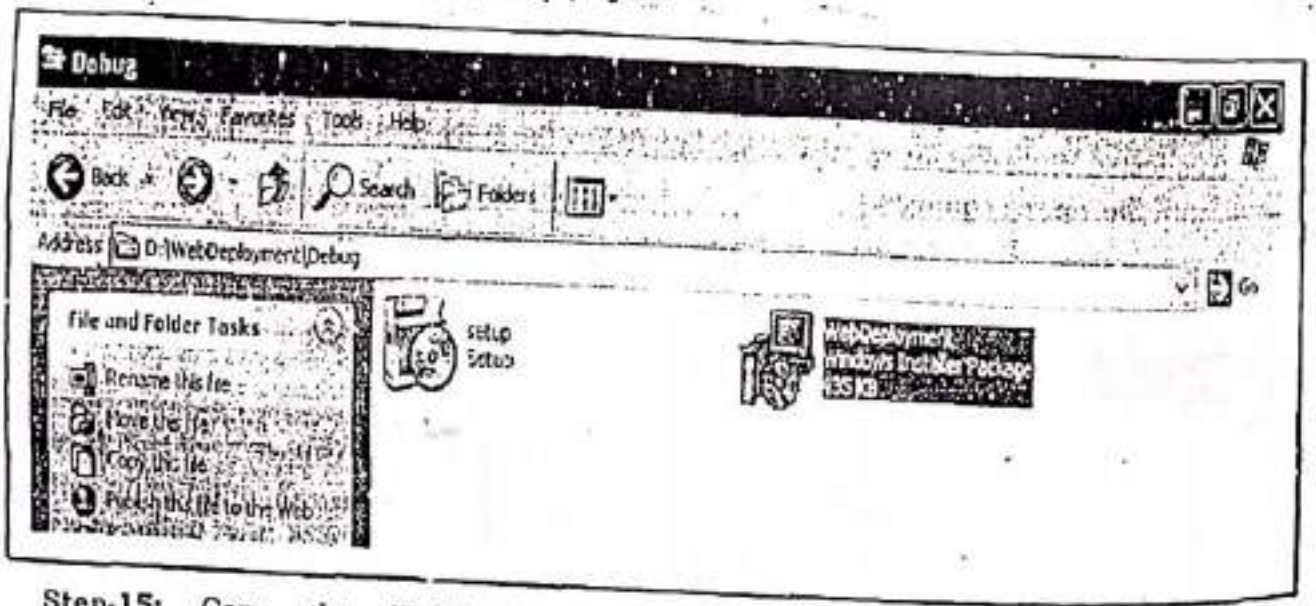
Step-12: In the same **Properties window** of the **Web Application Folder**, set the **DefaultDocument** property to **csWebPage1.aspx**. This property is used to set the default web page for the application.

Step-13: Build the solution by selecting **Build WebDeployment** from the **Build** menu.

While building solution the following process will happen.



Step-14: After the solution is built successfully, a **WebDeployment.msi** file is created in the **Debug** directory of the **WebDeployment** project.



Step-15: Copy the **WebDeployment.msi** file to the Web server computer **(c:\inetpub\wwwroot)** where we want to deploy the application.

Step-16: Double click the **WebDeployment.msi** file on the target computer to run the installer. After the installation is complete, we can run our application on the target computer.

To do so, start **Internet Explorer** and enter **http://<computer name>/WebDeployment** in the address box. The page that we developed is displayed.

4.7 WEB FORMS

ASP.NET provides services to allow the creation, deployment, and execution of Web Applications and Web Services. Web Applications are built using Web Forms. ASP.NET pages are known as Web Forms. Web Forms are the heart and soul of ASP.NET. These are the main building blocks for the ASP.NET applications. Web Forms are also called the **User Interface (UI)** elements through which the users can interact with the web application. Web Forms provide **properties, methods**, and **events** for the **controls** that are placed onto them. Web Forms are made up of two components: the **visual portion** (the .aspx file), and the **code behind the form**, which resides in a separate class file.

4.7.1 Characteristics of a Web Form

- A Web Form is built on Common Language Runtime, so, it provides managed execution environment, type safety, reusability and extendibility.
- It provides separation of HTML interface from application logic. This makes it much easier for the programmers and designers to collaborate efficiently.
- We can write our code in any CLR supported language such as C#.NET and VB.NET.
- A rich set of server-side controls that can detect the browser and send out appropriate markup language such as HTML
- It supports **session, request, state management** and **cashing** facilities.
- It provides **Event Driven** programming model.
- ASP.NET pages are precompiled to byte-code and Just In Time (JIT) compiled when first requested. Subsequent requests are directed to the fully compiled code, which is cached until the source changes.
- It facilitates the programmers to create their own custom controls provided with additional functionality.

4.8 WEB CONTROLS

ASP.NET provides a rich set of controls to design web pages. An ASP.NET control, is a .NET class that represent visual elements on a web form. These controls are small building blocks of the graphical user interface, which includes text boxes, buttons, check boxes, list boxes, labels and numerous other tools, using which users can enter data, make selections and indicate their preferences. ASP.NET controls also facilitates validation, data access, security, and data manipulation. All these controls are derived from the base **Control** class in the **System.Web.UI** namespace but these web controls are not based directly on the **Control** class. These web controls are based on the **WebControl** class, which is based on the **Control** class.

In Visual Studio .NET, we can design web pages by simply dragging and dropping the controls. We can also write ASP.NET code to add these controls to the web pages. These controls can be categorized as follows:

Standard Controls: The standard controls enable us to render standard form elements such as buttons, input fields, and labels.

Data Controls: The data controls enable us to connect to a database. It includes data source controls that we can drop onto a form and configure at design time (without using any code) and data display controls such as GridView.

Validation Controls: The validation controls enable us to validate form data before we submit the data to the server. For example, we can validate the input can't be empty, it must be a number, it must be greater than a certain value, and so on.

Navigation Controls: The navigation controls enable us to display standard navigation elements such as menus and tree views. These controls are designed to display site maps and allow the user to navigate from one page to another.

Login Controls: These controls provide prebuilt security solutions, such as login boxes and a wizard for creating users.

Rich Controls: The rich controls enable us to render things such as calendars, file upload buttons and multi-step wizards.

HTML Controls: These controls allow us to drag and drop static HTML elements. We can also use these controls to create server-side HTML controls.

The basic syntax for using these controls is:

```
<asp:controlType ID ="ControlID" runat="server" Property1=value1 [Property2=value2]>
</asp:controlType>
```

Or

```
<asp:controlType ID ="ControlID" runat="server" Property1=value1 [Property2=value2]
/>
```

However, visual studio .NET has the following features, which helps in error free coding:

- Dragging and dropping of controls in design view.
- IntelliSense feature that displays and auto completes the properties.
- The properties window to set the property values directly.

4.8.1 STANDARD CONTROLS

Standard controls enable us to display information, accepting user inputs, link pages, and submitting forms. It includes number of controls, such are as follows:

4.8.1.1 DISPLAY INFORMATION

In ASP.NET to display information on a page two controls are used i.e. Label control and Literal control. These can be described as follows:

(A) Using the Label Control

A **Label** Control is used to display text on a page dynamically which means that the text displayed by the **Label** control can be changed from one execution of a page to another. To display text on a **Label**, we use the **Text** property. The basic syntax to add a **Label** control to a web form using source code is as follows:

```
<asp:Label ID="Label1" runat="server" Text="Label" />

or

<asp:Label id=Label1 runat="server">Label</asp:Label>
```

We can also set the text of a Label control programmatically. For example, in C#, this can be done as follows:

```
Label1.Text="Label";
```

Table 4-A lists some commonly used properties of the Label control.

Table 4-A : Properties of the Label control	
Property	**Description**
BackColor	It enables to change the background color of a Label control.
BorderColor	It enables to set the color of a border rendered around the label.
BorderStyle	It enables to display a border around the label. Possible values are NotSet, None, Dotted, Dashed, Solid, Double, Groove, Ridge, Inset, and Outset.
Font	It enables to set the label's font properties.
ForeColor	It enables to set the font color of a Label control.
Text	Used to set the text to be displayed on a Label control.
Visible	It indicates whether or not a label is visible.

Take a look at the following program.

Program 4-3

Description of the Program 4-3: This program demonstrates how we can use a Label control to display information on a web page. In this program, the use of number of properties of the Label control is also demonstrated. Follow the steps given below.

Step-1: Create a new Web Application named "DisplayInformation".

Step-2: Add a Web Form to the application named "LabelControlDemo.aspx".

Step-3: Create the following design.

Label ——————→ Label: Label1

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
```

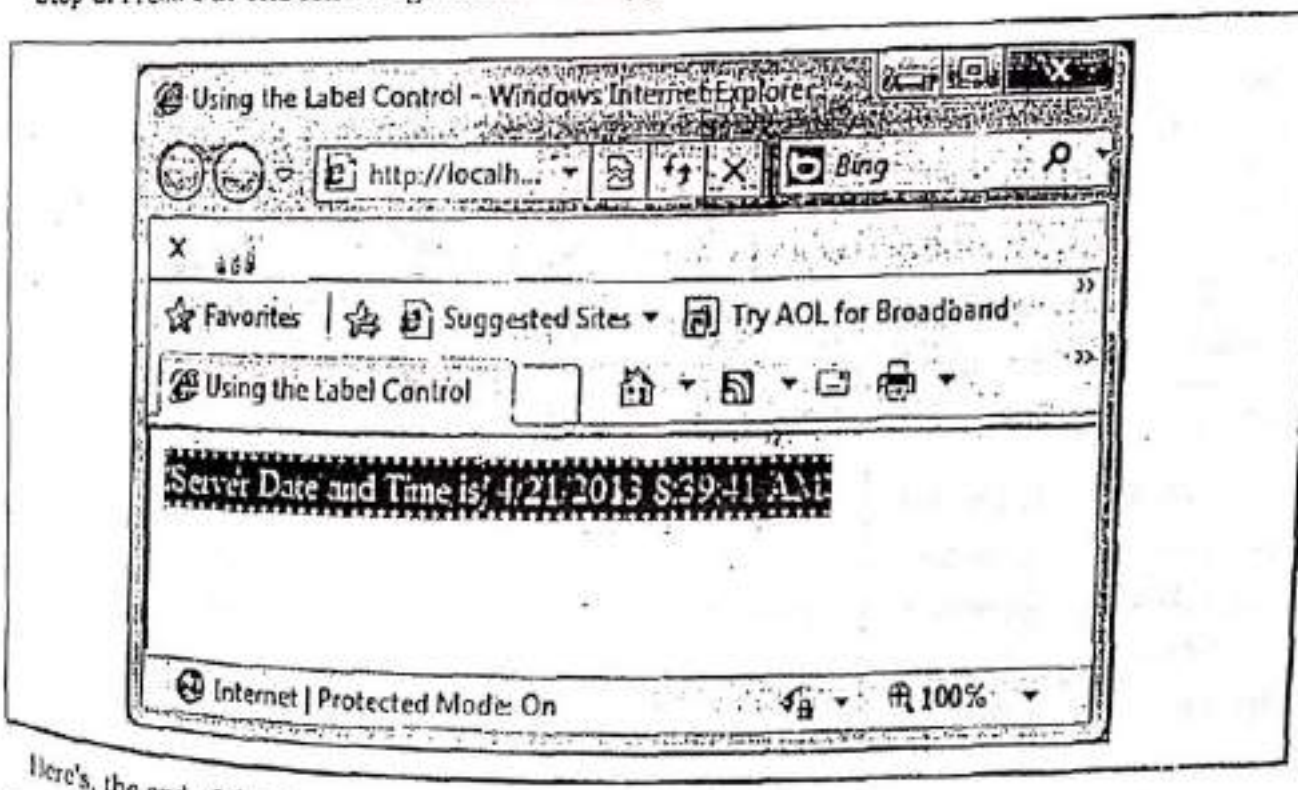
```
    CodeBehind="LabelControlDemo.aspx.cs"
    Inherits="LabelControlDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the Label Control</title>
    </head>
    <body>
        <form id="form1" runat="server">
            <div>
                <asp:Label ID="Label1"
                    runat="server"
                    Text="Label">
                </asp:Label>
            </div>
        </form>
    </body>
</html>
```

Step-4: Double click on the page. The code behind file will appear. Add the following code in **C# code behind file** named "**LabelControlDemo.aspx.cs**".

```
using System;                                   //Add namespace
using System.Drawing;                           //Add namespace
using System.Web.UI.WebControls;                //Add namespace

public partial class LabelControlDemo : System.Web.UI.Page
{

    protected void Page_Load(object sender, EventArgs e)
    {
        //Setting text of the label
        Label1.Text = "Server Date and Time is: "+DateTime.Now.ToString();

        //Setting back color of the label
```

```
        Label1.BackColor = Color.DarkGreen;

        //Setting font color of the label
        Label1.ForeColor = Color.Pink;

        //Setting Border color of the label
        Label1.BorderColor = Color.Yellow;

        //Setting Border Style of the label
        Label1.BorderStyle = BorderStyle.Dotted;
    }
}
```

Step-5: Press **F5**. The following window will appear.



Here's, the end of the **Program 4-3.**

(B) Using the Literal Control

The Literal control works very much like the Label control does. This control always used for text that we wanted to push out to the browser, but keep unchanged in the process (*a literal* stated. The basic syntax to add a **Literal** control to a web form using **source** code is as follows:

```
<asp:Literal ID="Literal1" runat="server"></asp:Literal>
```

A Label control alters the output by placing ****elements around the text as shown:

```
<span id="Label1">Here is some text</span>
```

The Literal control just outputs the text without the elements. One feature found in this control is the attribute **Mode**. This attribute enables us to dictate how the text assigned to the control is interpreted by the ASP.NET engine. If you place some HTML code in the string that is output (*for instance, Here is some text*), the Literal control outputs just that and the consuming browser shows the text as bold:

Here is some text

We can use the **Mode** attribute as illustrated here:

```
<asp:Literal ID="Literal1" runat="server" Mode="Encode"
Text="<b>Here is some text</b>"></asp:Literal>
```

Adding **Mode="Encode"** encodes the output before it is received by the consuming application. Now, instead of the text being converted to a bold font, the elements are displayed as follows:

Here is some text

This is ideal if we want to display code in our application. Other values for the **Mode** attribute include **Transform** and **PassThrough**. Transform looks at the consumer and includes or removes elements as needed. For instance, not all devices accept HTML elements so, if the value of the Mode attribute is set to Transform, these elements are removed from the string before it is sent to the consuming application. A value of PassThrough for the Mode property means that the text is sent to the consuming application without any changes being made to the string.

Take a look at the following program.

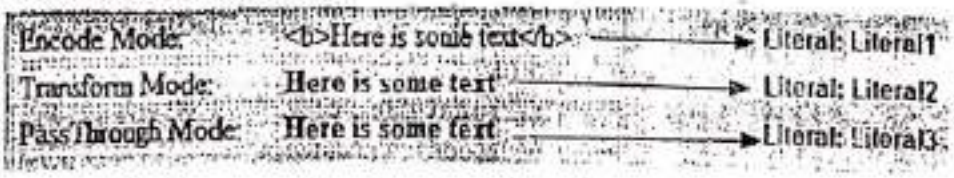
Program 4-4

Description of the Program 4-4: This program demonstrates how we can use a Literal control to display information on a web page. In this program, the use of Mode attribute of the Literal control is also demonstrated. Follow the steps given below.

Step-1: Create a new **Web Application** named **"DisplayInformation"**.

Step-2: Add a **Web Form** to the application named **"LiteralControlDemo.aspx"**.

Step-3: Create the following design.



Step-4: Set the following properties of various **Literal** controls:

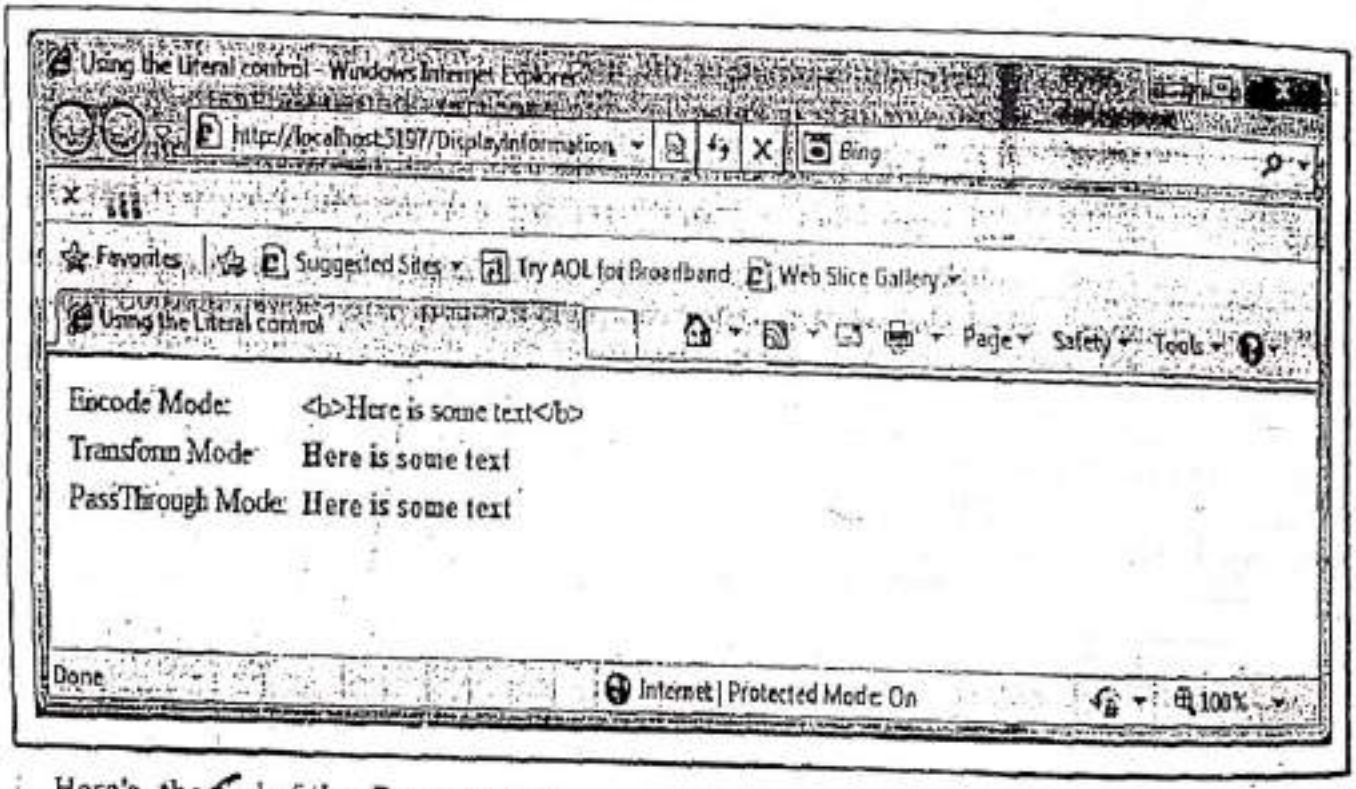
Control	Porperty	Value
Literal1	Mode	Encode
	Text	Here is some text
Literal2	Mode	Transform
	Text	Here is some text
Literal3	Mode	PassThrough
	Text	Here is some text

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="LiteralControlDemo.aspx.cs" Inherits="LiteralControlDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the Literal control</title>
    </head>

    <body>
        <form id="form1" runat="server">
            <div>
                <table>
                    <tr>
                    <td>Encode Mode:</td>
                    <td>
                        <asp:Literal ID="Literal1"
                                runat="server"
                                Mode="Encode"
                                 Text="<b>Here is some text</b>">
                            </asp:Literal>

                    </td>
                    </tr>
                    <tr>
                    <td>Transform Mode:</td>
                    <td>
                        <asp:Literal ID="Literal2"
                                runat="server"
                                Mode="Transform"
                                Text="<b>Here is some text</b>">
                        </asp:Literal>
                    </td>
                    </tr>
                    <tr>
                    <td>PassThrough Mode:</td>
                    <td>
                            <asp:Literal ID="Literal3"
                                    runat="server"
```

```
                                        Mode="PassThrough"
                                    Text="<b>Here is some text</b>">
                                </asp:Literal>
                            </tr>
                        </table>
                    </div>
                </form>
            </body>
</html>
```

Step-5: Press **F5**. The following window will appear.



Here's, the end of the **Program 4-4.**

4.8.1.2 ACCEPTING USER INPUT

The ASP.NET Framework includes several controls that we can use to gather user input. Such controls are **TextBox** control, **CheckBox** control, **RadioButton** control. These can be described as follows:

(A) Using the TextBox Control

TextBox control is used to accept input from the user. A **TextBox** control can accept one or more lines to text depending upon the setting of the **TextMode** property which can be **SingleLine**, **MultiLine** or **Password**. By default, a **TextBox** control is a **SingleLine** text box that allows users to type characters in a single line only. A **Password TextBox** control is similar to the **SingleLine** text box, but masks the characters that are typed by users and displays them as asterisks (*) or dots (.). A **MultiLineTextBox** control allows users to type multiple lines and wrap text.

The basic syntax to add a **TextBox** control to a web form using source code is as follows:

```
<asp:TextBox ID="TextBox1" runat="server" ntextchanged="TextBox1_TextChanged">
                                    </asp:TextBox>
```

Table 4-B lists some commonly used properties of the TextBox control.

Table 4-B : Properties of the TextBox control	
Property	**Description**
Enabled	Used to enable or disable the text box. If it is set to **False** then the text box is not editable i.e. text box is disable.
MaxLength	The maximum number of characters that can be entered into the text box.
ReadOnly	Determines whether the user can change the text in the text box. By default it is **False**, i.e., the user can change the text.
Text	Used to set the text to be displayed in the **TextBox** control.
Wrap	It represents the word wrap behavior for MultiLine text box. By default it is **True**.

The most commonly used method of **TextBox** control is **Focus()**. The Focus() method is used to set the initial form focus (cursor) to the text box. The Focus() method enables us to dynamically place the end user's cursor in an appointed form element such as TextBox, Button, CheckBox or any of the Web controls derived from the **WebControl** class. But, it is most probably used with the TextBox control, as follows:

```
protected void Page_Load(object sender, EventArgs e)
{
TextBox1.Focus();        //Set cursor in the TextBox1 whenever the page is loaded in the
browser.
}
```

The most commonly used **event of TextBox** control is **TextChanged.** This event is raised on the server when the contents of the text box are changed. The basic C# syntaxfor **TextChanged** event of a **TextBox** controlis as follows:

```
protected void TextBox1_TextChanged(object sender, EventArgs e)  //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

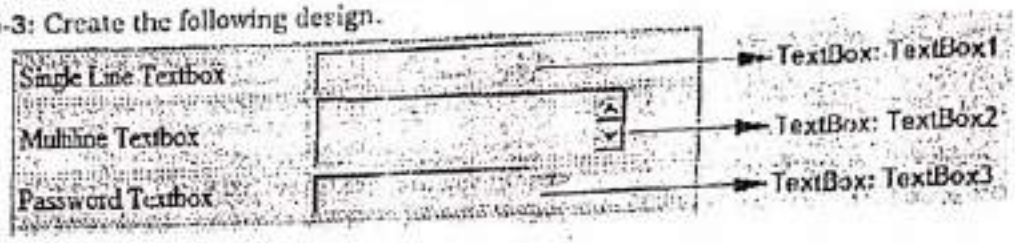
Program 4-5

Description of the Program 4-5: This program demonstrates how we can use a TextBox control to accept input from the user in various text modes (SingleLine, MultiLine, Password). Follow the steps given below.

Step-1: Create a new **Web Application** named "**AcceptingUserInput**".

Step-2: Add a **Web Form** to the application named "**TextBoxDemo.aspx**".

Step-3: Create the following design.



Step-4: Set the following properties of various **TextBox** controls:

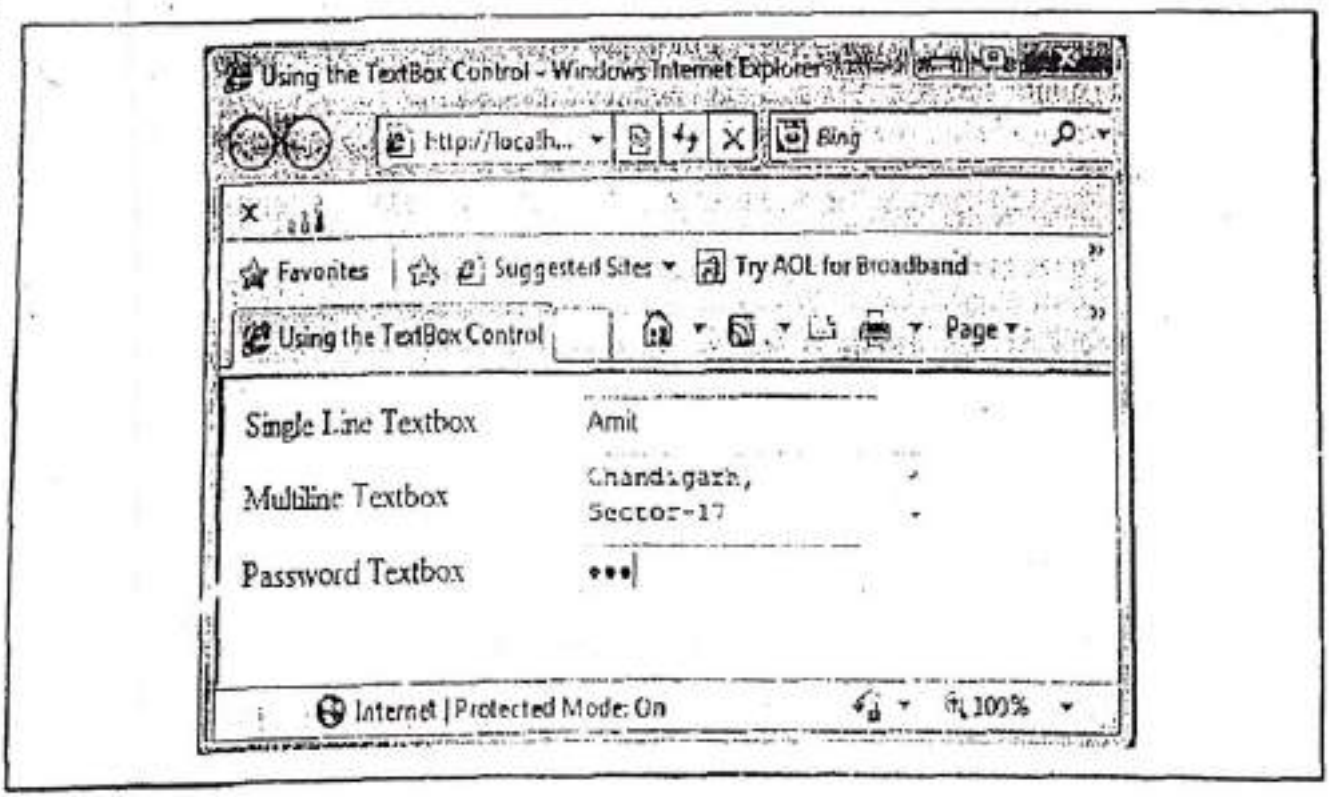
Control	Porperty	Value
TextBox1	TextMode	SingleLine *(Default)*
TextBox2	TextMode	MultiLine
TextBox3	TextMode	Password

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="TextBoxDemo.aspx.cs" Inherits="TextBoxDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the TextBox Control</title>
    </head>
    <body style="width: 396px">
        <form id="form1" runat="server">
            <div>
                <table>
                    <tr>
                        <td>Single Line Textbox</td>
                        <td>
                            <asp:TextBox ID="TextBox1"
                                runat="server">
                            </asp:TextBox>
                        </td>
                    </tr>

                    <tr>
                        <td>Multiline Textbox</td>
                        <td>
                            <asp:TextBox ID="TextBox2"
```

```
                                        runat="server"
                                        TextMode="MultiLine">
                                    </asp:TextBox>
                                </td>
                            </tr>
                            <tr>
                                <td>Password Textbox</td>
                                <td>
                                    <asp:TextBox ID="TextBox3"
                                        runat="server"
                                        TextMode="Password">
                                    </asp:TextBox>
                                </td>
                            </tr>
                        </table>
                    </div>
                </form>
            </body>
</html>
```

Step-5: Press **F5**. The following window will appear.



Here's the end of the Program 4-5.

(B) Using the CheckBox Control

A **CheckBox** control displays an option or choice that the user can either check or uncheck.

We can use number of checkboxes to make a group of choices, such as group of habits. It may allow user to make selections from a group of items. For example, user may select one or more habits from a group of habits. The basic syntax to add a **CheckBox** control to a web form using source code is as follows:

```
<asp:CheckBox ID="CheckBox1" runat="server"
oncheckedchanged="CheckBox1_CheckedChanged" Text="Habit" />
```

Table 4-C lists some commonly used properties of the CheckBox control.

Table 4-C : Properties of the CheckBox control

Property	Description
AutoPostBack	Used to post the form containing the **CheckBox** back to the server automatically when the **CheckBox** is checked or unchecked.
Checked	Used to get or set whether the **CheckBox** is checked.
Text	Used to set a label for the check box.

We can also set the **Checked** property of a **CheckBox** control programmatically. For example, in C#, this can be done as follows:

```
CheckBox1.Checked = true;
```

CheckedChanged is the most commonly used event of the **CheckBox** control. It is raised on the server when the check box is checked or unchecked. The basic C# syntax for **CheckedChanged** event of a **CheckBox** controlis as follows:

```
protected void CheckBox1_CheckedChanged(object sender, EventArgs e) //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

Program 4-6

Description of the Program 4-6: This program demonstrates how we can use a CheckBox control to accept input from the user. Follow the steps given below.

Step-1: Create a new **Web Application** named "**AcceptingUserInput**".

Step-2: Add a **Web Form** to the application named "**CheckBoxDemo.aspx**".

Step-3: Create the following design.

Step-4: Set the following properties of various **CheckBox** controls:

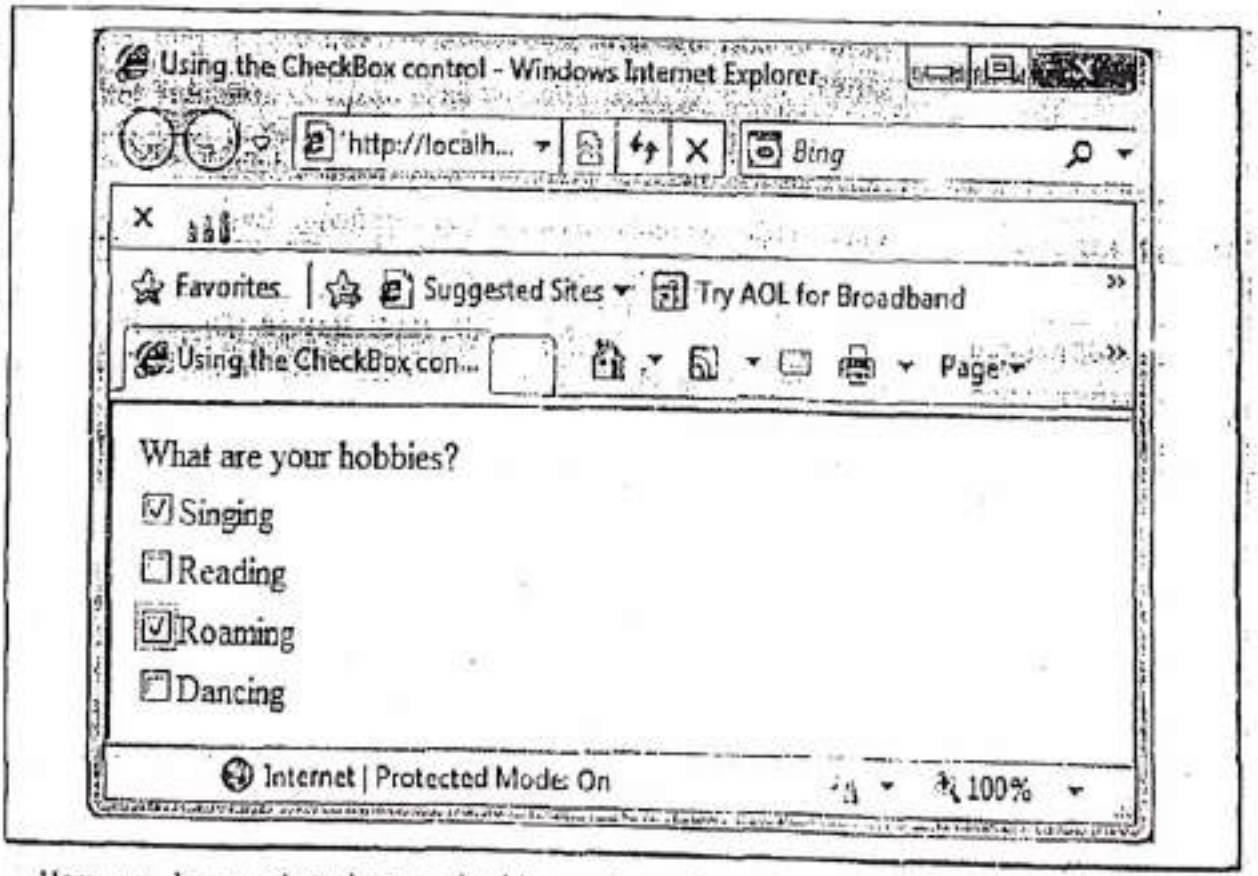
Control	Porperty	Value
CheckBox1	Text	Singing
CheckBox2	Text	Reading
CheckBox3	Text	Roaming
CheckBox4	Text	Dancing

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="CheckBoxDemo.aspx.cs" Inherits="CheckBoxDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the CheckBox control</title>
    </head>

    <body style="width: 350px">
        <form id="form1" runat="server">
            <div>
                <table>
                    <tr>
                        <td>What are your hobbies?</td>
                    </tr>
                    <tr>
                        <td>
                            <asp:CheckBox ID="CheckBox1"
                                runat="server"
                                Text="Singing" />
                        </td>
                    </tr>
                    <tr>
                        <td>
                            <asp:CheckBox ID="CheckBox2"
                                runat="server"
                                Text="Reading" />
                        </td>
                    </tr>
                    <tr>
                        <td>
```

```
                    <asp:CheckBox ID="CheckBox3"
                        runat="server"
                        Text="Roaming" />
                </td>
            </tr>
            <tr>
                <td>
                    <asp:CheckBox ID="CheckBox4"
                        runat="server"
                        Text="Dancing" />
                </td>
            </tr>
        </table>
    </div>
    </form>
</body>
</html>
```

Step-5: Press **F5**. The following window will appear.



Here we have selected two checkboxes from the given list. Here's, the end of the **Program 4-6.**

(C) Using the RadioButton Control

Radio buttons present a group of options from which the user can select just one option. **For example**, a list of courses contains four courses BCA, BBA, MCA, MBA. A student want to join a course then the student can opt only one course among the given list of courses.

To create a group of radio buttons, we can specify the same name for the **GroupName** property of each radio button in the group. If more than one group is required in a single form we can specify a different group name for each group. Initially, to select a radio button, set its **Checked** property to true. If the **Checked** property is set for more than one radio button in a group, then only the last one will be selected. The basic syntax to add a **RadioButton** control to a web form using source code is as follows:

```
<asp:RadioButton ID="RadioButton1" runat="server" Checked="True" GroupName="G1"
                                    Text="option1" />
```

Table 4-D lists some commonly used properties of the RadioButton control.

Table 4-D : Properties of the RadioButton control

Property	Description
AutoPostBack	Used to post the form containing the **RadioButton** back to the server automatically when the radio button is checked or unchecked.
Checked	Specifies whether the option is selected or not, default is false.
GroupName	Used to specify the name of a group of radio buttons.
Text	Used to set a label for the radio button.

CheckedChanged is the most commonly used event of the **RadioButton** control. It is raised on the server when the radio button is checked or unchecked. The basic C# syntaxfor **CheckedChanged** event of a **RadioButton** controlis as follows:

```
protected void RadioButton1_CheckedChanged(object sender, EventArgs e) //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

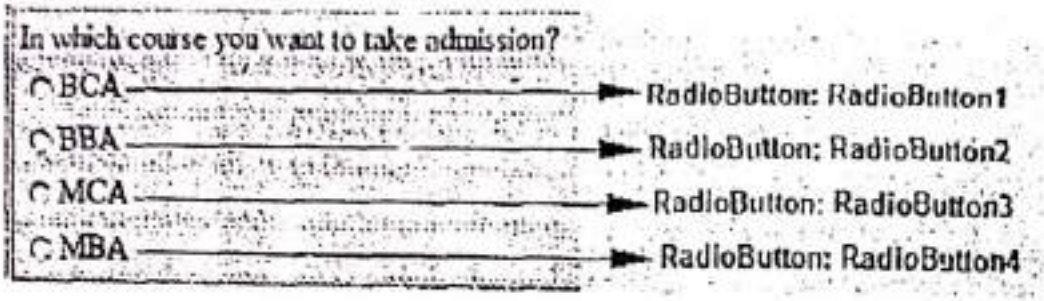
Program 4-7

Description of the Program 4-7: This program demonstrates how we can use a RadioButton control to accept input from the user. Follow the steps given below.

Step-1: Create a new Web Application named "AcceptingUserInput".

Step-2: Add a Web Form to the application named "RadioButtonDemo.aspx".

Step-3: Create the following design.



Step-4: Set the following properties of various **RadiouButton** controls:

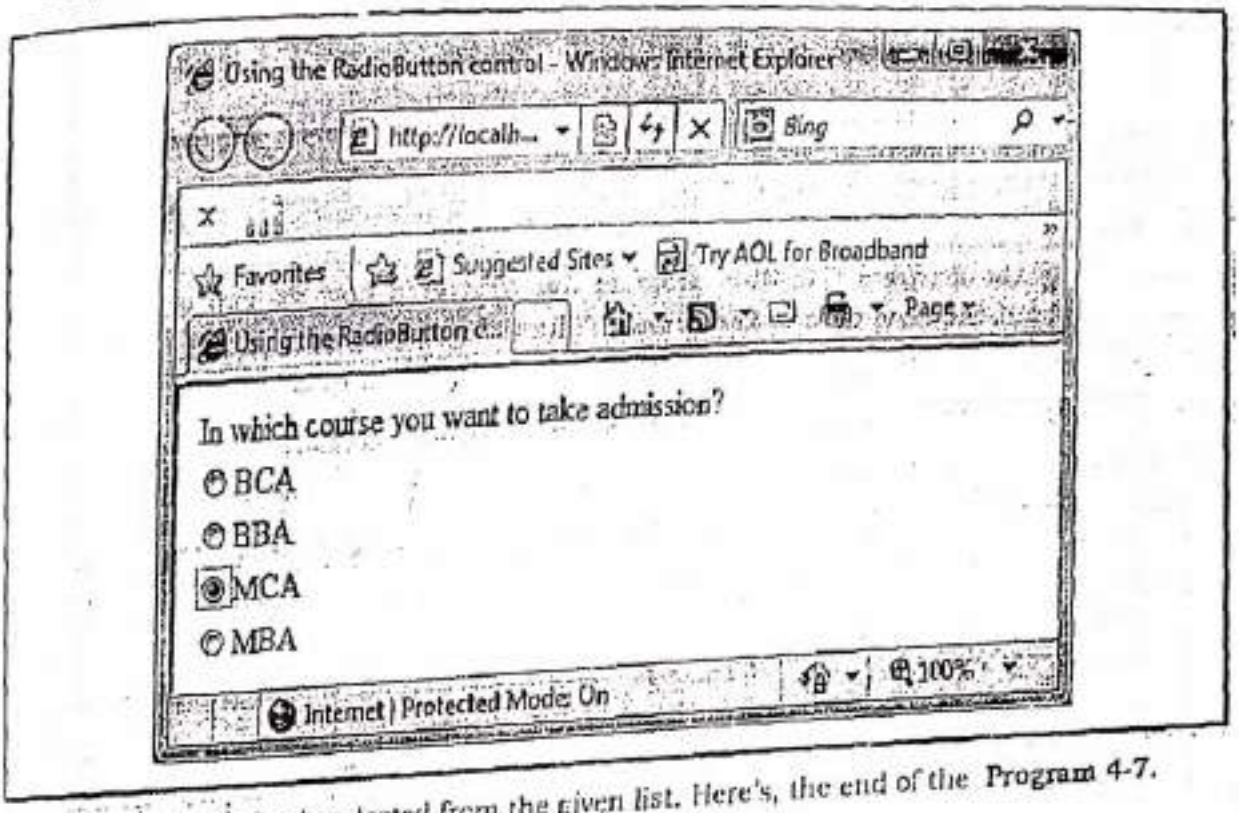
Control	Porperty	Value
RadioButton1	Text	BCA
RadioButton2	Text	BBA
RadioButton3	Text	MCA
RadioButton4	Text	MBA
Also set a property of all RadioButtons	GroupName	Course

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="RadioButtonDemo.aspx.cs" Inherits="RadioButtonDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the RadioButton control</title>
    </head>

    <body>
        <form id="form1" runat="server">
            <div>
                <table>
                    <tr>
                        <td>
                            In which course you want to take admission?
                        </td>
                    </tr>
```

```
                <tr>
                    <td>
                        <asp:RadioButton ID="RadioButton1"
                            runat="server"
                            GroupName="course"
                            Text="BCA" />
                    </td>
                </tr>
                <tr>
                    <td>
                        <asp:RadioButton ID="RadioButton2"
                            runat="server"
                            GroupName="course"
                            Text="BBA" />
                    </td>
                </tr>
                <tr>
                    <td>
                        <asp:RadioButton ID="RadioButton3"
                            runat="server"
                            GroupName="course"
                            Text="MCA" />
                    </td>
                </tr>
                <tr>
                    <td>
                        <asp:RadioButton ID="RadioButton4"
                            runat="server"
                            GroupName="course"
                            Text="MBA" />
                    </td>
                </tr>
            </table>

        </div>
    </form>
</body>
</html>
```

Step-5: Press F5. The following window will appear.



Here only one option is selected from the given list. Here's, the end of the **Program 4-7.**

4.8.1.3 SUBMITTING FORM DATA

The ASP.NET Framework includes the **Button**, **LinkButton**, and **ImageButton** controls which are used to submit a form to the server. These controls have the same function, but each control has a distinct appearance. These controls post a web page to the server. The following section describes these controls.

(A) Using the Button Control

The **Button** control is used to submit a form to the server. The basic syntax to add a **Button** control to a web form using **source** code is as follows:

```
<asp:Button ID="Button1" runat="server" onclick="Button1_Click" Text="Click" />
```

Table 4-E lists some commonly used properties of the Button control.

Table 4-E : Properties of the Button control

Property	Description
Enabled	Used to enable or disable the button.
PostBackUrl	It enables to post a from to a particular page.
TabIndex	Used to set the tab order of the **Button** control.
Text	Used to set the text to be displayed on the button.

The most commonly used **event** of **Button** control is **Click**. This event is raised when the Button control is clicked by the user. The basic C# syntax for **Click** event of a **Button** control is as follows:

```
protected void Button1_Click(object sender, EventArgs e)    //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

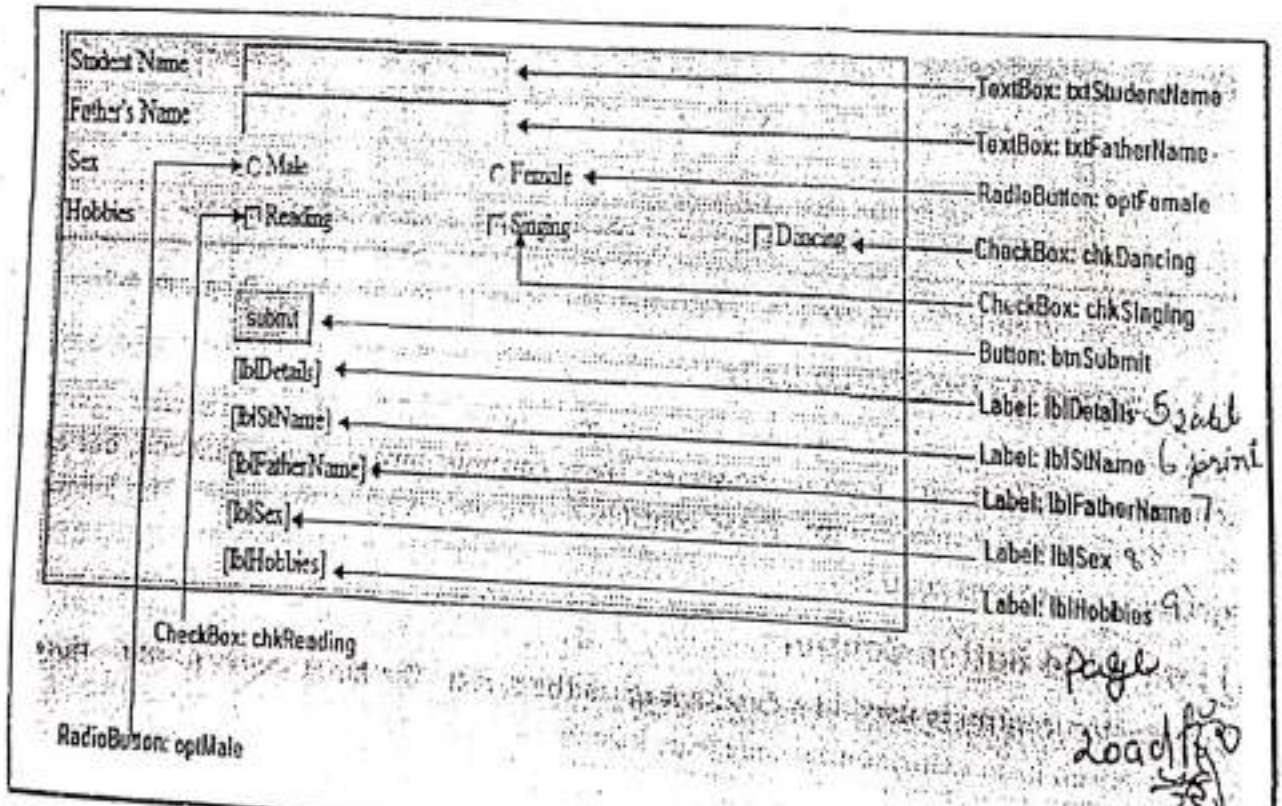
Program 4-8

Description of the Program 4-8: This program demonstrates how we can use a Button control to submit the form data to the server. Follow the steps given below.

Step-1: Create a new **Web Application** named "**SubmittingFormData**".

Step-2: Add a **Web Form** to the application named "**ButtonDemo.aspx**".

Step-3: Create the following design.



Step-4: Set the following properties of various controls:

Control	Porperty	Value
OptMale	GroupName	g1
	Text	Male
optFemale	GroupName	g1
	Text	Female
btnSubmit	Text	Submit

LblDetails	Text	(Blank: Set text value to empty)
lblStName		
lblFatherName		
LblSex		
lblHobbies		
LblDetails	Visible	False
lblStName		
lblFatherName		
LblSex		
lblHobbies		

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
      CodeBehind="ButtonDemo.aspx.cs" Inherits="ButtonDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Posting data using Button control</title>
    </head>

    <body>
        <form id="form1" runat="server">
            <div>
                <table>
                    <tr>
                      <td>Student Name</td>
                      <td colspan="3">
                          <asp:TextBox ID="txtStudentName"
                                       runat="server">
                          </asp:TextBox>
                      </td>
                    </tr>
                    <tr>
                      <td>Father&#39;s Name</td>
                      <td colspan="3">
                          <asp:TextBox ID="txtFatherName"
                                       runat="server">
                          </asp:TextBox>
                      </td>
```

```
</tr>
<tr>
  <td>Sex</td>
  <td>
      <asp:RadioButton ID="optMale"
                       runat="server"
                       GroupName="g1"
                       Text="Male" />
  </td>
  <td>
      <asp:RadioButton ID="optFemale"
                       runat="server"
                       GroupName="g1"
                       Text="Female" />
  </td>
  <td> </td>
</tr>
<tr>
  <td>Hobbies</td>
  <td>
      <asp:CheckBox ID="chkReading"
                       runat="server"
                       Text="Reading" />
  </td>
  <td>
      <asp:CheckBox ID="chkSinging"
                       runat="server"
                       Text="Singing" />
  </td>
  <td>
      <asp:CheckBox ID="chkDancing"
                       runat="server"
                       Text="Dancing" />
  </td>
</tr>
<tr>
  <td> </td>
  <td> </td>
  <td> </td>
  <td> </td>
</tr>
<tr>
```

```
        <td> </td>
        <td colspan="3">
            <asp:Button ID="btnSubmit"
                                runat="server"
                                onclick="btnSubmit_Click"
                                  Text="Submit" />
        </td>
      </tr>
      <tr>
        <td> </td>
        <td colspan="3">
            <asp:Label ID="lblDetails"
                        runat="server"
                        Visible="False">
            </asp:Label>
        </td>
      </tr>
      <tr>
        <td>
        </td>
        <td colspan="3">
            <asp:Label ID="lblStName"
                        runat="server"
                        Visible="False">
            </asp:Label>
        </td>
      </tr>
      <tr>
        <td> </td>
        <td colspan="3">
            <asp:Label ID="lblFatherName"
                        runat="server"
                        Visible="False">
            </asp:Label>
        </td>
      </tr>
      <tr>
        <td> </td>
        <td colspan="3">
            <asp:Label ID="lblSex"
                        runat="server"
                        Visible="False">
```

```
                                    </asp:Label>
                                </td>
                            </tr>
                            <tr>
                                <td> </td>
                                <td colspan="3">
                                    <asp:Label ID="lblHobbies"
                                               runat="server"
                                               Visible="False">
                                    </asp:Label>
                                </td>
                            </tr>
                        </table>
                    </div>
                </form>
            </body>
</html>
```

Step-5: Double click on page. The code behind file will appear. Add the following code in event handler **Page_Load(....)**.

```
    protected void Page_Load(object sender, EventArgs e)
    {
    //Setting default text of labels whenever the page loaded.
        lblDetails.Text = "----------Data Submitted----------";
        lblStName.Text = "Student Name: ";
        lblFatherName.Text="Father Name: ";
        lblSex.Text="Sex: ";
        lblHobbies.Text = "Hobbies: ";

}
```

Step-6: Double click on button **btnSubmit**. Add the following code in event handler **btnSubmit_Click(....)**.

```
    protected void btnSubmit_Click(object sender, EventArgs e)
    {
        //Making labels visible on page.
        lblDetails.Visible = true;
        lblStName.Visible = true;
        lblFatherName.Visible = true;
        lblSex.Visible = true;
        lblHobbies.Visible = true;

        //setting text property of labels
        lblStName.Text = "Student Name: "+txtStudentName.Text;
```

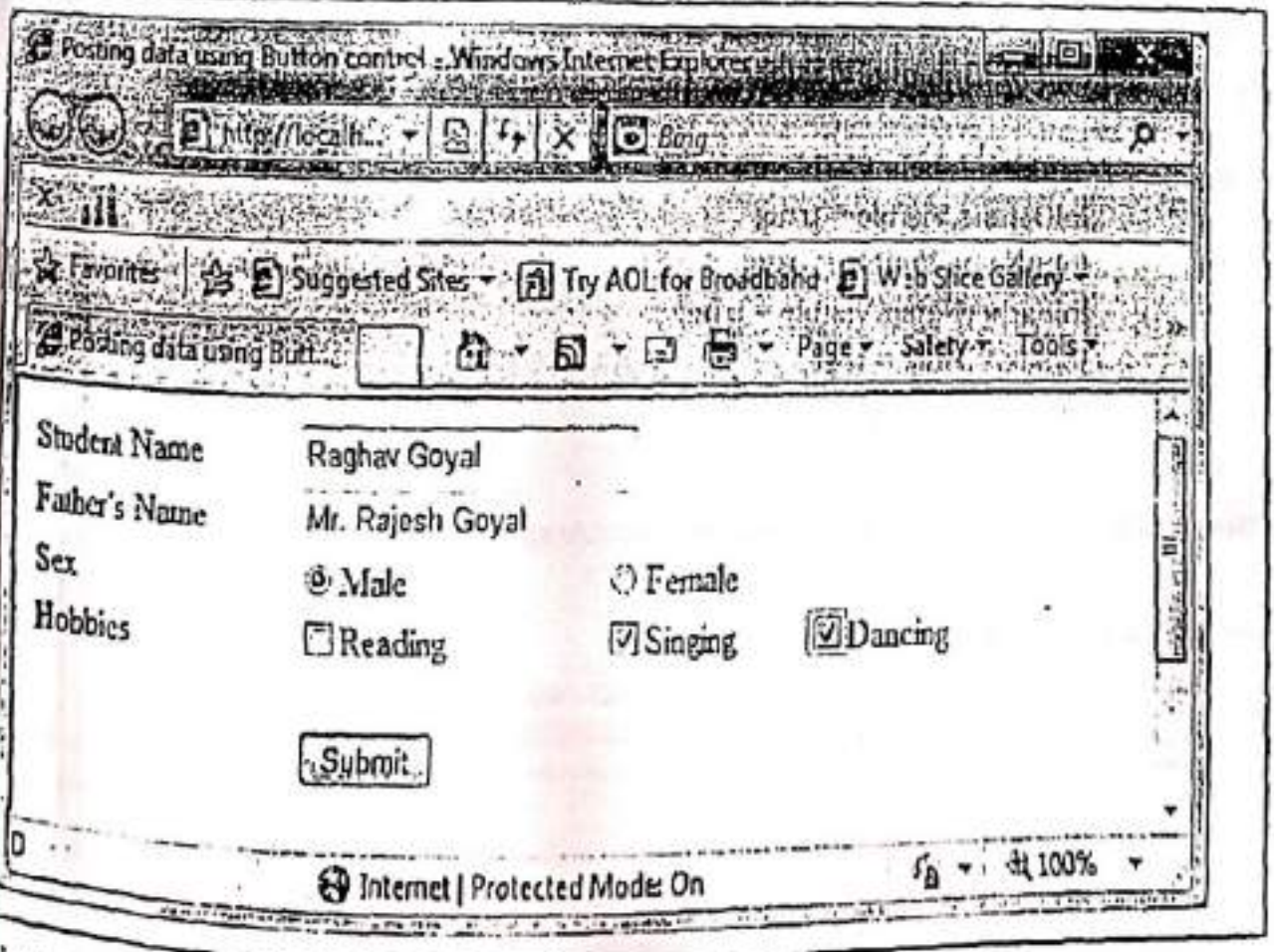
```
            lblFatherName.Text = "Father Name: "+txtFatherName.Text;

    if (optMale.Checked == true)
                lblSex.Text = "Sex: Male";
    else
            lblSex.Text = "Sex: Female";

    if(chkReading.Checked==true)
                lblHobbies.Text=lblHobbies.Text+" Reading ";

    if(chkSinging.Checked==true)
            lblHobbies.Text=lblHobbies.Text+" Singing ";

    if(chkDancing.Checked==true)
                lblHobbies.Text=lblHobbies.Text+" Dancing ";

    }
```

Note: The whole code behind file named **"ButtonDemo.aspx.cs"** will be as follows:

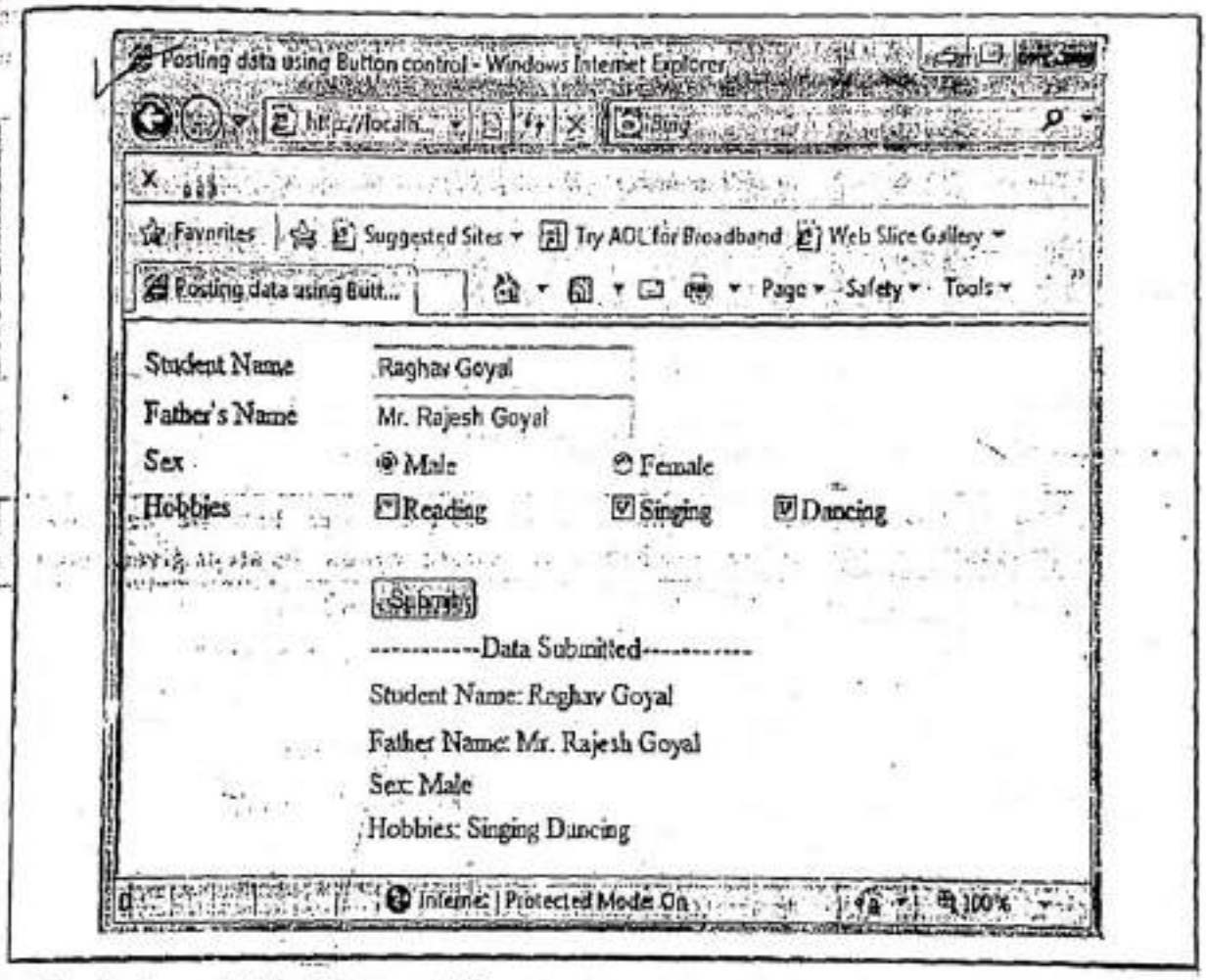
```
using System;
public partial class ButtonDemo : System.Web.UI.Page
{
protected void btnSubmit_Click(object sender, EventArgs e)
    {
      //Making labels visible on page.
        lblDetails.Visible = true;
        lblStName.Visible = true;
        lblFatherName.Visible = true;
        lblSex.Visible = true;
        lblHobbies.Visible = true;

      //setting text property of labels
        lblStName.Text = "Student Name: "+txtStudentName.Text;
        lblFatherName.Text = "Father Name: "+txtFatherName.Text;

if (optMale.Checked == true)
      lblSex.Text = "Sex: Male";
      else
          lblSex.Text = "Sex: Female";

if(chkReading.Checked==true)
              lblHobbies.Text=lblHobbies.Text+" Reading ";
```

```
if(chkSinging.Checked==true)
        lblHobbies.Text=lblHobbies.Text+" Singing ";

if(chkDancing.Checked==true)
        lblHobbies.Text=lblHobbies.Text+" Dancing ";
    }

protected void Page_Load(object sender, EventArgs e)
    {
//Setting default text of labels whenever the page loaded.
      lblDetails.Text = "----------Data Submitted----------";
      lblStName.Text = "Student Name: ";
      lblFatherName.Text="Father Name: ";
      lblSex.Text="Sex: ";
      lblHobbies.Text = "Hobbies: ";
    }
```

Step-7: Press **F5**. The output will be as follows:



When user clicks on the submit button, the output will be shown as follows:



Here's the end of the **Program 4-8.**

(B) Using the LinkButton Control

LinkButton is a button that look like a **hyperlink** in a page but actually it is not a hyperlink. A **LinkButton** control also behaves like a **Button** control. The basic syntax to add a **LinkButton** control to a web form using **source code** is as follows:

```
<asp:LinkButton ID="LinkButton1" runat="server" onclick="LinkButton1_Click"
                Text="Link"/>
```

Table 4-F lists some commonly used properties of the LinkButton control.

Table 4-F : Properties of the LinkButton control

Property	Description
Enabled	Used to enable or disable the **LinkButton** control.
PostBackUrl	It enables to post a from to a particular page.
TabIndex	Used to set the tab order of the **LinkButton** control.
Text	Used to set the text to be displayed on a **LinkButton** control.

The most commonly used **event** of **LinkButton** control is **Click**. This event raised when the **LinkButton** control is clicked by the user. The basic C# syntaxfor **Click** event of a **LinkButton** controlis as follows:

```
protected void LinkButton1_Click(object sender, EventArgs e)        //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

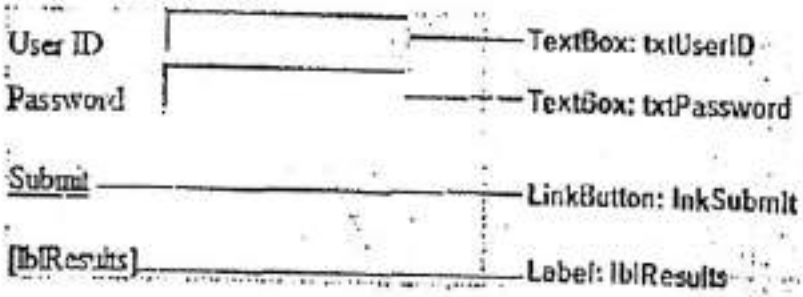
Program 4-9

Description of the Program 4-9: This program demonstrates how we can use a LinkButton control to submit the form data to the server. Follow the steps given below.

Step-1: Create a new **Web Application** named **"SubmittingFormData"**.

Step-2: Add a **Web Form** to the application named **"LinkButtonDemo.aspx"**.

Step-3: Create the following design.



Step-4: Set the following properties of various controls:

Control	Porperty	Value
txtPassword	TextMode	Password
LnkSubmit	Text	Submit
LblResults	Text	(Blank: No value)

Source will be as follows:

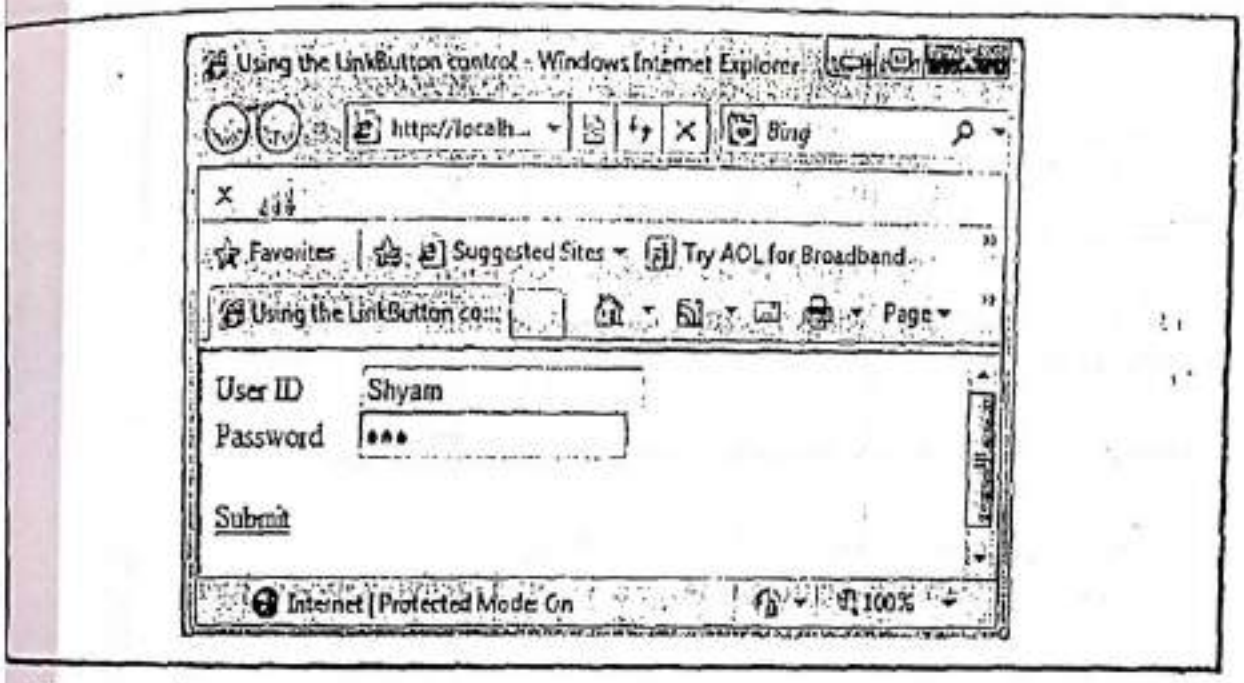
```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="LinkButtonDemo.aspx.cs" Inherits="LinkButtonDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the LinkButton control</title>
    </head>
```

```
<body>
    <form id="form1" runat="server">
        <div>User ID       
            <asp:TextBox id="txtUserID"
                Runat="server" />

            <br />Password    
            <asp:TextBox id="txtPassword"
                Runat="server"
                TextMode="Password" />
            <br /><br />

            <asp:LinkButton id="lnkSubmit"
                Text="Submit"
                OnClick="lnkSubmit_Click"
                Runat="server" />
            <br /><br />

            <asp:Label id="lblResults"
                Runat="server" />
        </div>
    </form>
</body>
</html>
```

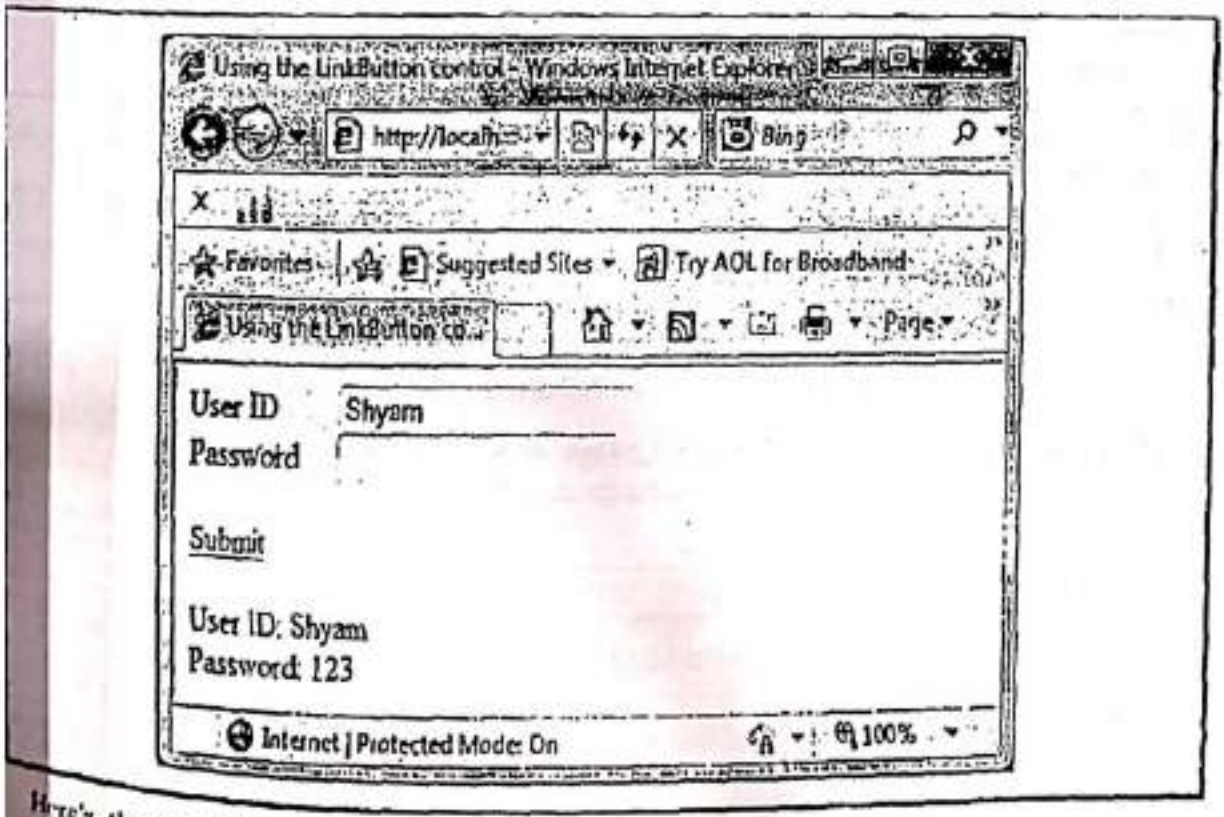
Step-5: Double click on the **lnkSubmit** button and write the following code in the code behind file named **"LinkButtonDemo.aspx.cs"**.

```
using System;
public partial class LinkButtonDemo : System.Web.UI.Page
{

protected void lnkSubmit_Click(object sender, EventArgs e)
  {
    lblResults.Text = "User ID: " + txtUserID.Text;
    lblResults.Text += "<br />Password: " + txtPassword.Text;
  }
}
```

Step-6: Press **F5**. The output will be as follows:



When user clicks on the submit button, the output will be shown as follows:



Here's the end of the **Program 4-9.**

(C) Using the ImageButton Control

ImageButton is a graphical button to provide a rich button appearance. We can set the **ImageUrl** property of **ImageButton** control to point to a specific image. It also causes the form to

be posted (submitted) to the server. The basic syntax to add an **ImageButton** control to a web form using source code is as follows:

```
<asp:ImageButton ID="ImageButton1" runat="server" ImageUrl="~/Image.bmp"
                 onclick="ImageButton1_Click" />
```

Table 4-G lists some commonly used properties of the ImageButton control.

Table 4-G : Properties of the ImageButton control

Property	Description
AlternateText	It enables to provide alternate text for the **ImageButton**. It displayed when the **ImageUrl** property is not set or image **URL** is invalid.
DescriptionUrl	Used to provide a link to a page that contains a detailed description of the image.
Enabled	Used to enable or disable the **ImageButton** control.
ImageAlign	Used to align the image relative to other elements in the page. Possible values are **NotSet, Left, Right, BaseLine, Top, Middle, Bottom, AbsBottom, AbsMiddle, and TextTop.**
ImageUrl	Used to specify the **URL** to the image to be displayed on **ImageButton.**
PostBackUrl	It enables to post a from to a particular page.
TabIndex	Used to set the tab order of the **ImageButton** control.

The most commonly used **event** of **ImageButton** control is **Click**. This event is raised when the **ImageButton** control is clicked by the user. The basic C# syntaxfor **Click** event of a **ImageButton** controlis as follows:

```
protected void ImageButton1_Click(object sender, ImageClickEventArgs e)      //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

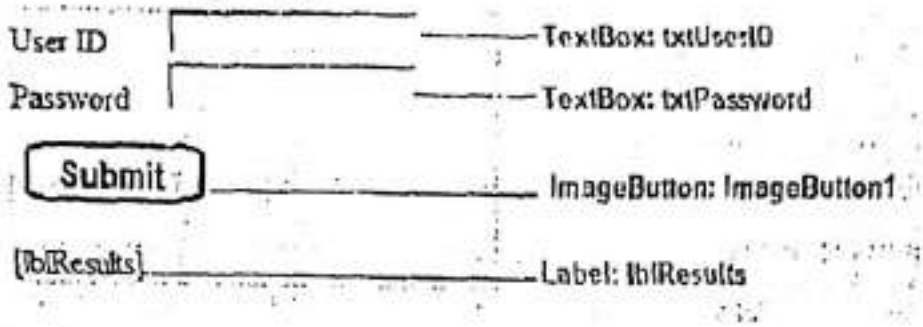
Program 4-10

Description of the Program 4-10: This program demonstrates how we can use a ImageButton control to submit the form data to the server. Follow the steps given below.

Step-1: Create a new **Web Application** named "**SubmittingFormData**".

Step-2: Add a **Web Form** to the application named **"ImageButtonDemo.aspx"** and an image named **"savebtn.png"**.

Step-3: Create the following design.



Step-4: Set the following properties of various controls:

Control	Porperty	Value
TxtPassword	TextMode	Password
ImageButton1	ImageUrl	~/savebtn.png
LblResults	Text	(Blank: No value)

Source will be as follows:

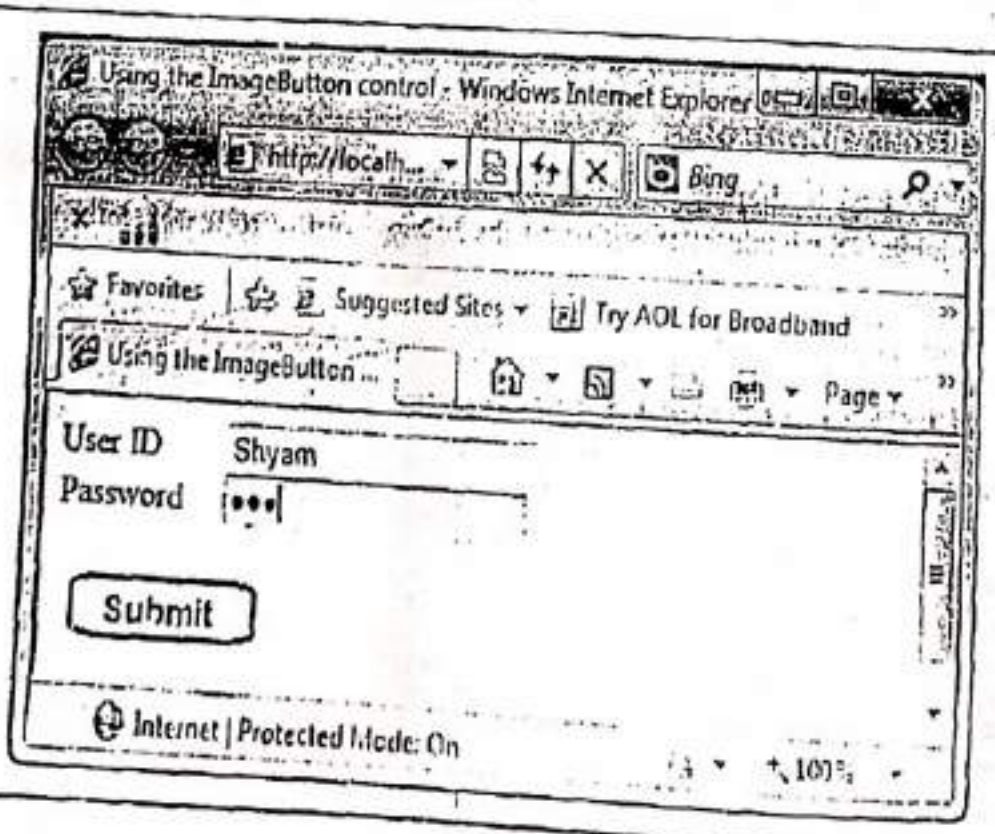
```
<%@ Page Languar .="C#" AutoEventWireup="true"
    CodeBehind="ImageButtonDemo.aspx.cs" Inherits="ImageButtonDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the ImageButton control</title>
    </head>
    <body>
        <form id="form1" runat="server">
            <div>User ID       
                <asp:TextBox id="txtUserID" Runat="server" />

                <br />Password    
                <asp:TextBox id="txtPassword"
                        Runat="server"
                        TextMode="Password" />
                <br /><br />

                <asp:ImageButton ID="ImageButton1"
                        runat="server"
                        ImageUrl="~/savebtn.png"
                        onclick="ImageButton1_Click" />
```

```
                    <br /><br />

                    <asp:Label id="lblResults"
                        Runat="server" />
                </div>
            </form>
        </body>
</html>
```

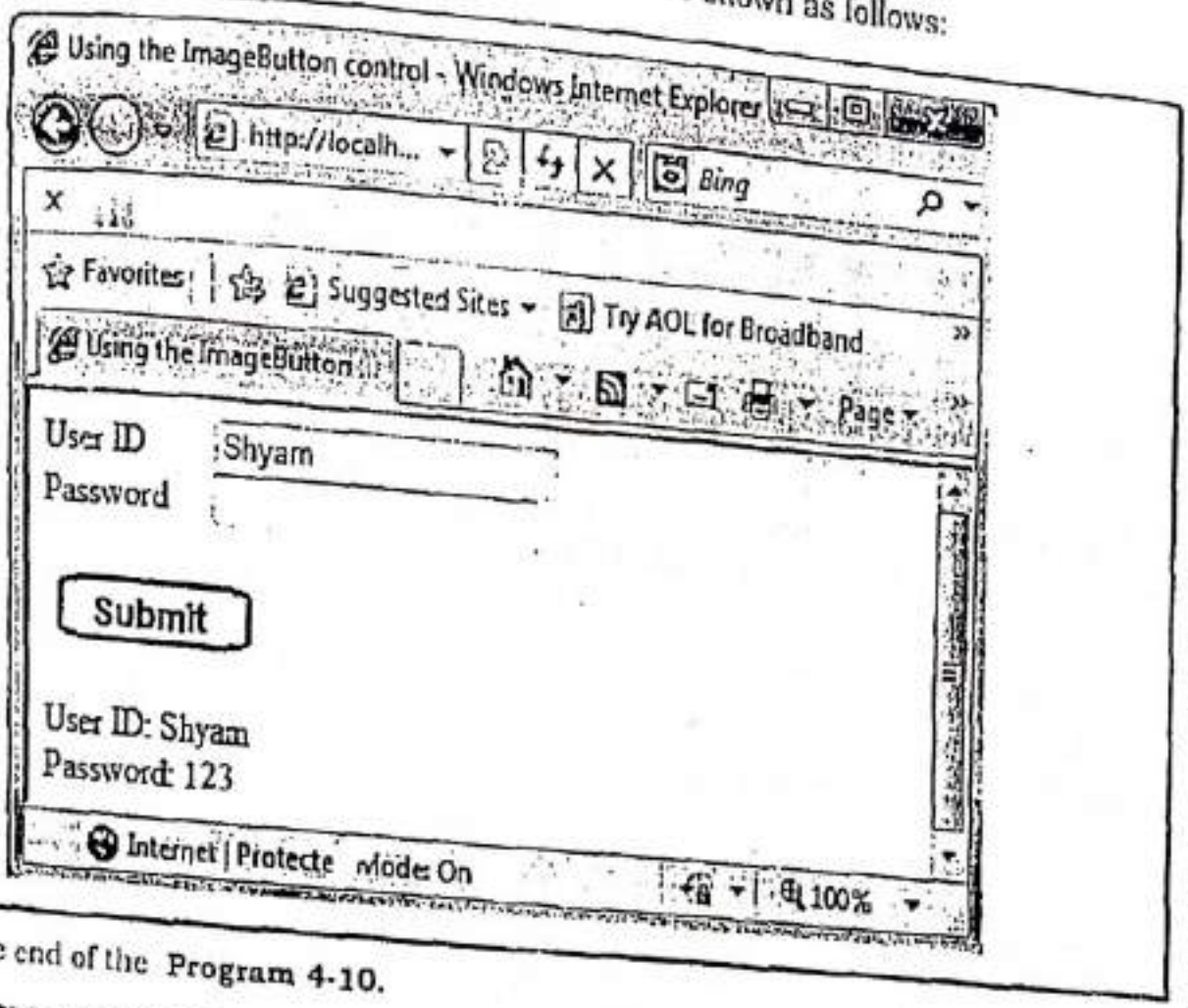
Step-5: Double click on the **ImageButton1** button and write the following code in the code behind file named **"ImageButtonDemo.aspx.cs"**.

```
using System;
using System.Web.UI;
public partial class ImageButtonDemo : System.Web.UI.Page
{
    protected void ImageButton1_Click(object sender, ImageClickEventArgs e)
    {
        lblResults.Text = "User ID: " + txtUserID.Text;
        lblResults.Text += "<br />Password: " + txtPassword.Text;

    }
}
```

Step-6: Press **F5**. The output will be as follows:



When user clicks on the submit button, the output will be shown as follows:



Here's, the end of the **Program 4-10.**

1.8.1.4 DISPLAYING IMAGES

ASP.NET provides two controls to display images. These controls are Image control and ImageMap control. These can be described as follows:

Using the Image Control

Image control is used to display an image on the web page, or some alternative text, if the image is not available. The basic syntax to add an Image control to a web form using source code is as follows:

```
<asp:Image ID="Image1" runat="server" ImageUrl="~/image.gif" />
```

Table 4-H lists some commonly used properties of the Image control

Table 4-H : Properties of the Image control

Property	Description
AlternateText	Alternate text to be displayed if image is not available.
ImageAlign	Used to specify alignment of the image.
ImageUrl	Used to specify the path of the image to be displayed by the control.

Take a look at the following program.

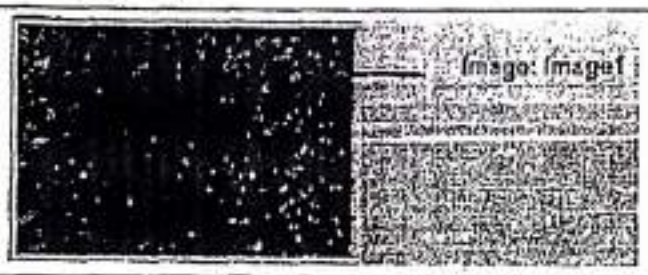
Program 4-11

Description of the Program 4-11: This program demonstrates how we can use an Image control to display image on the web page. Follow the steps given below.

Step-1: Create a new **Web Application** named "**DisplayImages**".

Step-2: Add a **Web Form** to the application named "**ImageControlDemo.aspx**" and an image named "**img1.jpg**"by selecting it from **Add Existing Item**option.

Step-3: Create the following design.



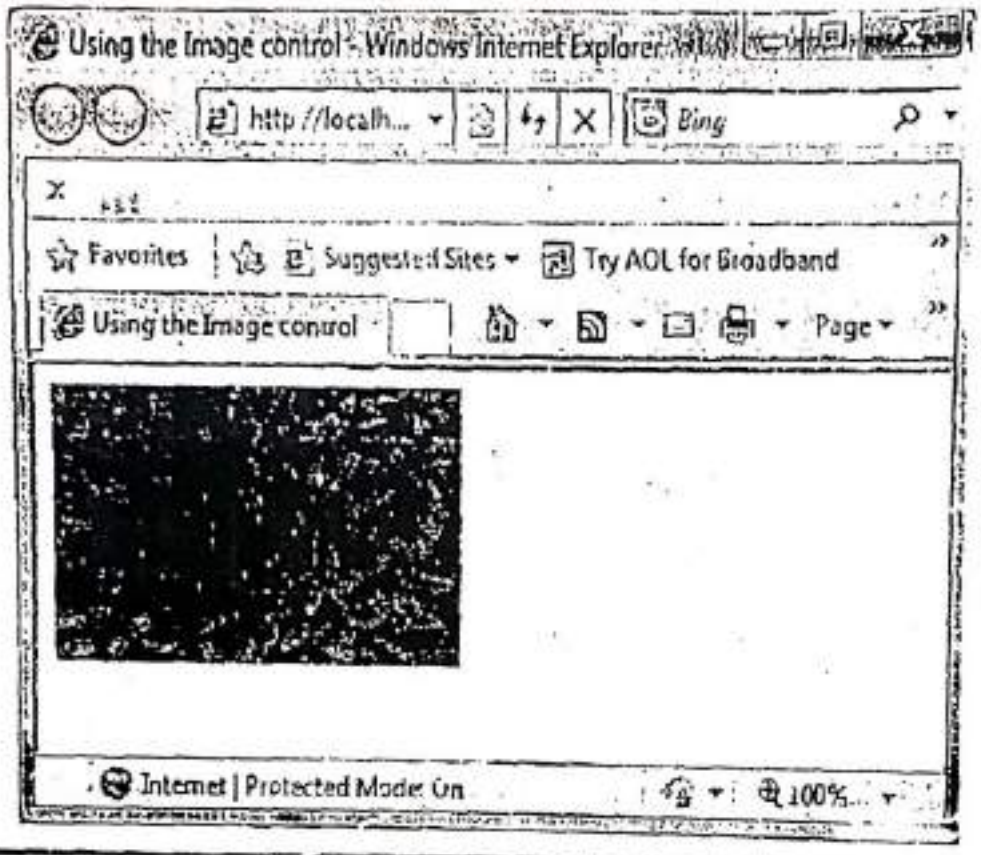
Step-4: Set the following properties of **Image** control:

Control	Porperty	Value
Image1	ImageUrl	~/img1.jpg

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="ImageControlDemo.aspx.cs"
Inherits="ImageControlDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the Image control</title>
    </head>

    <body>
        <form id="form1" runat="server">
            <div style="width: 198px">
                <asp:Image ID="Image1"
                           runat="server"
                           Height="123px"
                           ImageUrl="~/img1.jpg"
                           Width="197px" />
            </div>
        </form>
    </body>
</html>
```

Step-5: Press **F5**. The output will be as follows:



Here's, the end of the Program 4-11.

) Using the ImageMap Control

The **ImageMap** control enables us to create a **client-side** image map. An image map displays image. When we click different areas of the image, things happen. For example, we can use image map as a fancy navigation bar. In that case, clicking different areas of the image map rigates to different pages in our website.

ImageMap control is composed out of instances of the **HotSpot** class. A HotSpot defines the kable regions in an image map. The ASP.NET framework provides three HotSpot classes:

- **CircleHotSpot:** Enables you to define a circular region in an image map.
- **PolygonHotSpot:** Enables you to define an irregularly shaped region in an image map.
- **RectangleHotSpot:** Enables you to define a rectangular region in an image map.

With the ImageMap control, we can take a single image and specify particular hotspots on the ge using coordinates.

Take a look at the following program.

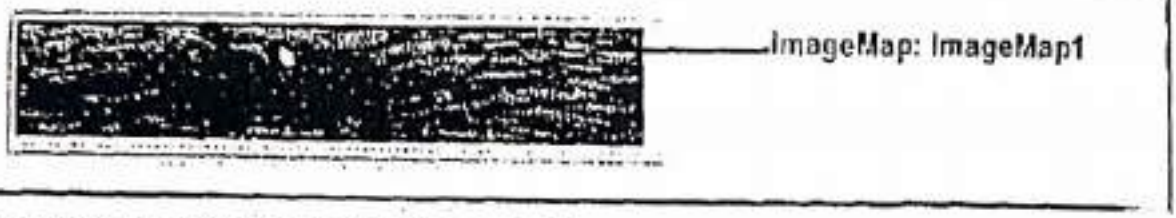
ogram 4-12

escription of the Program 4-12: This program demonstrates how we can use an ageMap control to display image on the web page. Follow the steps given below.

Step-1: Create a new **Web Application** named "**DisplayImages**".

Step-2: Add a Web Form to the application named "ImageMapDemo.aspx" and an image named "img1.jpg"by selecting it from **Add Existing Item** option.

Step-3: Create the following design.



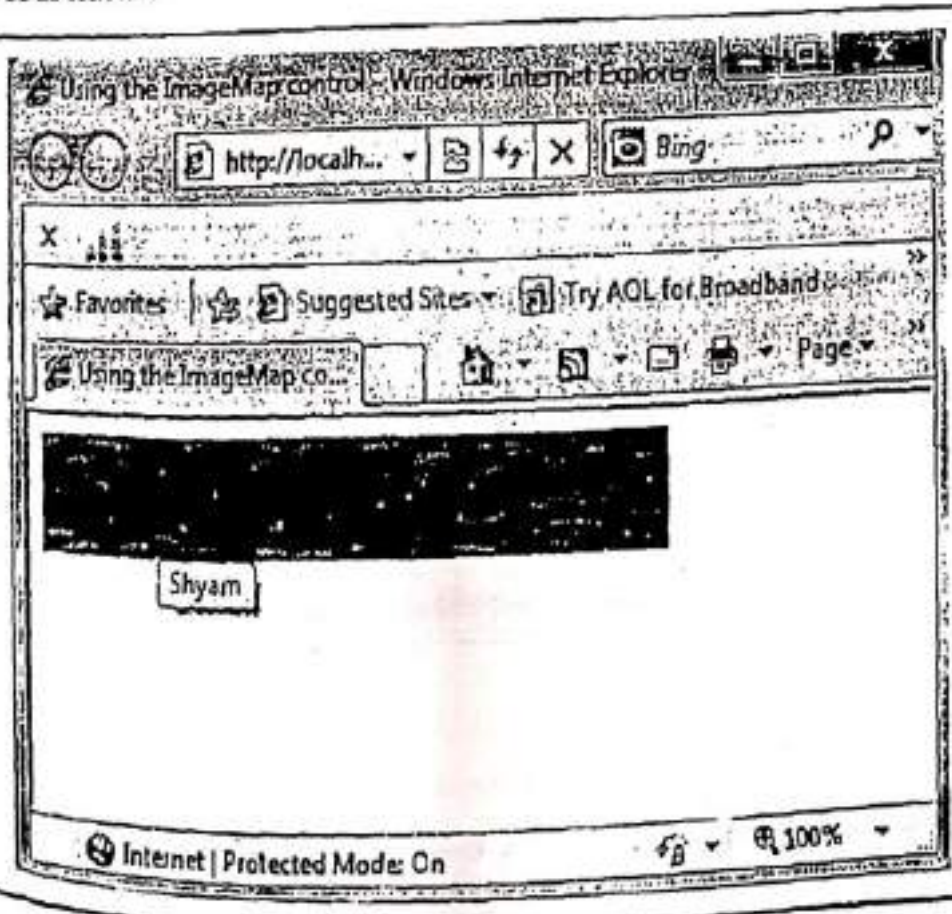
Step-4: Write the following code in the source file.

Source will be as follows:

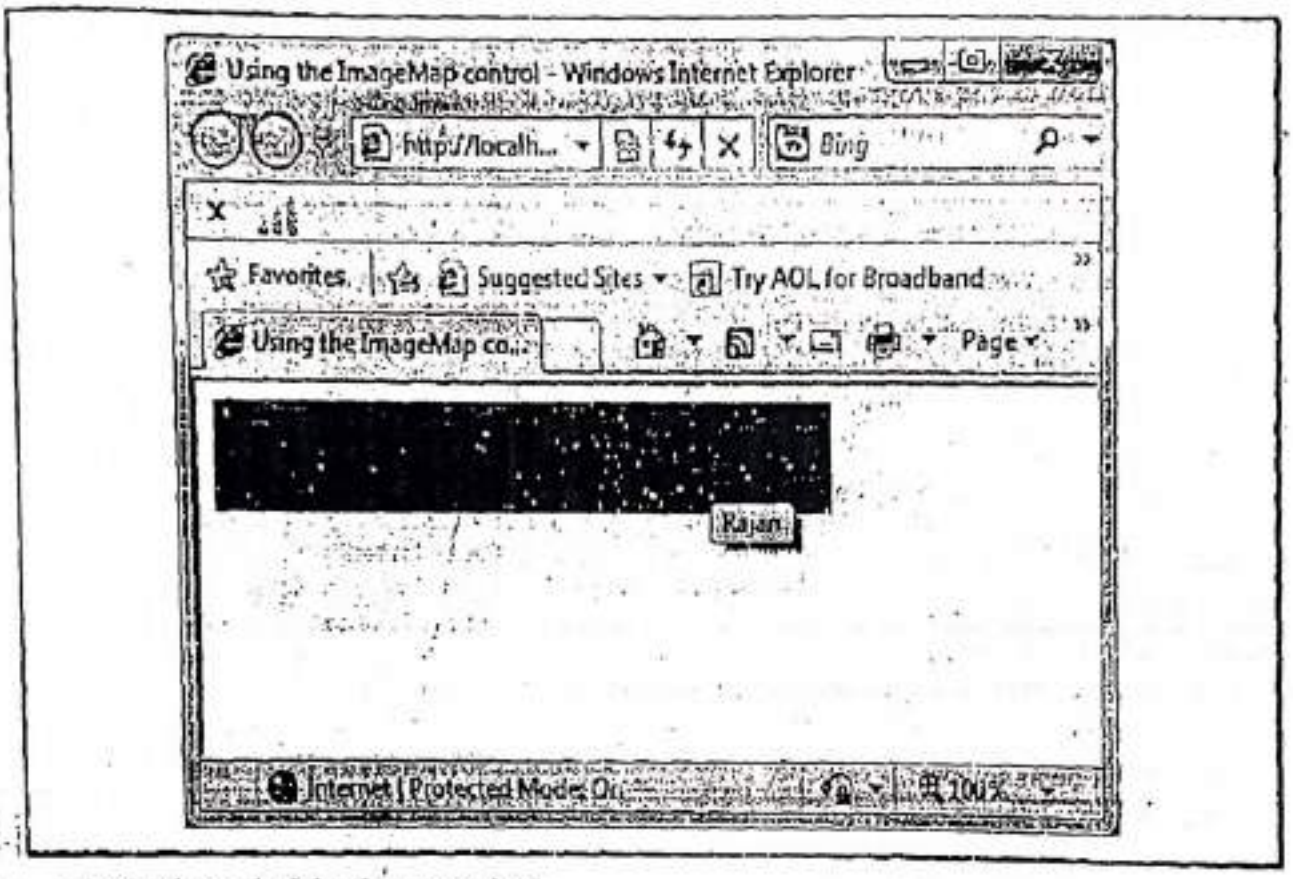
```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="ImageMapDemo.aspx.cs" Inherits="ImageMapDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the ImageMap control</title>
    </head>
    <body>
        <form id="form1" runat="server">
            <div>
                <asp:ImageMap ID="ImageMap1"
                    runat="server"
                    Height="47px"
                    ImageUrl="~/img1.jpg"
                     Width="300px"
                    OnClick="Imagemap1_Click"
                    HotSpotMode="PostBack">

                    <asp:RectangleHotSpot Top="0"
                        Bottom="225"
                        Left="0"
                        Right="150"
                        AlternateText="Shyam"
                        PostBackValue="Shyam">
```

```
                    </asp:RectangleHotSpot>

                    <asp:RectangleHotSpot Top="0"
                                          Bottom="225"
                                          Left="151"
                                          Right="300"
                                          AlternateText="Rajan"
                                          PostBackValue="Rajan">
                    </asp:RectangleHotSpot>
                </asp:ImageMap>
            </div>
        </form>
    </body>
</html>
```

Step-E: Press F5. When user moves cursor within the region from left 0 to right 150, the output will be as follows.



When user moves cursor within the region from left 150 to right 300, the output will be as follows:



Here's, the end of the **Program 4-12.**

4.8.1.5 USING THE PANEL CONTROL

The Panel control enables us to work with a group of ASP.NET controls. For example, we can use a Panel control to hide or show a group of ASP.NET controls.

Table 4-I lists some commonly used properties of the Panel control.

Table 4-I : Properties of the Panel control

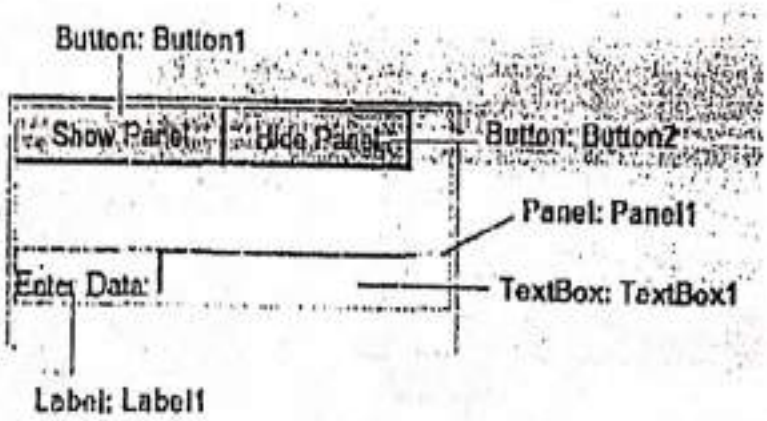
Property	Description
DefaultButton	It enables us to specify the default button in a Panel. The default button is invoked when you press the Enter button.
Direction	It enables us to get or set the direction in which controls that display text are rendered. Possible values are NotSet, LeftToRight, and RightToLeft.
GroupingText	It enables us to render the Panel control as a fieldset with a particular legend.
HorizontalAlign	Enables you to specify the horizontal alignment of the contents of the Panel. Possible values are Center, Justify, Left, NotSet, and Right.
ScrollBars	Enables you to display scrollbars around the panel's contents. Possible values are Auto, Both, Horizontal, None, and Vertical.

Take a look at the following program.

Program 4-13

Description of the Program 4-13: This program demonstrates how we can use a Panel control to display or hide a group of controls on the web page. Follow the steps given below.

Step-1: Create a new **Web Application** named **"PanelControlDemo"**.
Step-2: Add a **Web Form** to the application named **"PanelControl.aspx"**.
Step-3: Create the following design.



Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="PanelControl.aspx.cs" Inherits="PanelControl" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the Panel control</title>
    </head>

    <body style="width: 221px">
        <form id="form1" runat="server">
            <div>
                <asp:Button ID="Button1"
                            runat="server"
                            onclick="Button1_Click"
                             Text="Show Panel" />

                <asp:Button ID="Button2"
                            runat="server"
                            onclick="Button2_Click"
                             Text="Hide Panel" />
```

```
                    <br />
                    <br />   

                    <asp:Panel ID="Panel1" runat="server">
                        <asp:Label ID="Label1"
                                runat="server"
                                Text="Enter Data: ">
                        </asp:Label>

                        <asp:TextBox ID="TextBox1"
                                runat="server">
                        </asp:TextBox>
                    </asp:Panel>
                </div>
            </form>
        </body>
</html>
```

Step-4: Double click on the button named **Button1** and write the following code in C# code behind file named **"PanelControl.aspx.cs"**.

```
protected void Button1_Click(object sender, EventArgs e)
    {
        Panel1.Visible = true;
    }
```

Step-5: Double click on the button named **Button2** and write the following code in C# code behind file named **"PanelControl.aspx.cs"**.

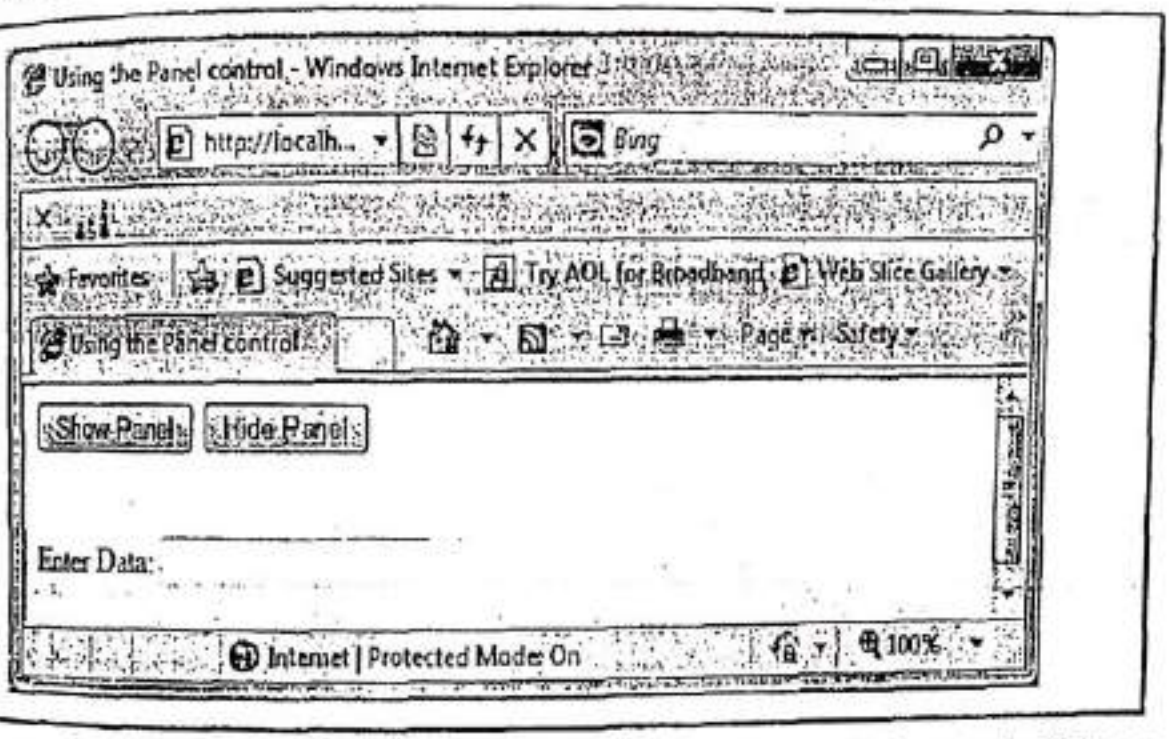
```
protected void Button2_Click(object sender, EventArgs e)
    {
        Panel1.Visible = false;
    }
```

Note: The whole code behind file named **"PanelControl.aspx.cs"** will be as follows:

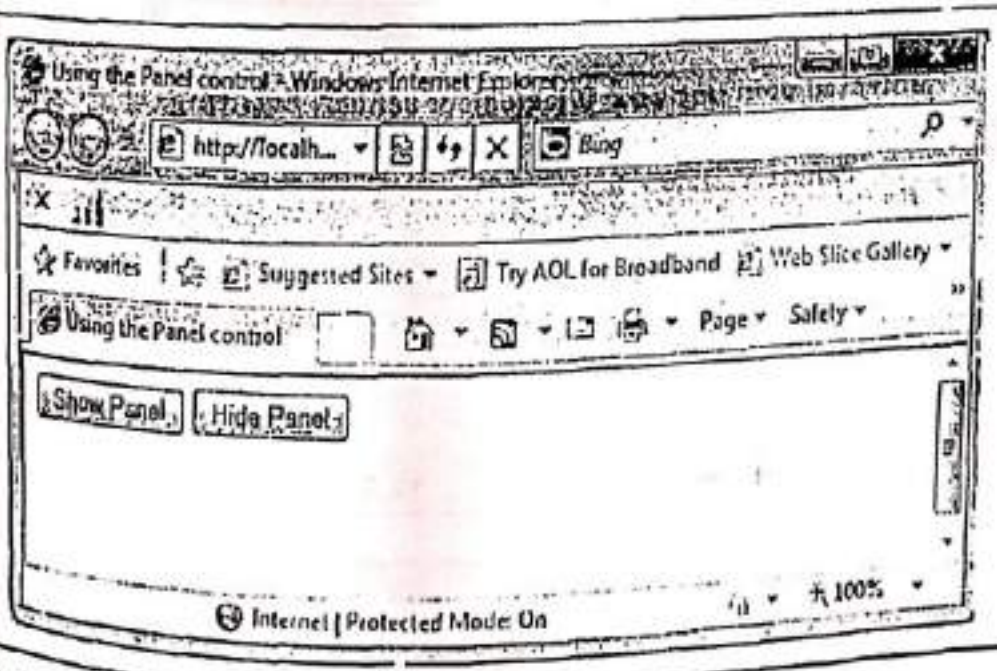
```
using System;
public partial class PanelControl : System.Web.UI.Page
{
    protected void Button1_Click(object sender, EventArgs e)
    {
        Panel1.Visible = true;
    }

    protected void Button2_Click(object sender, EventArgs e)
```

```
{
    Panel1.Visible = false;
}
}
```

Step-6: Press F5. The output will be as follows:



When user clicks on **Hide Panel** button (*Button2*) shown on the page, the output will be as follows:



Here's the end of the **Program 4-13.**

4.8.1.6 USING THE HYPERLINK CONTROL

HyperLink control creates links on a web page and allows users to navigate from one page to another in an application or an absolute URL. We can use text or an image to act as a link in a **HyperLink** control. When users click the control, the target page opens. **HyperLink** control does not submit a form to a server. The basic syntax to add a **HyperLink** control to a web form using source code is as follows:

```
<asp:HyperLink ID="HyperLink1" runat="server">HyperLink</asp:HyperLink>
```

Table 4-J lists some commonly used properties of the HyperLink control.

Table 4-J : Properties of the HyperLink control	
Property	**Description**
ImageUrl	Used to specify path of the image to be displayed by the control
NavigateUrl	Target page URL
Text	The text to be displayed as the link
Target	The window or frame which will load the linked page.

Take a look at the following program.

Program 4-14

Description of the Program 4-14: This program demonstrates how we can use a HyperLink control to link web pages. Follow the steps given below.

Step-1: Create a new **Web Application** named **"UsingHyperLink"**.

Step-2: Add two **Web Forms** to the application named **"Page1.aspx"** and **"Page2.aspx"**.

Step-3: Create the following design.

Page1.aspx

Page2.aspx

Step-4: Set the following properties:

Page1.aspx.

Control	Porperty	Value
HyperLink1	Text	Go to Page2
	NavigateUrl	~/Page2.aspx

Page2.aspx.

Control	Porperty	Value
HyperLink1	Text	Go to Page1
	NavigateUrl	~/Page1.aspx

Source will be as follows:

Page1.aspx

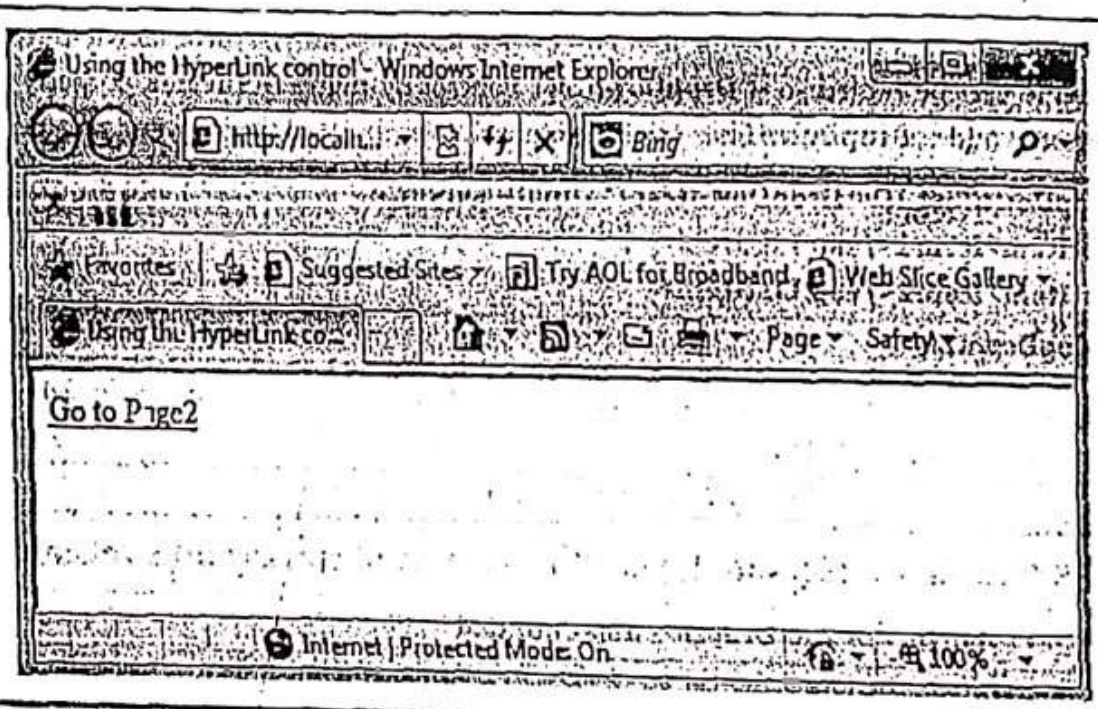
```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="Page1.aspx.cs" Inherits "Page1" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the HyperLink control</title>
    </head>

    <body>
        <form id="form1" runat="server">
            <div>
                <asp:HyperLink ID="HyperLink1"
                    runat="server"
                    NavigateUrl="~/Page2.aspx"> Go to Page2
                </asp:HyperLink>
            </div>
        </form>
    </body>
</html>
```

Page2.aspx

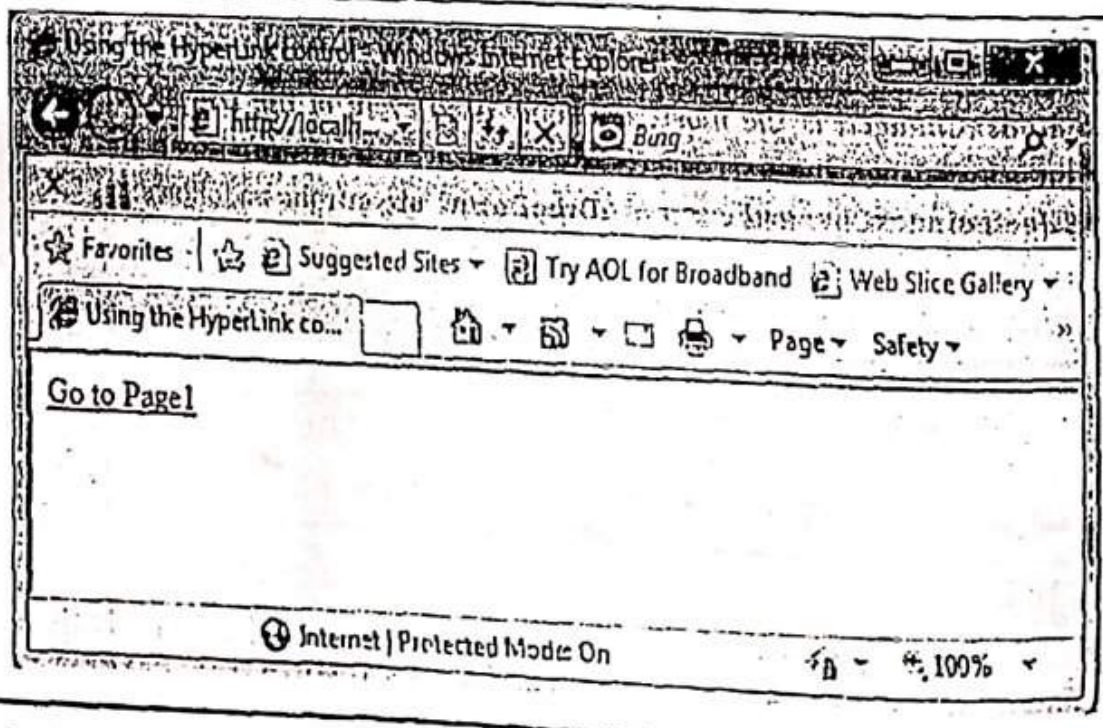
```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="Page2.aspx.cs" Inherits="Page2" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
    <head runat="server">
        <title>Using the HyperLink control</title>
    </head>
    <body>
        <form id="form1" runat="server">
            <div>
                <asp:HyperLink ID="HyperLink1"
                    runat="server"
                    NavigateUrl="~/Page1.aspx">Go to Page1
```

```
                    </asp:HyperLink>
                </div>
            </form>
        </body>
</html>
```

Step-5: Press F5. The output will be as follows:



When the user clicks on Hyperlink the output will be as follows:



Here's, the end of the **Program 4-14.**

4.8.2 LIST CONTROLS

The List controls enable us to display lists of items or collection. We can display a group of radio buttons, checkboxes, list items, bulleted items, links etc. There are number of controls which are used to represent collection of items such are as follows:

4.8.2.1 DropDownList Control

DropDownList control is a collection of items from which the users can select one item at a time, and the list of items remains hidden until a user clicks the drop-down button of the control. Items can be added to a **DropDownList** control by using the **Items** property of it. The basic syntax to add a **DropDownList** control to a web form using **source** code is as follows:

```
<asp:DropDownList ID="DropDownList1" runat="server">
<asp:ListItem>Item1</asp:ListItem>
<asp:ListItem>Item2</asp:ListItem>
</asp:DropDownList>
```

We can also add items to the **DropDownList** control dynamically through code. **For example,** in C#, this can be done as follows:

```
DropDownList1.Items.Add("Item");
```

Table 4-K lists some commonly used properties of the DropDownList control.

Table 4-K : Properties of the DropDownList control

Property	Description
Items	Used to add collection of items to the **DropDownList** control.
SelectedIndex	Used to get or set the index of the currently selected item.
SelectedValue	Used to get or set the value of the currently selected item.

SelectedIndexChanged is the most commonly used event of the **DropDownList** control. It raised on the server when the user selects a different item from **drop-down list.** The basic C# syntaxfor **SelectedIndexChanged** event of a**DropDownList**controlis as follows:

```
protectedvoidDropDownList1_SelectedIndexChanged(object sender, EventArgs e)//Event
Handler
{
    //Write Code Here
}
```

Note:To work with the items in a **drop-down list** or **list box,** we use the **Items** property of the control. This property returns a **ListItemCollection** object which contains all the items of the list.

Take a look at the following program.

Program 4-15

Description of the Program 4-15: This program demonstrates how to add items in a DropDownList control. Follow the steps given below.

Step-1: Create a new **Web Application** named "**ListControls**".

Step-2: Add a **Web Form** to the application named "**DropDownDemo.aspx**".

Step-3: Create the following design:

Step-4: Set the following properties of DropDownList1 control:

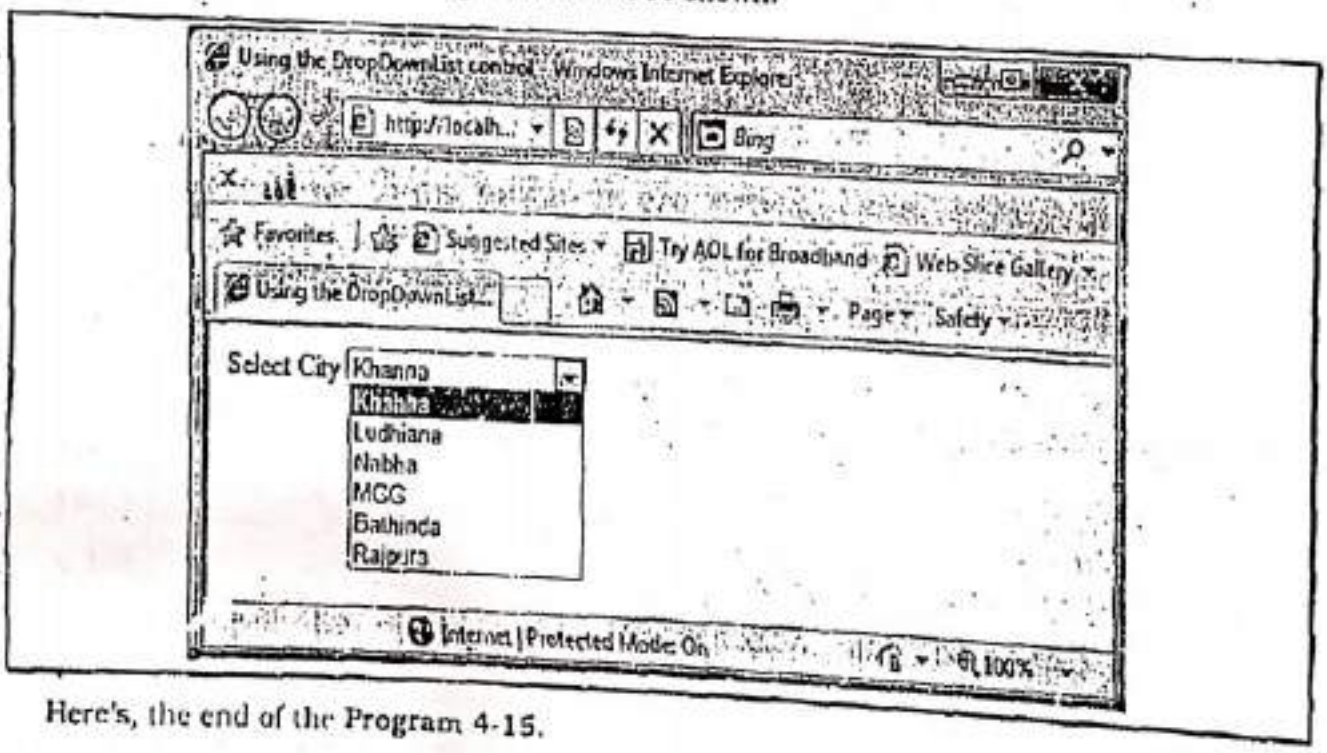
Control	Property	Value
DropDownList1	Items	(Collection) Khanna Ludhiana Nabha MGG Bathinda Rajpura

Source of the page will be as follows:

```
<%@ Page Langunge="C#" AutoEventWireup="true" CodeBehind="DropDownDemo.aspx.cs"
Inherits="DropDownDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using the DropDownList control</title>
</head>
<body>
<form id="form1" runat="server">
<div>
<table>
<tr>
<td>Select City</td>
<td>
<asp:DropDownList ID="DropDownList1"
                  runat="server"
                  Height="19px"
                  Width="140px">

                  <asp:ListItem>Khanna</asp:ListItem>
```

```
                <asp:ListItem>Ludhiana</asp:ListItem>
                <asp:ListItem>Nabha</asp:ListItem>
                <asp:ListItem>MGG</asp:ListItem>
                <asp:ListItem>Bathinda</asp:ListItem>
                <asp:ListItem>Rajpura</asp:ListItem>
        </asp:DropDownList>

</td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-4: Press **F5.** The following window will be shown:



Here's, the end of the Program 4-15.

4.8.2.2 RadioButtonList Control

A **RadioButtonList** control displays a collection of radio buttons on a Web page. These options are mutually exclusive. This control contains a collection of ListItem objects that could be referred to through the **Items** property of the control. The basic syntax to add a **RadioButtonList** control to a web form using source code is as follows:

```
<asp:RadioButtonList ID="RadioButtonList1" runat="server">
<asp:ListItem>option1</asp:ListItem>
<asp:ListItem>option2</asp:ListItem>
</asp:RadioButtonList>
```

We can also add items to the **RadioButtonList** control dynamically through code. For example, in C#, this can be done as follows:

```
RadioButtonList1.Items.Add("Item");
```

SelectedIndexChanged is the most commonly used event of the **RadioButtonList** control. It is raised on the server when the user selects a different item from **radio-buttons list**. The basic C# syntaxfor **SelectedIndexChanged** event of a **RadioButtonList** controlis as follows:

```
protected void RadioButtonList1_SelectedIndexChanged(object sender, EventArgs e) //Event Handler
{
    //Write Code Here
}
```

Take a look at the following program.

Program 4-16

Description of the Program 4-16: This program demonstrates how to add Items in a RadioButtonList control. Follow the steps given below.

Step-1: Create a new **Web Application** named "ListControls".

Step-2: Add a **Web Form** to the application named **"RadioButtonListDemo.aspx".**

Step-3: Create the following design:

Step-4: Set the following properties of RadioButtonList1 control:

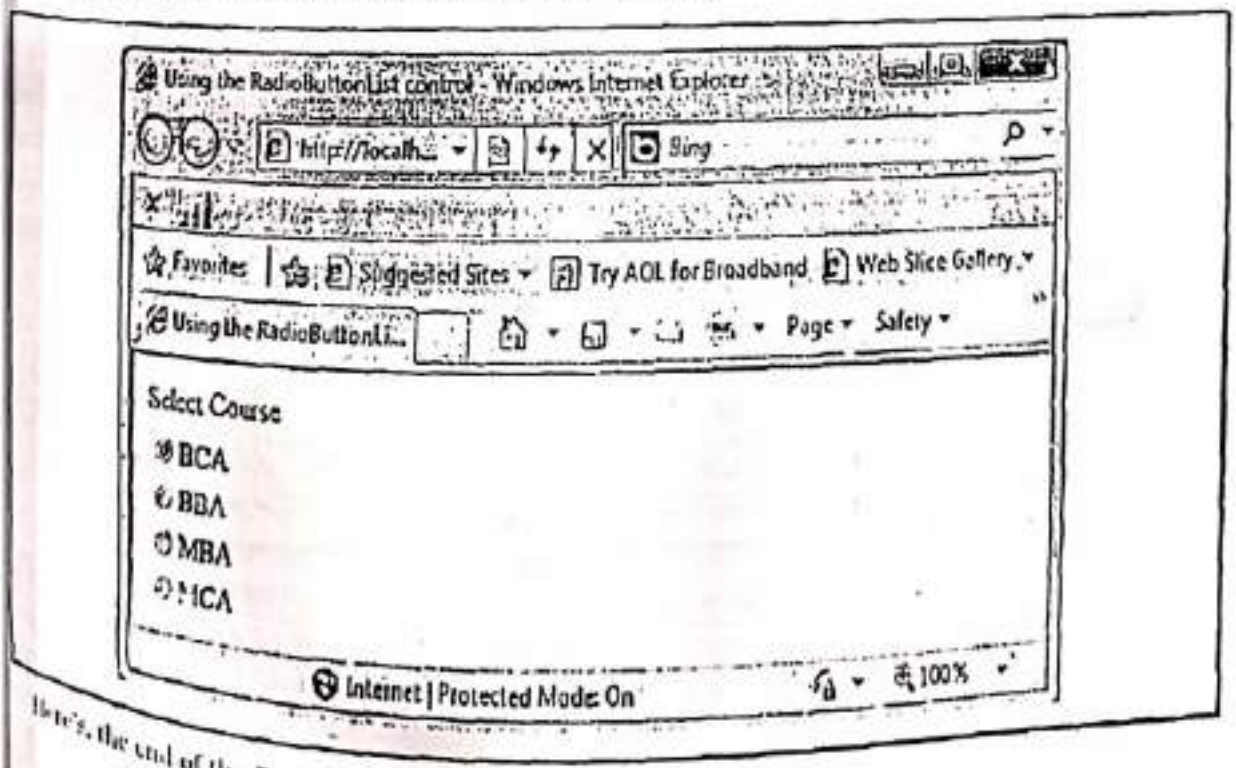
Control	Porperty	Value
RadioButtonList1	Items	(Collection) BCA BBA MBA MCA

Source of the page will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="RadioButtonListDemo.aspx.cs" Inherits="RadioButtonListDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
<title>Using the RadioButtonList control</title>
</head>
<body>
<form id="form1" runat="server">
<div>
<table>
<tr>
<td>Select Course</td></tr>
<tr><td>
        <asp:RadioButtonList ID="RadioButtonList1" runat="server">
            <asp:ListItem Selected="True">BCA</asp:ListItem>
            <asp:ListItem>BBA</asp:ListItem>
            <asp:ListItem>MBA</asp:ListItem>
            <asp:ListItem>MCA</asp:ListItem>
        </asp:RadioButtonList>
</td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-4: Press F5. The following window will be shown:

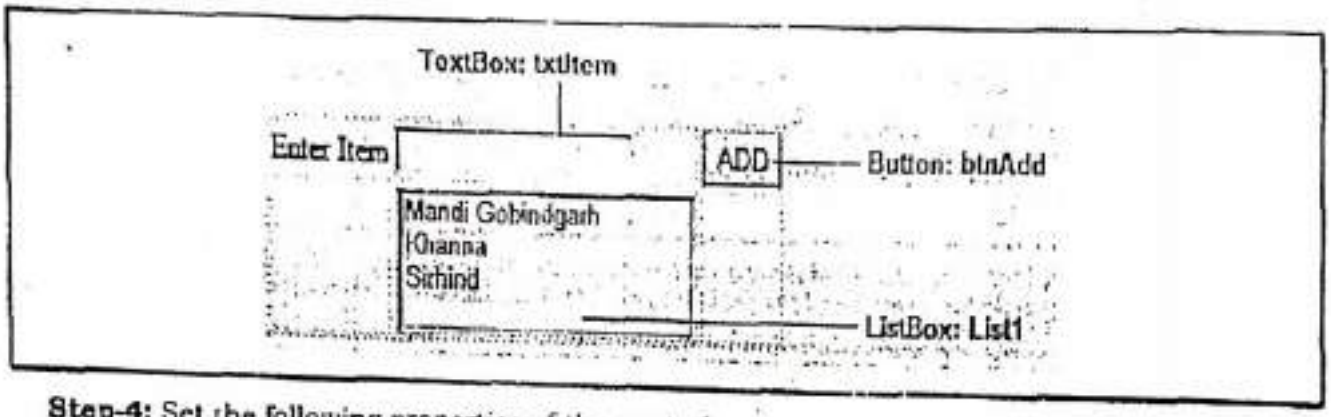


Hence, the end of the Program 4-16.

Step-1: Create a new **Web Application** named **"ListControls"**.

Step-2: Add a **Web Form** to the application named **"ListBoxDemo.aspx"**.

Step-3: Create the following design:



Step-4: Set the following properties of the controls:

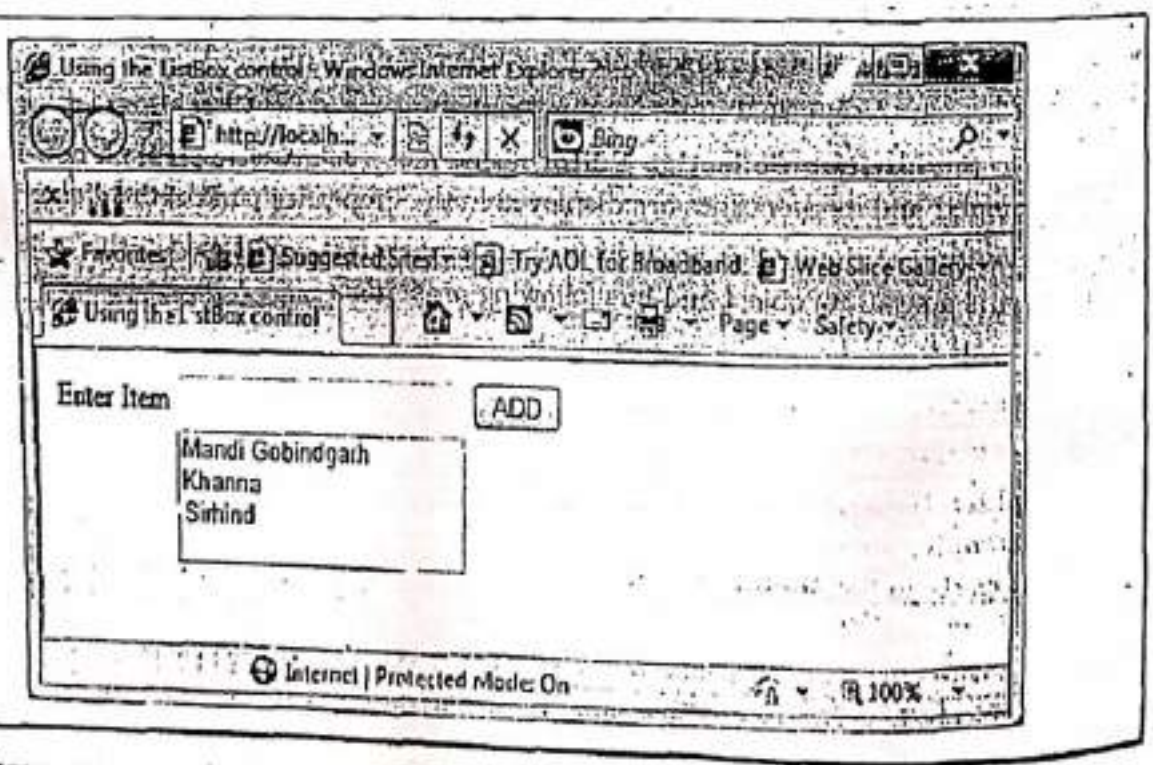
Control	Porperty	Value
btnAdd	Text	ADD
List1	Items	**(Collection)** Mandi Gobindgarh Khanna Sirhind

Source of the page will be as follows:

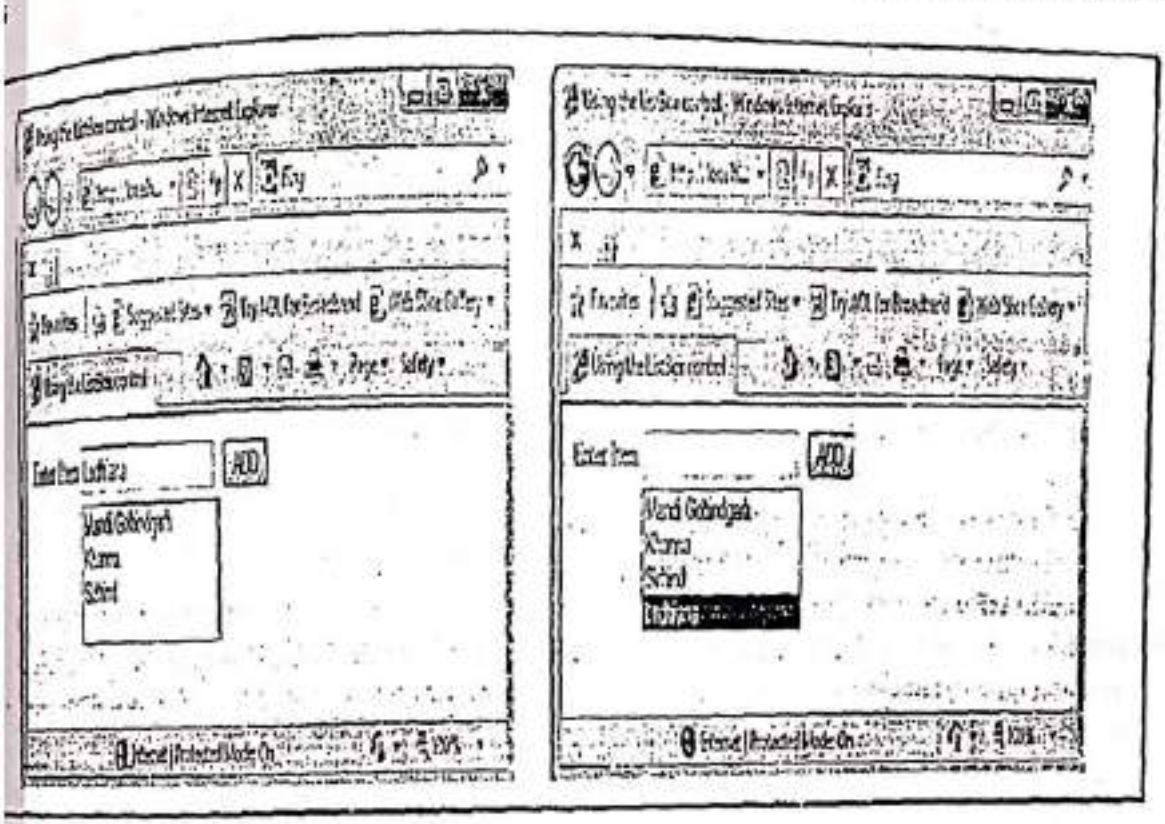
```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="ListBoxDemo.aspx.cs" Inherits="ListBoxDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Using the ListBox control</title>
</head>
<body>
<form id="form1" runat="server">
<div>
<table>
<tr>
<td>Enter Item</td>
<td>
<asp:TextBox ID="txtItem" runat="server"></asp:TextBox>
</td>
<td>
<asp:Button ID="btnAdd"
```

```
                    runat="server"
                    onclick="btnAdd_Click"
                    Text="ADD" />
</td>
</tr>
<tr>
<td> </td>
<td>
<asp:ListBox ID="List1" runat="server" Width="160px">
<asp:ListItem>Mandi Gobindgarh</asp:ListItem>
<asp:ListItem>Khanna</asp:ListItem>
<asp:ListItem>Sirhind</asp:ListItem>
</asp:ListBox>
</td>
<td> </td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-5: Press F5. The following window will be shown:



Suppose user enters "Ludhiana" in the textbox as a new item to be inserted into the list, then clicks on Add button. The output will be as follows:



Here's, the end of the Program 4-17.

3.2.4 CheckBoxList Control

A **CheckBoxList** displays a group of check boxes that all work together. This control ntains a collection of related items, and each item being a checkbox. These items need to be ecl, and to do so, follow the steps given below:

ep-1 :Open **Properties window of the CheckBoxList** control.

ep-2 :Click on the ellipsis button for the **Items** property of the **CheckBoxList** control.

ep-3 :In the **ListItem Collection Editor** dialog box, click **Add** to create a new item. A new em is created and its properties are displayed in the Properties pane of the dialog box.

ep-4 :Verify that the item is selected in the **Members** list, and then set the item properties. ch item is a separate object and has following properties:

- **Enabled:** Used to enable or disable the item in the list.
- **Selected:** A Boolean value that indicates whether or not the item is selected.
- **Text:** Used to set the text to be displayed for the item in the list.
- **Value:** Used to set the value associated with the item without displaying it. **For example,** you can set the **Text** property of an item as the **Student Name** and the **Value** property to the **Mobile Numb .r** of the student. Thus, you can keep the Text and Value properties different when you do not want the actual value to be displayed to the user.

e basic syntax to add a **CheckBoxList** control to a web form using source code is as follows:

```
sp:CheckBoxList ID="CheckBoxList1" runat="server">
asp:ListItem Value="Value1">Item1</asp:ListItem>
asp:ListItem Value="Value2">Item2</asp:ListItem>
/asp:CheckBoxList>
```

We can also add items to the **CheckBoxList** control dynamically through code. **For example,** in C#, this can be done as follows:

```
CheckBoxList1.Items.Add("Item");
```

SelectedIndexChanged is the most commonly used event of the **CheckBoxList** control. It is raised on the server when the user selects a different item from **check-box list.** The basic C# syntaxfor **SelectedIndexChanged** event of a **CheckBoxList** controlis as follows:

```
protected void CheckBoxList1_SelectedIndexChanged(object sender, EventArgs e)  //Event Handler
{
      //Write Code Here
}
```

Program 4-18

Description of the Program 4-18: This program demonstrates how to use CheckBoxList control. Follow the steps given below.

Step-1: Create a new **Web Application** named **"ListControls".**

Step-2: Add a **Web Form** to the application named **"CheckBoxListDemo.aspx".**

Step-3: Create the following design:

Step-4. Set the following properties of the CheckBoxList1 control.

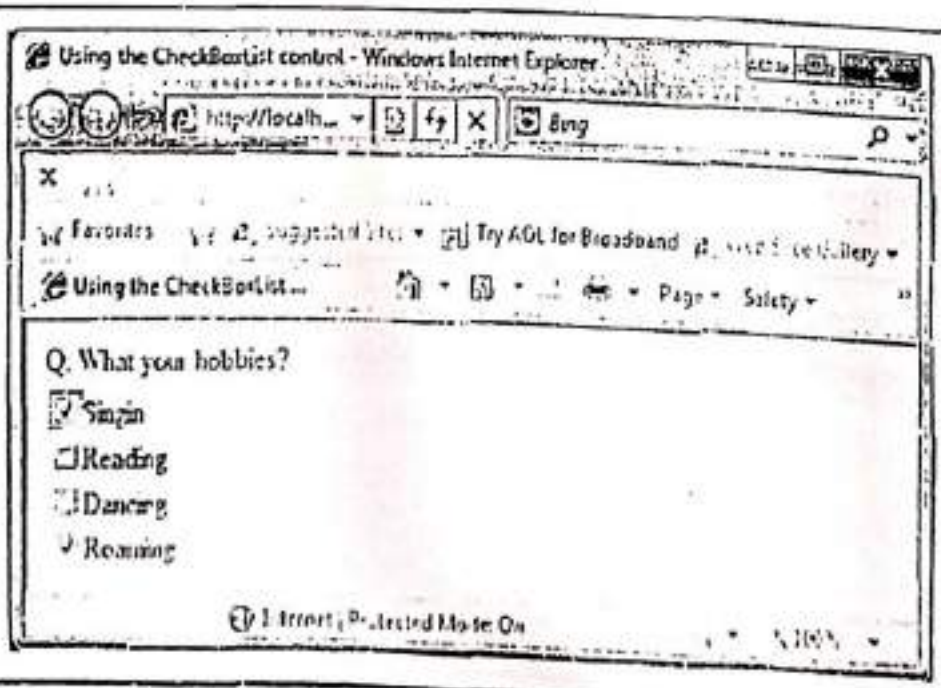
Control	Porperty	Value
CheckBoxList1	Items	(Collection) Singing Reading Dancing Roaming

Source of the page will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="CheckBoxListDemo.aspx.cs"
Inherits="CheckBoxListDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
```

```
<head runat="server">
<title>Using the CheckBoxList control</title>
</head>
<body>
<form id="form1" runat="server">
<div>

<table>
<tr>
<td> Q: What your hobbies?</td>
</tr>
<tr>
<td>
<asp:CheckBoxList ID="CheckBoxList1" runat="server">
<asp:ListItem>Singing</asp:ListItem>
<asp:ListItem>Reading</asp:ListItem>
<asp:ListItem>Dancing</asp:ListItem>
<asp:ListItem>Roaming</asp:ListItem>
</asp:CheckBoxList>
</td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-5: Press **F5**. The following window will be shown:



Here's, the end of the **Program 4-18.**

4.8.2.5 BulletedList Control

The BulletedList control renders either an unordered (bulleted) or ordered (numbered)list. Each list item can be rendered as plain text, a LinkButton control, or a link toanother web page.

We can control the appearance of the bullets that appear for each list item with theBulletStyle property. This property accepts the following values:

- Circle
- CustomImage
- Disc
- LowerAlpha
- LowerRoman
- NotSet
- Numbered
- Square
- UpperAlpha
- UpperRoman

We can set **BulletStyle** to **Numbered** to display a numbered list. If we set this propertyto the value **CustomImage** and assign an image path to the **BulletImageUrl** property, then we can associate an image with each list item.

We can modify the appearance of each list item by modifying the value of theDisplayMode property. This property accepts one of the following values from the**BulletedListDisplayMode** enumeration:

- **HyperLink:** Each list item is rendered as a link to another page.
- **LinkButton:** Each list item is rendered by a LinkButton control.
- **Text:** Each list item is rendered as plain text.

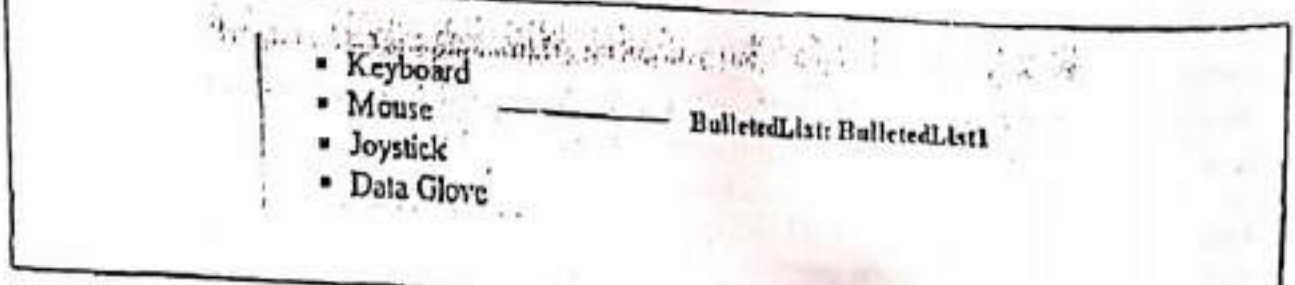
Program 4-19

Description of the Program 4-19: This program demonstrates how to use BulletedList control. Follow the steps given below.

Step-1: Create a new **Web Application** named "**ListControls**".

Step-2: Add a **Web Form** to the application named "**BulletedListDemo.aspx**".

Step-3: Create the following design:



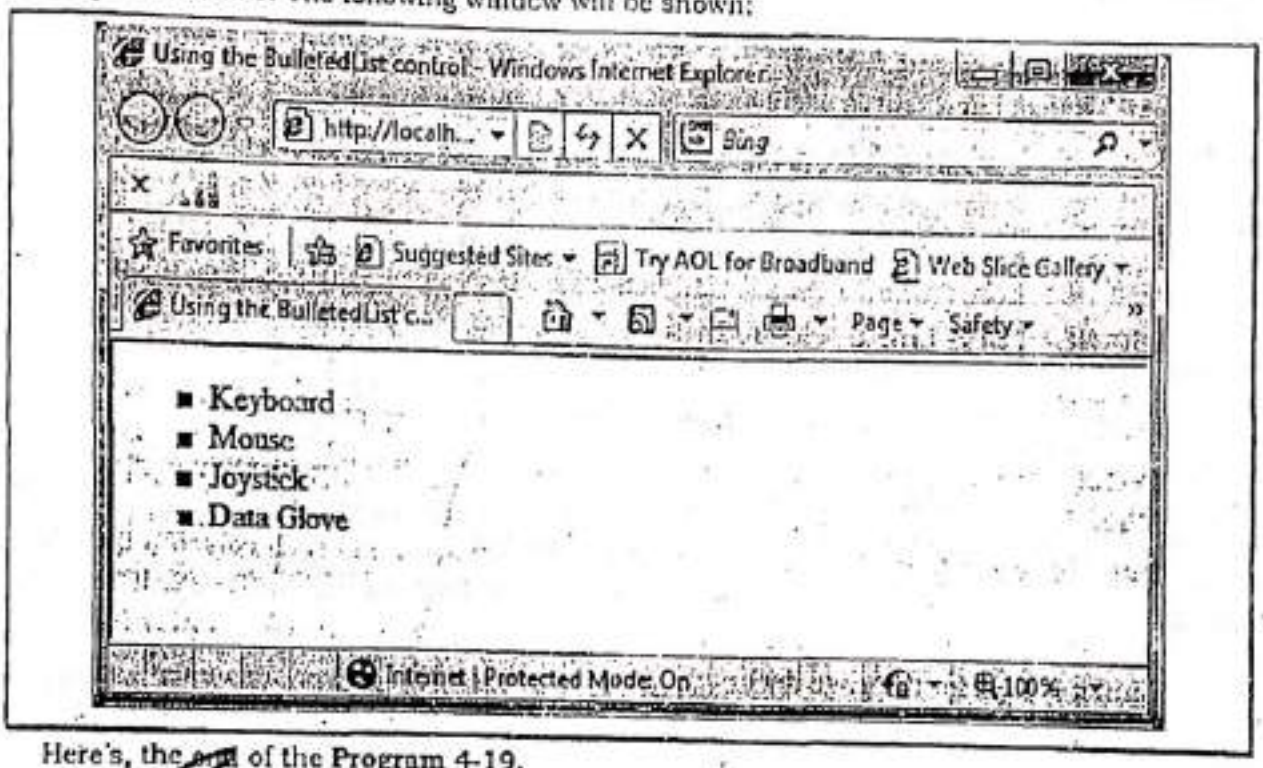
Step-4: Set the following properties of the BulletedList1 control.

Control	Porperty	Value
BulletedList1	Items	(Collection) Singing Reading Dancing Roaming
	BulletStyle	Square

Source of the page will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="BulletedListDemo.aspx.cs"
Inherits="BulletedListDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using the BulletedList control</title>
</head>
<body>
<form id="form1" runat="server">
<div>

<asp:BulletedList ID="BulletedList1" runat="server" BulletStyle="Square">
<asp:ListItem>Keyboard</asp:ListItem>
<asp:ListItem>Mouse</asp:ListItem>
<asp:ListItem>Joystick</asp:ListItem>
<asp:ListItem>Data Glove</asp:ListItem>
</asp:BulletedList>

</div>
</form>
</body>
</html>
```

Step-5: Press **F5**. The following window will be shown:



Here's, the end of the Program 4-19.

4.9 WORKING WITH EVENTS IN ASP.NET

Event is an action, initiated either by the user or the computer. An event generates when a user interacts with the web form elements. Some of the activities that cause event to be generated are pressing a button, entering a character via keyboard, selecting an item in a list, and clicking the mouse. Events may also occur that are not directly caused by interactions with a user interface but they are generated internally. For example, an event may be generated when a timer expires, a counter exceeds a value, software or hardware failure occurs, running out of memory, an interrupt occurs.

In ASP.Net when user interacts with the various web controls on a page, events are raised. In client-based web applications, the events are raised and handled on the client side. However in web forms applications, the events are raised either on the client or on the server, but are always handled on the server.

The majority of the ASP.NET controls support one or more events. For example, the ASP.NET **Button** control supports the **Click** event. The **Click** event is raised on the server when the **Button** control rendered on a web form is clicked by the user.

When an event is raised, it needs to be handled for processing. The server has a procedure that is executed when an event occurs. This procedure is called as **event handler**. It describes what to do when the event is raised. Therefore, when the event message is transmitted to the server for **Click** event, it checks whether the **Click** event has an associated **event handler**, and if it has, the event handler is executed.

ASP.Net event handlers generally take two parameters and return void. The first parameter represents the object raising the event and the second parameter is called the **event argument**. The general syntax of an event is as follows:

```
protected void EventName(object sender, EventArgs e)
{
    //Write Code Here
}
```

4.9.1 ASP.NET Page Life Cycle Events

When a page request is sent to the web server, the page is run through a series of events during its creation and disposal. These events are explained as follows:

1. **PreInit:** The entry point of the page life cycle is the pre-initialization phase called **"PreInit"**.
2. **Init:** This event fires after each control has been initialized and each control's **UniqueID** is set. We can use this event to change initial values for the controls. The **"Init"** event is fired first for the most bottom control in the hierarchy, and then fired up the hierarchy until it is fired for the page itself.
3. **InitComplete:** This event is raised after all initializations of the page and its controls have been completed. But, The **viewstate** values are not yet loaded, so we can use this event to make changes to view state that we want to make sure are persisted after the **next postback.**
4. **PreLoad:** This event is raised after the page loads **view state** for itself and all controls, and after it processes **postback data** that is included with the **Request** instance.

   Note: During this phase of the page creation, form data that was posted to the server (termed postback data in ASP.NET) is processed against each control that requires it. Hence, the page fires the **LoadPostData** event and parses through the page to find each control and updates the control state with the correct postback data. ASP.NET updates the correct control by matching the control's unique **ID** with the name/value pair in the **NameValueCollection**. This is one reason that ASP.NET requires unique IDs for each control on any given page.
5. **Load:** Code inside the page load event typically checks for **PostBack** and then sets control properties appropriately. This is the first place in the page lifecycle where all values are restored. Most probably the value of **IsPostBack** is checked in this event to avoid unnecessarily resetting state.
6. **Control (PostBack) event(s):** ASP.NET now calls any events on the page or its controls that caused the PostBack to occur. This might be a button's click event or a dropdown's selectedindexchange event. These are the events, for which the code is written in our code-behind class(.cs file).
7. **LoadComplete:** This event signals the end of Load.
8. **PreRender** Allows final changes to the page or its control. This event takes place after all regular PostBack events have taken place. This event takes place before saving

ViewState, so any changes made here are saved. **For example,** after this event, you cannot change any property of a button or change any viewstate value. Because, after this event, SaveStateComplete and Render events are called.

9. **SaveStateComplete** Prior to this event the view state for the page and its controls is set. Any changes to the page's controls at this point or beyond are ignored.
10. **Render** This is a method of the page object and its controls (and not an event). At this point, ASP.NET calls this method on each of the page's controls to get its output. The Render method generates the client-side HTML, Dynamic Hypertext Markup Language (DHTML), and script that are necessary to properly display a control at the browser.
11. **UnLoad** This event is used for cleanup code. It raised when the page is unloaded from memory. During this event, you should destroy any objects or references you have created in building the page including the Page object. Cleanup can be performed on:
    (a) Instances of classes i.e. objects
    (b) Opened files
    (c) Opened database connections.

These events can be demonstrated by the **Program 4-20.**

Program 4-20

Description of the Program 4-20: In this program, we are going to demonstrate the page life cycle events. For this, various events of the page are overridden in the program. To do so, follow the steps given below.

Step-1: Create a new **Web Application** named " **PageLifeCycle**".

Step-2: Add a **Web Form** to the application named **"LifeCycleEvents.aspx".**

Step-3: Place a **Button** on the page and set its **Text** property as **"PostBack".**

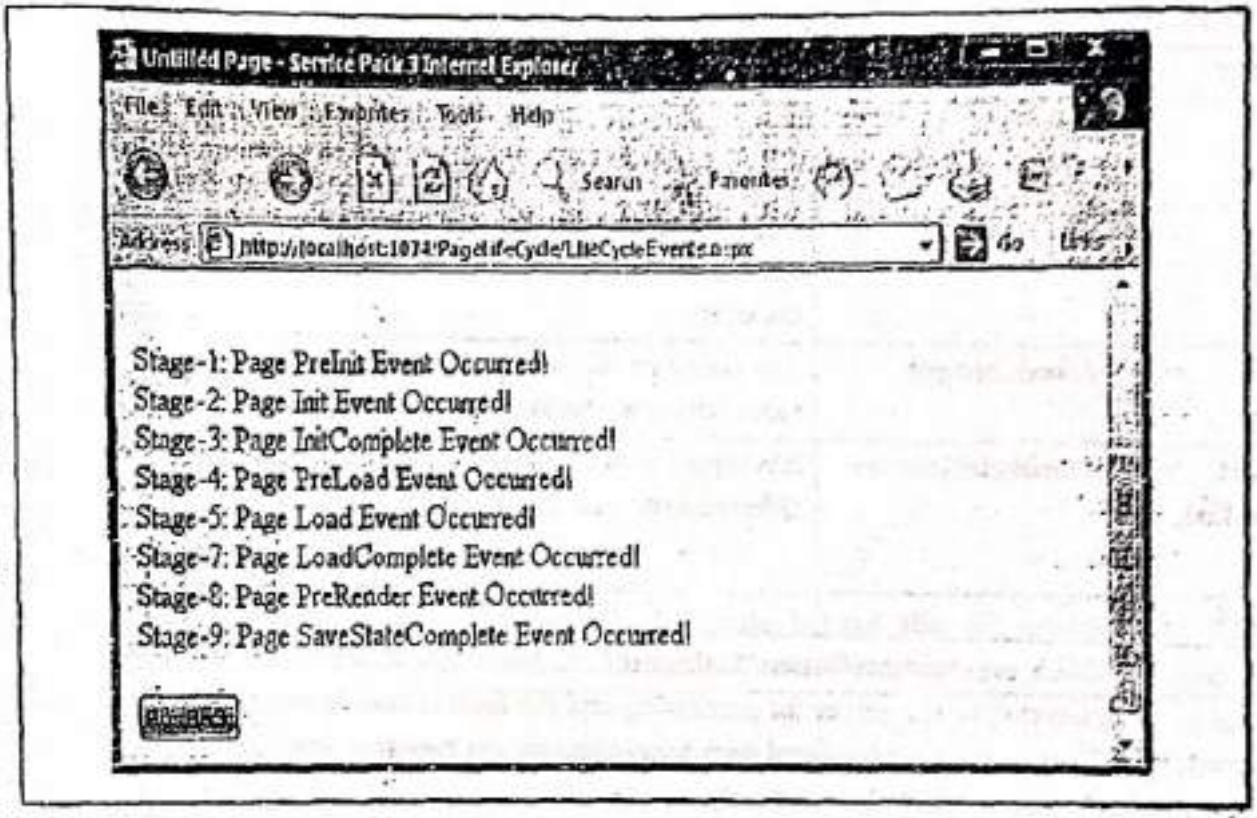
Button1

Step-4: Add the following code in **C#.NET code behind file.**

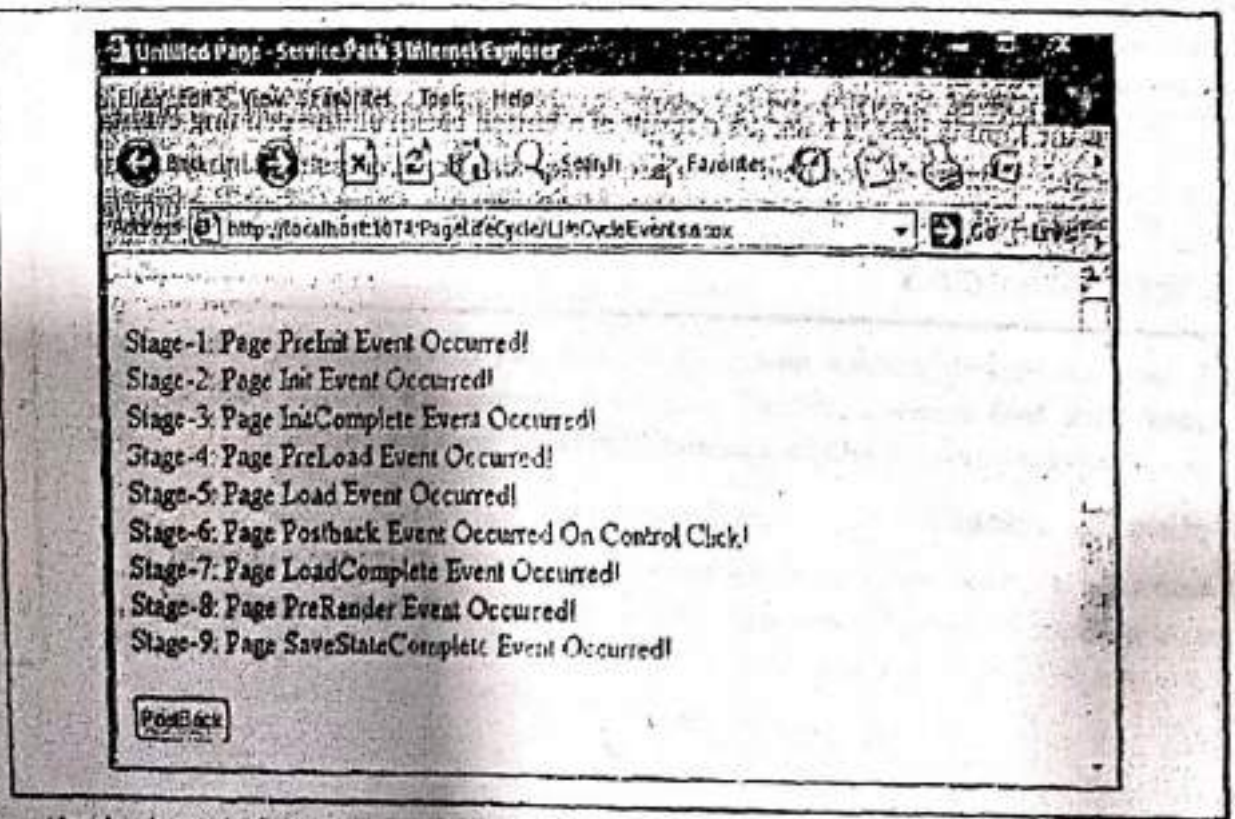
```
using System;          PostBack
public partial class LifeCycleEvents : System.Web.UI.Page
{
    protected void Page_PreInit(object sender, EventArgs e)
    {
        Response.Write("<br>Stage-1: Page PreInit Event  Occurred!");
    }
    protected void Page_Init(object sender, EventArgs e)
    {
```

```
        Response.Write("<br>Stage-2: Page Init Event Occurred!");
    }
    protected void Page_InitComplete(object sender, EventArgs e)
    {
        Response.Write("<br>Stage-3: Page InitComplete Event Occurred!");
    }
    protected override void OnPreLoad(EventArgs e)
    {
        Response.Write("<br>Stage-4: Page PreLoad Event Occurred!");
    }
    protected void Page_Load(object sender, EventArgs e)
    {
        Response.Write("<br>Stage-5: Page Load Event Occurred!");
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        Response.Write("<br>Stage-6: Page Postback Event Occurred On Control Click!");
    }
    protected void Page_LoadComplete(object sender, EventArgs e)
    {
        Response.Write("<br>Stage-7: Page LoadComplete Event Occurred!");
    }
    protected override void OnPreRender(EventArgs e)
    {
        Response.Write("<br>Stage-8: Page PreRender Event Occurred!");
    }
    protected override void OnSaveStateComplete(EventArgs e)
    {
        Response.Write("<br>Stage-9: Page SaveStateComplete Event Occurred!");
    }

/*
    protected void Page_UnLoad(object sender, EventArgs e)
    {
        // This event occurs for each control and then for the page. In controls, use this
        event to do final cleanup for specific controls, such as closing control-specific
        database connections.
        // During the unload stage, the page and its controls have been rendered, so you
        cannot make further changes to the response stream.
        //If you attempt to call a method such as the Response.Write method, the page
        will throw an exception.
        Response.Write("<br>Stage-10: Page Unload Event Occurred!");    //Try this
    }
*/
```

Step-5: To run the application, press F5. First, the following window will appear.



See, Stage-6 is not there i.e. the PostBack event not yet occurred. Now click on the button, see the PostBack event fires and the output will be shown as follows:



Here's, the end of the **Program 4-20.**

4.9.2 Handling Control Events

All ASP.Net controls have one or more events which are fired when user performs certain action on them. To handle these events, in-built attributes and event handlers are there. To respond to an event, the event handler is coded. **Table 4-M** describes the most probably used events associated with different ASP.NET web controls.

Table 4-M : Events associated with ASP.NET controls

Control(s)	Event	Description
TextBox	TextChanged	This event is raised on the server when the contents of the text box are changed.
Button, LinkButton, ImageButton	Click	This event is raised when the Button, LinkButton or ImageButton control is clicked by the user.
CheckBox, RadioButton	CheckedChanged	It is raised on the server when the check box or radio button is checked or unchecked.
CheckBoxList, RadioButtonList, ListBox, DropDownList	SelectedIndexChanged	It is raised on the server when the user selects a different item from any of the list controls.

By default, only the **Click** event of the Button, LinkButton, and ImageButton web controls causes the form to be submitted to the server for processing and the form is said to be posted back to the server. The **Change** events associated with other controls are captured and cached and do not cause the form to be submitted immediately.

When the form is posted back (as a result of a button click), all the pending events are raised and processed. But there is no particular sequence exists for processing **Change** events, such as TextChanged and CheckChanged on the server. The **Click** event is processed only after all the **Change** events are processed.

Note: We can set the change events of server controls to result in the form post back to the server. To do so, modify the **AutoPostBack** property of the control to **True**.

4.10 RICH WEB CONTROLS

In ASP.NET, some of the web controls have more complex and rich functionality. These controls are called **Rich Web** controls. ASP.NET includes numerous rich controls such as **AdRotator** control, **Calendar** control, FileUpload control, **MultiView** andView Control.

4.10.1 Accepting file uploads

The **FileUpload** control enables users to upload files to the web server. Once the web server receives the posted file data, it's up to the web application to examine it, ignore it, or save it to a back-end database or a file on the web server.

The basic syntax to add a **FileUpload** control to a web form using source code is as follows:

```
<asp:FileUpload ID="FileUpload1" runat="server" '>
```

Table 4-N lists some commonly used properties of the FileUpload control.

Table 4-N : Properties of the Calendar control

Property	Description
Enabled	Used to enable or disable the **FileUpload** control.
FileBytes	Used to get the uploaded file contents as a byte array.
FileContent	Used to get the uploaded file contents as a stream.
FileName	Used to get the name of the file uploaded.
HasFile	Returns True when a file has been uploaded.
PostedFile	Used to get the uploaded file wrapped in the **HttpPostedFile** object.

The **FileUpload** control also supports **SaveAs()** method to save the uploaded file to the file system. The use of **FileUpload** control can be demonstrated by **Program 4-21**

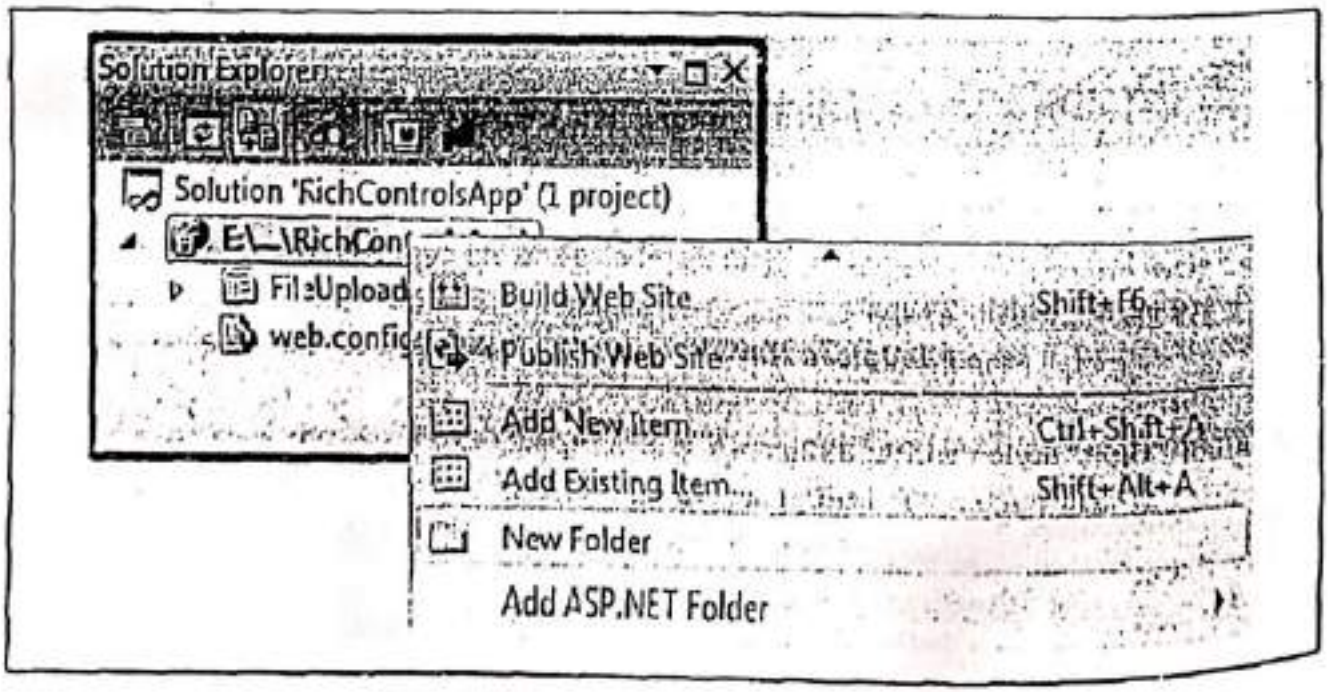
Program 4-21

Description of the Program 4-21: In this program, a file is selected using FileUpload control. On a button click selected file is to be uploaded into the web application's directory named UploadedFiles. Follow the steps given below.

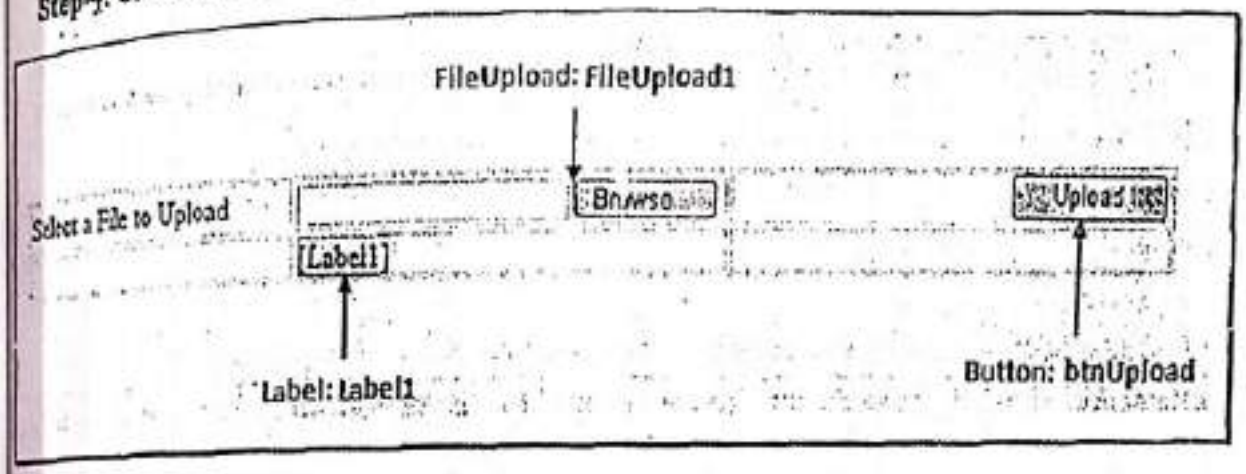
Step-1: Create a new **Web Application** named **"RichControlsApp"**.

Step-2: Add a **Web Form** to the application named **"FileUploadControl.aspx"**.

Step-3: Add a folder to the application named **"UploadedFiles"** where all the images will be uploded.



Step-4: Create the following design.



Source will be as follows:

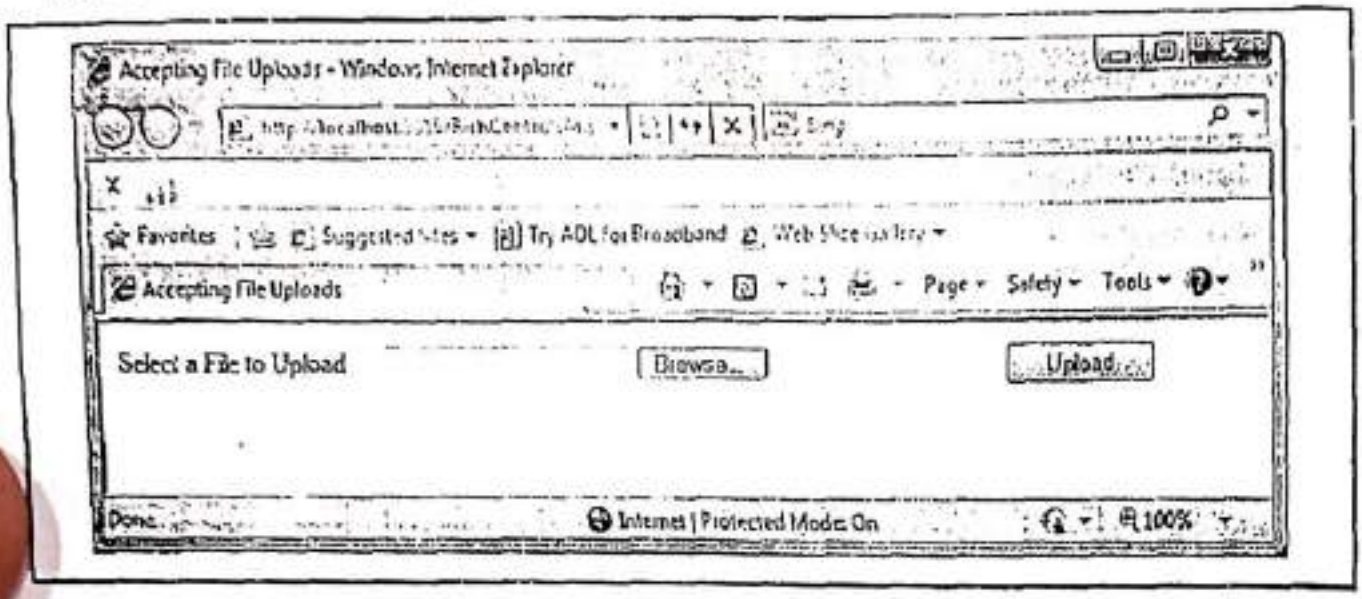
```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="FileUploadControl.aspx.cs" Inherits="FileUploadControl" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
<title>Accepting File Uploads</title>
</head>
<body style="width: 636px">
<form id="form1" runat="server">
<div>
<table>
<tr>

<td style="width: 300px">Select a File to Upload</td>

    <td style="width: 242px">
            <asp:FileUpload ID="FileUpload1" runat="server" /></td>

<td align="right" style="width: 392px">
        <asp:Button ID="btnUpload" runat="server"
                    OnClick="btnUpload_Click" Text="Upload"
</tr>          Width="90px" /></td>
```

```
<tr>
<td style="width: 300px">
</td>
<td style="width: 242px">
<asp:Label ID="Label1" runat="server"></asp:Label></td>
<td style="width: 392px">
</td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-5: Double click on button **btnUpload**. The **code behind file** will appear. Add the following code in **C# code behind file** named "**FileUploadControl.aspx.cs**".

```
using System;
public partial class FileUploadControl : System.Web.UI.Page
{
    protected void btnUpload_Click(object sender, EventArgs e)
    {
        if (FileUpload1.HasFile)
        {
          //UploadFiles : Name of the directory in which the files to be uploaded.
          //Mapping to the UploadedFiles directory
           string s = Server.MapPath("~/UploadedFiles");

          //Setting location and name of the file.
            s = s + "\\" + FileUpload1.FileName;

          //Saving a file or uploading a file.
            FileUpload1.SaveAs(s);
            Label1.Text = "Uploaded Successfully!";
        }
        else
        {
            Label1.Text = "No File Selected!";
        }
    }
}
```

Step-6: Press F5. The following window will appear.



Click on **Browse** to select a file that you want to upload then click on **Upload** button. The file will be uploaded to the web application. Here's, the end of the **Program 4-21.**

4.10.2 Displaying a Calendar

A calendar can be displayed on a web page using a **Calendar Control.** Calendar control creates a functionally rich and good-looking calendar box that shows one month at a time. The user can move from month to month, select a date, and even select a range of days (if multiple selections are allowed). The basic syntax to add a **Calendar** control to a web form using **source code** is as follows:

```
<asp:Calendar ID="Calendar1" runat="server"></asp:Calendar>
```

The Calendar control has many properties that, taken together, allow you to change almost every part of this control. For example, you can fine-tune the foreground and background colors, the font, the title, the format of the date, the currently selected date, and so on.

Table 4-0 lists some commonly used properties of the Calendar control.

Table 4-0 : Properties of the Calendar control	
Property	**Description**
CellPadding	Specifies the space between cells.
CellSpacing	Specifies the space between the contents of a cell and the cell's border.
DayNameFormat	Used to specify the appearance of the days of the week. Possible values are **FirstLetter, FirstTwoLetters, Full, Short,** and **Shortest.**
FirstDayOfWeek	Used to set a value for the day of the week that will be displayed in the calendar's first column.
ShowNextPrevMonth	Takes a Boolean value and specifies whether or not the calendar is capable of displaying next and previous month links.
NextMonthText	Shows the HTML text for the 'Next Month' navigation link if the **ShowNextPrevMonth** property is set to **true.**

NextPrevFormat	Used to specify the format of the next month and previous month links. Possible values are **CustomText, FullMonth,** and **ShortMonth.**
PrevMonthText	Used to specify the text that appears for the previous month link.
SelectedDate	Used to get or set the selected date.
SelectedDates	Allows us to get or set a collection of selected dates.
SelectionMode	Used to specify how dates are selected. Possible values are **Day, DayWeek, DayWeekMonth,** and **None.**
SelectMonthText	Allows us to specify the text that appears for selecting a month.
SelectWeekText	Allows us to specify the text that appears for selecting a week.
ShowDayHeader	Allows us to hide or display the day names at the top of the Calendar control.
TodaysDate	Used to specify the current date. This property defaults to the current date on the server.
ShowTitle	Enables to hide or display the title bar displayed at the top of the calendar.
TitleFormat	Used to format the title bar. Possible values are **Month** and **MonthYear.**
VisibleDate	Used to specify the month displayed by the Calendar control. This property defaults to displaying the month that contains the date specified by **TodaysDate.**

The Calendar also provides events that enable you to react when the user changes the current month **(VisibleMonthChanged)**, when the user selects a date **(SelectionChanged)**, and when the Calendar is about to render a day **(DayRender)**.

The most important Calendar event is **SelectionChanged**, which fires every time a user selects a date. Here's a basic event handler that responds to the **SelectionChanged** event and displays the selected date is as follows:

```
protected void Calendar1_SelectionChanged(object sender, EventArgs e)
{
    Response.Write("Selected Date Is:"+Calendar1.SelectedDate.ToLongDateString());
}
```

Note: Every user interaction with the calendar triggers a **postback**. This allows you to react to the selectionevent immediately, and it allows the Calendar to rerender its interface, thereby showing a new month or newlyselected dates. The Calendar does not use the **AutoPostBack** property.

Take a look at the following program.

Program 4-22

Description of the Program 4-22: In this program, a Calendar control will be displayed on a web page. When a user selects a date from the calendar the selected date will be displayed on the web page. Follow the steps given below.

Step-1: Create a new **Web Application** named "**RichControlsApp**".

Step-2: Add a **Web Form** to the application named "**CalendarControl.aspx**".

Step-3: Add a **calendar control** on the web form.

Calendar: Calendar1

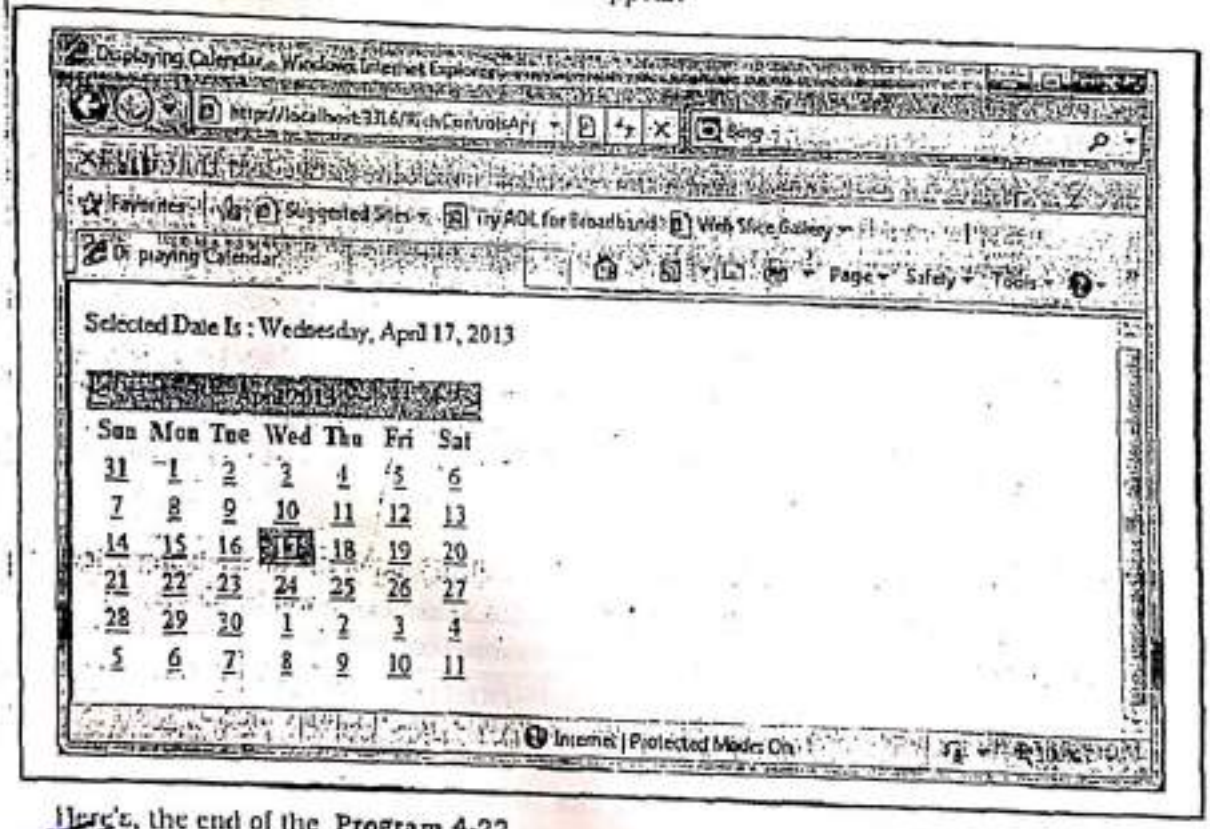
Source of this is as follows:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="CalendarControl.aspx.cs"
Inherits="CalendarControl" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
<title>Displaying Calendar</title>
</head>
<body>
<form id="form1" runat="server">
<div>

<asp:Calendar ID="Calendar1" runat="server"
        OnSelectionChanged="Calendar1_SelectionChanged">
</asp:Calendar>

</div>
</form>
</body>
</html>
```

Step-4: Double click on button **Calendar1**. The **code behind file** will appear. Add the following code in **C# code behind file** named "**CalendarControl.aspx.cs**".

```
using System;
public partial class CalendarControl : System.Web.UI.Page
{
    protected void Calendar1_SelectionChanged(object sender, EventArgs e)
    {
        Response.Write("Selected Date Is : "+Calendar1.SelectedDate.ToLongDateString());
    }
}
```

Step-5: Press **F5**. The following window will appear.



Here's, the end of the **Program 4-22.**

4.10.3 Displaying Advertisement

The **AdRotator** control is used to display a series of advertisements to the end users on web pages. These advertisements includes flashing ads, such as banner ads and news flashes on Web pages. The control is capable of displaying ads randomly, because the control refreshes the display every time the web page is refreshed. So, it is possible to display different ads for different users. We can also assign priorities to the ads so that certain ads are displayed more frequently than others.

With this control, we can use advertisement data from an **XML** (Extensible Markup Language) file or from other sources. If we are using an XML source for the ad information, then first we have to create an **XML** advertisement file. For example, a banner ad that displays one

out of a set of images based on a predefined scheduled that's saved in an XML file. The basic syntax to create an XML advertisement file is as follows:

```
<Advertisements>
<Ad>
    <ImageUrl>
        Absolute or relative URL of the image to be displayed on the web page.
    </ImageUrl>

    <NavigateUrl>
        URL of the page to navigate to, if a user clicks the advertisement image. This is
        an optional parameter.
    </NavigateUrl>

    <AlternateText>
        This is an optional parameter that specifies some alternative text that will be
        displayed if the image specified in the ImageUrl parameter is not accessible. In
        some browsers, the AlternateText parameter appears as a ToolTip for the ads.
    </AlternateText>

    <Keyword>
        This is an optional parameter that specifies categories of ads, such as computers,
        books, and magazines that can be used to filter for specific ads.
    </Keyword>

    <Impressions>
        Takes a numerical value that indicates the likelihood of the image being selected
        for display. The larger the number, the more often the ad will be displayed.
    </Impressions>
</Ad>
<!-- More ads can go here. -->
</Advertisements>
```

The basic syntax to add an AdRotator control to a web form using source code is as follows:

```
<asp:AdRotatorID="AdRotator1"runat="server"AdvertisementFile="~/XMLFile.xml"Height
    ="200px"OnAdCreated="AdRotator1_AdCreated"Width="300px"/>
```

Table 4-P lists some commonly used properties of the AdRotator control.

Table 4-P : Properties of the AdRotator control

Property	Description
AdvertisementFile	The AdvertisementFile property represents the path to an Advertisement file. The Advertisement file is a well-formed XML document that contains information about the image to be displayed for advertisement and the page to which a user is redirected when the user clicks the banner or image.

DataMember	Used to bind to a particular data member in the data source.
DataSource	Used to specify a data source programmatically for the list of banner advertisements.
DataSourceID	Used to bind to a data source declaratively.
Height	Specifies the height of the ads.
ImageUrlField	Used to specify the name of the field for the image URL for the banner advertisement. The default value for .his field is ImageUrl.
KeywordFilter	The KeywordFilter property specifies a category filter to be passed to the source of the advertisement. A keyword filter allows the AdRotator control to display ads that match a given keyword.
NavigateUrlField	Enables to specify the name of the field for the advertisement link. The default value for this field is NavigateUrl.
Target	The Target property specifies the name of the browser window or frame in which the advertisement needs to be displayed.
Width	Specifies the width of the ads.

The use of **AdRotator** control can be demonstrated by **Program 4-23**

Program 4-23

Description of the Program 4-23: In this program, an XML advertisement file is created which contains three ads for different websites. An AdRotator control is used to display ads. Different ad may be displayed every time the page is refreshed. To do so, follow the steps given below.

Step-1: Create a new **Web Application** named **"RichControlsApp"**.

Step-2: Add an **XML File** to the application named **"XMLFile.xml"** and write the following advertisement code into it.

```
<?xml version="1.0" encoding="utf-8" ?>
<Advertisements>
<Ad>
        <ImageUrl>Google.gif</ImageUrl>
        <NavigateUrl>http://www.Google.com</NavigateUrl>
        <AlternateText>Google Image Not Available</AlternateText>
        <Impressions>20</Impressions>
        <Keyword>Google</Keyword>
</Ad>
<Ad>
        <ImageUrl>Yahoo.gif</ImageUrl>
        <NavigateUrl>http://www.Yahoo.com</NavigateUrl>
        <AlternateText>Yahoo Image Not Available</AlternateText>
```

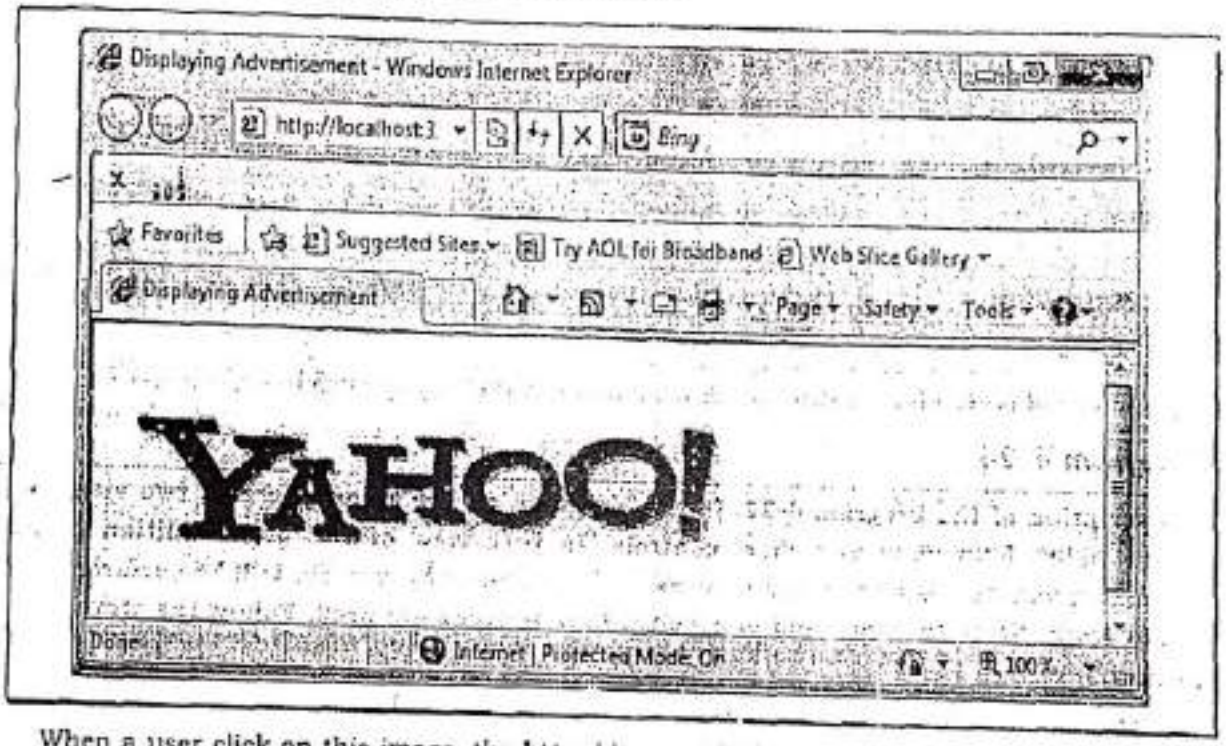
```
        <Impressions>20</Impressions>
        <Keyword>Yahoo</Keyword>
    </Ad>
    <Ad>
        <ImageUrl>Facebook.gif</ImageUrl>
        <NavigateUrl>http://www.facebook.com</NavigateUrl>
        <AlternateText>Facebook Image Not Available</AlternateText>
        <Impressions>20</Impressions>
        <Keyword>Facebook</Keyword>
    </Ad>
    <!-- More ads can go here. -->
    </Advertisements>
```

Step-3: Add a **Web Form** to the application named **"AdRotatorControl.aspx"**.

Step-4: Place an **AdRotator** control on the page and set value of its **AdvertisementFile** property to **"~/XMLFile.xml"**, **Height** as **"150px"** and **Width** as **"400px"**. Its source will be as follows:

```
%@ Page Language="C#" AutoEventWireup="true" CodeBehind="AdRotatorControl.aspx.cs"
Inherits="AdRotatorControl" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >

    <head runat="server">
    <title>Untitled Page</title>
    </head>

    <body>
    <form id="form1" runat="server">
        <div>
        <asp:AdRotator ID="AdRotator1" runat="server"
                       AdvertisementFile="~/XMLFile.xml"
                       Height="150px"
                       OnAdCreated="AdRotator1_AdCreated"
                       Width="400px" />
        </div>
    </form>
    </body>
</html>
```

Step-5: Press **F5**, the following window will appear.



When a user click on this image, the **http://www.yahoo.com** will be opened because of the **NavigateUrl** parameter of the image (set in XML File). It is also possible that a user refreshes the page and image may get changed on refresh.

Here's, the end of the **Program 4-23.**

4.10.4 Displaying different page views

MultiView control allows us to create pages with multiple views. MultiViewcontrol enables to hide and display different views of a page. The MultiView control contains one or more **View** controls. We can use the **MultiView** control to select a particular **View** control to be rendered using **ActiveViewIndex** property of MultiView control. The selected View control is the called as the **Active View**. The contents of the remaining View controls will be hidden. We can render only one View control at a time. The basic syntax to add a MultiView and View controls to a web form using **source** code is as follows:

```
<asp:MultiView ID="MultiView1" runat="server" ActiveViewIndex="0">

<asp:View ID="View1" runat="server">
</asp:View>

<asp:View ID="View2" runat="server">
</asp:View>

</asp:MultiView>
```

Table 4-Q lists some commonly used properties of the MultiView control.

Table 4-Q : Properties of the MultiView control	
Property	**Description**
ActiveViewIndex	Used to select the View control to render by index (starting from 0).
Views	Used to retrieve the collection of View controls contained in the MultiView control.
GetActiveView	Used to retrieve the selected View control.
SetActiveView	Used to select the active view.

The use of **MultiView** control can be demonstrated by **Program 4-24.**

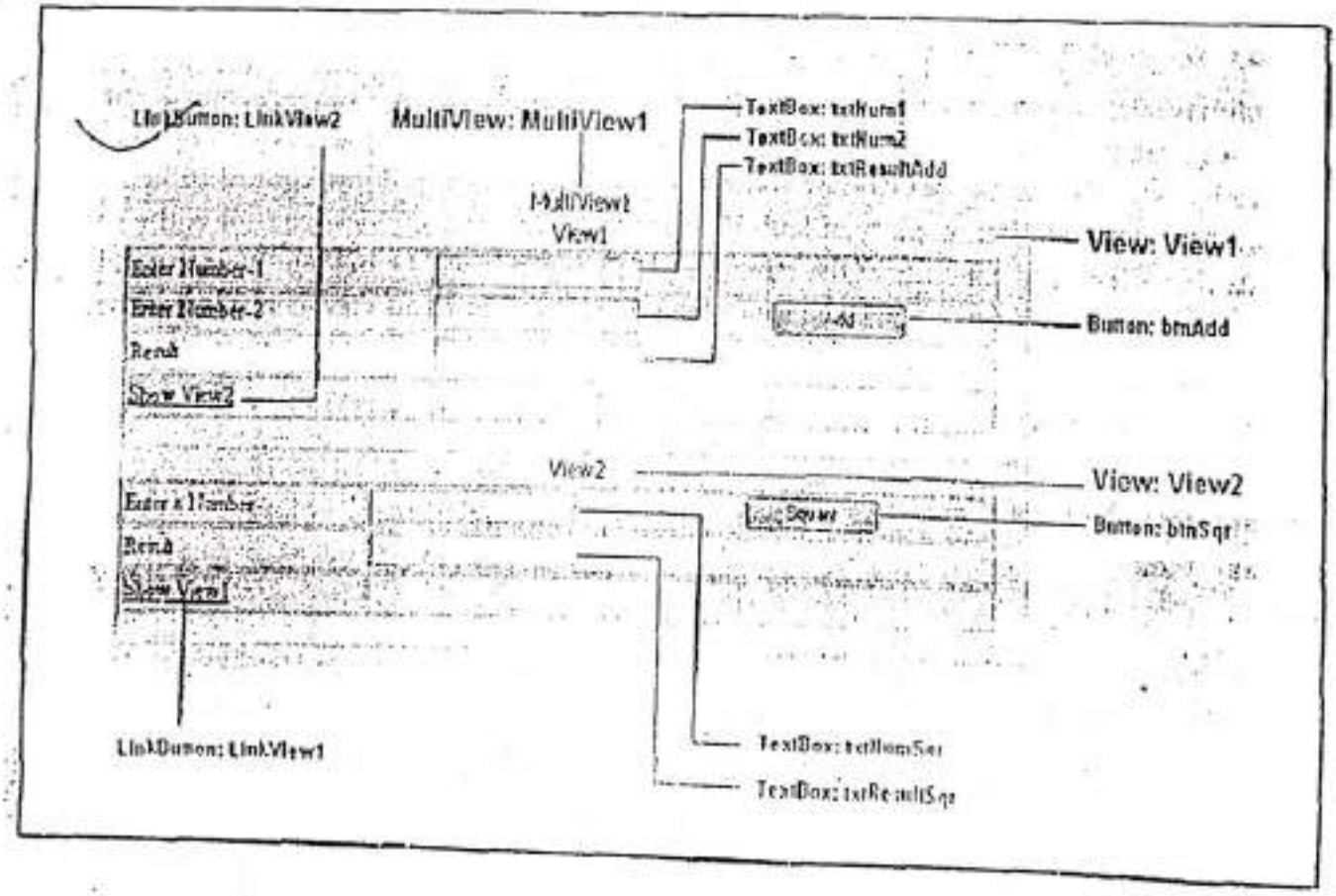
Program 4-24

Description of the Program 4-24: In this program, we are going to create two views of a page using MultiView and View controls. In first view of the page addition of two numbers will be performed and in second view square of a number will be performed. To shift from View1 to View2 and vice versa, Link Buttons are used. Follow the steps given below.

Step-1: Create a new **Web Application** named **"RichControlsApp"**.

Step-2: Add a **Web Form** to the application named **"MultiViewControl.aspx"**.

Step-3: Create design, first, add a **MultiView** Control to the web page then create two different views in **MultiView** control using **View** control as shown below.



Set MultiView control's ActiveViewIndex property as 0. Source of the page is as follows:

```
<%@ Page Langunge="C#" AutoEventWireup="true" CodeBehind="MultiViewControl.aspx.cs"
Inherits="MultiViewControl" %>
  /
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
<title>Displaying Different Page Views</title>
</head>
<body>
<form id="form1" runat="server">

<div>
<asp:MultiView ID="MultiView1" runat="server" ActiveViewIndex="0">

<asp:View ID="View1" runat="server">
<table style="width: 664px">
<tr>
<td style="width: 140px; height: 26px">Enter Number-1</td>
<td style="width: 36px; height: 26px">
<asp:TextBox ID="txtNum1" runat="server"></asp:TextBox></td>
<td style="width: 27px; height: 26px">
</td>
</tr>
<tr>
<td style="width: 140px">Enter Number-2</td>
<td style="width: 36px">
<asp:TextBox ID="txtNum2" runat="server"></asp:TextBox></td>
<td style="width: 27px">
<asp:Button ID="btnAdd" runat="server" OnClick="btnAdd_Click"
                        Text="Add" Width="103px" /></td>
</tr>
<tr>
<td style="width: 140px; height: 26px">Result</td>
<td style="width: 36px; height: 26px">
<asp:TextBox ID="txtResultAdd" runat="server"></asp:TextBox></td>
```

```
10
d style="width: 27px; height: 26px">
td>
tr>
>
d style="width: 140px; height: 26px">
asp:LinkButton ID="LinkView2" runat="server"
                        OnClick="LinkView2_Click">
                        Show View2
        </asp:LinkButton></td>
        <td style="width: 36px; height: 26px"></td>
d style="width: 27px; height: 26px"></td>
/tr>
/table>
/asp:View>

asp:View ID="View2" runat="server">
able style="width: 664px">
tr>
d style="width: 100px; height: 26px">
            Enter a Number</td>
d style="width: 36px; height: 26px">
asp:TextBox ID="txtNumSqr" runat="server"></asp:TextBox></td>
d style="width: 27px; height: 26px">
aspButton ID="btnSqr" runat="server" OnClick="btnSqr_Click"
                    Text="Square" Width="103px" /></td>
/tr>
tr>
d style="width: 100px; height: 26px">
            Result</td>
d style="width: 36px; height: 26px">
asp:TextBox ID="txtResultSqr" runat="server"></asp:TextBox></td>
d style="width: 27px; height: 26px">
/td>
/tr>
tr>
d style="width: 100px; height: 26px">
asp:LinkButton ID="LinkView1" runat="server"
            OnClick="LinkView1_Click">Show View1</asp:LinkButton></td>
```

```
<td style="width: 36px; height: 26px">
</td>
<td style="width: 27px; height: 26px">
</td>
</tr>
</table>
</asp:View>

</asp:MultiView>    

</div>
</form>
</body>
</html>
```

Step-4: Double click on button **btnAdd**. The code behind file will appear. Add the following code in event handler **btnAdd_Click(....)**.

```
protected void btnAdd_Click(object sender, EventArgs e)
{
    int a, b,c;
    /*
     converting values of txtNum1 and txtNum2 into integer
     and storing them into variable a and b respectively
    */
    a = Convert.ToInt32(txtNum1.Text);
    b = Convert.ToInt32(txtNum2.Text);

    //Add two a and b
    c = a + b;

    //storing the result of addition in txtResultAdd as a string
    txtResultAdd.Text = c.ToString();
}
```

Step-5: Double click on button **btnSqr** and add the following code in event handler **btnSqr_Click(....)**.

```
protected void btnSqr_Click(object sender, EventArgs e)
{
    int a, sqr;

    //converting value of txtNumSqr into integer and store it in variable a.
```

```
        a = Convert.ToInt32(txtNumSqr.Text);

        //calculating square
        sqr = a * a;

        //storing result of square into txtResultSqr as a string.
        txtResultSqr.Text = sqr.ToString();
    }
```

Step-6: Double click on link button LinkView2 and add the following code in event handler **LinkView2_Click(....).**

```
    protected void LinkView2_Click(object sender, EventArgs e)
    {
        MultiView1.ActiveViewIndex = 1;
    }
```

Step-7: Double click on link button LinkView1 and add the following code in event handler **LinkView1_Click(....).**

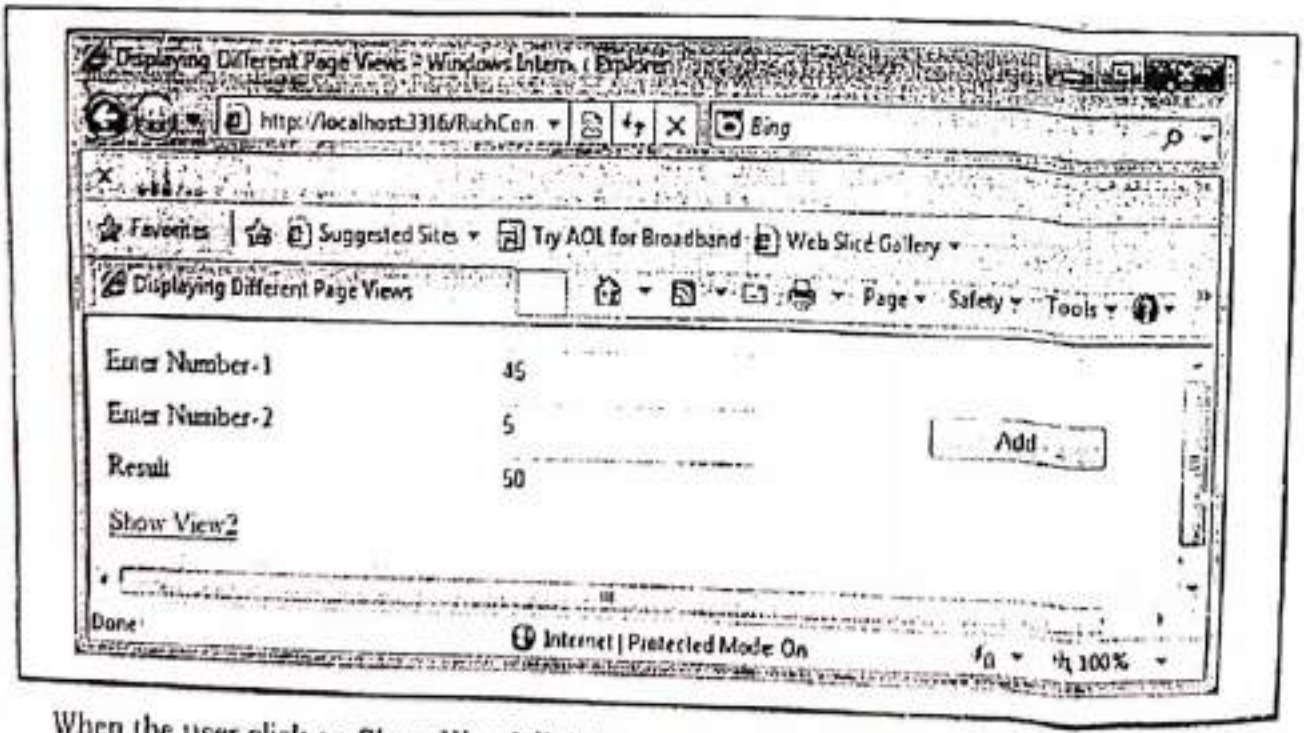
```
    protected void LinkView1_Click(object sender, EventArgs e)
    {
        MultiView1.ActiveViewIndex = 0;
    }
```

Note: The whole code behind file named "MultiViewControl.aspx.cs" will be as follows:

```
using System;
public partial class MultiViewControl : System.Web.UI.Page
{
    protected void LinkView1_Click(object sender, EventArgs e)
    {
            MultiView1.ActiveViewIndex = 0;
    }
    protected void LinkView2_Click(object sender, EventArgs e)
    {
            MultiView1.ActiveViewIndex = 1;
    }
    protected void btnAdd_Click(object sender, EventArgs e)
    {
        int a, b,c;
        /*
          converting values of txtNum1 and txtNum2 into integer
          and storing them into variable a and b respectively
         */
        a = Convert.ToInt32(txtNum1.Text);
```

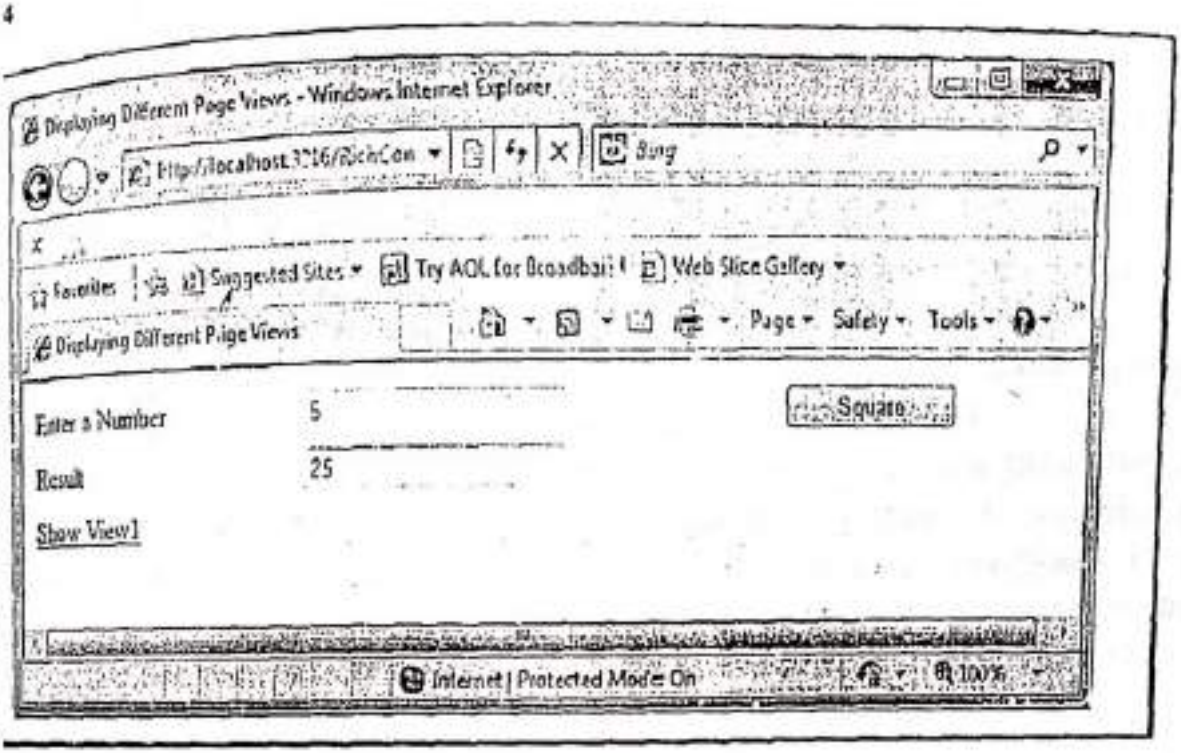
```
        b = Convert.ToInt32(txtNum2.Text);

        //Add two a and b
        c = a + b;

        //storing the result of addition in txtResultAdd as a string
        txtResultAdd.Text = c.ToString();
    }
    protected void btnSqr_Click(object sender, EventArgs e)
    {
        int a, sqr;
        //converting value of txtNumSqr into integer and store it in avariable a.
        a = Convert.ToInt32(txtNumSqr.Text);

        //calculating square
        sqr = a * a;

        //storing result of square into txtResultSqr as a string.
        txtResultSqr.Text = sqr.ToString();
    }
}
```

Step-8: Press **F5**. The Output will be as follows:



When the user click on **Show View2** link button the second view of the page will be shown as follows:

4



Here's, the end of the **Program 4-24.**

0.5 Displaying a Wizard

The Wizard control is a more glamorous version of the **MultiView** control. It also supports wing one of several views at a time, but it includes a fair bit of built-in yet customizable avior, including navigation buttons, a sidebar with step links, styles, and templates.

Usually, wizards represent a single task, and the user moves linearly through them, moving the current step to the one immediately following it. The ASP.NET Wizard control also ports nonlinear navigation, which means it allows you to decide to ignore a step based on the rmation the user supplies.

The Wizard control contains one or more **WizardStep** child controls. Only one **WizardStep** is layed at a time. The basic syntax to add a **Wizard** control to a web form using **source** code is llows:

```
p:Wizard ID="Wizard2" runat="server">
izardSteps>
    <asp:WizardStep runat="server" title="Step 1"></asp:WizardStep>
        <asp:WizardStep runat="server" title="Step 2"></asp:WizardStep>
WizardSteps>
sp:Wizard>
```

Table 4-R lists some commonly used properties of the Wizard control.

ble 4-R : Properties of the Wizard control

Property	Description
liveStepIndex	Enables you to set or get the index of the active **WizardStep** control.

CancelDestinationPageUrl	Enables you to specify the URL where the user is sent when the Cancel button is clicked.
DisplayCancelButton	Enables you to hide or display the Cancel button.
DisplaySideBar	Enables you to hide or display the Wizard control's sidebar. The sidebar displays a list of all the wizard steps.
FinishDestinationPageUrl	Enables you to specify the URL where the user is sent when the Finish button is clicked.
HeaderText	Enables you to specify the header text that appears at the top of the Wizard control.
WizardSteps	Enables you to retrieve the WizardStep controls contained in the Wizard control.

Wizard Styles and Templates

The Wizard control's greatest strength is the way it lets you customize its appearance. This means that if we want the basic model (*a multistep process with navigation buttons and various events*), we aren't locked into the default user interface. Depending on how radically we want to change the wizard, we have different options. For less dramatic modifications, we can set various top-level properties. For example, we can control the colors, fonts, spacing, and border style, as we can with any ASP.NET control. We can also tweak the appearance of every button. There various styles supported by a Wizard control such are as follows:

Wizard Styles

Style Description

ControlStyle	Applies to all sections of the Wizard control.
HeaderStyle	Applies to the header section of the Wizard control, which is visible only if you set some text in the HeaderText property.
SideBarStyle	Applies to the sidebar area of the Wizard control.
SideBarButtonStyle	Applies to just the buttons in the sidebar.
StepStyle	Applies to the section of the control where you define the step content.
NavigationStyle	Applies to the bottom area of the control where the navigation buttons are displayed.
NavigationButtonStyle	Applies to just the navigation buttons in the navigation area.
StartNextButtonStyle	Applies to the next navigation button on the first step (when StepType is Start).
StepNextButtonStyle	Applies to the next navigation button on intermediate steps (when StepType is Step).
StepPreviousButtonStyle	Applies to the previous navigation button on intermediate steps (when StepType is Step).

FinishPreviousButtonStyle	Applies to the previous navigation button on the last step (when StepType is Finish).
CancelButtonStyle	Applies to the cancel button, if you have Wizard.DisplayCancel-Button set to true.

The Wizard control supports the following templates:

HeaderTemplate: Defines the content of the header region.

SideBarTemplate: Defines the sidebar, which typically includes navigation links for each :ep.

StartNavigationTemplate: Defines the navigation buttons for the first step (*when StepType : Start*).

StepNavigationTemplate: Defines the navigation buttons for intermediate steps (*when tepType is Step*).

FinishNavigationTemplate: Defines the navigation buttons for the final step (*when StepType : Finish*).

Take a look at the following program.

rogram 4-25

Description of the Program 4-25: This program demonstrates the use of Wizard control. Follow the steps given below.

Step-1: Create a new **Web Application** named "**RichControlsApp**".

Step-2: Add a **Web Form** to the application named "**WizardControl.aspx**".

Step-3: Write the following code in the source of the page:

```
<%@ Page Langur ge="C#" AutoEventWireup="true" CodeBehind="WizardControl.aspx.cs"
nherits="WizardControl" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
'http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Displaying a Wizard</title>
</head>
<body>
<form id="form1" runat="server">
<div>
<asp:Wizard ID="Wizard1" runat="server" Width="467px"
            BackColor="#EFF3FB" BorderColor="#B5C7DE" BorderWidth="1px"
            ActiveStepIndex="0">
<WizardSteps>
```

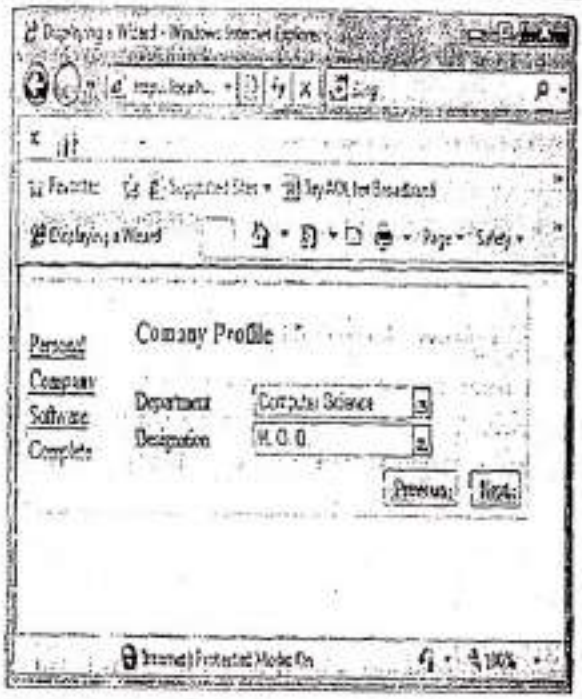
```
<asp:WizardStep ID="WizardStep1" runat="server" Title="Personal">
<h3>Personal Profile</h3>
<table>
<tr>
<td>Name:</td>
<td><asp:TextBox ID="TextBox2" runat="server"></asp:TextBox></td>
</tr>
<tr>
<td>Address:</td>
<td>
      <asp:DropDownList ID="DropDownList2" runat="server"
                        Height="20px"  Width="125px">
      <asp:ListItem>Khanna</asp:ListItem>
      <asp:ListItem>Ludhiana</asp:ListItem>
      <asp:ListItem>Nabha</asp:ListItem>
      <asp:ListItem>MGG</asp:ListItem>
      </asp:DropDownList>
</td>
</tr>
</table>
<br />
</asp:WizardStep>
<asp:WizardStep ID="WizardStep2" runat="server" Title="Company">
      <h3>Comany Profile</h3>
      <table>
      <tr>
      <td>
            Department</td>
      <td>
      <asp:DropDownList ID="DropDownList3" runat="server"
                        Height="20px" Width="165px">
      <asp:ListItem>Computer Science</asp:ListItem>
      <asp:ListItem>Management</asp:ListItem>
      <asp:ListItem>Media</asp:ListItem>
      <asp:ListItem>Animation</asp:ListItem>
      </asp:DropDownList>
      </td>
      </tr>
```

```
    <tr>
    <td>Designation</td>
    <td>
    <asp:DropDownList ID="DropDownList4" runat="server"
                                Height="20px" Width="165px">
    <asp:ListItem>H. O. D.</asp:ListItem>
    <asp:ListItem>Professor</asp:ListItem>
    <asp:ListItem>Assistant Professor</asp:ListItem>
    <asp:ListItem>Senior Lecturer</asp:ListItem>
    <asp:ListItem>Lecturer</asp:ListItem>
    </asp:DropDownList>
    </td>
    </tr>
    </table>
</asp:WizardStep>
<asp:WizardStep ID="WizardStep3" runat="server" Title="Software">
    <h3>Specialization Profile</h3>
    <asp:CheckBoxList ID="lstTools" runat="server">
        <asp:ListItem>ASP.NET</asp:ListItem>
        <asp:ListItem>JAVA</asp:ListItem>
        <asp:ListItem>MARKETTING</asp:ListItem>
        <asp:ListItem>FINANCE</asp:ListItem>
        <asp:ListItem>GRAPHICS DESIGNING</asp:ListItem>
        <asp:ListItem>MASS COMMUNICATION</asp:ListItem>
    </asp:CheckBoxList>
</asp:WizardStep>
<asp:WizardStep ID="Complete" runat="server" Title="Complete"
        StepType="Complete">
    <br />
        Thank you for completing this survey. <br />
    </asp:WizardStep>
</WizardSteps>
</asp:Wizard>
</div>
</form>
</body>
</html>
```

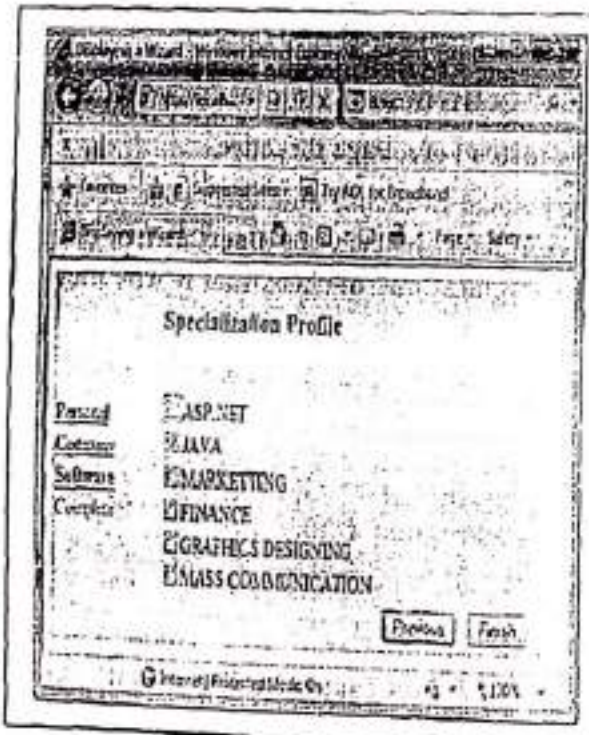
Step-4: Press F5. The following views can be shown:

View1

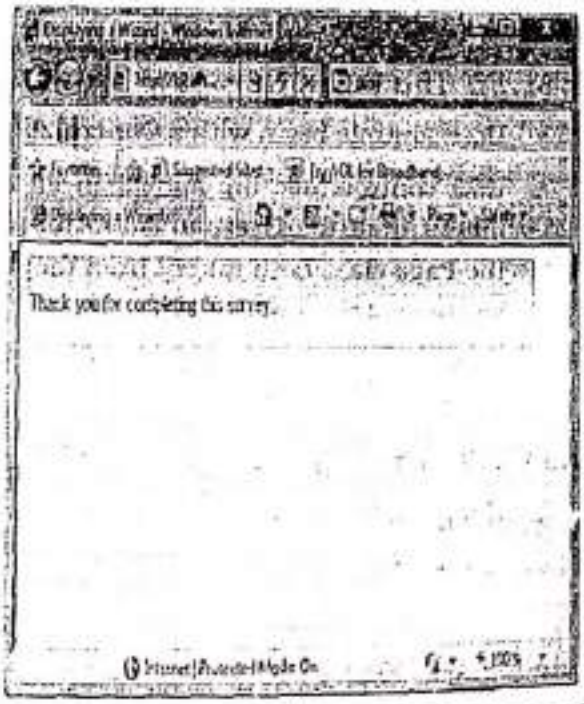
View2



View3



View4



Here's, the end of the **Program 4-25.**

1.11 CUSTOM WEB CONTROLS

ASP.NET provides number of controls to design web forms (web pages). These controls provide a wide range of functionality and allow developers to design and develop a user-friendly nterface for their applications. But, sometimes these controls do not meet developer's needs. To overcome this limitation ASP.NET allow developers to create their own controls to be used in web orms. This can be done in two ways;

- User Controls
- Custom Controls

1.11.1 User Controls

A user control is similar to a web page that can include static HTML code and web server ontrols. The advantage of user controls is that once we create a user control, we can reuse it in multiple pages in the same web application. We can even add our own properties, events, and nethods.

We can develop the user controls almost exactly the same way that we develop Web Forms ages. So, it is easy to create user controls, but they can be less convenient to use in advanced cenarios. User controls cannot be added to the Toolbox, and they are represented by a simple laceholder when added to a page. This makes user controls harder to use if you are accustomed o full Visual Studio .NET design-time support, including the **Properties** window and **Design iew** previews. Also, the only way to share the user control between applications is to put a eparate copy in each application, which takes more maintenance if you make changes to the ontrol.

1.11.1.1 The key differences between user controls and a web pages

- User controls begin with a **Control** directive instead of a **Page** directive.
- User controls use the file extension .ascx instead of .aspx, and their code behind files inherit from the **System.Web.UI.UserControl** class. In fact, the **UserControl** class and the **Page** class both inherit from the same **TemplateControl** class, that is why they share so many of the same methods and events.
- User controls can't be requested directly by a client. Instead, user controls are embedded inside other web pages.

1.11.1.2 Creating a Simple User Control

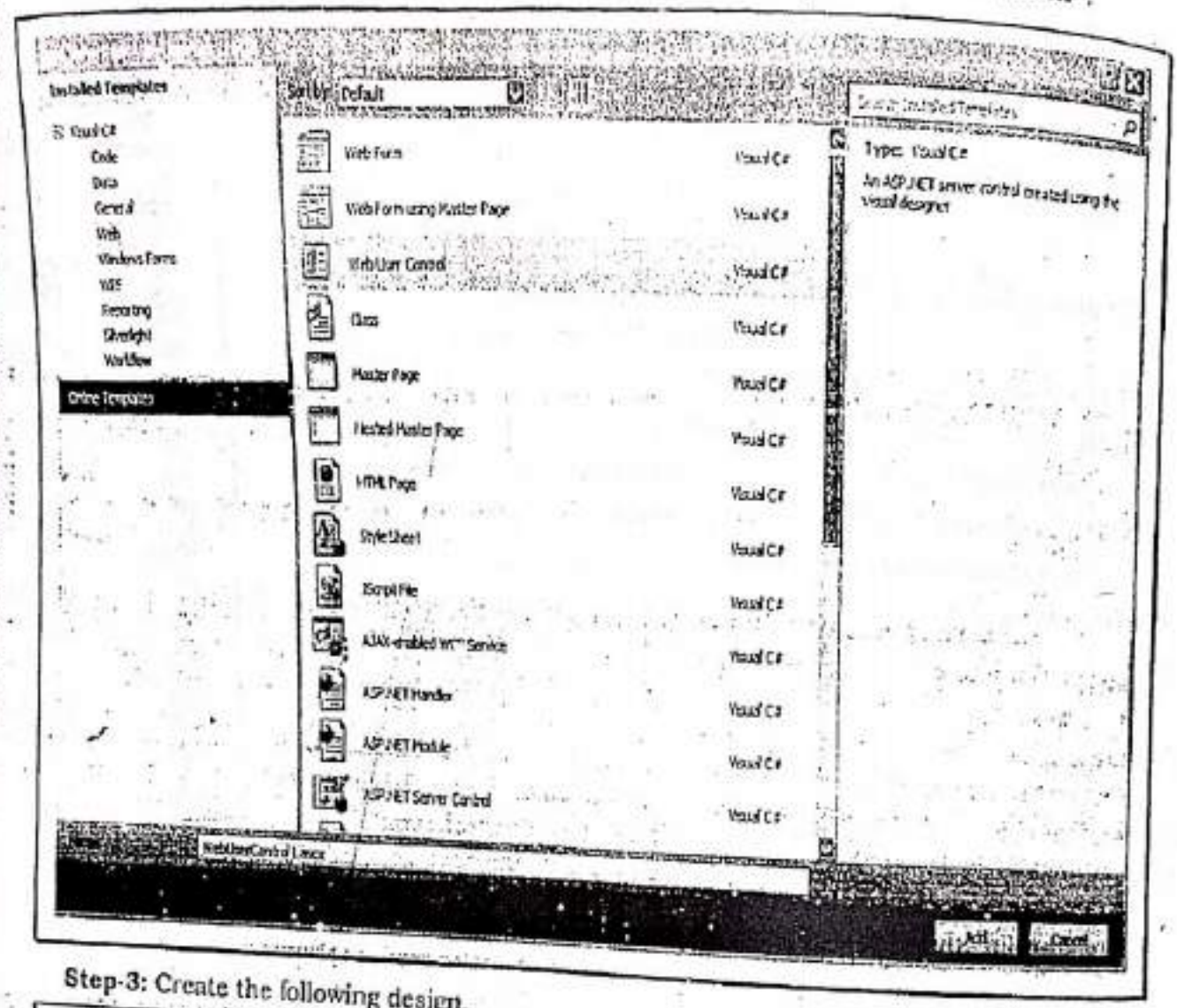
Creating user controls can be demonstrated by the **Program 4-26.**

Program 4-26

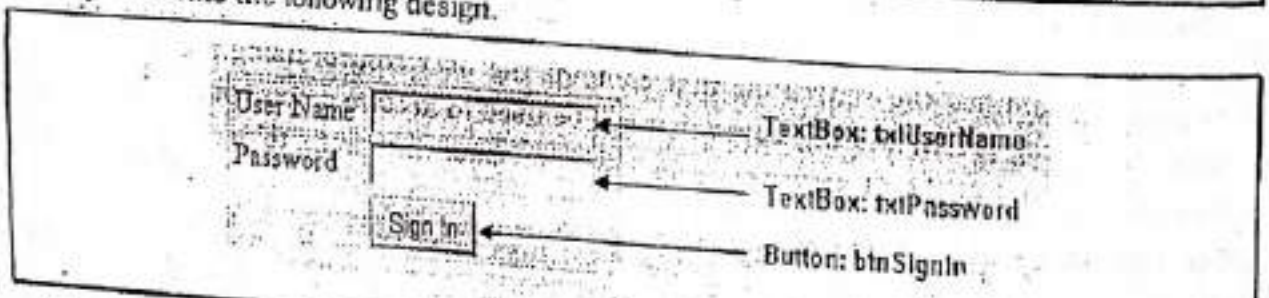
Description of Description of the Program 4-26: In this program, we are going to create a simple user control. It will be Login Control for the web pages. To create this control, take the following steps.

Step-1: Create a new **Web Application** named " **MyUserControl**".

Step-2: Add a **Web User Control** to the application named **"WebUserControl1.ascx"**.



Step-3: Create the following design.



The source file of this will be as follows:

```
<%@ Control Language="C#" AutoEventWireup="true" CodeBehind="WebUserControl1.ascx.cs"
Inherits="MyUserControl.WebUserControl1" %>

<style type="text/css">
    .style1
    {
        width: 33%;
```

```
.a1{
}
</style>

<table class="style1">
 <tr>
    <td> User Name</td>
    <td> <asp:TextBox ID="txtUserName" runat="server"></asp:TextBox> </td>
 </tr>
 <tr>
    <td> Password</td>
    <td><asp:TextBox ID="txtPassword" runat="server"></asp:TextBox></td>
 </tr>

 <tr>
    <td>  </td>
    <td><asp:Button ID="btnSignIn" runat="server" onclick="btnSignIn_Click" Text="Sign
In" /></td>
 </tr>
</table>
```

Step-4: Now to test the control, we need to place it on a **Web Forms** page. So, add a new **Web Forms** page to the application named **"UsingUserControl.aspx"**.

Step-5: Now, we need to tell the ASP.NET page that we plan to use that user control with the **<%@Register** directive, as shown here:

```
<%@ Register TagPrefix="UserControl" TagName
="Login" Src="~/WebUserControl1.ascx" %>
```

This line identifies the source file that contains the user control using the **Src** attribute. It also defines a **tag prefix** and **tag name** that will be used to declare a new control on the page. In the same way that ASP.NET server controls have the **<asp: ... >** prefix to declare the controls as follows:

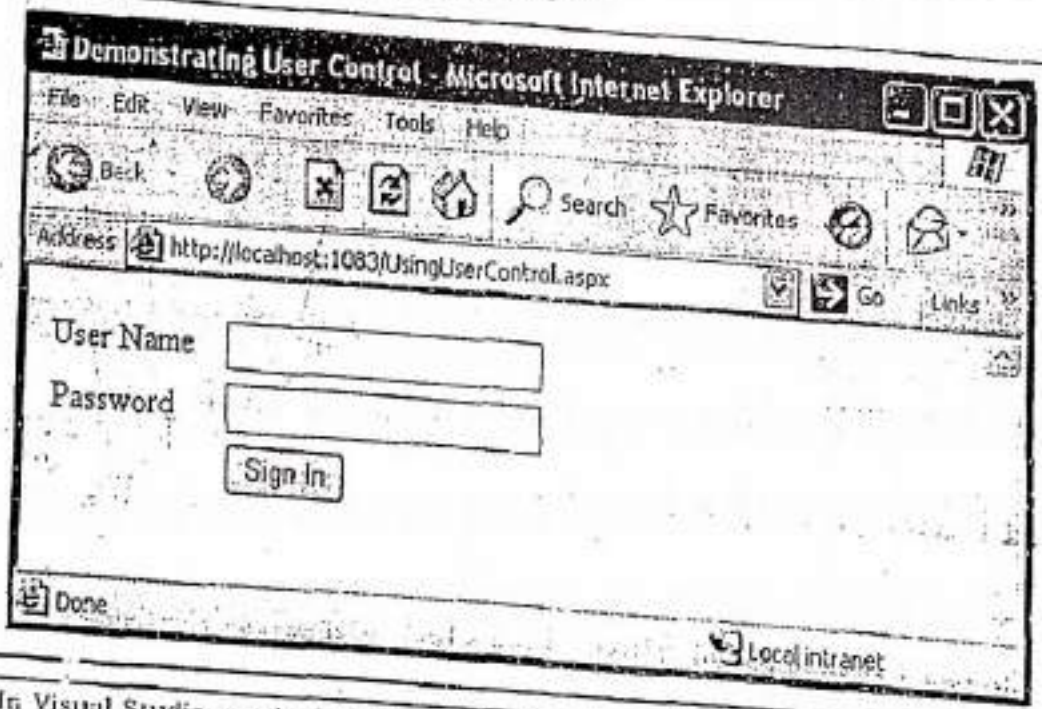
```
<asp:TextBox>.
```

We can use our own tag prefixes to distinguish the controls we have created. This example uses a tag prefix of **UserControl** and a tag named **Login**. The source file of the web form will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="UsingUserControl.aspx.cs"
    Inherits="MyUserControl.UsingUserControl" %>

<%@ Register TagPrefix="UserControl" TagName="Login" Src="~/WebUserControl1.ascx" %>

<html xmlns="http://www.w3.org/1999/xhtml">
```

```
    <head runat="server">
        <title>Demonstrating User Control</title>
    </head>

    <body>
      <form id="form1" runat="server">
        <div>
            <UserControl:Login id="Login1" runat="server"> </UserControl:Login>
        </div>
      </form>
    </body>
</html>
```

Step-5: Press **F5**. The following window will appear.



Note: In Visual Studio, we don't need to code the **Register** directive manually. Instead, once we have created our user control, we can simply select the **.ascx** in the **Solution Explorer** and drag it onto the design area of a web form. Visual Studio will automatically add the **Register** directive, as well as an instance of the user control tag.

Here's, the end of the **Program 4-26.**

4.11.2 Custom Controls

Web custom controls are compiled components that run on the server and that encapsulate user interface and other related functionality into reusable packages. They can include all the

design-time features of standard ASP.NET server controls, including full support for Visual Studio design features such as the **Properties window**, the **visual designer**, and the **Toolbox**. Custom controls can be created by the developers by their own, completely from scratch, or by extending an existing control's behavior.

Custom controls are easy to use but difficult to create. Once we have created the control, we can add it to the Toolbox and display it in a visual designer with full Properties window support and all the other design-time features of ASP.NET server controls. In addition, we can install a single copy of the custom control in the global assembly cache (GAC) and share it between applications, which make maintenance easier.

Custom controls are not **User controls**. User controls are easier to create than custom server controls, but server controls are far more powerful. So, the main difference between the two controls lies in ease of creation versus ease of use at design time. Some of the differences between the two types are outlined in the **Table 4-S**

Table 4-S: Differences between User controls and Custom controls

User Controls	Custom Controls
Easier to create.	Harder to create.
Limited support for consumers who use a visual design tool.	Full visual design tool support for consumers.
A separate copy of the control is required in each application.	Only a single copy of the control is required, in the global assembly cache.
Cannot be added to the Toolbox in Visual Studio.	Can be added to the Toolbox in Visual Studio.
Good for static layout.	Good for dynamic layout.

4.11.2.1 Creating a Custom Control

There are several ways to create custom controls:

I. We can create a custom control that combines the functionality of two or more existing controls. For example, if we need a control that encapsulates a button and a text box, we can create it by compiling the existing controls together. This type of custom control is known as **Composite Control**.

II. If an existing server control almost meets our requirements but lacks some required features, we can customize the control by deriving from it and overriding its **properties**, **methods**, and **events**.

III. If none of the existing Web controls (or their combinations) meet our requirements, we can create a custom control by deriving from one of the base control classes. These classes provide all the basic functionality of Web controls, so we can focus on programming the features we need.

Note: At design time in Visual Studio, our code always runs fully trusted, even if the code will eventually be in a project where it would receive less-than-full trust at run time. This means that our custom control might work properly when we are testing it on our own computer, but might fail due to lack of adequate permissions in a deployed application. So, we must be sure to test our controls in the security context in which they will run in real world applications.

Ideally, we can create our custom controls in a separate class library project and compile the project into a separate DLL assembly. Although we can create a custom control and place the source code directly in the **App_Code** directory. But, this limits our ability to reuse the control in pages written in different languages. If we place controls in a separate assembly, we will also have better design-time support when we use them in pages. In the following section, we walk through the steps of authoring a simple custom server control.

(A) Define a class that directly or indirectly derives from **System.Web.UI.Control**.

```
using System;
using System.Web.UI;
public class FirstControl : Control{...}
```

The **using** directive allows code to reference types from a namespace without having to use the fully qualified name. Thus **Control** resolves to **System.Web.UI.Control.**

(B) Enclose custom control in a namespace. We can define a new namespace or use an existing namespace. The namespace name is the value of the **namespace** pseudo-attribute in the **Register** page directive. For example:

```
<%@ Register TagPrefix="Custom" Namespace="CustomControls"  Assembly =
                                "CustomControls" %>
```

So, enclose custom control in a namespace as follows:

```
namespace CustomControls
{
    public class FirstControl : Control {...}
...
}
```

(C) Define properties needed by our control. The following code fragment defines a property named **Message**. Properties are like smart fields and have accessor methods.

```
private String message = "Custom Control";
public virtual String Message          //The Message property.
{
    get {
        return message;
    }

    set{
```

```
            message = value;
        }
    }
```

(d) Override the **Render** method that our control inherits from **Control**. This method provides the logic for sending HTML to a client browser. The HTML that our control sends to the client is passed as a string argument to the **Write** method of an instance of **System.Web.UI.HtmlTextWriter**, as in the following example.

```
protected override void Render( HtmlTextWriter writer)
{
    writer.Write("<font> "+ this.Message + "<br>" +"The date and time on the server: "
                + System.DateTime.Now.ToLongTimeString()+ "</font>");
}
```

The class **System.DateTime** that is accessed in this code fragment is a utility class that provides date and time information. This class is invoked on the server and hence returns the time on the server.

In the code fragment, raw **HTML** is simply passed as a string argument to the **Write** method of **HtmlTextWriter**.

(e) Save, compile, and deploy the control by executing the following steps.

(a) Create a subdirectory named /bin in your application's root directory.

(b) Compile the source file into an assembly (**.dll**) and save the assembly in your application's /bin subdirectory.

For example, if our source code is in C# and we save it to a file named **FirstControl.cs**, we can execute the following command from the directory containing the source file.

```
csc /t[arget]:library /out:[path to bin]bin\CustomControls.dll
[r[eference]:System.Web.dll /r:System.dll [path to FirstControl.cs]FirstControl.cs
```

The /r option informs the compiler which assemblies are referenced by our control.

Our control is now compiled and ready to be used from any ASP.NET page in our application's root directory (or any of its subdirectories).

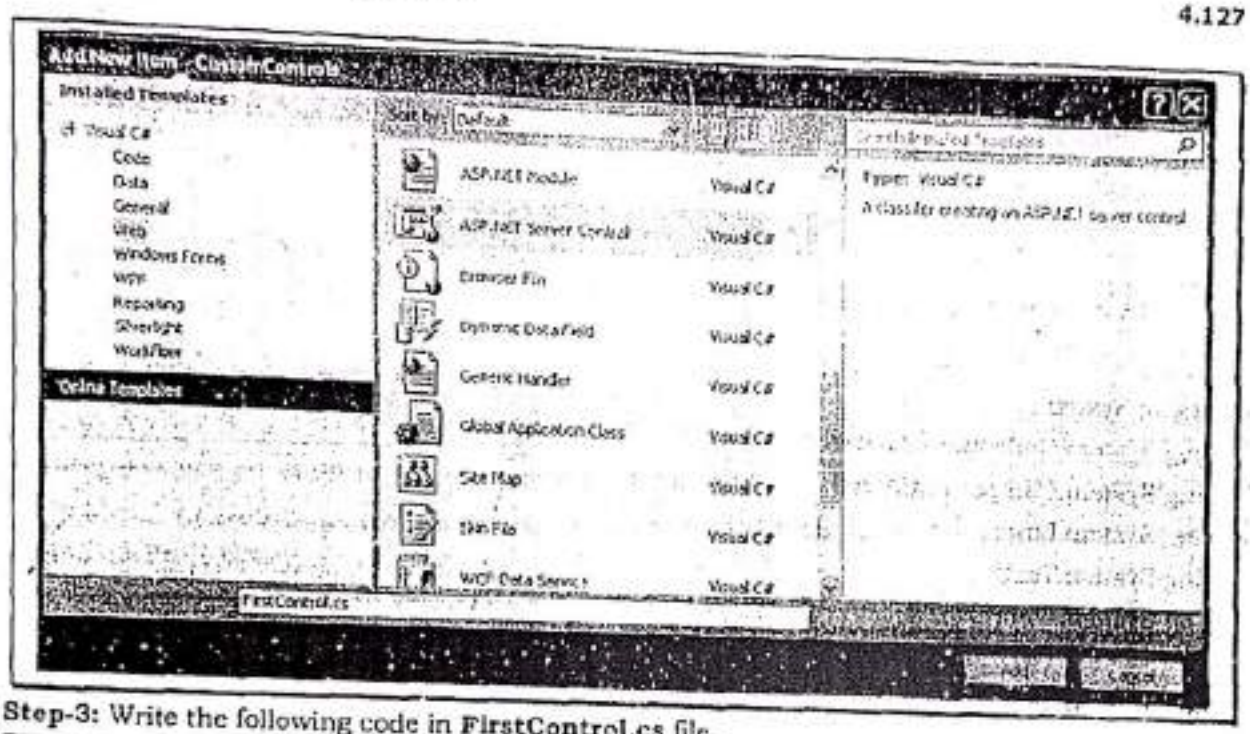
Creating custom controls can be demonstrated by the **Program 4-27.**

Program 4-27

Description of the Program 4-27: In this program, we are going to create a simple custom control that will be used to display current time of the server system. To create this control, follow the steps given below.

Step1: Create a new Web Application named "CustomControls".

Step2: Add an ASP.NET Server Control to the application named "FirstControl.cs".



Step-3: Write the following code in **FirstControl.cs** file.

```
using System;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace CustomControls
{
   public class FirstControl : Control
   {
       private String message = "Custom Control";
       public virtual String Message      //The Message property
       get
          {
               return message;
          }
          set
          {
               message = value;
          }
   }

   protected override void Render(HtmlTextWriter writer)
```

```
    {
            writer.Write("<font> " + this.Message + "<br>" + "The time on the server is " +
                                    System.DateTime.Now.ToLongTimeString() + "</font>");
    }

  }
}

/*using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Text;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace CustomControls
{
  [DefaultProperty("Text")]
  [ToolboxData("<{0}:FirstControl runat=server></{0}:FirstControl>")]
  public class FirstControl : WebControl
  {
    [Bindable(true)]
    [Category("Appearance")]
    [DefaultValue("")]
    [Localizable(true)]
    public string Text
    {
      get
      {
        String s = (String)ViewState["Text"];
        return ((s == null) ? String.Empty : s);
      }

      set
      {
        ViewState["Text"] = value;
```

```
        }
    }

    protected override void RenderContents(HtmlTextWriter output)
    {
        output.Write(Text);
    }
}
}
*/
```

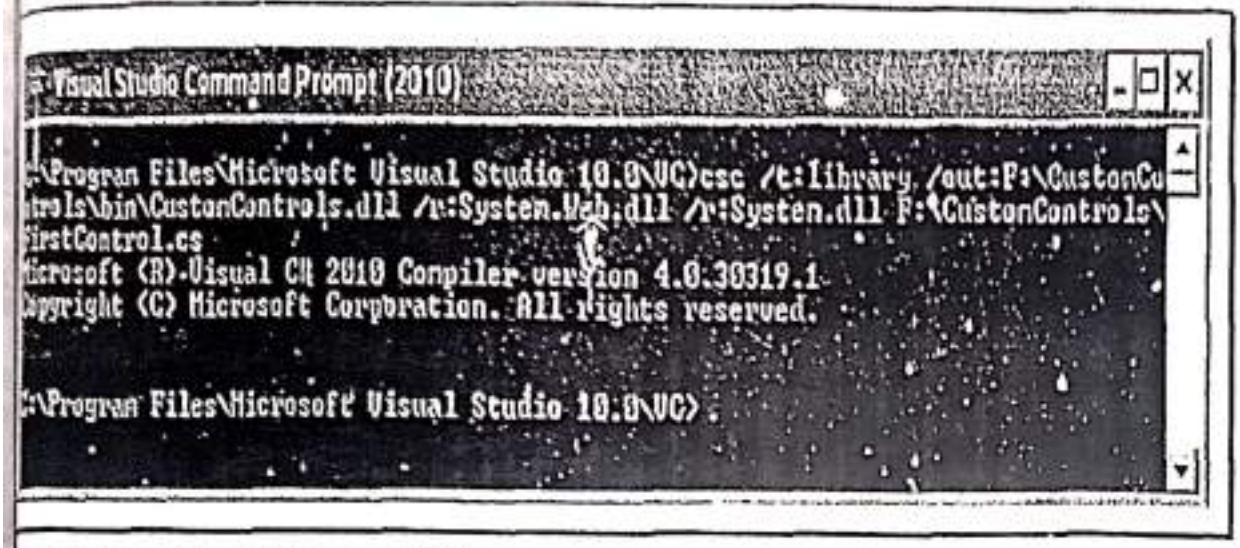
Step-4: Click on **Build** menu and select **Build Solution.**

See, the **CustomControls.dll** file will be created in the **bin** directory of the application as highlighted as follows:

Our custom control is now ready to be used. We can use this control on other web pages by simply dragging and dropping or by using HTML code.

Note: This can also be done by compiling the code in **Visual Studio Command Prompt (2010)** by using the following command.

```
csc /t:library /out: F:\CustomControls\bin\CustomControls.dll /r:System.Web.dll
/r:System.dll F:\CustomControls\FirstControl.cs
```

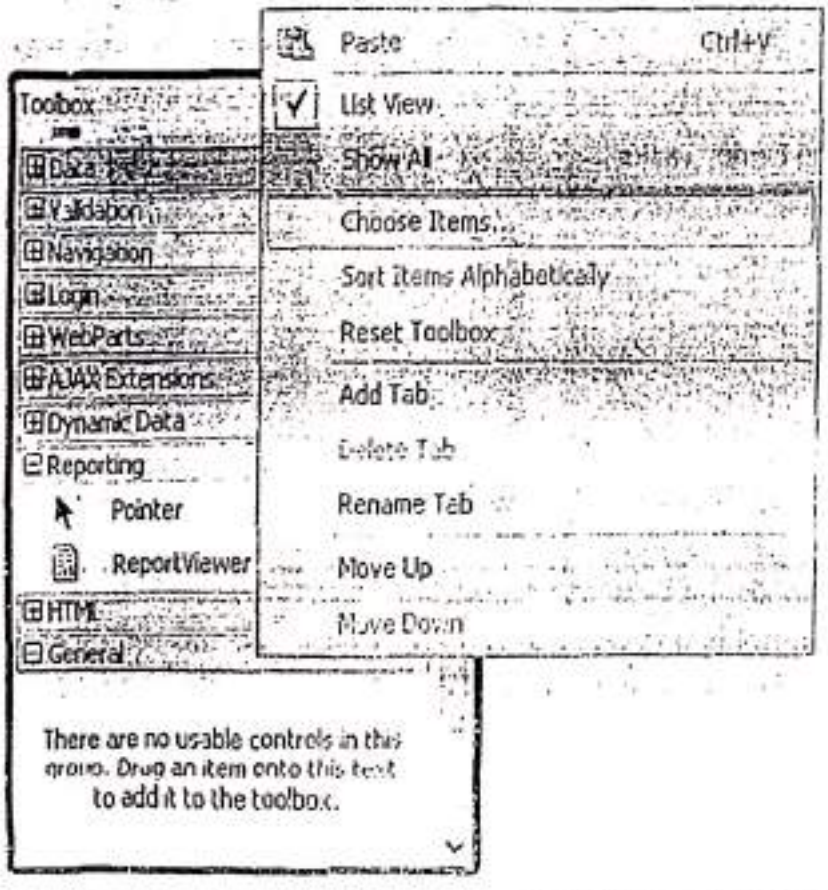


Here's, the end of the **Program 4-27.**

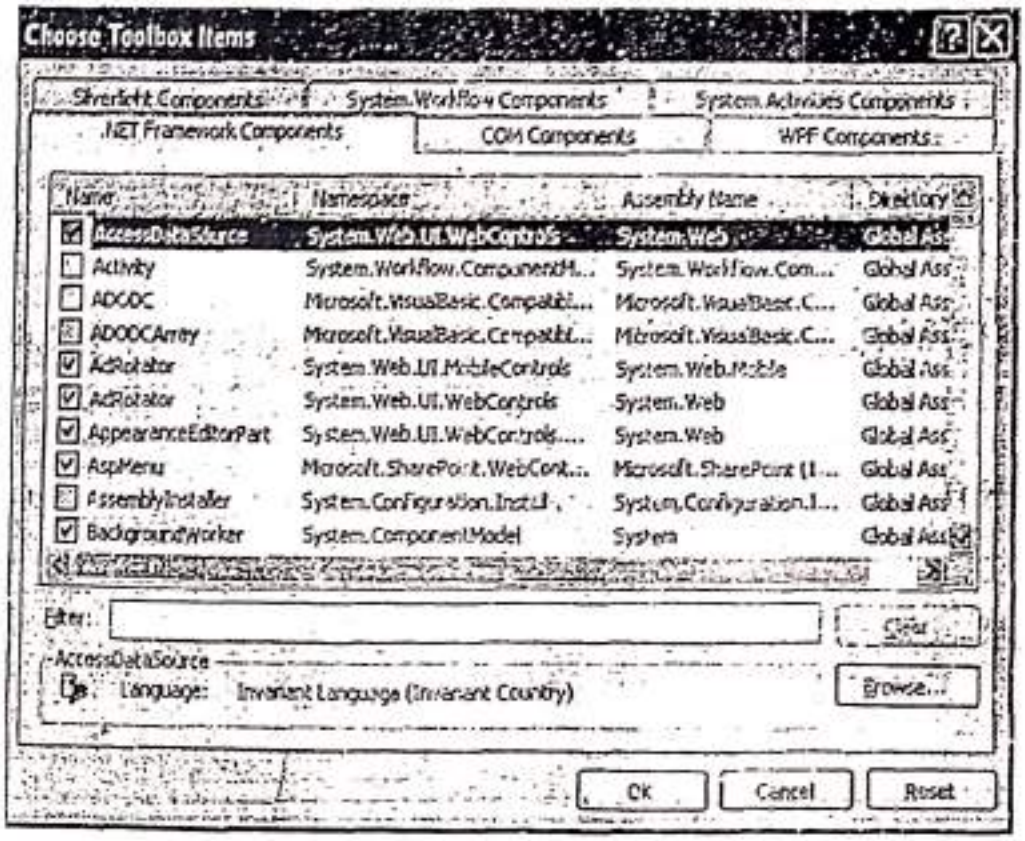
1.2.2 Adding a Custom Control in Toolbox

To use custom controls on web pages, we have to add these controls in Visual Studio IDE lbox. To add a custom control in Toolbox, follow the steps given below.

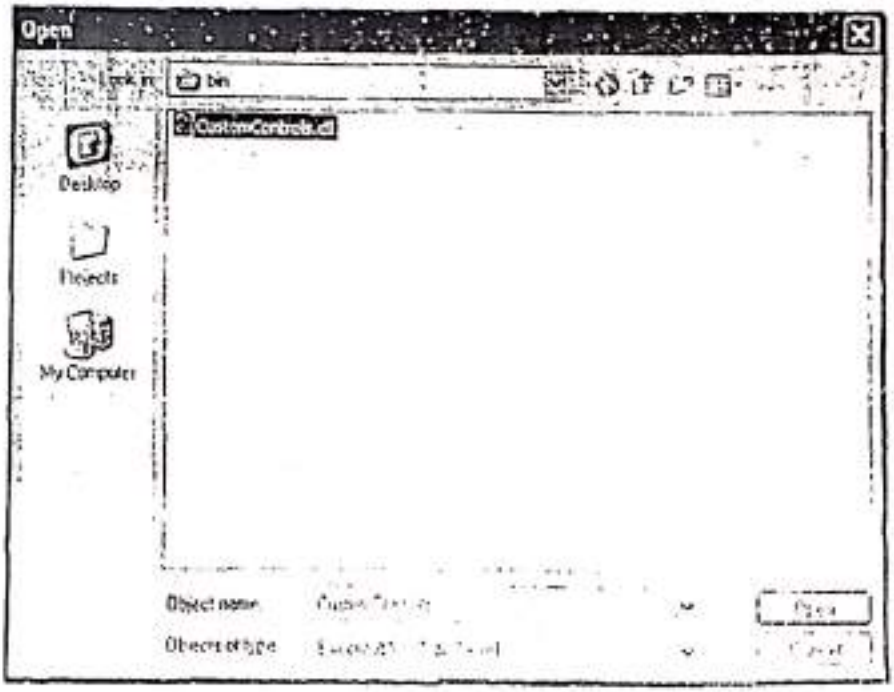
Step-1: Right click on any tab of the Toolbox and select **Choose Items...** option as shown as ows:



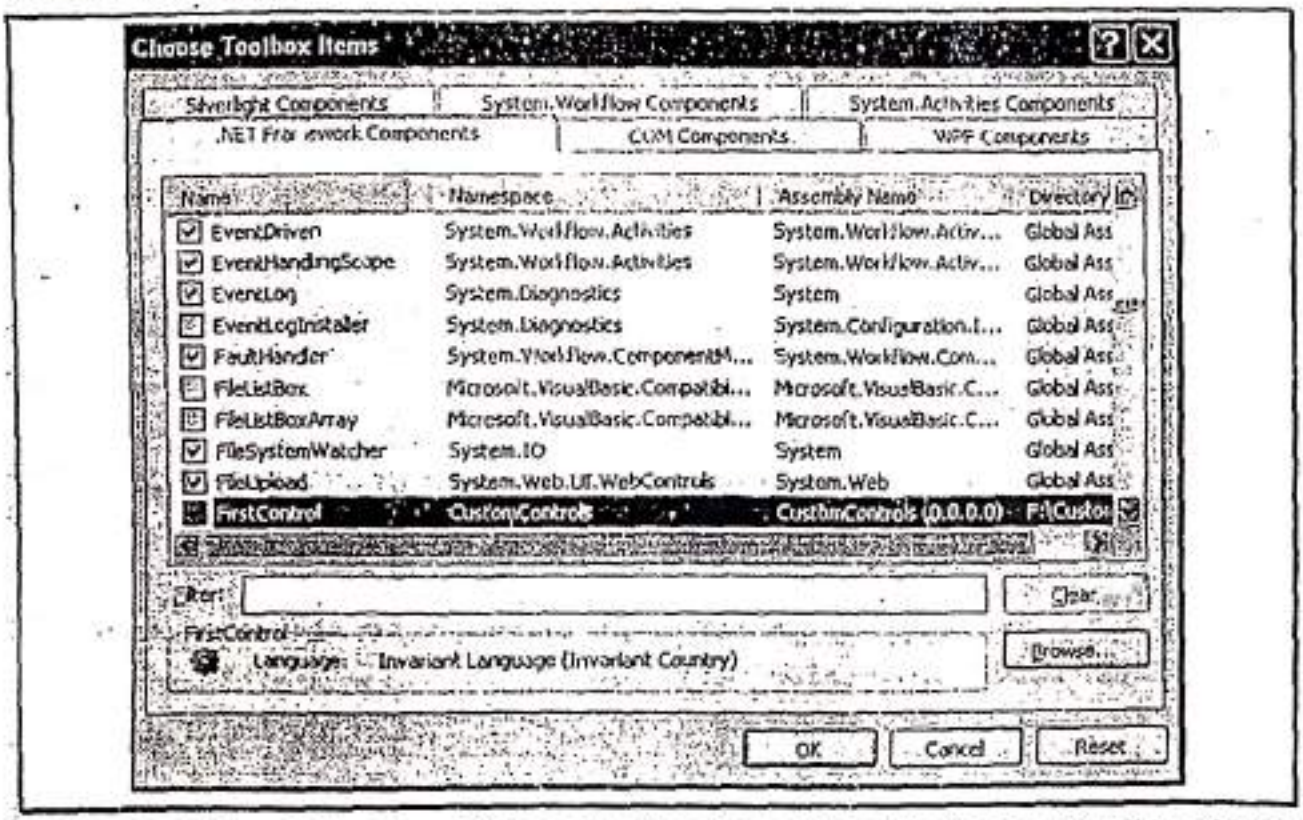
Step-2: Choose Toolbox Items dialog will appear as follows:



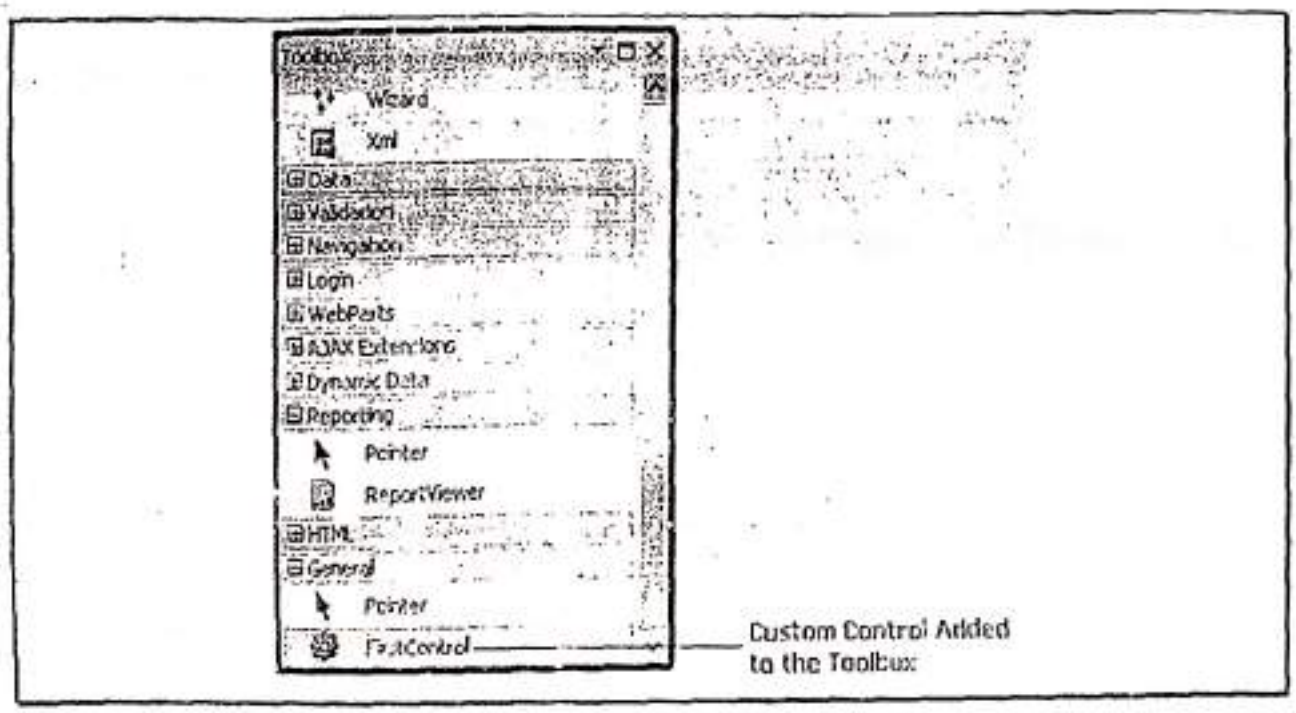
Step-3: Click on **Browse** button the **Open** dialog will appear as follows:



Step-4: In **Open** dialog, select **CustomControls.dll** file and click on **Open** button. See the custom control will be added into the list of **.NET Framework Componets** tab in **Choose Toolbox Items** dialog.



Step-5: Click on **Ok** button of **Choose Toolbox Items** dialog. See the control will be added in the Toolbox tab as shown as follows:



Now, we can use this control as we use other ASP.NET standard controls

4.11.2.3 Using a Custom Control

Once the custom control is created we can use it all web pages. This can be demonstrated by the **Program 4-28**

Program 4-28

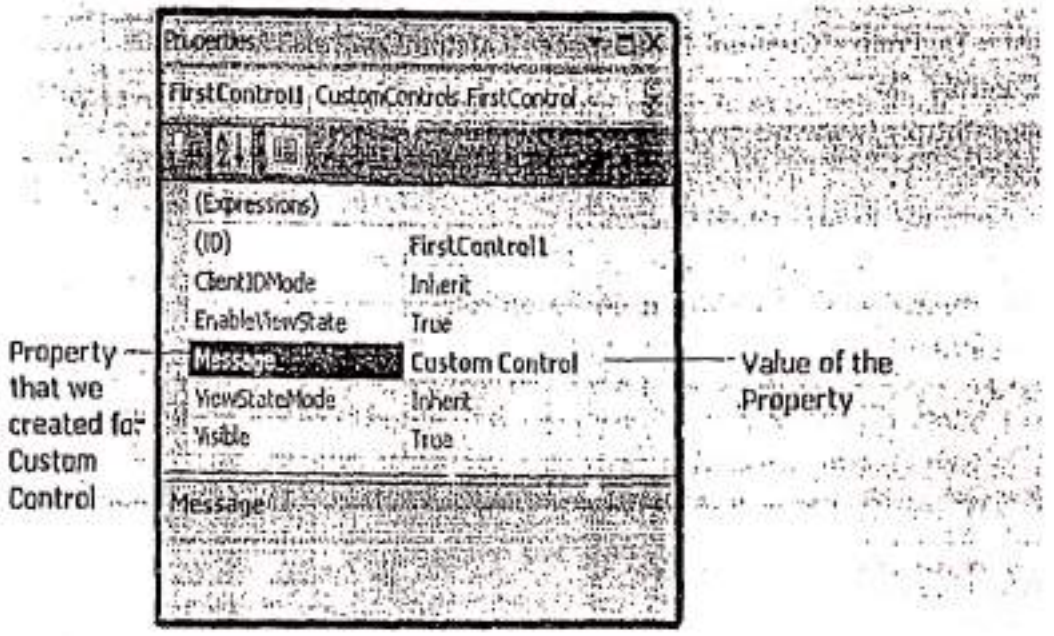
Description of the Program 4-28: In this program, we are going to use a custom control that we have created above in Program 4-27 (FirstControl), on a web page. Follow the steps given below.

Step-1: Create a new **Web Application** named **"UsingCustomControl"**.

Step-2: Add a **Web Form** to the application named **"MyCustomControl.aspx"**.

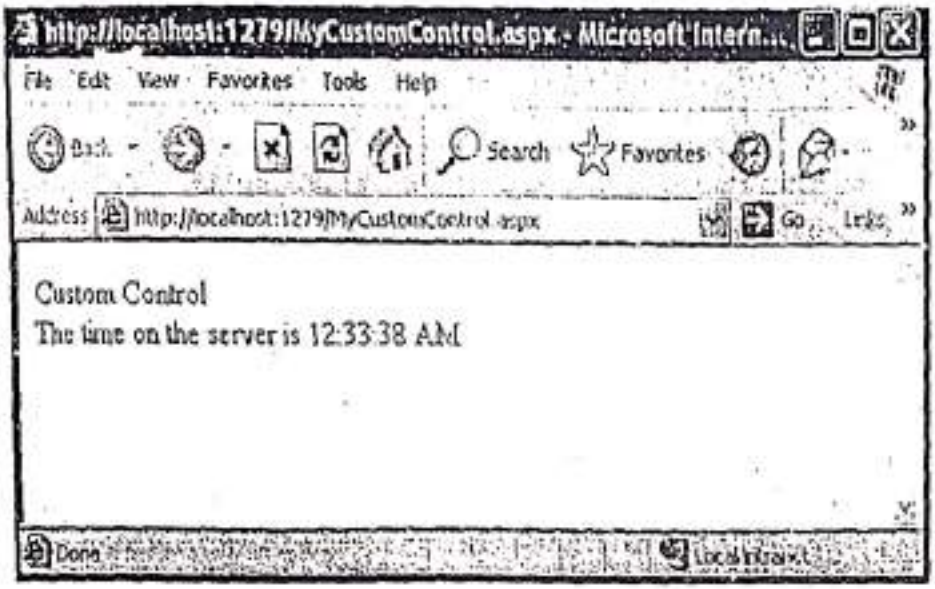
Step-3: Add the custom control in **Toolbox** and then simply drag and drop on the page as we do with the other controls of ASP.NET. The design of the page will look like as follows after adding the custom control to it.

Note: See the Properties Window of the custom control



We can change the value of the Message property of custom control.

Step-4: Press F5. The output will be as follows:



Here's, the end of the **Program 4-28.**

Note: We can use custom controls by using the **Register** page directive that allows us to create an alias for a namespace and also provides ASP.NET with the name of the assembly that contains the control. The example creates the alias **Custom** for the **CustomControls** namespace.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="MyCustomControl.aspx.cs"
        Inherits="UsingCustomControl.MyCustomControl" %>

<%@ Register TagPrefix="Custom" Namespace="CustomControls" Assembly =
"CustomControls" %>
<html xmlns="http://www.w3.org/1999/xhtml">
    <body>
        <form id="form1" runat="server">
                <Custom:FirstControl ID="FirstControl1"
          runat="server"></Custom:FirstControl>
        </form>
    </body>
</html>
```

4.12 VALIDATION CONTROLS

A web page may ask a user for some information and then store it in a back-end database. In almost every case, this information must be validated to ensure that invalid and unauthenticated data do not get stored. Ideally, the validation of user input take place on the

client side so that the user is immediately informed that there is something wrong with the input before the form is posted back to the server. To do so, ASP.NET provides a set of Validation controls that also reduce the headache of writing lengthy code to validate user input.

Validation controls are used to validate form fields before a form is submitted to the web server. The value of a field is compared with the given set of rules. So, if the value of all the fields satisfies their given set of rule only then the form is post back to the server, otherwise, it may display an error message. It means that validation controls prevent users from submitting the wrong type of data into a database table. For example, a user can not submit the value "123" for an Email ID field.

More than one validation control can also be used for the same input control. ASP.NET provides following validation controls.

4.12.1 Using the RequiredFieldValidator Control

RequiredFieldValidator control used to check whether a user entered a value for a required input field and if the input field has no value then it will display an error message set by **ErrorMessage** property of the **RequiredFieldValidator** control. It is generally tied to a text box. The basic syntax to add this control to a web form using source code is as follows:

```
<asp:RequiredFieldValidator ID="RequiredFieldValidator1" runat="server"
    ControlToValidate="TextBox1" ErrorMessage="Field Must Not Be Empty">
</asp:RequiredFieldValidator>
```

Table 4-T lists some commonly used properties of the RequiredFieldValidator control.

Table 4-T : Properties of the RequiredFieldValidator control	
Property	**Description**
ControlToValidate	It indicates the input control which is to be validated.
ErrorMessage	Used to set error message that is to be displayed if the field is empty.

Take a look at the following program.

Program 4-29

Description of the Program 4-29: In this program, a textbox is validated using RequiredFieldValidator. If textbox is empty and user clicks on submit button an error message will be display to inform the user that the field must not be empty. Follow the steps given below.

Step-1: Create a new **Web Application** named "**ValidationControls**".

Step-2: Add a **Web Form** to the application named "**RequiredFieldDemo.aspx**".

Step-3: Create the following design.

Step-4: Set the following properties:

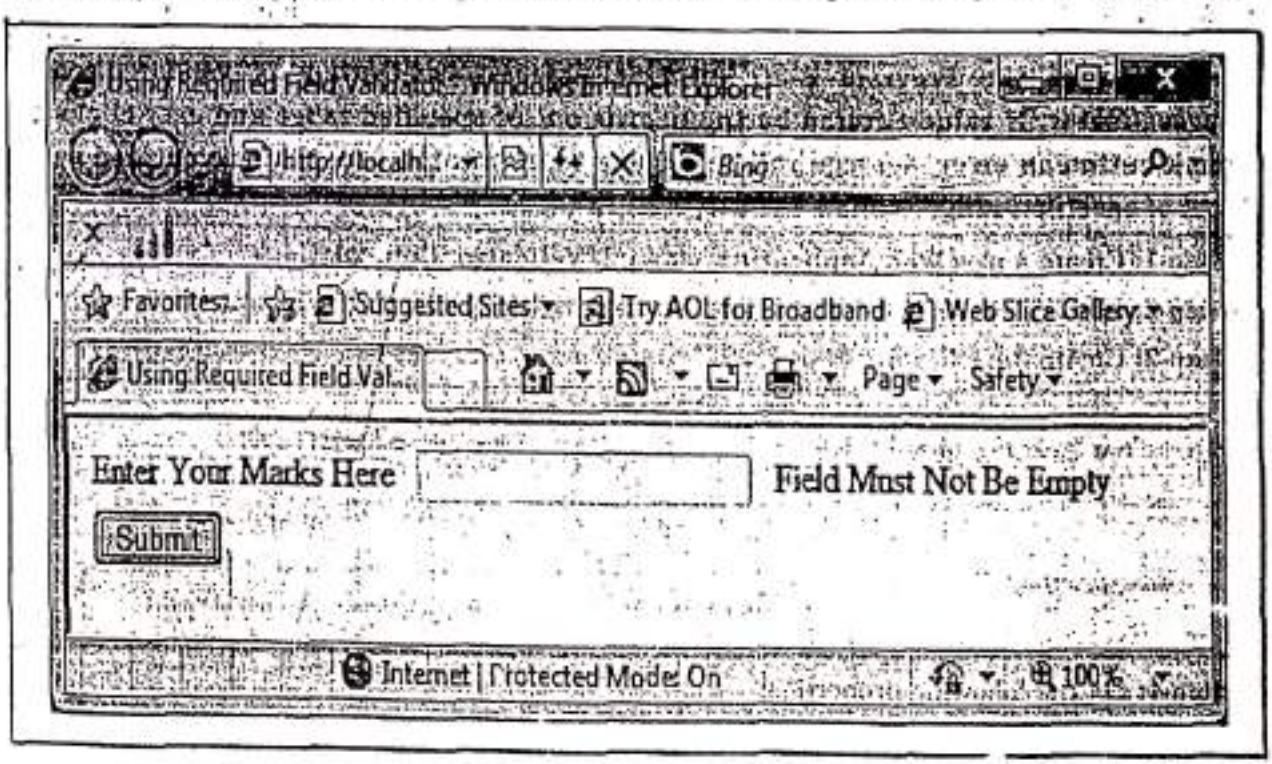
Control	Porperty	Value
RequiredFieldValidator1	ControlToValidate	TextBox1
	ErrorMessage	Field Must Not Be Empty

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
 CodeBehind="RequiredFieldDemo.aspx.cs" Inherits="RequiredFieldDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using Required Field Validator</title>
</head>
<body style="width: 492px">
<form id="form1" runat="server">
<div>
<table>
<tr>
<td>Enter Your Marks Here</td>
<td>
<asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
</td>
<td>
<asp:RequiredFieldValidator ID="RequiredFieldValidator1"
                            runat="server"
                               ControlToValidate="TextBox1"
                            ErrorMessage="Field Must Not Be Empty">
        </asp:RequiredFieldValidator>
</td>
</tr>
```

```
<tr>
<td colspan="3">
<asp:Button ID="btnSubmit" runat="server" Text="Submit" />
</td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-4: Press **F5**. When **TextBox1** is left empty by the user and the user clicks on **submit** button, **RequiredFieldValidator1** will generate an error message. The output will be as follows:



Here's the end of the **Program 4-29.**

4.12.2 Using the RangeValidator Control

The **RangeValidator** control verifies that the input value falls within a certain minimum value and maximum value. The basic syntax to add this control to a web form using **source code** is as follows:

```
<asp:RangeValidator ID="RangeValidator1" runat="server"
    ControlToValidate="TextBox1" ErrorMessage="Enter Value in Range (1-100)"
    MaximumValue="100" MinimumValue="1" Type="Integer">
</asp:RangeValidator>
```

Table 4-U lists some commonly used properties of the RangeValidator control.

Table 4-U : Properties of the RangeValidator control	
Property	Description
ControlToValidate	It indicates the input control which is to be validated.
ErrorMessage	Used to set error message that is to be displayed if the field value does not falls within the given range.
MaximumValue	It specifies the maximum value of the range.
MinimumValue	It specifies the minimum value of the range.
Type	It specifies the type of data in which the range is to be checked.

Take a look at the following program.

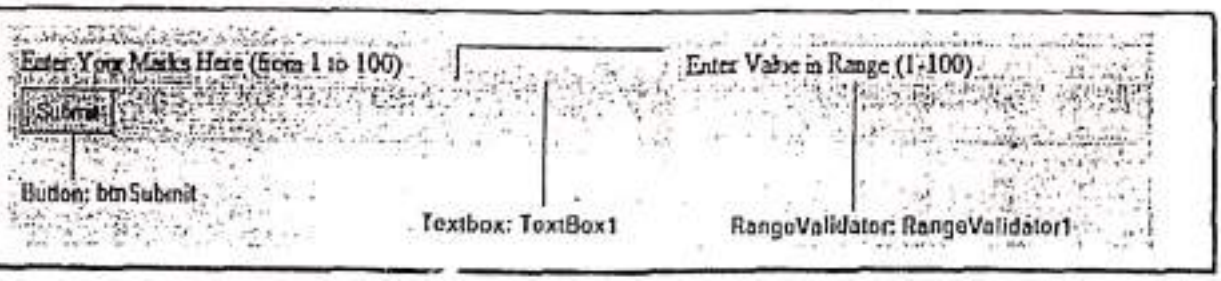
Program 4-30

Description of the Program 4-30: In this program, a textbox is validated using RangeValidator. If value entered by the user is out of specified range and user clicks on submit button an error message will be display to inform the user that the entered value is out of specified range. Follow the steps given below.

Step-1: Create a new **Web Application** named **"ValidationControls"**.

Step-2: Add a **Web Form** to the application named **"RangeValidatorDemo.aspx"**.

Step-3: Create the following design.



Step-4: Set the following properties:

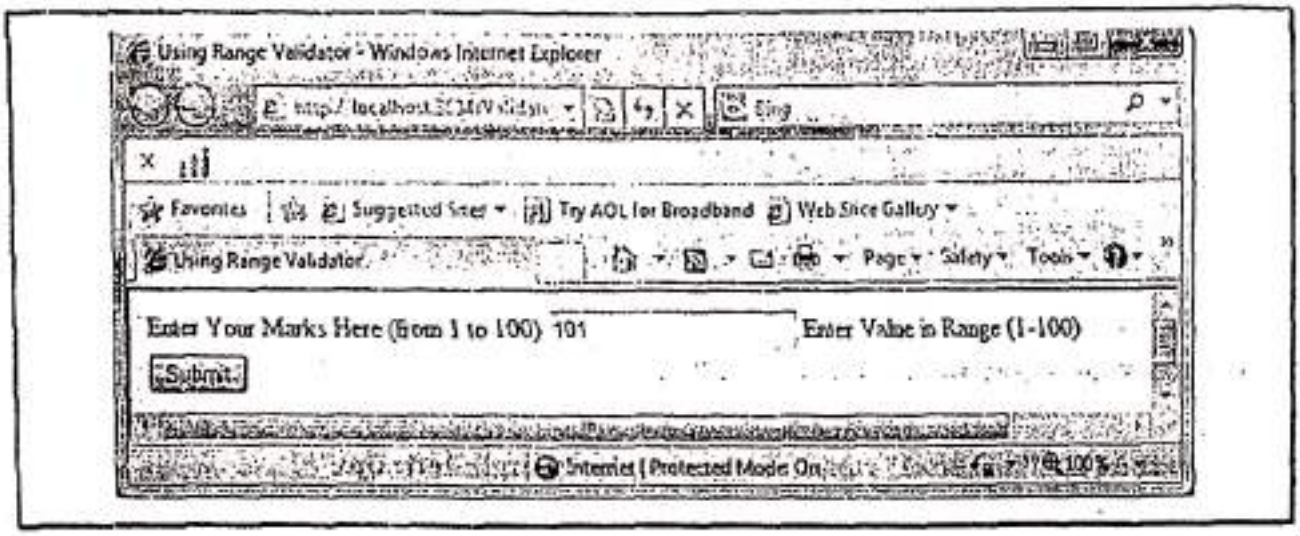
Control	Porperty	Value
RangeValidator1	ControlToValidate	TextBox1
	ErrorMessage	Enter Value in Range (1-100)
	MaximumValue	100
	MinimumValue	1
	Type	Integer

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="RangeValidatorDemo.aspx.cs" Inherits="RangeValidatorDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using Range Validator</title>
</head>
<body style="width: 724px; height: 76px;">
<form id="form1" runat="server">
<div>
<table>
<tr>
<td>Enter Your Marks Here (from 1 to 100)</td>
<td>
<asp:TextBox ID="TextBox1" runat="server" style="margin-left: 0px">
        </asp:TextBox>
</td>
<td>
<asp:RangeValidator ID="RangeValidator1" runat="server"
                ControlToValidate="TextBox1"
                        ErrorMessage="Enter Value in Range (1-100)"
                         MaximumValue="100"
                        MinimumValue="1"
                        Type="Integer">
            </asp:RangeValidator>
</td>
</tr>
<tr>
<td colspan="3">
<asp:Button ID="btnSubmit" runat="server" Text="Submit" />
</td>
</tr>
</table>

</div>
</form>
</body>
</html>
```

Step-5: Press **F5**. When user enters a value in **TextBox1** which is out of range and incompatible type and clicks on submit button, **RangeValidator1** will generate an error message. The output will be as follows:



Here's, the end of the **Program 4-30.**

4.12.3 Using the CompareValidator Control

The **CompareValidator** control compares a value in one control with a fixed value, or, a value in another control. The basic syntax to add this control to a web form using **source code** is as follows:

```
<asp:CompareValidator ID="CompareValidator1" runat="server"
                ControlToCompare="TextBox1" ControlToValidate="TextBox2"
                ErrorMessage="Text Does Not Match">
</asp:CompareValidator>
```

Table 4-V lists some commonly used properties of the CompareValidator control.

Table 4-V : Properties of the CompareValidator control	
Property	**Description**
ControlToCompare	It specifies the input control to compare with.
ControlToValidate	It indicates the input control which is to be validated.
ErrorMessage	Used to set error message that is to be displayed if the field value does not match with the value of the input control (**ControlToCompare**)or the constant value (**ValueToCompare**).
Operator	It specifies the comparison operator, the available operators are: Equal, NotEqual, GreaterThan, GreaterThanEqual, LessThan, LessThanEqual and DataTypeCheck.
ValueToCompare	It specifies the constant value to compare with.

Take a look at the following program

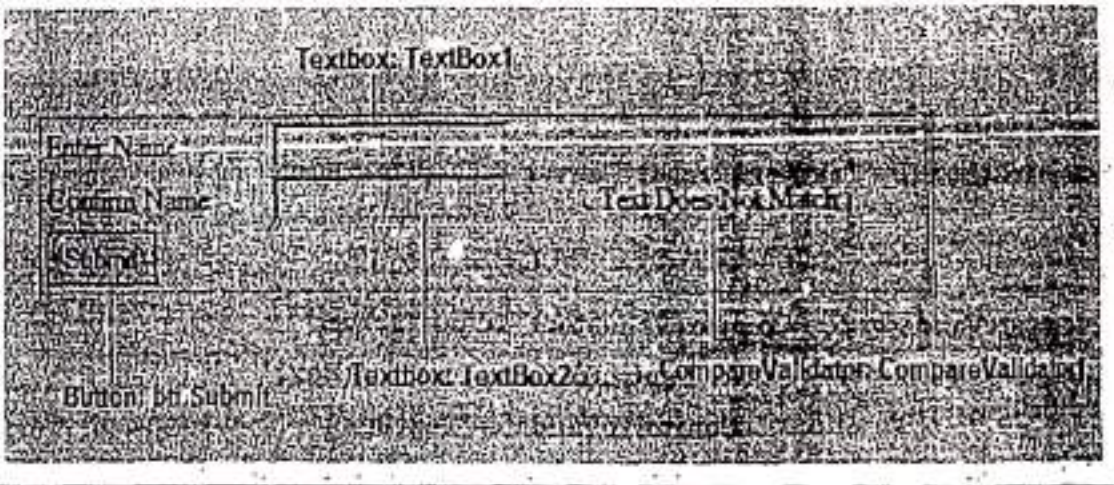
Program 4-31

Description of the Program 4-31: This program demonstrates the use of CompareValidator control. Follow the steps given below.

Step-1: Create a new **Web Application** named "**ValidationControls**".

Step-2: Add a **Web Form** to the application named "**CompareValidatorDemo.aspx**".

Step-3: Create the following design.



Step-4: Set the following properties:

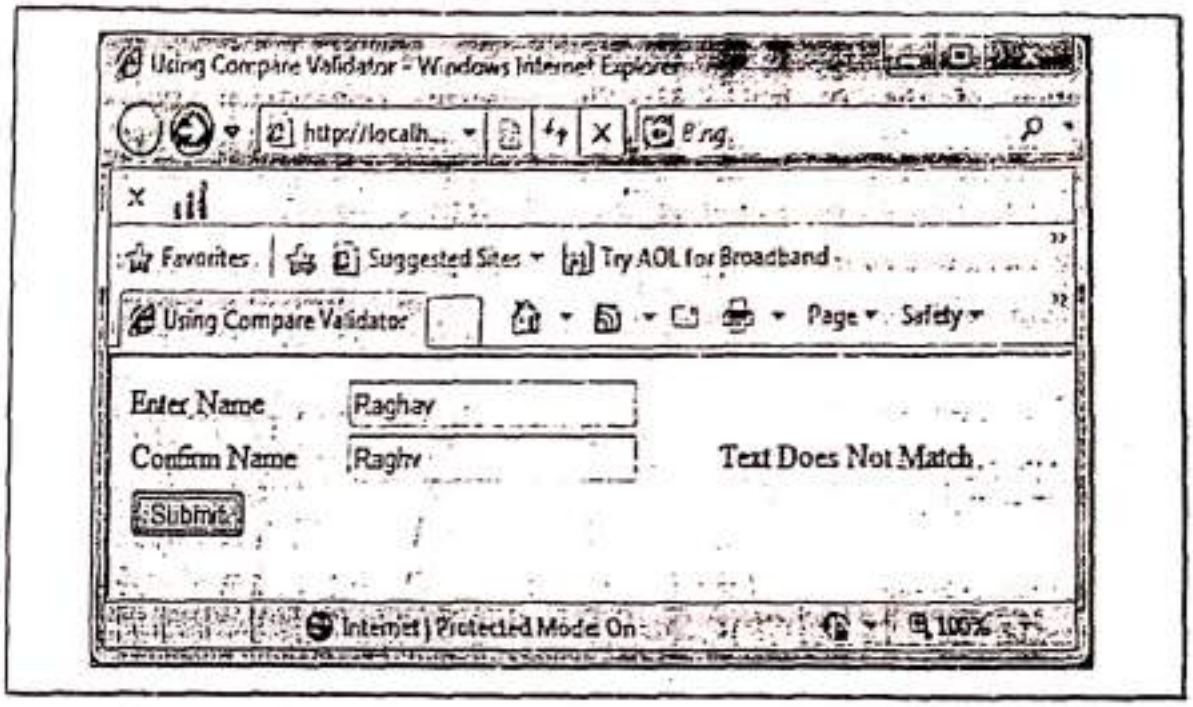
Control	Porperty	Value
CompareValidator1	ControlToCompare	TextBox1
	ControlToValidate	TextBox2
	ErrorMessage	Text Does Not Match

Source of this is as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="CompareValidatorDemo.aspx.cs" Inherits="CompareValidatorDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using Compare Validator</title>

</head>
<body style="width: 492px">
```

```
m id="form1" runat="server">
>
ble>
>
>Enter Name</td>
><asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
d>
> </td>
>
><td>Confirm Name</td>
><asp:TextBox ID="TextBox2" runat="server"></asp:TextBox>
d>
>
p:CompareValidator ID="CompareValidator1" runat="server"
                                ControlToCompare="TextBox1"
                              ControlToValidate="TextBox2"
                                 ErrorMessage="Text Does Not Match">
    </asp:CompareValidator>
d>
r>
>
colspan="3">
p:Button ID="btnSubmit" runat="server" Text="Submit"
lick="btnSubmit_Click" />
d>
r>
able>
div>
form>
body>
html>
```

Step-5: Press F5. If user enters different values in **TextBox1** and **TextBox2** and clicks on Submit button then **CompareValidator1** will generate an error message. The output will be as follows:



Here's, the end of the **Program 4-31.**

4.12.4 Using the RegularExpressionValidator Control

The **RegularExpressionValidator** allows validating the input text by matching it against a regular expression. There is a set of regular expression in the **ValidationExpression** property. The basic syntax to add this control to a web form using **source code** is as follows:

```
<asp:RegularExpressionValidator ID="RegularExpressionValidator1" runat="server"
    ControlToValidate="TextBox1" ErrorMessage="Regular Expression Does Not Match"
    ValidationExpression="\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*">
</asp:RegularExpressionValidator>
```

Here's the **ValidationExpression** is for **Internet E-Mail address.**

Table 4-W lists some commonly used properties of the RegularExpressionValidator control.

Table 4-W : Properties of the RegularExpressionValidator control	
Property	**Description**
ControlToValidate	It indicates the input control which is to be validated.
ErrorMessage	Used to set error message that is to be displayed if the field value expression does not match with the specified **ValidationExpression.**
ValidationExpression	Used to specify the pattern to be matched with the pattern of input control value.

Take a look at the following program.

ogram 4-32

escription of the Program 4-32: This program demonstrates the use of egularExpressionValidator control. Follow the steps given below.

Step-1: Create a new **Web Application** name: "**ValidationControls**".

Step-2: Add a **Web Form** to the application named "**RegularExpressionDemo.aspx**".

Step-3: Create the following design.

Step-4: Set the following properties:

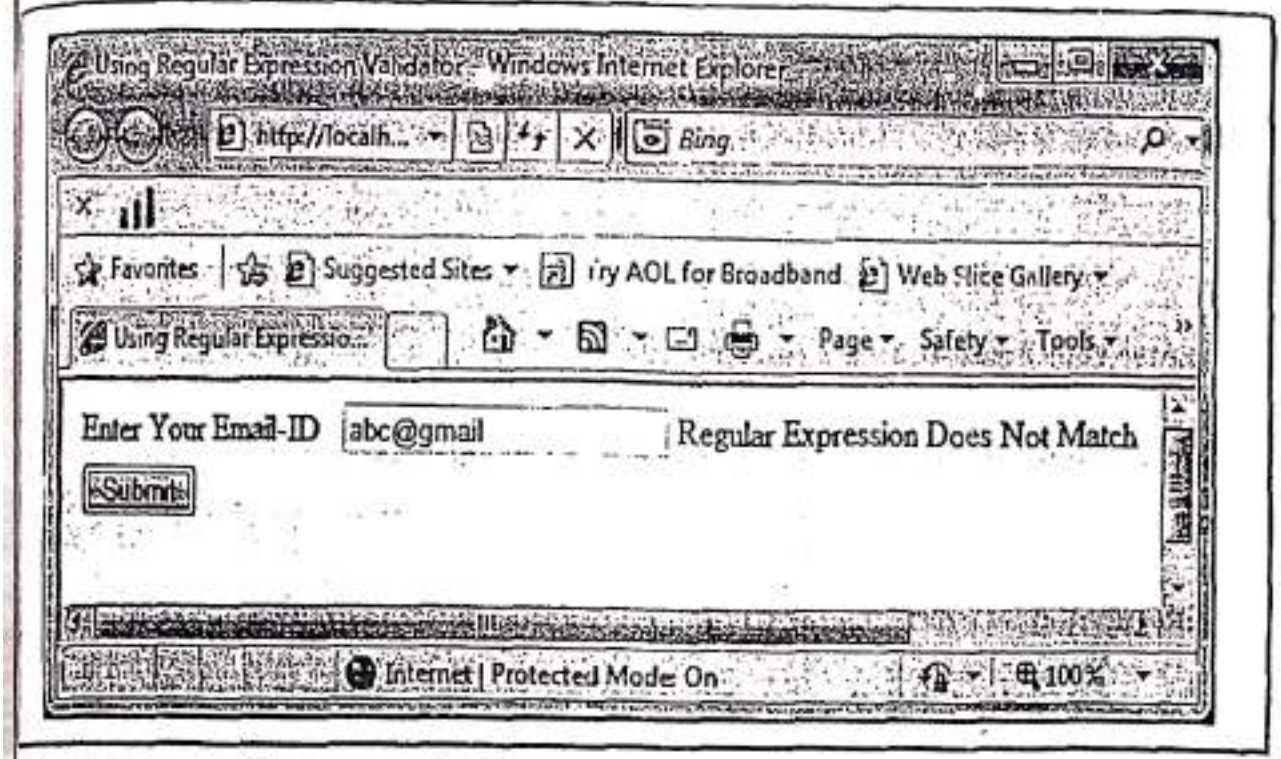
ontrol	Porperty	Value
gularExpression	ControlToValidate	TextBox3
	ErrorMessage	Regular Expression Does Not Match
	ValidationExpression	\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)
		Internet e-mail address

Source of this is as follows:

```
&@ Page Language="C#" AutoEventWireup="true"
deBehind="RegularExpressionDemo.aspx.cs" Inherits="RegularExpressionDemo" %>

OCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
tp://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

tml xmlns="http://www.w3.org/1999/xhtml">
ead runat="server">
tle>Using Regular Expression Validator</title>

ead>
```

```
<body style="width: 675px">
<form id="form1" runat="server">
<div>

<table>
<tr>
<td>Enter Your Email-ID  </td>
<td>
<asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
</td>
<td>
<asp:RegularExpressionValidator ID="RegularExpressionValidator1"
        runat="server"
        ControlToValidate="TextBox1"
        ErrorMessage="Regular Expression Does Not Match"
        ValidationExpression="\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*">
        </asp:RegularExpressionValidator>
</td>
</tr>
<tr>
<td colspan="3">
<asp:Button ID="btnSubmit" runat="server" Text="Submit" />
</td>
</tr>
</table>

</div>
</form>
<p> </p>
</body>
</html>
```

Step-4: Press F5. If user enters an invalid Email-ID in TextBox1 and clicks on submit button then RegularExpressionValidator1 will generate an error message. The output will be as follows:



Here's, the end of the **Program 4-32.**

12.5 Using the CustomValidator Control

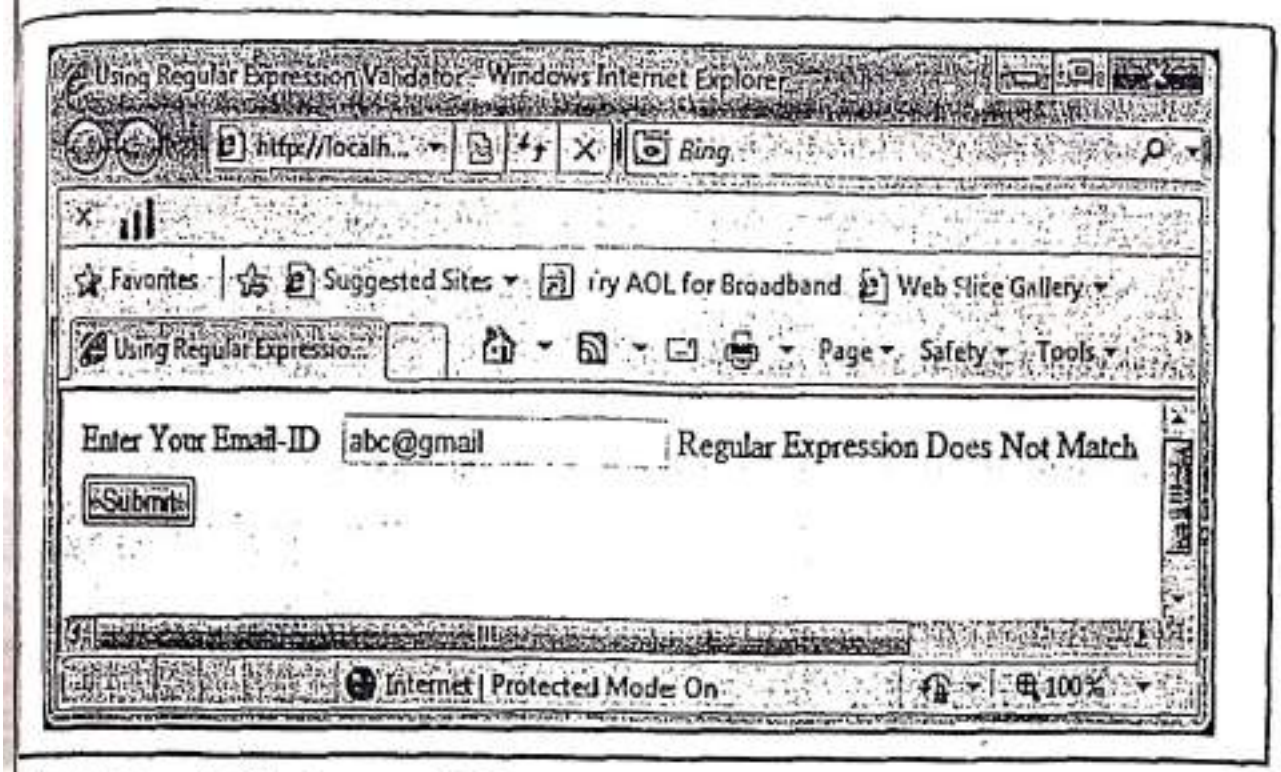
The **CustomValidator** control allows writing application specific custom validation routines for both the client side and the server side validation. The basic syntax to add this control to a web form using **source code** is as follows:

```
<asp:CustomValidator id="CustomValidator1" ControlToValidate="TextBox1"
    ErrorMessage="Custom Validation Does Not Meet"
    ClientValidationFunction="Function_ClientValidate"
    OnServerValidate="Function_ServerValidate" runat="server">
</asp:CustomValidator>
```

Table 4-X lists some commonly used properties of the CustomValidator control.

Table 4-X : Properties of the CustomValidator control	
Property	**Description**
ControlToValidate	It indicates the input control which is to be validated.
ErrorMessage	Used to set error message that is to be displayed if the field value does not satisfies the client side validation or server side validation
ClientValidationFunction	Access function of client side validation.
OnServerValidate	Access function of server side validation.

Take a look at the following program.



Here's, the end of the **Program 4-32.**

12.5 Using the CustomValidator Control

The **CustomValidator** control allows writing application specific custom validation routines for both the client side and the server side validation. The basic syntax to add this control to a web form using **source code** is as follows:

```
<asp:CustomValidator id="CustomValidator1" ControlToValidate="TextBox1"
    ErrorMessage="Custom Validation Does Not Meet"
    ClientValidationFunction="Function_ClientValidate"
    OnServerValidate="Function_ServerValidate" runat="server">
</asp:CustomValidator>
```

Table 4-X lists some commonly used properties of the CustomValidator control.

Table 4-X : Properties of the CustomValidator control	
Property	**Description**
ControlToValidate	It indicates the input control which is to be validated.
ErrorMessage	Used to set error message that is to be displayed if the field value does not satisfies the client side validation or server side validation
ClientValidationFunction	Access function of client side validation.
OnServerValidate	Access function of server side validation.

Take a look at the following program.

Program 4-33

Description of the Program 4-33: This program demonstrate the use of CustomValidatorControl. Follow the steps given below.

Step-1: Create a new **Web Application** named "**ValidationControls**".

Step-2: Add a **Web Form** to the application named "**CustomValidationDemo.aspx**".

Step-3: Create the following design.

Step-4: Set the following properties:

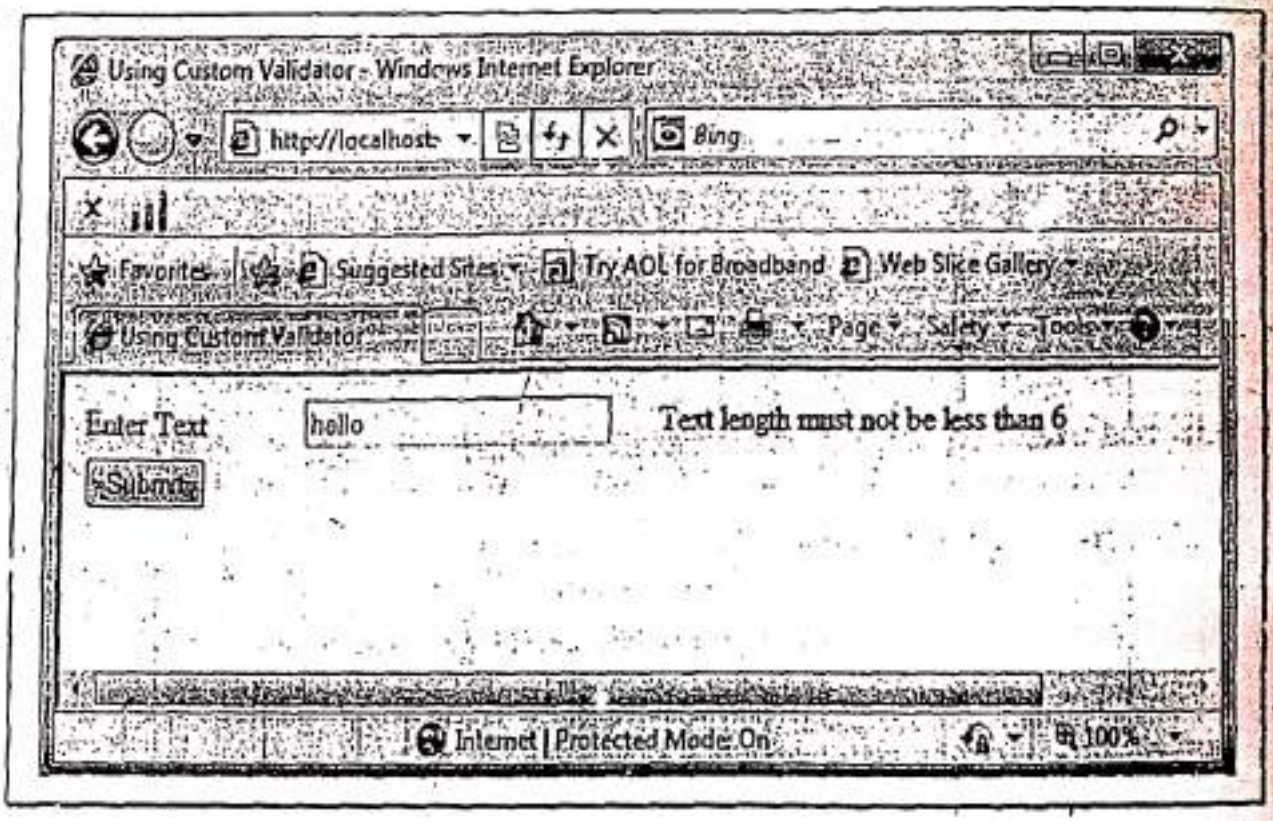
Control	Porperty	Value
CustomValidator1	ControlToValidate	TextBox1
	ErrorMessage	Text length must not be less than 6
	ServerValidate **Note:** It's an event	CustomServerValidate

Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="CustomValidationDemo.aspx.cs" Inherits="CustomValidationDemo" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using Custom Validator</title>

<script runat="server">
void CustomServerValidate(Object source, ServerValidateEventArgs args)
  {
if (args.Value.Length < 6)
      args.IsValid = false;
```

```
o
        args.IsValid = true;

cript>
ead>
dy style="width: 675px">
m id="form1" runat="server">
v>

ble>
>
>Enter Text</td>
>
p:TextBox ID="TextBox1" runat="server"></asp:TextBox>
d>
>
p:CustomValidator ID="CustomValidator1" runat="server"
                    OnServerValidate="CustomServerValidate"
                ControlToValidate="TextBox1"
                ErrorMessage="Text length must not be less than 6" >
  </asp:CustomValidator>
d>
r>

 colspan="3">
p:Button ID="btnSubmit" runat="server" Text="Submit"  />
d>
>
able>

iv>
rm>
dy>
tml>
```

Step-5: Press **F5**. When user enter text in **TextBox1** whose length is less than 6, and the user clicks on **submit** button, **CustomValidator1** will generate an error message. The output will be as follows:



Here's, the end of the **Program 4-33.**

4.12.6 Using the ValidationSummary Control

The **ValidationSummary** control does not perform any validation but shows a summary of all errors in the page. The basic syntax to add this control to a web form using **source code** is as follows:

```
<asp:ValidationSummary ID="ValidationSummary1" runat="server" />
```

Take a look at the following program.

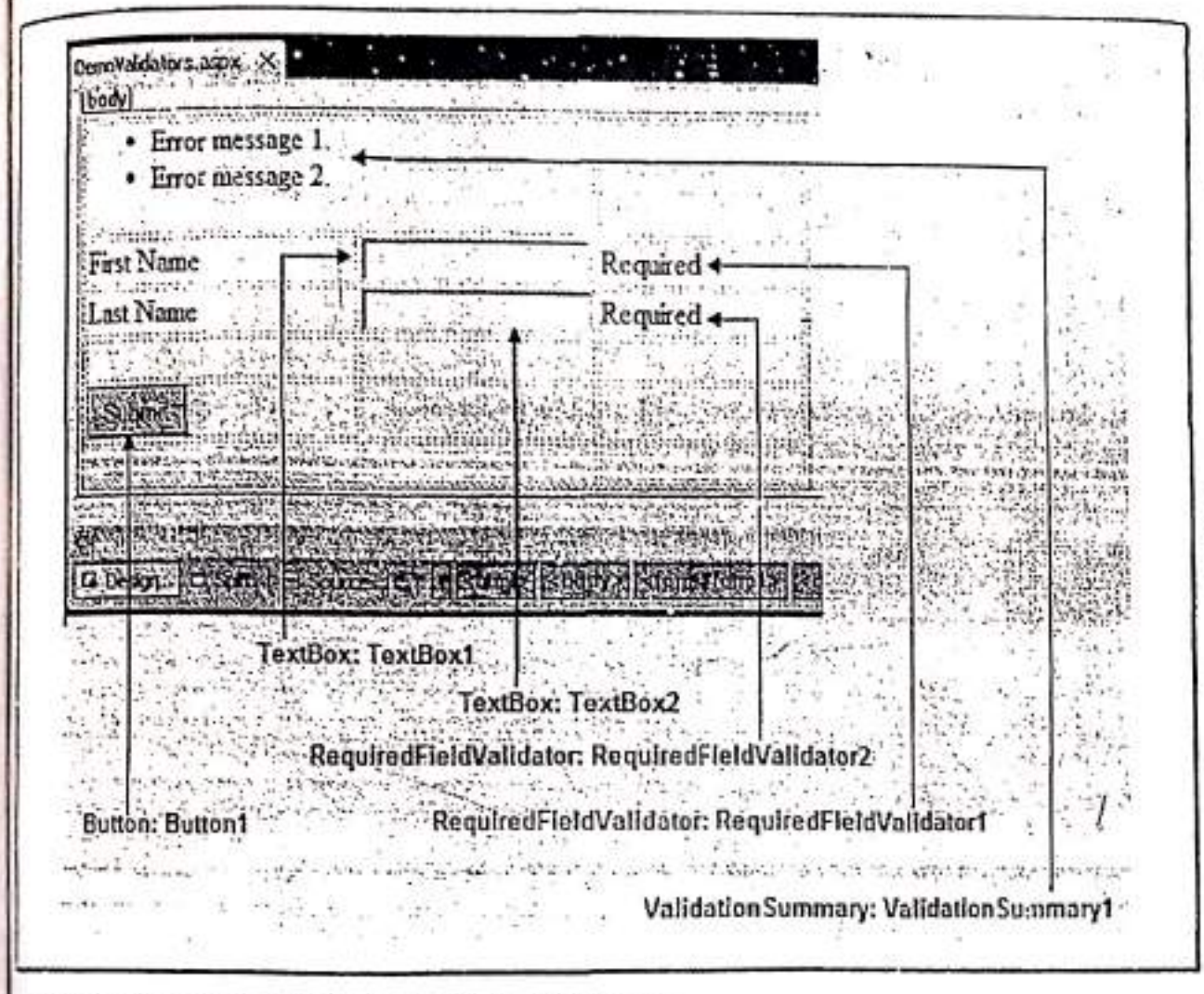
Program 4-34

Description of the Program 4-34: This program demonstrates the use of ValidationSummary controls. Follow the steps given below.

Step-1: Create a new **Web Application** named **"ValidationControls"**.

Step-2: Add a **Web Form** to the application named **"SummaryValidatorDemo.aspx"**.

Step-3: Create the following design.



Step-4: Set following properties of various Controls.

Control	Porperty	Value
RequiredFieldValidator1	ControlToValidate	TextBox1
	ErrorMessage	First Name Required
	Text	Required
RequiredFieldValidator2	ControlToValidate	TextBox2
	ErrorMessage	Last Name Required
	Text	Required
Button1	Text	Submit

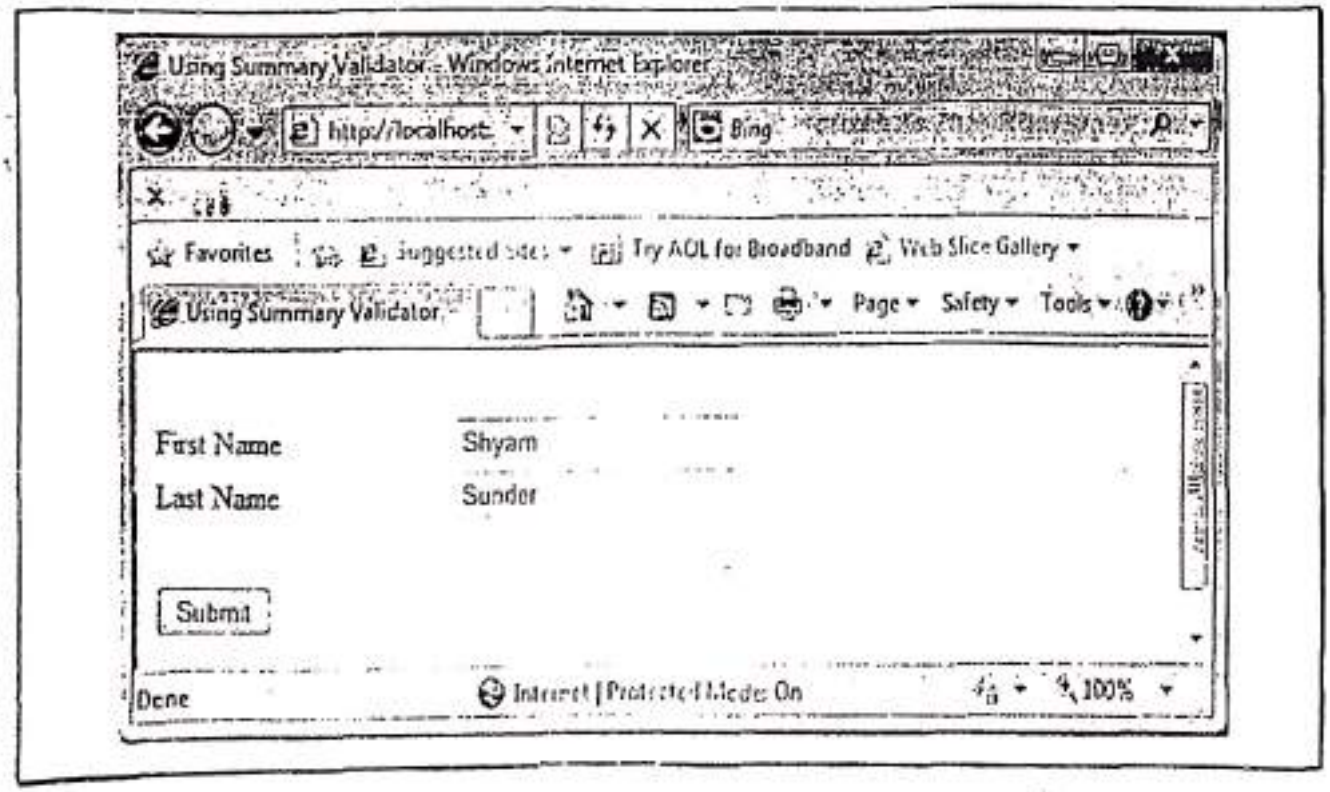
Source will be as follows:

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="SummaryValidatorDemo.aspx.cs" Inherits="_Default" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Using Summary Validator</title>
</head>
<body>
<form id="form1" runat="server">
<div>
<table>
<tr>
<td colspan="2">
<asp:ValidationSummary ID="ValidationSummary1" runat="server" />
</td>
<td> </td>
</tr>
<tr>
<td>First Name</td>
<td>
<asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
</td>
<td>
<asp:RequiredFieldValidator ID="RequiredFieldValidator1" runat="server"
                            ControlToValidate="TextBox1"
                           ErrorMessage="First Name Required">Required
      </asp:RequiredFieldValidator>
</td>
</tr>
<tr>
<td>Last Name</td>
<td>
            <asp:TextBox ID="TextBox2" runat="server"></asp:TextBox>
</td>
<td>
<asp:RequiredFieldValidator ID="RequiredFieldValidator2" runat="server"
                            ControlToValidate="TextBox2"
                           ErrorMessage="Last Name Required">Required
      </asp:RequiredFieldValidator>
</td>
```

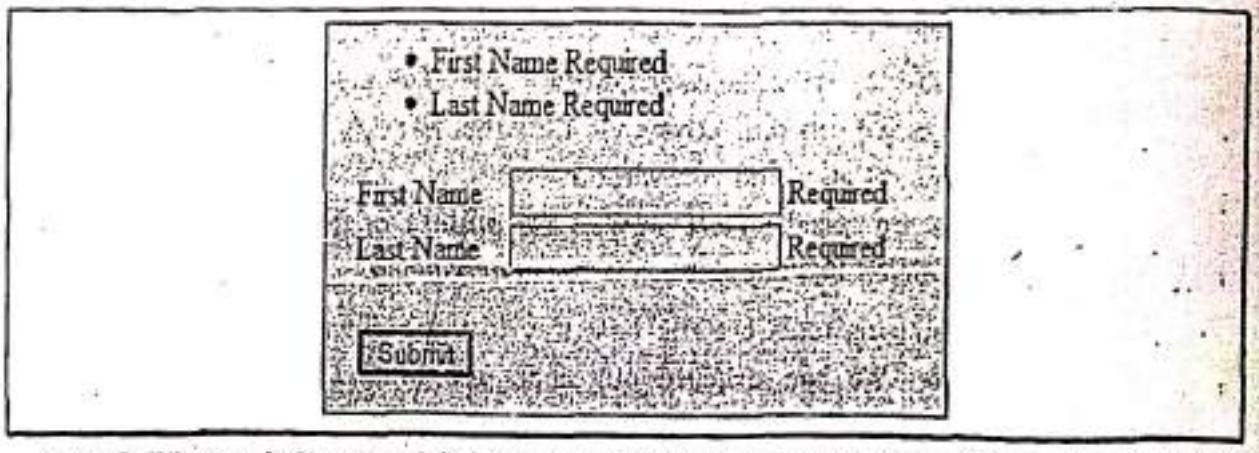
```
</tr>
<tr>
<td></td>
<td></td>
<td> </td>
</tr>
<tr>
<td>
<asp:Button ID="Button1" runat="server" Text="Submit" />
</td>
<td> </td>
<td> </td>
</tr>
<tr>
<td> </td>
<td> </td>
<td> </td>
</tr>
</table>
</div>
</form>
</body>
</html>
```

Step-5: Press **F5**. The output will be as follows:

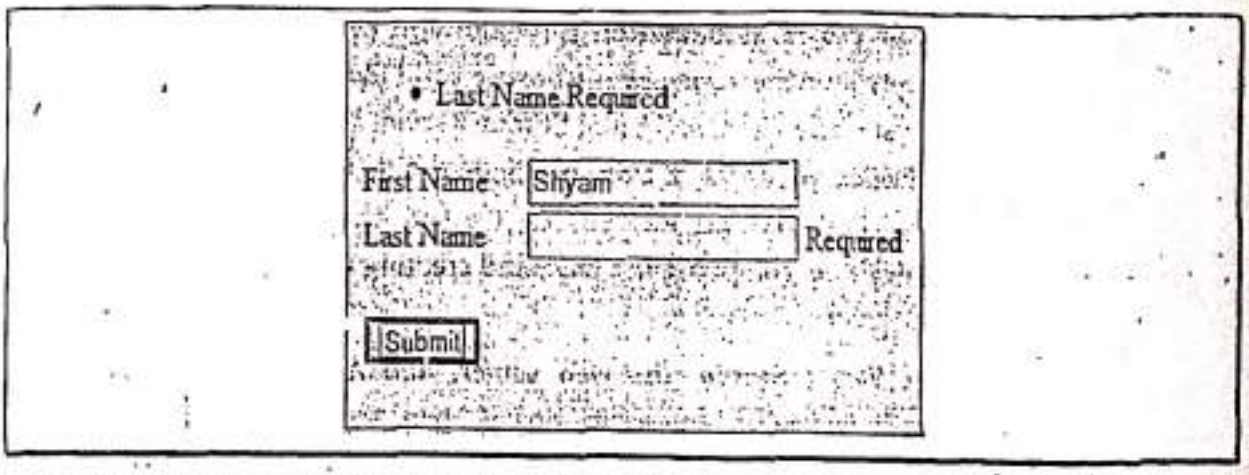


Since both the fields have values in the given example so, there is no error message generated by any of the validation control. Now, we are going to discuss some cases when one or both the fields are left empty by the user. These are as follows:

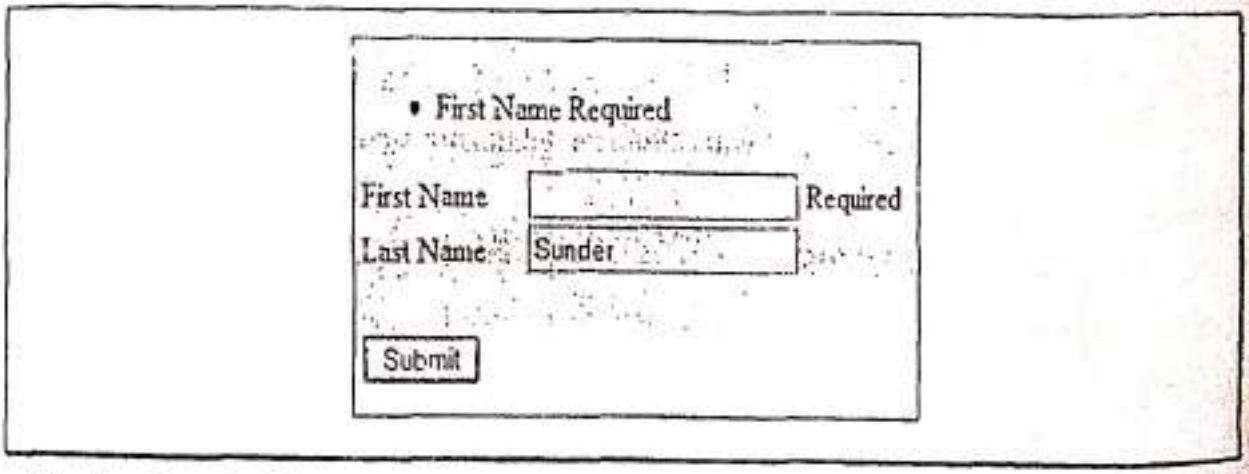
Case-1: When both the textboxesareleft empty by the user and the user clicks on the button. The output will be as follows:



Case-2: When only **TextBox2** is left empty by the user and the user clicks on the button. The output will be as follows:



Case-3: When only **TextBox1** is left empty by the user and the user clicks on the button. The output will be as follows:



Here's, the end of the **Program 4-34.**

13 DEBUGGING ASP .NET PAGES

In the above sections, we have discussed how to design and develop an ASP.NET application using Web Forms. After designing and developing the application, it becomes critical to check for the desired functionality. While we are developing the application, the code editor catches out syntax errors. However, the errors that cannot be caught during application development cause the application to display error messages at run time. The errors that occur while the application is running are called run-time errors. On the other hand, if there is a problem in the programming logic, the application would run without errors, but it will not provide the desired functionality. Such errors are called logical errors. *The process of going through the code to identify the root cause of an error in an application is called debugging.*

The Visual Studio debugging tools are the best way to track down mysterious errors and diagnose strange behavior. We can execute our code one line at a time, set intelligent breakpoints that we can save for later use, and view current in-memory information at any time.

To debug a specific web page in Visual Studio, select that web page in the Solution Explorer, and click the Start Debugging button on the toolbar ( ). What happens next depends on the location of our project. If our project is stored on a remote web server or a local IIS virtual directory, Visual Studio simply launches our default browser and directs us to the appropriate URL. If we have used a file system application, Visual Studio starts its integrated web server on a dynamically selected port (which prevents it from conflicting with IIS, if it's installed). Then Visual Studio launches the default browser and passes it a URL that points to the local web server. Either way, the real work i.e. compiling the page and creating the page objects, is passed along to the ASP.NET worker process.

The separation between Visual Studio, the web server, and ASP.NET allows for a few interesting tricks. For example, while our browser window is open, we can still make changes to the source code and design of our web pages. Once we have completed our changes, just save the page, and click the Refresh button in our browser to re-request it. Although we will always be forced to restart the entire page to see the results of any changes we make, it's still more convenient than rebuilding our whole project.

Fixing and restarting a web page is handy, but what about when we need to track down an elusive error? In these cases, we need Visual Studio's debugging smarts, which are described in the following sections.

13.1 Single-Step Debugging

Single-step debugging allows us to execute our code's one line at a time. It's incredibly easy to use. Just follow the steps given below:

Step- 1: Find a location in code where we want to pause execution, and start single-step debugging (we can use any executable line of code but it must not be a variable declaration,

13 DEBUGGING ASP .NET PAGES

In the above sections, we have discussed how to design and develop an ASP.NET application using Web Forms. After designing and developing the application, it becomes critical to check for the desired functionality. While we are developing the application, the code editor catches out syntax errors. However, the errors that cannot be caught during application development cause the application to display error messages at run time. The errors that occur while the application is running are called run-time errors. On the other hand, if there is a problem in the programming logic, the application would run without errors, but it will not provide the desired functionality. Such errors are called logical errors. *The process of going through the code to identify the root cause of an error in an application is called debugging.*

The Visual Studio debugging tools are the best way to track down mysterious errors and diagnose strange behavior. We can execute our code one line at a time, set intelligent breakpoints that we can save for later use, and view current in-memory information at any time.

To debug a specific web page in Visual Studio, select that web page in the Solution Explorer, and click the Start Debugging button on the toolbar ( ). What happens next depends on the location of our project. If our project is stored on a remote web server or a local IIS virtual directory, Visual Studio simply launches our default browser and directs us to the appropriate URL. If we have used a file system application, Visual Studio starts its integrated web server on a dynamically selected port (which prevents it from conflicting with IIS, if it's installed). Then Visual Studio launches the default browser and passes it a URL that points to the local web server. Either way, the real work i.e. compiling the page and creating the page objects, is passed along to the ASP.NET worker process.

The separation between Visual Studio, the web server, and ASP.NET allows for a few interesting tricks. For example, while our browser window is open, we can still make changes to the source code and design of our web pages. Once we have completed our changes, just save the page, and click the Refresh button in our browser to re-request it. Although we will always be forced to restart the entire page to see the results of any changes we make, it's still more convenient than rebuilding our whole project.

Fixing and restarting a web page is handy, but what about when we need to track down an elusive error? In these cases, we need Visual Studio's debugging smarts, which are described in the following sections.

13.1 Single-Step Debugging

Single-step debugging allows us to execute our code's one line at a time. It's incredibly easy to use. Just follow the steps given below:

Step- 1: Find a location in code where we want to pause execution, and start single-step debugging (we can use any executable line of code but it must not be a variable declaration,

comment, or blank line). Click in the margin before line of code, and a red breakpoint will appear (see Figure 4.16).

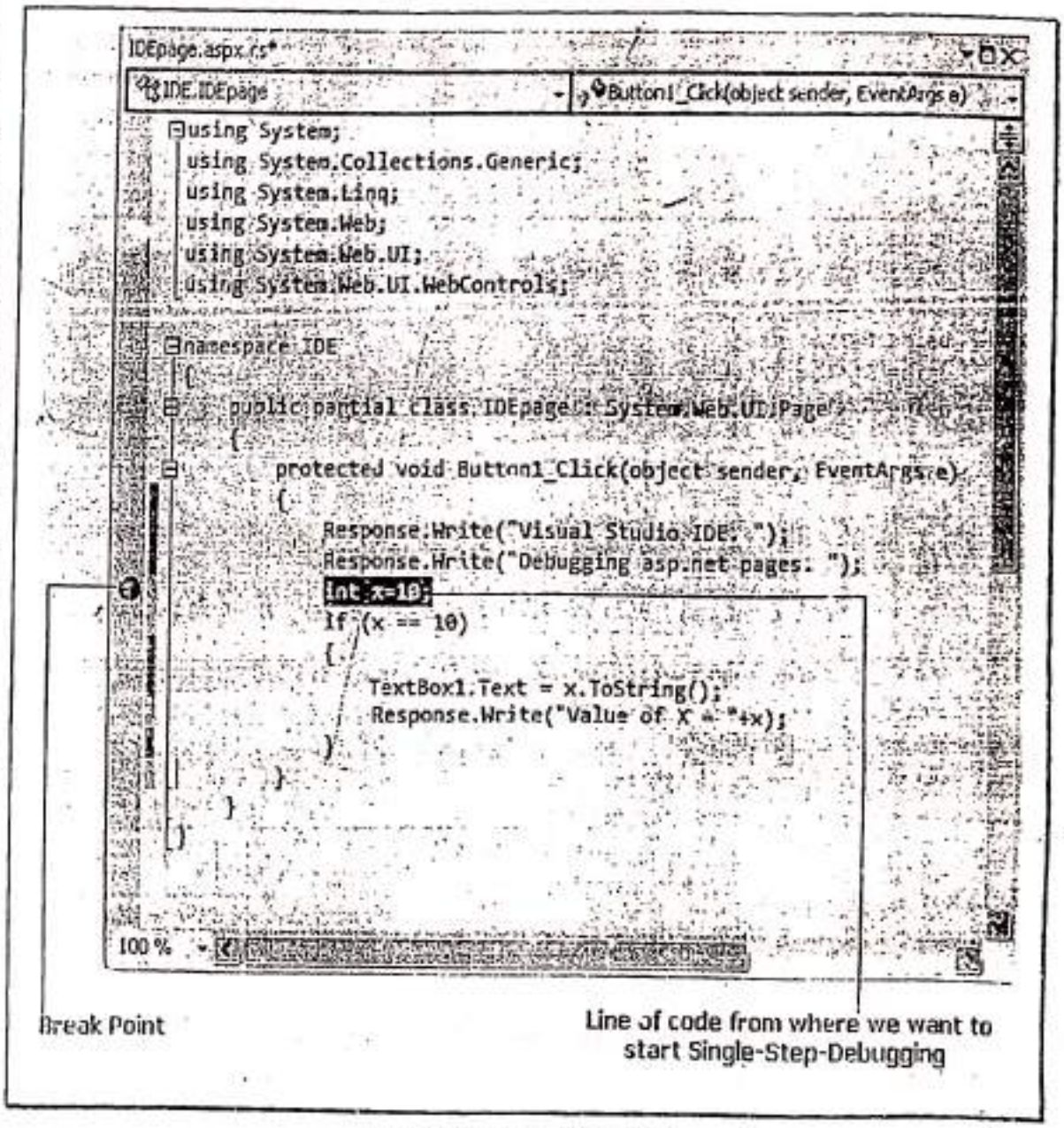


Figure 4.16: Setting a breakpoint

Step-2: Start program as we would ordinarily. When the program reaches our breakpoint, execution will pause, and we will be switched back to the Visual Studio code window. The breakpoint statement won't be executed.

Step-3: At this point, we have several options. We can execute the current line by pressing **F11**. The following line in our code will be highlighted with yellow color and the breakpoint will be with a yellow arrow, indicating that this is the next line that will be executed. We can continue

ke this through our program, running one line at a time by pressing F11 and following the ode's path of execution. Or, we can exit break mode and resume running our code by pressing '5.

Step-4: Whenever the code is in break mode, we can hover over variables to see their current ontents. This allows us to verify that variables contain the values we expect **(see Figure 4.17).**

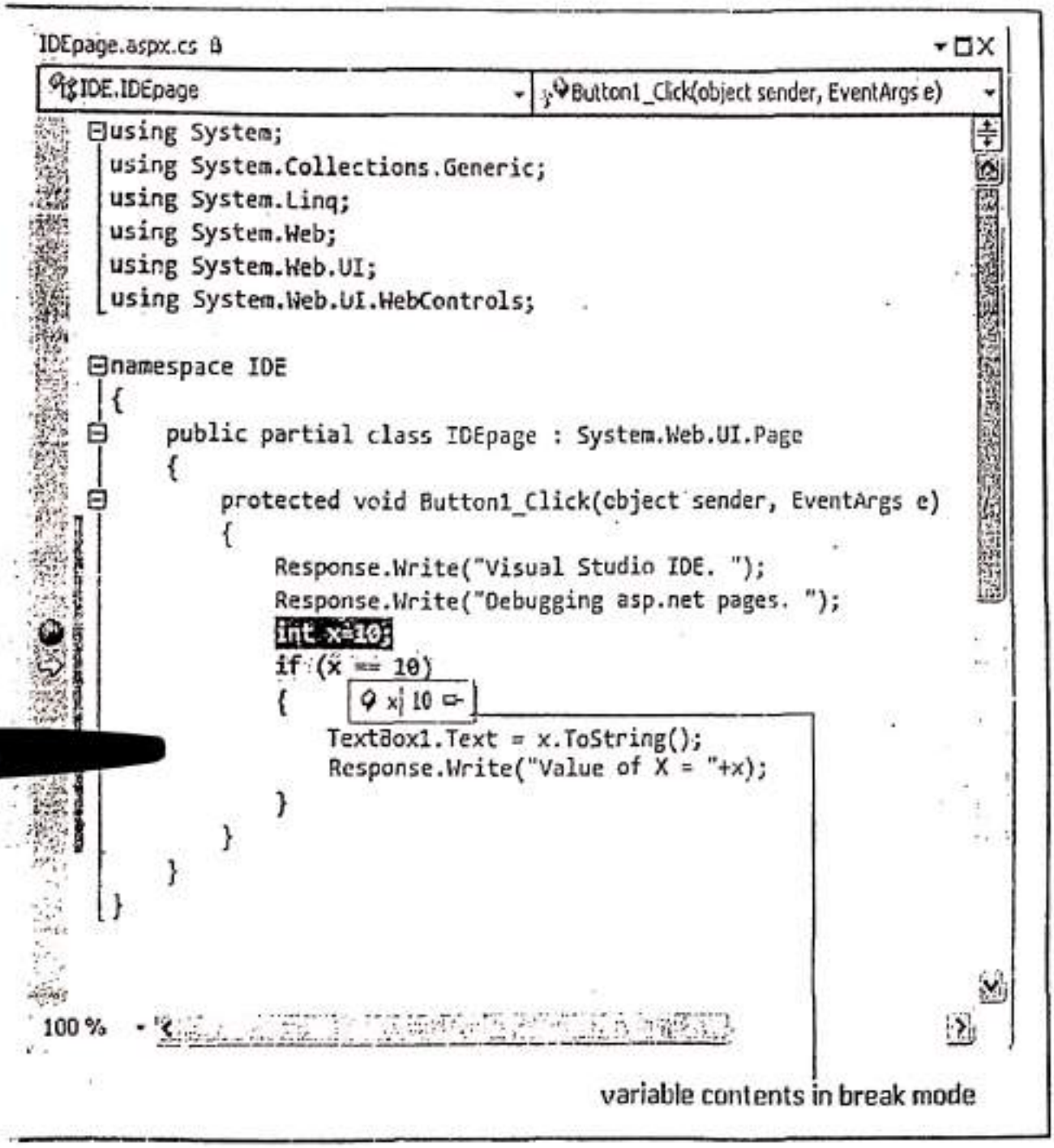


Figure 4.17: View Variable contents in break mode

If we hover over an object we can drill down into all the individual properties by clicking a nall plus symbol to expand it **(see Figure 4.18)**

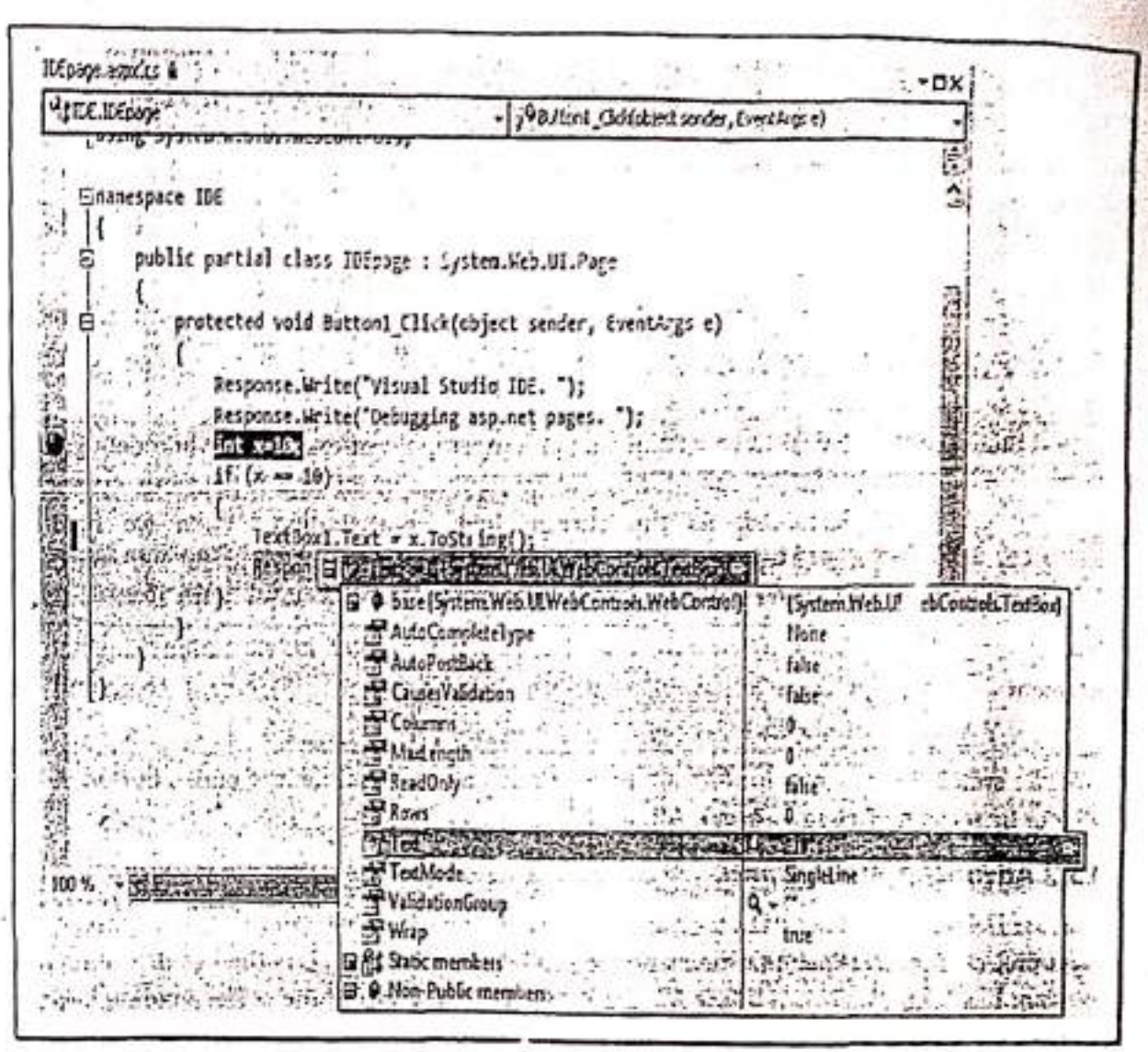


Figure 4.18: Viewing object properties in break mode

5. We can also use any of the commands listed in **Table 4-Y1** while in break mode. These commands are available from the context menu by right-clicking the code window or by using the associated hot key.

Table 4-Y1: Commands Available in Break Mode

Command (Hot Key)	Description
Step into (F11)	Executes the currently highlighted line and then pauses. If the currently highlighted line calls a procedure, execution will pause at the first executable line inside the method or function (which is why this feature is called **stepping into**).
Step Over (F10)	The same as Step Into, except that it runs procedures as though they are a single line. If we select the Step Over command while a procedure call is highlighted, the entire procedure will be executed. Execution will pause at the next executable statement in the current procedure.

Step Out (Shift+F11)	Executes all the code in the current procedure and then pauses at the statement that immediately follows the one that called this method or function. In other words, this allows us to step "out" of the current procedure in one large jump.
Continue (F5)	Resumes the program and continues to run it normally without pausing until another breakpoint is reached.
Run to Cursor	Allows us to run all the code up to a specific line (where our cursor is currently positioned). We can use this technique to skip a time consuming loop.
Set Next Statement	Allows us to change our program's path of execution while debugging. It causes our program to mark the current line (where our cursor is positioned) as the current line for execution. When we resume execution, this line will be executed, and the program will continue from that point.
Show Next Statement	Moves focus to the line of code that is marked for execution. This line is marked by a yellow arrow. The Show Next Statement command is useful if we lose our place while editing.

We can switch our program into break mode at any point by clicking the pause button in the toolbar or by selecting **Debug >>Break All**.

13.2 Advanced Breakpoints

Choose Debug >>Windows >>Breakpoints to see a window that lists all the breakpoints in our current project. The Breakpoints window provides a hit count, showing us the number of times a breakpoint has been encountered **(see Figure 4.19)**. We can jump to the corresponding location in code by double-clicking a breakpoint. We can also use the Breakpoints window to disable a breakpoint without removing it. That allows us to keep a breakpoint to use in testing later, without leaving it active. Breakpoints are automatically saved with the hidden solution file.

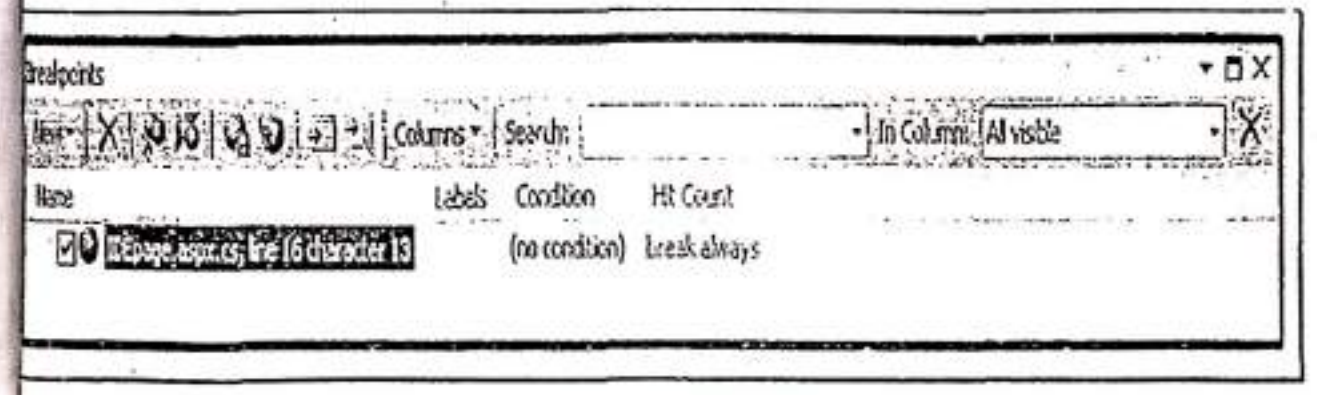


Figure 4.19: The Breakpoints window

Visual Studio allows us to customize breakpoints so they occur only if certain conditions are true. To customize a breakpoint, right-click it, and we can take one of the following actions:

- **Location:** Break execution when the program reaches the location in a file as shown in the following figure (Figure 4.20).

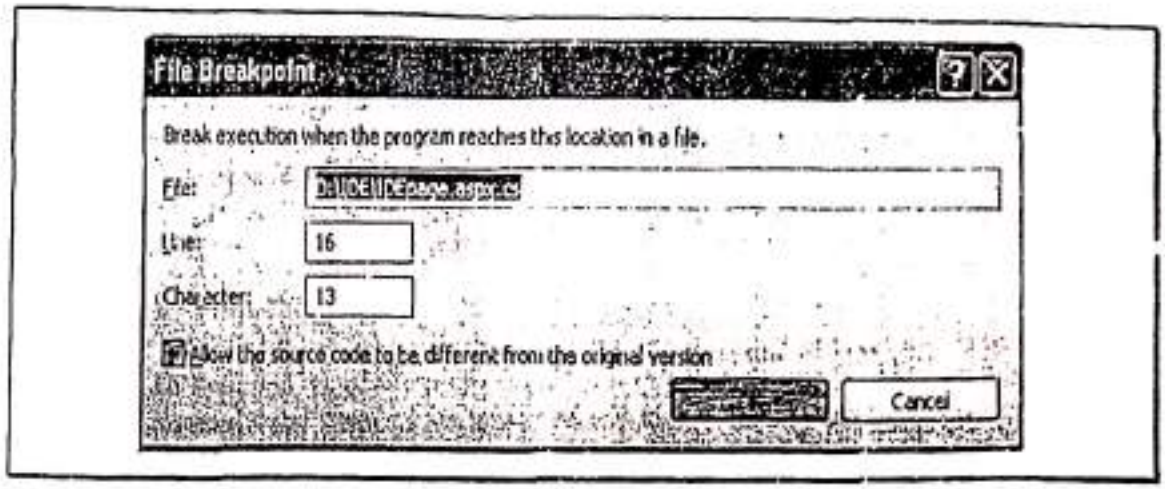


Figure 4.20: Location Breakpoint

- **Condition:** Used to set an expression. We can choose to break when this expression is true or when it has changed **(see Figure 4.21).**

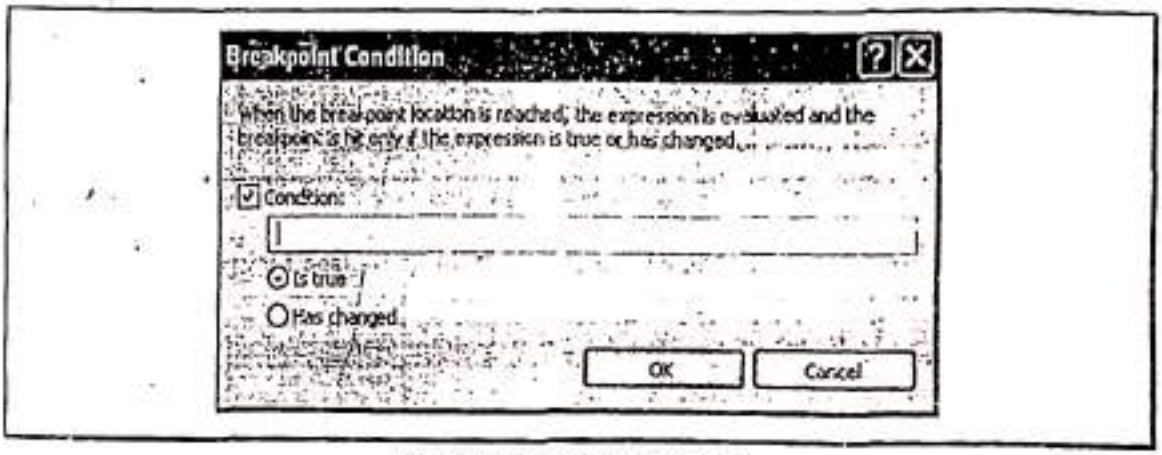


Figure 4.21: Condition Breakpoint

- **Hit Count:** Used to create a breakpoint that pauses only after a breakpoint has been hit a certain number of times (for example, at least 10) or a specific multiple of times (for example, every second time) as shown in the following figure **(Figure 4.22).**

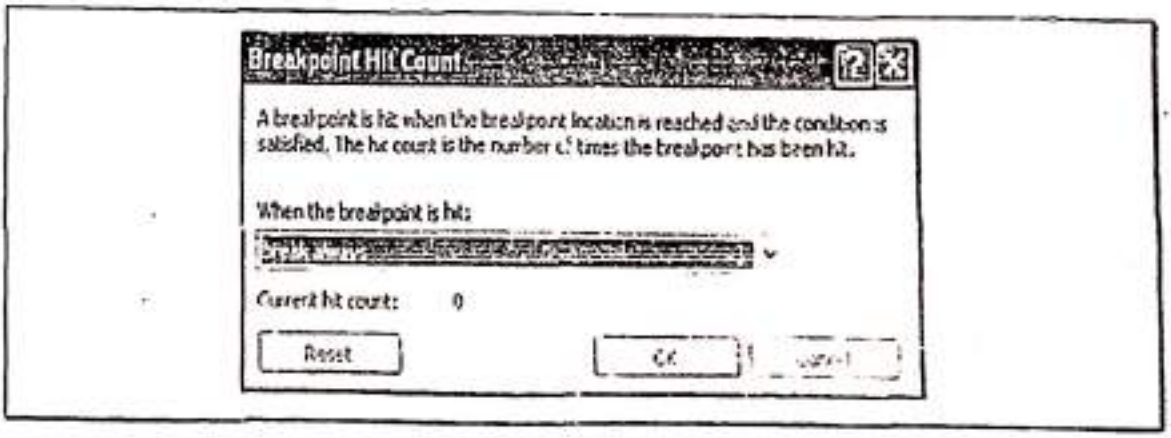


Figure 4.22: Hit Count Breakpoint

- **Filter:** Used to restrict the breakpoint to only being set in certain processes and threads (see Figure 4.23).

Figure 4.23: Filter Breakpoint

- **When Hit:** It specifies what to do when the breakpoint is hit (see Figure 4.24).

Figure 4.24: When Hit Breakpoint

4.13.3 Variable Watches

In some cases, we might want to track the status of a variable without switching into break mode repeatedly. In this case, it's more useful to use the Locals, Autos, and Watch windows, which allow us to track variables across an entire application. **Table 4-Y2** describes these windows.

Table 4-Y2: Variable Tracking Windows

Window	Description
Locals	Automatically displays all the variables that are in scope in the current procedure. This offers a quick summary of important variables.
Autos	Automatically displays variables that Visual Studio determines are important for the current code statement. For example, this might include variables that are accessed or changed in the previous line.
Watch	Displays variables we have added. Watches are saved with our project, so we can continue tracking a variable later. To add a watch, right-click a variable in code, and select Add Watch, alternatively, double-click the last row in the Watch window, and type in the variable name.

Each row in the Locals, Autos, and Watch windows provides information about the type or class of the variable and its current value. If the variable holds an object instance, we can expand the variable and see its private members and properties. For example, in the Locals window we will see this variable, which is a reference to the current page class. If we click the plus symbol next to this, a full list will appear that describes many page properties (and some system values), as shown in the following figure (**Figure 4.25**)

Figure 4.25: Viewing Locals window

The Locals, Autos, and Watch windows allow us to change variables or properties while our program is in break mode. Just double-click the current value in the Value column, and type in a new value. If one of the watch windows is missing, then we can show it manually by selecting it from the **Debug -> Windows submenu**.

4.13.4 Debugging ASP.NET pages using the ASP.NET Trace functionality

When it comes to catching programming errors, the debugger is a developer's best friend. ASP.NET tracing, however, is a nice complement to the debugger and shouldn't be overlooked. It enables our ASP.NET code to emit messages during processing, offering valuable information that can prove quite useful for detecting problems.

The two techniques are a bit different. Using a debugger is an inherently interactive technique that relies on being able to pause execution and examine current state. Tracing is less intrusive, simply tracking information being emitted by the code (*similar to the classic "printf" style of debugging*). At the end of the execution, we take the output produced by the tracing activity and analyze it.

The ASP.NET tracing subsystem exposes an API through which any control, page, or component can add its own trace output. We can output any information available at run time that makes sense to us.

In ASP.NET, tracing is tightly integrated with the page and the runtime pipeline. We can enable tracing by turning on an attribute in individual pages. To enable tracing for all pages in an application, we have to change a value in the configuration file (*Discussed in the following section*). This means that it is possible to enable tracing for an application after it has been deployed.

ASP.NET tracing enables us to view diagnostic information about a single request for an ASP.NET page simply by enabling it for a page or application. It enables us to follow a page's execution path, display diagnostic information at run time, and debug our application.

Through tracing, we can analyze data communicated between a browser and a Web server, including request details, timing information, server variables, cookies, headers, session state, and so on. When tracing for a page is enabled, it can append diagnostic information to the page's output. The trace output is divided into 12 distinct sections, as outlined in Table 4-Z1.

Table 4-Z1: Trace Output Sections

Section Name	What It Displays
Request Details	Information about the current request, such as status code, verb, session ID, and timestamp.
Trace Information	System- and user-defined messages sorted by time or category.
Control Tree	The hierarchy of the controls in the page, including each corresponding view state size.
Session State	All the items stored in Session.
Application State	All the items stored in the Application intrinsic object.
Request Cookies Collection	Name and content of all cookies attached to the request.
Response Cookies Collection	Name and content of all cookies attached to the response.
Headers Collection	Name and content of all request headers.
Response Headers Collection	Name and content of all headers attached to the response.

Form Collection	Name and content of all input fields packed in the HTTP response.
Querystring Collection	Name and content of all query string parameters
Server Variables	Name and content of all server variables available

Trace capability of ASP.NET is declared in the *TraceContext* class. The *TraceContext* class is a non-inheritable public class of the .NET Framework that is used for capturing the execution details of a web request and presenting the data on the page. We can use the methods of the *TraceContext* class to emit specific messages (*custom diagnostic messages*) and information in the *Trace Information* section. We can use the following code:

```
Trace.Write("Write Message Here!");
```

TraceContext class also features a *Warn* method. *Warn* differs from *Write* in that it renders its text in *red* color. It can be used as follows:

```
Trace.Warn("Warning!");
```

Both the *Write* and *Warn* methods have a few overloads, given below:

```
/*---------- Write overloads ----------*/
// To write a Trace Message to the trace log.
    void TraceContext.Write(string message)

//To write Trace Information to the trace log, including trace message and any user defined category.
    void TraceContext.Write(string category, string message)

//To write Trace Information to the trace log, including trace message, any user defined category and error Information.
    void TraceContext.Write(string category, string message, Exception errorInfo)

/*---------- Warn overloads ----------*/
// To write a Trace Message to the trace log. All warnings are appear in the log as red text.
    void TraceContext.Warn(string message)

//To write Trace Information to the trace log, including trace message and any user defined category. All warnings are appear in the log as red text.
    void TraceContext.Warn(string category, string message)

//To write Trace Information to the trace log, including trace message, any user defined category and error Information. All warnings are appear in the log as red text.
    void TraceContext.Warn(string category, string message, Exception errorInfo)
```

4.13.4.1 The <trace> Element

Most of the tracing attributes can be controlled from within the configuration file. There is an option that allows us to capture trace information without emitting text in the page served to the

browser. This feature is extremely useful for tracking data inconsistencies, monitoring the flow, and asserting conditions on applications already deployed in a production environment. The configuration section of interest is <trace>, which looks like as follows:

```
<trace enabled="true/false"
       localOnly="true/false"
       mostRecent="true/false"
       requestLimit="integer"
       pageOutput="true/false"
       traceMode="SortByTime/SortByCategory"
       writeToDiagnosticsTrace="true/false" />
```

These can be explained as in **Table 4-Z2**

Table 4-Z2: Attributes of the Trace section

Section Name	What It Indicates
Enabled	Whether tracing is enabled through the *trace.axd* viewer. This attribute alone is not sufficient to append trace information to all pages. By default it is *false*.
LocalOnly	Whether the *trace.axd* viewer should be available only on the local Web server. By default it is *true*.
MostRecent	The tracing system only maintains tracing information on a specific number of requests, as determined by *requestLimit*. When *mostRecent* is *true*, information from newer requests overwrites that from older requests when the limit has been reached. By default it is *false*, meaning that trace data is recorded only until the limit is reached, causing subsequent traces to be lost.
PageOutput	Whether trace output is rendered inside each page of the application. If *false*, trace output is only accessible through the *trace.axd* viewer. By default it is *false*.
requestLimit	The number of trace requests to store on the server. By default it is *10* and the maximum is *10000*.
TraceMode	The order in which trace information is displayed: sorted by *category* or by *time*. By default it is *SortByTime*.
writeToDiagnosticsTrace	Whether trace output should be forwarded to any .NET trace listeners that are registered. By default it is *false*

4.13.4.2 Tracing Levels

Tracing can be enabled for a page or an application. So, ASP.NET allows tracing at two levels:

- Page-level Tracing.
- Application-level Tracing.

(A) Page-level Tracing

We can control whether tracing is enabled or disabled for a page with the *Trace* attribute of the *@Page* directive. To enable page level tracing use the following statement:

```
<%@ Page Trace="true" %>
```

Security Note: When tracing is enabled for a page, trace information is displayed on any browser that makes a request for the page from the server. Tracing displays *sensitive* information, such as the values of server variables, and can therefore represent a security threat. So, be sure to disable page tracing for the page before porting application to a production server. We can do this by setting the *Trace* attribute to *false*, or by removing it.

Regardless of whether we write messages to the trace log, when we enable tracing and the page is requested, ASP.NET appends a series of tables containing diagnostic information about the page request. *(see Program 4-35)*

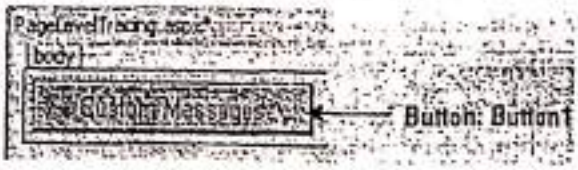
Program 4-35

Description of the Program 4-25: In this program, we are going to demonstrate the use of Page Level Tracing with Custom messages and Warnings. Follow the steps given below.

Step-1: Create a new **Web Application** named **"PageTrace"**.

Step-2: Add a **Web Form** to the application named **"PageLevelTracing.aspx"**.

Step-3: Create the following design.



Step-4: Set following properties of various Controls.

Control	Porperty	Value
Button1	Text	Custom Message

Step-5: Set Trace attribute of *@Page* directive of the page in source i.e. *Trace="true"*.

```
//Note: The whole source code will be as follows:
<%@ Page Language="C#" AutoEventWireup="true"
        CodeBehind="PageLevelTracing.aspx.cs"  Inherits="PageTrace.PageLevelTracing"
        Trace="true"
%>

<html xmlns="http://www.w3.org/1999/xhtml">
<body>
  <form id="form1" runat="server">
  <div>
```

```
    <asp:Button ID="Button1" runat="server" onclick="Button1_Click"
    Text="Custom Messages" />
  </div>
  </form>
</body>
</html>
```

Step-6: Double click on page. Add the following code in event handler **Page_Load(....).**

```
    protected void Page_Load(object sender, EventArgs e)
    {
        Trace.Write("Custom Message Traced on Page Load!");
}
```

Step-7: Double click on button Button1. Add the following code in event handler **Button1_Click(....).**

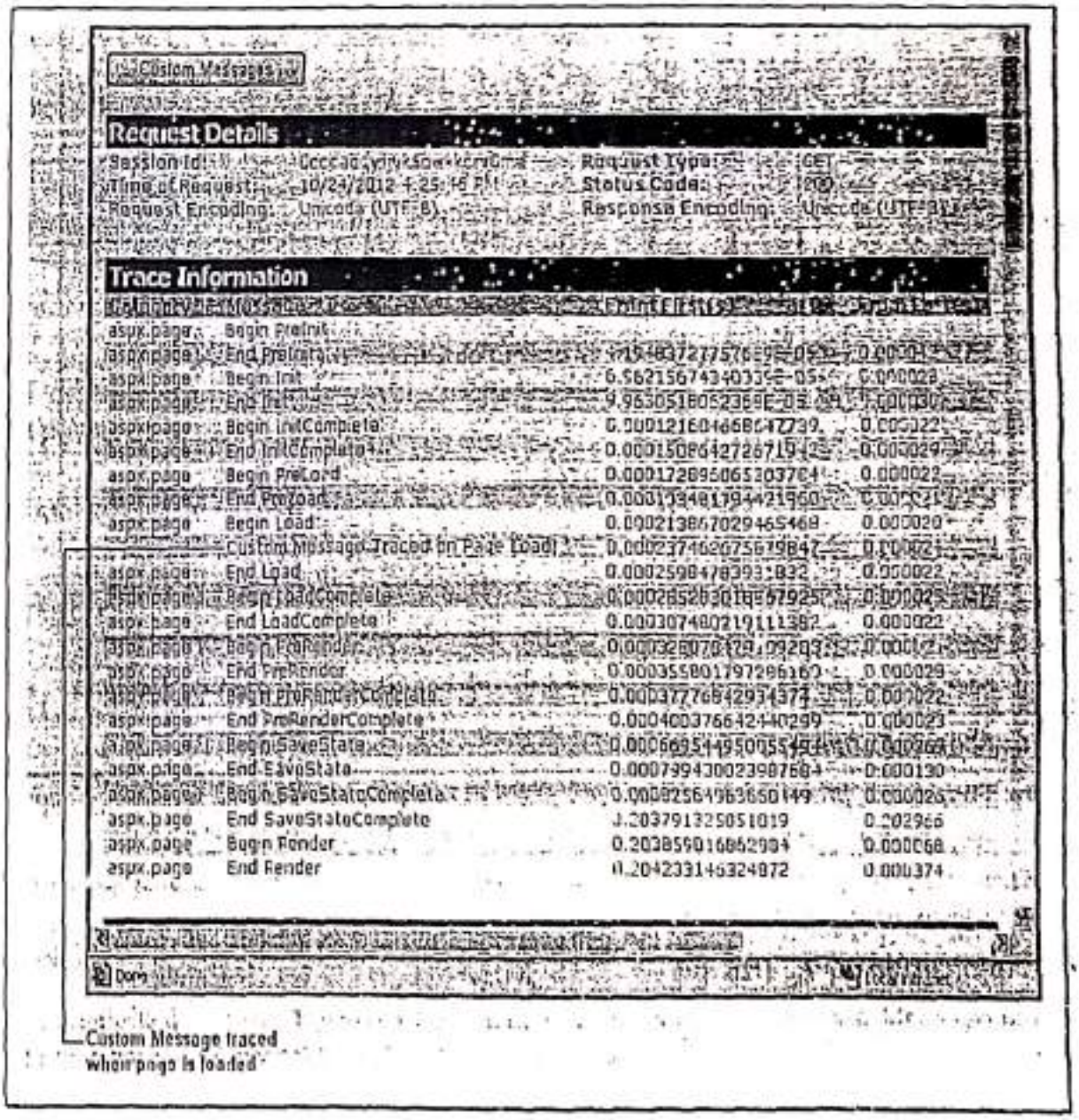
```
    protected void Button1_Click(object sender, EventArgs e)
    {
        Trace.Write("My Message being traced!");
        Trace.Warn("My Warning","This is my first Warning!");
}
```

Note: The whole code behind file will be as follows:

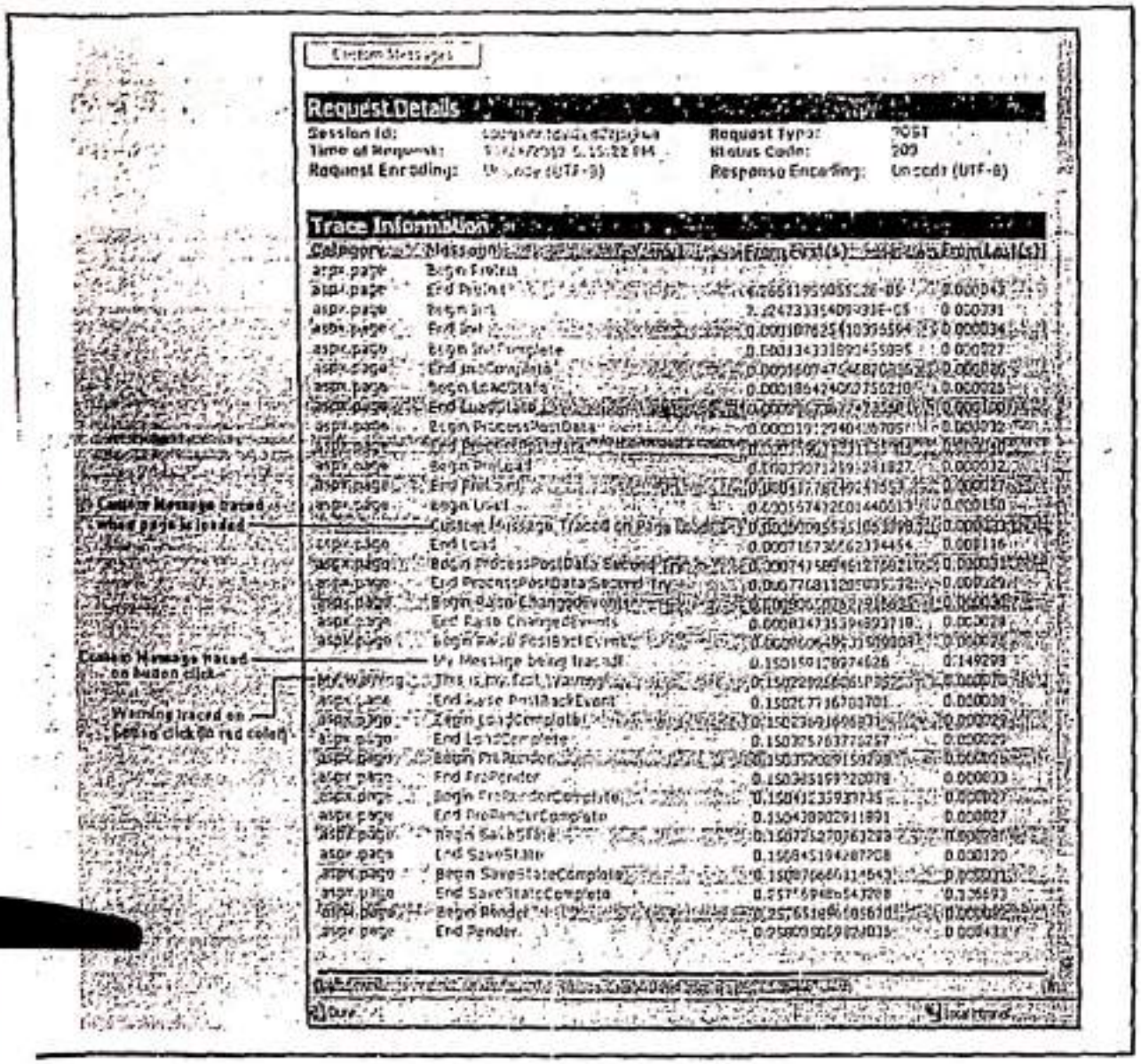
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace PageTrace
{
  public partial class PageLevelTracing : System.Web.UI.Page
  {
    protected void Page_Load(object sender, EventArgs e)
    {
        Trace.Write("Custom Message Traced on Page Load!");
    }

    protected void Button1_Click(object sender, EventArgs e)
    {
```

```
        Trace.Write("My Message being traced!");
        Trace.Warn("My Warning", "This is my first Warning!");
    }
}
```

Step-8: Press **F5**. The output will be as follows:



In output window, see, the custom message that we set in **Page_Load(....)** event is traced but not the message and warning set in **Button1_Click(....)** event. This is because *Click()* event of button is not yet fired. So, click on button to trace the message and warning that we set in **Button1_Click(....)** event and see the output as follows:



Here's the end of **Program 4-35.**

Note: Suppose we have added two or more pages in a web application and we want to apply page level tracing on them then we have set *Trace="true"* in *@Page* directive of each page.

When Tracing is disabled in our page, trace information is not gathered, and trace statements e write do not appear on the page.

B) Application-level Tracing

You can enable tracing for an entire application in the *Web.config* file in the application's root rectory. It means we need not to set *Trace* attribute to *true*, in *@page* directive of each page of a eb application containing multiple pages. This can be done as follows.

```
<configuration>
    <system.web>
        <trace enabled="true"
```

```
            pageOutput="true"
            traceMode="SortByTime"/>

    </system.web>
</configuration>
```

By default, application-level tracing can be viewed only on the local Web server computer. You must set the *localOnly* attribute to *false* in the *Web.config* file to make application-level trace information visible from remote computers.

Security Note: To keep our Web application secure, use the remote trace capability only when we are developing or deploying our application. Be sure to disable it before we transfer our application onto production Web servers. To disable remote tracing, set the *localOnly* attribute to *true* in the *Web.config* file.

The following program ***(Program 4-36)*** shows an application trace configuration that collects trace information for up to 20 requests and allows browsers on computers other than the origin server to display the trace viewer.

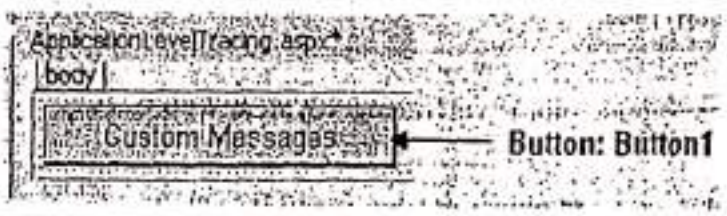
Program 4-36

Description of the Program 4-36: In this program, we are going to demonstrate the use of Application Level Tracing with Custom messages and Warnings. Follow the steps given below.

Step-1: Create a new **Web Application** named **"ApplicationTrace"**.

Step-2: Add a **Web Form** to the application named **"ApplicationLevelTracing.aspx"**.

Step-3: Create the following design.



Step-4: Set following properties of various Controls.

Control	Porperty	Value
Button1	Text	Custom Message

Step-5: Add the following code to the *Web.config* file.

```
<configuration>
    <system.web>
        <trace enabled="true"
               requestLimit="10"
               pageOutput="true"
               traceMode="SortByTime"/>
    </system.web>
</configuration>
```

Step-6: Double click on page. Add the following code in event handler **Page_Load(....).**

```
protected void Page_Load(object sender, EventArgs e)
{
        Trace.Write("Custom Message Traced on Page Load!");
}
```

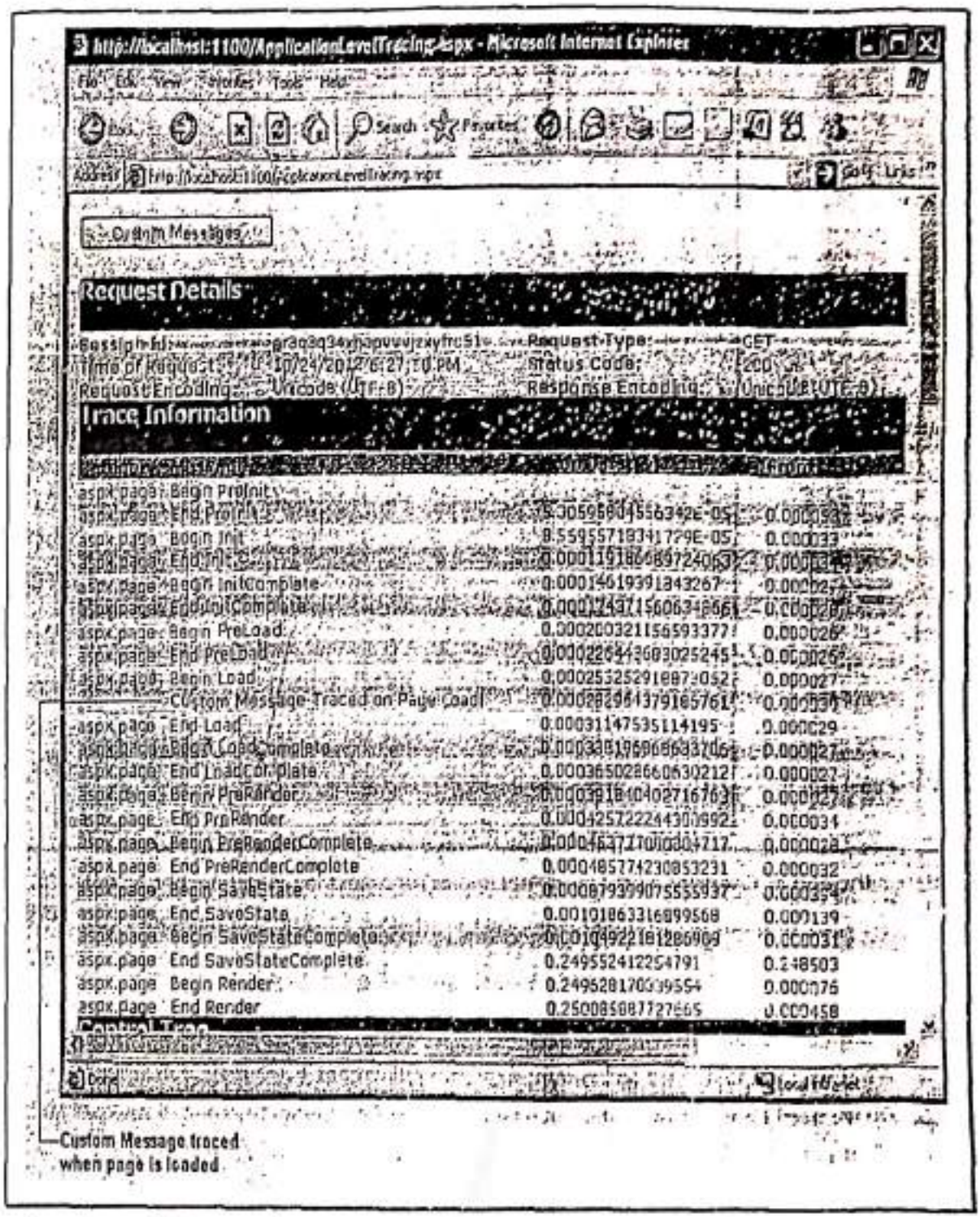
Step-7: Double click on button Button1. Add the following code in event handler **Button1_Click(....).**

```
protected void Button1_Click(object sender, EventArgs e)
{
        Trace.Write("My Message being traced!");
        Trace.Warn("My Warning","This is my first Warning!");
}
```

Note: The whole code behind file will be as follows:

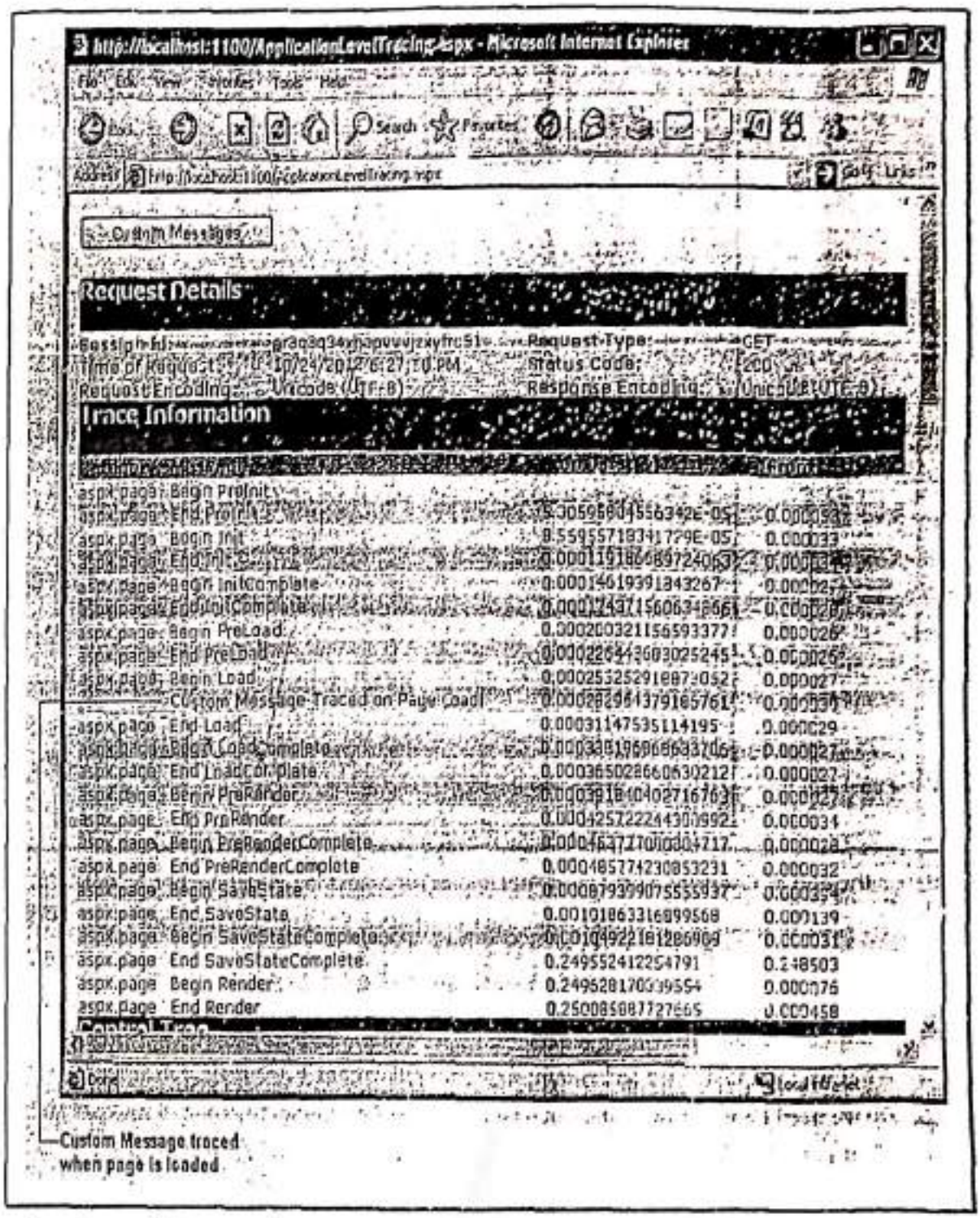
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace ApplicationTrace
{
  public partial class AppplicationLevelTracing : System.Web.UI.Page
  {
      protected void Page_Load(object sender, EventArgs e)
      {
          Trace.Write("Custom Message Traced on Page Load!");
      }

      protected void Button1_Click(object sender, EventArgs e)
      {
          Trace.Write("My Message being traced!");
          Trace.Warn("My Warning","This is my first Warning!");
      }
  }
}
```

Step-8: Press **F5**. The output will be as follows:



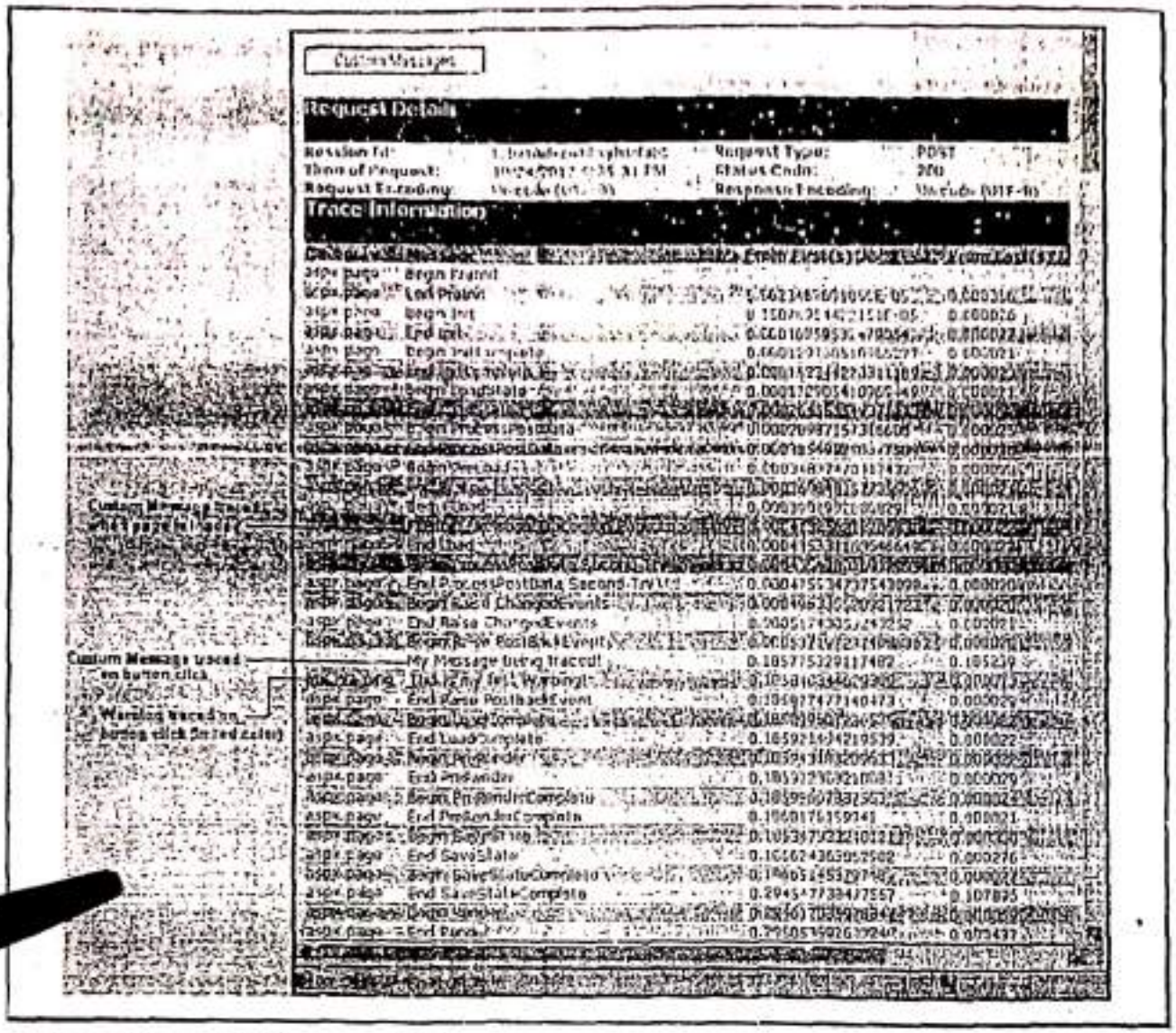
In output window, see, the custom message that we set in **Page_Load(....)** event is traced but not the message and warning set in **Button1_Click(....)** event. This is because *Click()* event of button is not yet fired. So, click on button to trace the message and warning that we set in **Button1_Click(....)** event and see the output as follows:

Step-8: Press **F5**. The output will be as follows:



In output window, see, the custom message that we set in **Page_Load(....)** event is traced but not the message and warning set in **Button1_Click(....)** event. This is because *Click()* event of button is not yet fired. So, click on button to trace the message and warning that we set in **Button1_Click(....)** event and see the output as follows:

to trace the message and warning that we set in



Here's the end of *Program 4-36.*

REVIEW QUESTIONS

1. What does an asp net application includes?Explain the various features of asp.net.
2. What we need to develop and run the asp.net applications?
3. Do we need IIS to run Web applications?
4. What is PostBack in asp.net?
5. Explain web page elements. What does the code runat="server" means?
6. Explain the elements of Visual Studio IDE?
7. How IntelliSense and Outlining helps to code an application?
8. What do you mean by Error Underlining?

9. What are the various ways to create an asp.net application? Demonstrate each by creating an asp.net application.
10. What is deployment? How we can deploy an asp.net application?
11. Explain User Interface (UI) elements. What are the characteristics of UI elements?
12. Which class is the base class for all asp.net web controls?
13. List the categories of asp.net controls?
14. Explain various standard controls of asp.net?
15. Create an application to demonstrate the use of various standard controls of asp.net.
16. What is event? Explain the stages of page life cycle events in order. Create a web application to demonstrate the concept.
17. What do you mean by event handler? Generally which event handler is used to handle the click event of a button control?
18. Which asp.net control is used to display a series of advertisements? Explain by creating a suitable application
19. Explain various properties of calendar control.
20. How we can upload a file to the web server using a web application?
21. How we can create multiple views of page? Demonstrate the concept by giving step by step creation of a web application.
22. What is a user control?
23. What are the differences between user controls and web pages?
24. What are the differences between user controls and custom controls?
25. What are the various ways to create a custom control?
26. What are the various steps to steps of authoring a custom server control?
27. Develop a web application to create a custom control.
28. How to add a custom control in a Toolbox tab? How to use a custom control?
29. Why validation is required? Explain various validation controls.
30. Should validation occur server-side or client-side? Why?
31. Which control would you use if you needed to make sure the values in two different Controls matched? Explain by creating a web application for this.
32. What is debugging? Explain single-step-debugging.
33. Explain breakpoints and variable watches.